INSTRUCTOR KIT · MYSQL FOUNDATIONS · COMPLETE FACILITATOR GUIDE

MySQL Foundations

The full teaching manual for the 14-session MySQL Foundations kit (Modules 1 to 8, in two parts; one 3-hour session, two lessons each). Timing, live-demo scripts, the mistakes to pre-empt, lab facilitation, and checks for understanding, for every session.

14 sessions · 25 lessons | 14 sessions × 3 h · 42 h | Instructor copy · not for students

CONTENTS

PART 1: SQL FOUNDATIONS · SESSIONS 1 TO 7

S01 **What Databases Are and Installing MySQL**

- 1.1 What Is a Database?
- 1.2 Installing MySQL

S02 **The MySQL Client and Your First Tables**

- 1.3 MySQL CLI & phpMyAdmin
- 2.1 Creating Databases & Tables

S03 **Data Types, Constraints and CRUD**

- 2.2 Data Types & Constraints
- 2.3 CRUD Operations

S04 **Filtering, Sorting and Limiting**

- 3.1 WHERE Clauses, Operators & Filtering
- 3.2 Sorting & Limiting Results

S05 **Aggregates and Subqueries**

- 3.3 Aggregate Functions & GROUP BY
- 3.4 Subqueries & Nested Queries

S06 **Relationships and Foreign Keys**

- 4.1 Table Relationships
- 4.2 Foreign Keys & Referential Integrity

S07 **Joins and the Part 1 Assessment**

4.3 Joins

PART 2: BEYOND THE BASICS · SESSIONS 8 TO 14

S08 CTEs and Window Functions

5.1 CTEs & Recursive Queries

5.2 Window Functions

S09 Indexes and Views

6.1 Indexes & Query Optimization

6.2 Views

S10 Normalization and Stored Programs

6.3 Database Design & Normalization

7.1 Stored Procedures & Functions

S11 Transactions and JSON

7.2 Transactions

7.3 JSON Columns

S12 Users, Permissions and Backups

7.4 User Management & Permissions

8.1 Importing & Exporting Data

S13 Final Project Kickoff and Studio I

8.2 Final Project: Design and Build a Database

S14 Final Project Studio II, Showcase and Next Steps

8.2 Final Project: Design and Build a Database

8.3 Next Steps & Best Practices

MYSQL FOUNDATIONS · FACILITATOR GUIDE

Part 1: SQL Foundations

Sessions 1 to 7. Modules 1 to 4: install MySQL, build and change tables, query, summarize and join. Runs on its own as a short course and ends with the Part 1 assessment.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 1: What Databases Are and Installing MySQL

How to teach the first session of MySQL Foundations. Welcome a room of beginners, find out which machines have a working MySQL 9.7 and what the room already knows about spreadsheets and files, then show why databases exist (a sheet that cannot keep one customer's name straight, a server that refuses bad rows with numbered errors) and make every laptop ready: an install clinic, connect and verify, and a health check learners can rerun all course.

2 lessons | 3 hrs session | Lessons 1.1-1.2 | Session 1 of 14 · Part 1

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's ideas) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Type, do not paste. Every SQL block below was run to produce the output printed under it, on MySQL 9.7.2 with the default settings. Ask for a prediction before every statement that prints something, and read error messages from the left, out loud: the number, then the reason.
- The lessons are the reference. Keep `lesson_01.html` and `lesson_02.html` open in browser tabs for the diagrams (the application, the server and the files; client, port and server) and the per-platform installation sections. Labs live in `labs/module-01/` : `_starter.sql` files are handouts, `_solution.sql` files are yours.
- Session 1 is different in two ways. It opens with 30 minutes of welcome, environment check and diagnostic instead of the usual 10-minute warm-up, and Lesson 1.2's room time is mostly an **install clinic**: installing MySQL is prework (the kit's setup guide follows Lesson 1.2), so the room fixes what went wrong instead of watching progress bars.
- Part 1 (Sessions 1 to 7) can run on its own as a short course; nothing in this session depends on Part 2. Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (1.1, 1.2) match the course site.

> **THE #1 RISK OF SESSION 1: THE INSTALL EATS THE SESSION**
>
> MySQL installs differently on every platform, and each one has its own trap: the Windows MSI does not put `mysql` on the PATH; Homebrew's `mysql@9.7` is keg-only; Ubuntu's own package is 8.0 or 8.4 and logs root in through `sudo`; Docker needs a free port 3306. Done together on venue Wi-Fi, installation fills three hours and nobody writes a line of SQL. **Before:** send the setup guide a week ahead (install MySQL 9.7 with Lesson 1.2's section for your platform, then run the environment check) and ask every learner to reply with the check's last table. **In the room:** the check gets 12 minutes in the opening; a NOT READY machine gets a ready partner until the 25-minute install clinic at 1:45, not your attention during Lesson 1.1. **After:** whoever is still not ready leaves with a numbered plan and runs the check again before Session 2.

BEFORE THE SESSION

- ☐ **A week before:** email the setup guide (install MySQL 9.7 LTS following Lesson 1.2's section for your platform, set a strong root password, then run `labs/module-01/l1_0-orientation/check_environment.sh` on macOS and Linux or `labs/module-01/l1_0-orientation/check_environment.ps1` on Windows). Ask learners to reply with the last table it prints. No reply means an install risk: chase it.
- ☐ **Files to hand out** (shared folder or USB stick, in case the network fails): the whole `labs/` folder **without** the `*_solution.sql` files, and the workbook. Learners keep `labs/` in the course folder the setup guide has them create (`mysql-course` in their home folder) and open their terminal there, so `SOURCE labs/...` paths work. Put the platform installers on the stick too (Windows MSI, macOS DMG for both processor types): a 200 MB download per learner over venue Wi-Fi is how a clinic stalls.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-01` from this kit's folder. Every line should say PASS (13 files, a few seconds). Then run the environment check on the projector machine so you can show what READY looks like.
- ☐ **Downloads:** none for the course database today (Lesson 2.1 builds `bookstore` in Session 2). The Docker image `mysql:9.7` (about 250 MB) if you will demonstrate Docker.
- ☐ **Projector machine:** MySQL 9.7 running, a terminal open in your course folder (the one with `labs/`) at 18 pt or larger, logged out (you log in during the 1.2 demo), and a second terminal already in the `mysql>` client for the 1.1 demo. Browser tabs: the course home page, `lesson_01.html`, `lesson_02.html`.
- ☐ **Board, before learners arrive:** three boxes, "client (you type)" → "server (mysqld, port 3306)" → "data on disk", with "a database lives inside the server" written under the middle box; the error triage line "command not found = PATH · 1045 = credentials · 2002/2003 = server stopped or wrong address"; and an empty box headed "CRUD".
- ☐ **Print or share:** the workbook (its Start Here page has the environment checklist and the diagnostic), sticky notes for help flags and the exit ticket, and paper for the 1.1 project sketch.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
Windows: `mysql` "is not recognized as the name of a cmdlet"	The MSI does not add the client to the PATH	Lesson 1.2, Windows Step 4: add `C:\Program Files\MySQL\MySQL Server 9.7\bin` to the user Path, then open a NEW PowerShell window.
macOS: `zsh: command not found: mysql` after `brew install mysql@9.7`	Versioned formulae are keg-only	The `export PATH="$(brew --prefix)/opt/mysql@9.7/bin:$PATH"` line from the lesson, in `~/.zshrc`, then a new Terminal window.
`SELECT VERSION();` says 26.x	`brew install mysql` or `mysql:latest` gave an innovation release	Fine for today; reinstall `mysql@9.7` / `mysql:9.7` at home to match the lesson outputs.
Ubuntu: version 8.0.x, or root works only with `sudo mysql`	The distribution's own package (and its auth_socket root)	Add MySQL's APT repository (Lesson 1.2), or set a password with the lesson's ALTER USER statement.
`ERROR 1045 (28000): Access denied`	Wrong password, a missing `-p`, or a Docker container whose password was set on an earlier first start	Retype carefully; Docker: `MYSQL_ROOT_PASSWORD` counts only on the very first start of a volume.
`ERROR 2002` (socket) or `ERROR 2003` (port)	The server is stopped, or the client asks the wrong place (`localhost` with Docker on macOS/Linux)	Start it (`Start-Service`, `brew services start mysql@9.7`, `sudo systemctl start mysql`, `docker start mysql97`); Docker: `-h 127.0.0.1 -P 3306`.
`docker run` fails: port is already allocated	Another MySQL, MariaDB, XAMPP or MAMP already listens on 3306	Stop the old server, or publish on another port: `-p 3307:3306`, then connect with `-P 3307`.
The check says `CHECK MariaDB, not MySQL`	XAMPP or a distribution's default package	Fine to follow along with the version notes; install MySQL 9.7 at home for matching outputs.
Windows: the `.ps1` check "cannot be loaded because running scripts is disabled"	PowerShell's execution policy	Run it with `powershell -ExecutionPolicy Bypass -File ...` as the README shows (nothing to change system-wide).
A locked work laptop: nothing can be installed	No administrator rights	Pair today. Docker Desktop or a personal laptop at home; MySQL's "ZIP Archive" for

SYMPTOM	CAUSE	FIX
		Windows runs without installing but needs manual setup (instructor help, not in the room).

SESSION 1 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	30	Welcome, environment check and diagnostic	Introductions, how the course runs, `check_environment` on every machine, a six-question spreadsheet-to-SQL diagnostic (not graded).
0:30	10	**1.1** Frame and mental model	The bookshop that outgrew its spreadsheet; the application, the server and the files; tables and keys.
0:40	20	**1.1** Live demo	The lesson's first taste of SQL typed live: a practice database, CRUD, three refused rows, cleanup.
1:00	15	**1.1** Guided lab	`labs/module-01/l1_1-what-is-a-database/order_sheet_starter.sql` in pairs: what one big sheet gets wrong.
1:15	10	**1.1** Start the lesson project	Sketch the bookstore on paper, Steps 1 and 2; Steps 3 to 5 go home.
1:25	10	Break	NOT READY machines line up for the install clinic. Leave the server, client and database picture on the board.
1:35	10	**1.2** Frame and mental model	Server, client, port; LTS versus innovation releases; one install path per laptop.
1:45	25	**1.2** Install clinic	Fix the machines that were NOT READY by platform; ready learners help, then rerun the check.
2:10	15	**1.2** Live demo	Connect and verify; the health-check queries; two pieces of old advice that fail on purpose.
2:25	15	**1.2** Guided lab	`labs/module-01/l1_2-installing-mysql/server_facts_starter.sql`: ask your server who it is.
2:40	10	**1.2** Start the lesson project	The installation health check, Steps 3 and 4; Step 5 goes home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 2.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 75 + 90 = 165 minutes for a solo learner, and after the 30-minute opening, the break and the wrap-up the room has 130. Lesson 1.1 keeps its full shape (model, demo, lab, project start). Lesson 1.2's installation sections are prework from the setup guide: the room uses its time for an install clinic, a connect-and-verify demo, a short lab and the start of the health check.

Goes home (about 2 to 3 hours). The bookstore sketch Steps 3 to 5 and the health check Step 5 (stop, fail, start); each lesson's Hands-on Exercises; both Quick Quizzes; both journal entries; Lesson 1.2's "Before the Next Lesson" list. Anyone still NOT READY finishes the installation with the numbered plan from the clinic.

Protect, in this order: the environment check result for every learner (nobody leaves without knowing whether their machine is ready and what to do next); the install clinic; the 1.1 demo through the three refused rows (the database saying no is the idea the whole course leans on); the order-sheet lab's TODO 3 (Dune Messiah). **Cut first:** the 1.2 demo Steps 9 and 10 (the old advice; hand out `debug_old_advice.sql` instead), then the 1.1 project start (send all five steps home), then the 1.2 project start (the lab already ran most of those queries).

OPENING SEGMENT

0:00 Welcome, Environment Check and Diagnostic — 30 min

TIMING

5m	Welcome and introductions
5m	How the course runs: 14 sessions in two parts, two lessons a session, the room and home split, the journal
12m	Environment check: `check_environment` ends with READY on every machine, or the machine gets a partner and a clinic slot
8m	Diagnostic: six spreadsheet and computer questions, not graded

SEQUENCE AND TALKING POINTS

- **Welcome (5m).** One minute on yourself, including a time when data in the wrong place cost someone (a spreadsheet with two versions of the truth works well). Then a quick round: name, where they meet data today (spreadsheets, a web shop, reports), and one question they wish they could ask their data. Write three or four on the board; they come back in the 1.1 journal prompt. Housekeeping in one breath: break at 1:25, where the lab files are, and "if you are stuck, put a sticky note on your laptop lid and keep trying while I get to you."
- **How the course runs (5m).** Show the course home page. Say: "Fourteen sessions of three hours in two parts. Part 1, the first seven sessions, takes you from nothing to joining tables; Part 2 goes beyond the basics and ends with a database you design yourself. Every lesson uses the same online bookstore, so by the end you will know its books, customers and orders well." Then the deal: "Each lesson was written for someone working alone for 75 to 110 minutes, so two do not fit in three hours. In this room we do what is hard alone: the mental model, a live demo, a guided lab, and the first steps of each project. At home you finish the projects and do the exercises, quiz and journal: plan on two to three hours a week." Scroll once through `lesson_01.html`: objectives, sections with real output, common mistakes, a project in steps, exercises with hidden solutions, a quiz, a journal prompt.
- **Environment check (12m).** Say: "Before any new ideas, we find out which machines are ready." On the projector, then theirs: **1** open a terminal in the course folder (the one with `labs/`); **2** macOS/Linux: `bash labs/module-01/l1_0-orientation/check_environment.sh`, Windows: `powershell -ExecutionPolicy Bypass -File labs\module-01\l1_0-orientation\check_environment.ps1`; **3** type the root password when asked. Done = the last table says `READY for MySQL Foundations` (`READY WITH NOTES` on 8.4 or MariaDB is fine for today). Anyone who saw READY at home runs it again and becomes a helper. NOT READY falls into three piles: client not found (PATH: often one minute, fix now), 1045 / 2002 / 2003 (password or server stopped: two

minutes, fix now), nothing installed (do not start a download now: pair with a ready neighbor; install clinic at 1:45). Write the third pile's names on a sticky note.

- **Diagnostic (8m).** Say: "Six quick questions on the spreadsheet and computer skills SQL builds on. Not a test, nobody collects it. It tells you what to practice and tells me what to slow down for." Learners answer D1 to D6 on the workbook's Start Here page (about four minutes), then show fingers (1 to 4 for A to D) as you reveal each. Keep a tally of wrong answers per question on a sticky note.

DIAGNOSTIC ANSWERS AND HOW TO READ THE TALLY

Answers: **D1** B) 2; **D2** C) Dune; **D3** A) 1; **D4** D) 44.97; **D5** B) 5; **D6** B) `Downloads/bookstore.sql`.

- **D1 (filter) and D2 (sort)** are WHERE and ORDER BY in disguise: the shape of every query in Module 3. Misses here are rare; if a third of the room misses either, read the 1.1 demo's SELECT aloud as a filter ("keep the rows where the city is Chicago").
- **D3 (AND versus OR)** is the single best predictor of trouble in Lesson 3.1. Two or more misses: in the 1.1 lab, do TODO 1 together and say the WHERE line aloud each time.
- **D4 (a formula column) and D5 (empty cells)** come back as expressions in SELECT and as NULL. Misses on D5 are common and not urgent: Lessons 2.2 and 3.3 teach NULL properly.
- **D6 (paths)** predicts trouble with `SOURCE labs/...` today and with running `.sql` files in Session 2. Learners who missed it get the typing role in the lab pair, with you nearby for the first SOURCE.
- **Three or more wrong:** not a problem to announce. Pair them with a strong neighbor today, and at the wrap-up hand them `labs/module-01/l1_0-orientation/diagnostic_starter.sql`: the same six moves done in SQL, one small gap each (`diagnostic_solution.sql` is your reference). The workbook's Start Here page says where each skill comes back.

WATCH-OUTS

- **Do not debug one machine in front of the room.** Two minutes per machine during the check, then pair and move on: the install clinic at 1:45 exists so that you can say "we will fix that at 1:45" with a straight face.
- **Mixed backgrounds.** Some learners live in spreadsheets and have never opened a terminal; some write code and have never designed a table. The diagnostic and the terminal steps are for both groups.
- **"I already use SQL."** Welcome them and give them a job: helper during the check and the clinic, and the stretch tasks in each lab. Ask them to watch for the habits today is about (reading error numbers, checking which server answered), which experienced people skip most.
- **MariaDB and other versions.** XAMPP and some hosts ship MariaDB under the name MySQL. Today it only changes a few outputs; the check says so in a CHECK line. Do not reinstall in the room.

LESSON PLANS

1.1 What Is a Database? — 55 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain what a database and a database management system (DBMS) are, and name three problems a database solves that a spreadsheet does not
- Tell relational databases apart from the main non-relational kinds, and pick a sensible type for a simple scenario
- Point to the table, columns, rows, primary key and foreign key in a small example
- Describe what SQL is and match the four CRUD operations to INSERT, SELECT, UPDATE and DELETE
- Sketch the tables of a small online bookstore on paper, with keys and relationships

TIMING IN THE ROOM

10m	Frame: the bookshop that outgrew its spreadsheet; application, server, files; tables, rows, keys
20m	Live demo: the lesson's first taste of SQL, three refused rows, cleanup
15m	Guided lab: `order_sheet_starter.sql` in pairs
10m	Start the lesson project: sketch the bookstore, Steps 1 and 2

MENTAL MODEL TO SAY

Tell the story while you draw it: "A small online bookstore keeps everything in one spreadsheet: one row per order, with the customer's name, email and city, and the books typed into a cell. It works for fifty orders. At five thousand, Alice's name is spelled three ways, Bob lives in two cities at once, and two people saving the file overwrite each other."

Then the three boxes on the board: "Your application never touches the data files. It sends SQL to a **server**, the DBMS, and the server reads and writes the files, enforces the rules and answers. MySQL is that server. A **database** is your data inside it."

Then the one idea of relational design, pointing at two small tables you draw: "Store each fact once, and point to it with a key. Alice lives in the customers table exactly once, with id 1. Every order stores only her id. Change her email in one place and every order is right." Write **primary key** (tells rows apart) and **foreign key** (points to another table's row) next to the two columns.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

This is the lesson's "A First Taste of SQL" and its Common Mistakes, typed live in the `mysql>` client on the projector. Ready learners type along; the rest watch and get their turn after the clinic. Ask for a prediction

before every statement that prints something; read every error number aloud.

1. **A practice database, and a table (3m).** "Everything I type goes to the MySQL server; the server does the reading and writing. First a practice database, then a table for customers. Every column gets a name, a type and its rules." Read the column lines aloud: AUTO_INCREMENT hands out ids, NOT NULL means required, UNIQUE means no two alike.

```
CREATE DATABASE preview_shop;    -- a new, empty database
USE preview_shop;                -- work inside it from now on
CREATE TABLE customers (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50)  NOT NULL,
    last_name  VARCHAR(50)  NOT NULL,
    email      VARCHAR(100) NOT NULL UNIQUE,
    city       VARCHAR(50)
);
```

```
Database changed
```

MySQL only answers `Database changed` after `USE`; the other two statements succeed quietly. Ready learners type along; the others watch (they install at 1:45).

2. **Create: three customers (2m).** "INSERT is the C in CRUD. I leave out id: MySQL fills it in."

```
-- @status
INSERT INTO customers (first_name, last_name, email, city)
VALUES ('Alice', 'Johnson',  'alice@example.com', 'New York'),
       ('Bob',   'Smith',    'bob@example.com',   'Chicago'),
       ('Carol', 'Williams', 'carol@example.com', 'Los Angeles');
```

```
Query OK, 3 rows affected
Records: 3  Duplicates: 0  Warnings: 0
```

3. **Read: one city (2m).** "SELECT is the R. You say WHAT you want, never HOW to find it: that is what declarative means." Ask for a prediction first: how many rows?

```
SELECT first_name, last_name, city
FROM customers
WHERE city = 'Chicago';
```

```
+------------+-----------+---------+
| first_name | last_name | city    |
+------------+-----------+---------+
| Bob        | Smith     | Chicago |
+------------+-----------+---------+
```

Read the statement as English: select first name, last name and city from customers where the city is Chicago.

4 **Update: Bob moves (2m).** "The WHERE line picks the one row to change. Without it, everybody moves to Boston."

```
-- @status
UPDATE customers
SET city = 'Boston'
WHERE id = 2;
```

```
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0
```

5 **Delete, then look at everything (2m).** "DELETE is the D. SELECT star means all columns." Point at the gap: Carol's id 3 is gone and will not be reused.

```
DELETE FROM customers
WHERE id = 3;
SELECT * FROM customers;
```

```
+----+------------+-----------+-------------------+----------+
| id | first_name | last_name | email             | city     |
+----+------------+-----------+-------------------+----------+
|  1 | Alice      | Johnson   | alice@example.com | New York |
|  2 | Bob        | Smith     | bob@example.com   | Boston   |
+----+------------+-----------+-------------------+----------+
```

6 **Refused 1: a duplicate key (2m).** "Now I break the rules on purpose. A spreadsheet takes anything; a database says no, with a number. Read the message from the left: number, then the reason."

```
INSERT INTO customers (id, first_name, last_name, email)
VALUES (1, 'Dave', 'Brown', 'dave@example.com');
```

```
ERROR 1062 (23000): Duplicate entry '1' for key 'customers.PRIMARY'
```

id 1 is Alice's. Fix: leave id out and let AUTO_INCREMENT choose.

7. **Refused 2: text where a number belongs (1m).** "id is a whole number. 'abc' is not."

```
INSERT INTO customers (id, first_name, last_name, email)
VALUES ('abc', 'Dave', 'Brown', 'dave@example.com');
```

```
ERROR 1366 (HY000): Incorrect integer value: 'abc' for column 'id' at row 1
```

8. **Refused 3: a required value missing (2m).** "last_name was declared NOT NULL and has no default." Then the silent fourth problem from the lesson, said aloud: copying a customer's name into every order raises no error at all; that is what the lab is about.

```
INSERT INTO customers (first_name, email)
VALUES ('Dave', 'dave@example.com');
```

```
ERROR 1364 (HY000): Field 'last_name' doesn't have a default value
```

9. **Clean up (1m).** "DROP DATABASE deletes a database and everything in it, with no undo. Read the name twice before you press Enter."

```
DROP DATABASE preview_shop;
```

No output on success. Draw the server / client / database picture again: preview_shop lived INSIDE the server and is now gone; the server is still running.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Server, client and database blur together.** "I installed MySQL but I can't find my database" or "is MySQL the database?" Coach with the board: point at the three boxes and ask "which box are you talking about?" Then the lesson's own line: the server runs in the background, the client is what you type into, a database lives inside the server and you create it.
- **A missing semicolon.** The client shows `->` and waits; learners think it froze. Coach: "What does the prompt look like? `->` means MySQL is waiting for the end of the statement: type `;` and Enter." (Lesson 1.3 covers the prompts properly.)
- **Treating every column of a sheet as one table.** In the project, a learner draws one big table with `book1`, `book2`, `book3` or a comma-separated list. Coach with the lab's TODO 3: "How would you find every order that contains Dune? What about Dune Messiah?" Then: "a list in a cell needs its own table, one row per item."

GUIDED LAB

File: learners open `labs/module-01/l1_1-what-is-a-database/order_sheet_starter.sql`; your reference is `labs/module-01/l1_1-what-is-a-database/order_sheet_solution.sql`. It practices the lesson's first section (why a spreadsheet is not enough) on a new sheet: each TODO changes one value in a ready-made query and reveals one spreadsheet problem.

- **SOURCE it first, as shipped (2m).** Show the command on the projector: `SOURCE labs/module-01/l1_1-what-is-a-database/order_sheet_starter.sql` from a client started in the course folder. Everyone should see the six-row sheet, three `Empty set` answers and a 0-row answer. A learner who sees `Failed to open file` started the client in another folder: `exit`, `cd` into the course folder, log in again.
- **TODO 1 together on the projector (3m), TODOs 2 to 4 in pairs (8m).** Each TODO is a one-word edit in a text editor, then SOURCE again. Circulate for quotes deleted along with the placeholder (an `ERROR 1064` syntax error: show them the quotes in the original) and for edits made in the client instead of the file.
- **Expected:** TODO 1 three spellings of Alice (orders 1, 2 and 6); TODO 2 Bob in Chicago (order 3) and Boston (order 4); TODO 3 four rows (orders 1, 3, 4, 5), and order 4 is Dune Messiah, a different book; TODO 4 order 5 with `about 26` in the total column.
- **Debrief (2m):** one sentence per TODO, from the pairs: copies drift apart (1 and 2), a list in a cell cannot be searched reliably (3), a column with no type accepts anything (4). Then: "Every one of these problems disappears when each fact is stored once, in the right type, and connected with keys. That is what the next twenty-three lessons build." The solution file's step 5 shows the two-table fix in miniature: show its last two results if time allows.
- **Fast finishers:** `labs/module-01/l1_1-what-is-a-database/crud_tour_starter.sql` (the four CRUD statements on a books table, turned off with `--` until the learner predicts the result), then `labs/module-01/l1_1-what-is-a-database/debug_rules.sql`: four INSERTs that MySQL refuses with errors 1062, 1364, 1366 and 1406.

START THE LESSON PROJECT

The lesson project is **Sketch the Bookstore**: a paper design of the database this course builds. In the room, do Steps 1 and 2 only (10 minutes).

- Read the store's description aloud from the lesson page (Step 1). Learners underline the nouns and turn them into about five table names.
- Step 2 in pairs: four to six columns per table, each with a type (text, whole number, decimal, date, yes/no).
- **Done in the room** = five tables (customers, orders, books, authors, categories) with typed columns. Do not show the course design yet.

- **Goes home:** Step 3 (primary keys: why a title or an email makes a poor one), Step 4 (foreign keys and the two linking tables) and Step 5 (compare with the course design on the lesson page). They keep the sketch: Sessions 2 and 6 compare it with the real tables.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold-call gently, or think-pair-share):

- Is MySQL the database? **Answer:** no: MySQL is the server software (a DBMS); a database such as `preview_shop` lives inside it.
- Which CRUD letter is SELECT? And DROP DATABASE? **Answer:** SELECT is Read. DROP DATABASE is not CRUD at all: it removes the structure, not rows.
- Why does an orders table store `customer_id` instead of the customer's name? **Answer:** the name lives once in customers; the id points to it, so a correction happens in one place.
- Which of the three refused rows would a spreadsheet have accepted? **Answer:** all three: the duplicate id, the text in a number column and the missing last name.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** B) MySQL is a database management system: server software that stores databases, enforces their rules and answers SQL; **Q2** C) `author_id`; **Q3** Two rows: `Tom` and `Fido`; **Q4** A) `ERROR 1062 (23000): Duplicate entry '2' for key 'pets.PRIMARY'`; **Q5** D) A key-value store such as Redis; **Q6** B) `DELETE FROM customers WHERE id = 3;`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners whose machines are not ready do the lab with a ready neighbor (one keyboard, two heads, the NOT READY learner types the edits). For learners new to the terminal, write `SOURCE labs/module-01/...` on the board and leave it there; tab completion of the path helps.

Stretch: `crud_tour_starter.sql` and `debug_rules.sql`; then the lesson's Exercise 4 (design a table for a collection they know, and spot the column that is secretly a list) and the Bonus Challenge (five tables of an app they use every day).

1.2 Installing MySQL

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Choose the right MySQL release (9.7 LTS) and installation method for your computer, and recognize when a package gives you something else (an older MySQL, or MariaDB)
- Install MySQL 9.7 LTS with the Windows MSI, the macOS package or Homebrew, the MySQL APT repository, or the official Docker image
- Start and stop the server, and set a strong root password
- Connect with mysql -u root -p and verify the version, user and character set with SQL
- Diagnose the four classic first-day errors: command not found, 1045, 2002 and 2003

TIMING IN THE ROOM

10m	Frame: server, client, port; LTS versus innovation releases; one installation per laptop
25m	Install clinic: the machines that were NOT READY, by platform
15m	Live demo: connect and verify, the health-check queries, two pieces of old advice
15m	Guided lab: `server_facts_starter.sql`
10m	Start the lesson project: the health check, Steps 3 and 4

MENTAL MODEL TO SAY

Back to the three boxes: "Installing MySQL means two programs. The **server**, `mysqld`, runs in the background, owns the data files and listens on a **port**, 3306, like a shop with a door number. A **client**, the `mysql` command, knocks on that door and carries your SQL in. They can be on different machines; today they are on the same laptop."

Then versions, in one breath: "MySQL ships LTS releases that stay the same for years, and innovation releases with new features every few months. This course uses **9.7 LTS**. 'Latest' on a download page usually means an innovation release, numbered like 26.x; Ubuntu's own package is older; XAMPP gives you MariaDB, a close relative. Whatever you installed, the first thing you ask after logging in is SELECT VERSION()."

Then the triage line on the board, which is worth the whole lesson: **command not found = PATH · 1045 = the server answered and refused you · 2002/2003 = nothing answered.**

INSTALL CLINIC

Learners from the opening's NOT READY pile sit together by platform; ready learners pair with them as helpers. You move from table to table; each fix ends with the environment check run again. Twenty-five

minutes is enough to fix configuration and start an install, not to download and install on slow Wi-Fi: use the USB stick.

PLATFORM	MOST LIKELY STATE	CLINIC FIX (IN THIS ORDER)
Windows	MSI installed, `mysql` not recognized; or Configurator never finished	PATH (Lesson 1.2 Step 4, a NEW PowerShell window); Services app: is MySQL97 running? Re-run MySQL Configurator if the service does not exist.
macOS (Homebrew)	`brew install mysql` gave 26.x; or `mysql@9.7` not on the PATH; root with no password	The keg-only PATH line; `brew services start mysql@9.7`; `mysql_secure_installation` for the root password. 26.x: fine today, reinstall at home.
macOS (DMG)	Installed, `/usr/local/mysql/bin` not on the PATH	The export PATH line; start the server from the MySQL pane in System Settings.
Linux	Ubuntu's package: 8.0/8.4, root through `sudo mysql` only	Today: `sudo mysql`, then the lesson's ALTER USER statement to set a password. At home: the MySQL APT repository for 9.7.
Docker	Port 3306 taken, or connecting with localhost	`-p 3307:3306` and `-h 127.0.0.1 -P 3307`; or `docker exec -it mysql97 mysql -u root -p`.
Nothing installed	Setup guide not done	Docker Desktop if already installed; otherwise the installer from the USB stick, started now, finished at home. Pair for the rest of today.

Done for each machine = the environment check prints READY (or READY WITH NOTES). Anyone not done by 2:10 leaves with a numbered plan written in their workbook's Start Here box, and you check it at the start of Session 2.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Connect and verify, from a fresh terminal on the projector. The two shell steps print text that differs on every machine (the lesson shows each platform's version); everything else is real output from a MySQL 9.7 server.

1. **Client version: no server needed (1m).** "First the program on my laptop. This works even with the server stopped."

   ```
   mysql --version
   ```

   The output is the lesson's "Output" card for your platform (for example `mysql Ver 9.7.2 for Linux on x86_64 (MySQL Community Server - GPL)`). Your exact text will differ.

2. **Log in (1m).** "-u is the user, -p means ask me for the password. Nothing appears while I type it: that is normal."

```
mysql -u root -p
```

Docker: `docker exec -it mysql97 mysql -u root -p`.

3. **Which server answered? (1m).** "Now ask the SERVER."

```
SELECT VERSION();
```

```
+-----------+
| VERSION() |
+-----------+
| 9.7.2     |
+-----------+
```

Any 9.7.x is right. 8.4, 26.x or a version containing MariaDB means another server than intended: Common Mistake 5.

4. **What is in it? (1m).** "A fresh server holds four system databases. Never put your own tables in them."

```
SHOW DATABASES;
```

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
```

5. **Who am I? (1m).** "root, connected from this computer."

```
SELECT CURRENT_USER();
```

```
+----------------+
| CURRENT_USER() |
+----------------+
| root@localhost |
+----------------+
```

Through Docker's published port with `-h 127.0.0.1` it says `root@%`.

6 **The health check, part 1 (2m).** "These are the lesson project's Step 3 queries. @@ names read server settings." (The lesson's version also prints @@port: show it on your server; it is 3306 on a standard install.)

```
SELECT VERSION() AS version, @@version_comment AS edition;
```

```
+---------+------------------------------+
| version | edition                      |
+---------+------------------------------+
| 9.7.2   | MySQL Community Server - GPL |
+---------+------------------------------+
```

7 **The health check, part 2 (2m).** "utf8mb4 is full Unicode: every language and emoji. It is the default since 8.0."

```
SELECT
    CURRENT_USER()         AS logged_in_as,
    @@character_set_server AS charset,
    @@collation_server     AS collation;
```

```
+----------------+---------+--------------------+
| logged_in_as   | charset | collation          |
+----------------+---------+--------------------+
| root@localhost | utf8mb4 | utf8mb4_0900_ai_ci |
+----------------+---------+--------------------+
```

8 **How does root log in? (2m).** "Accounts live in the mysql.user table. Look, never edit by hand."

```
SELECT user, host, plugin
FROM mysql.user
WHERE user = 'root'
ORDER BY host;
```

```
+------+-----------+-----------------------+
| user | host      | plugin                |
+------+-----------+-----------------------+
| root | localhost | caching_sha2_password |
+------+-----------+-----------------------+
```

Docker shows a second row, root with host %. Ubuntu's own package shows auth_socket until a password is set.

9. **Old advice 1, on purpose (2m).** "A forum says: create your app's user with mysql_native_password. On 9.7:"

```
CREATE USER 'legacy_app'@'localhost'
    IDENTIFIED WITH mysql_native_password BY 'placeholder-password';
```

```
ERROR 1524 (HY000): Plugin 'mysql_native_password' is not loaded
```

The plugin is gone in 9.x (disabled in 8.4). Fix: leave IDENTIFIED WITH out; update the client instead of weakening the server. No account was created.

10. **Old advice 2, on purpose (1m).** "And the setting old tutorials tell you to check:"

```
SELECT @@default_authentication_plugin;
```

```
ERROR 1193 (HY000): Unknown system variable 'default_authentication_plugin'
```

Removed too; the modern setting is `@@authentication_policy`.

11. **Leave cleanly (1m).** "exit, or Ctrl+D on macOS and Linux."

```
exit
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Asking the client instead of the server.** "`mysql --version` says 9.7, so my server is 9.7." Coach: "Which box does that command ask? Log in and ask the server." (Self-check Q7 and the lesson's Exercise 1.)
- **Reading the wrong half of an error.** Learners stop at "ERROR" and guess. Coach the triage line: "Which number? 1045, the server answered and said no: check the password. 2002 or 2003, nobody answered: check that the server runs and where you are knocking."
- **Weakening the server to fix a client.** Someone finds advice to switch root to `mysql_native_password` (or tries it and gets 1524). Coach: "That method was removed in 9.x for good reasons; update the tool instead." Never run anything that changes root's login method in class.

GUIDED LAB

- **File:** `labs/module-01/l1_2-installing-mysql/server_facts_starter.sql`; reference `labs/module-01/l1_2-installing-mysql/server_facts_solution.sql`. Read-only: it asks

the server what it is and how it is set up. Questions 1 and 2 run as shipped; TODOs 3 to 6 each replace a placeholder SELECT with a real one (TODO 3 gives the full statement, TODO 4 and 5 only the setting names).

- **Expected:** `9.7.2` and `MySQL Community Server - GPL`; `is_mysql_97` 1 and `is_mariadb` 0; `utf8mb4` / `utf8mb4_0900_ai_ci`; `InnoDB` and `lower_case_names` 0 (Windows prints 1, macOS 2: a good moment to say that table names may be case-sensitive on Linux servers); `max_connections` 151; the four system databases.
- **Circulate for:** a missing comma between two columns (`ERROR 1064`), `@@` written as `@` (a user variable: NULL, not an error), and learners on MariaDB (`is_mariadb` 1 and another collation: say "that is the version note in action").
- **Fast finishers:** `labs/module-01/l1_2-installing-mysql/accounts_check_starter.sql` (who may log in, and how: three locked helper accounts, no native-password accounts), then `labs/module-01/l1_2-installing-mysql/debug_old_advice.sql` (the two errors from the demo, to explain in writing).
- **Debrief (1m):** "Write the version, the character set and root's plugin on your health-check page. When something breaks in week five, these three lines tell you in a minute whether the server changed."

START THE LESSON PROJECT

The lesson project is the **Installation Health Check**: a few queries that describe the server, saved in the journal. Steps 1 and 2 (install, secure root) were the prework and the clinic. In the room: Steps 3 and 4 (10 minutes).

- Everyone runs Step 3's two queries and Step 4's query from the lesson page and copies the answers into the journal.
- **Done in the room** = version, port, charset (`utf8mb4`), collation and root's plugin (`caching_sha2_password`) written down; Docker users have a second root row with host `%`.
- **Goes home:** Step 5: stop the server, see the 2002 or 2003 error on purpose, start it again, and write the start and stop commands for their platform in the journal.

CHECKS FOR UNDERSTANDING

Ask these out loud during the clinic, the demo and the lab:

- Your client prints 9.7.2 but `SELECT VERSION();` prints 8.4.x. Which is the server? **Answer:** 8.4: only the query asks the server.
- 1045 or 2003: which one means the server is running? **Answer:** 1045: it answered and refused the credentials.
- Why connect to a Docker container with `127.0.0.1` rather than `localhost`? **Answer:** on macOS and Linux, `localhost` makes the client use a local socket file and ignore the port.

- Is root without a password acceptable on a laptop? **Answer:** no: any program on the computer could connect. A strong password costs nothing.

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** C) The mysql client program is installed and on your PATH; **Q8** A) The user name or the password; **Q9** `1` (in a column named `on_97`); **Q10** D) `brew install mysql@9.7`; **Q11** C) `mysql -h 127.0.0.1 -P 3307 -u root -p`; **Q12** `caching_sha2_password`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners who spent the clinic installing do the lab at home; give them the README's expected outputs. For anyone overwhelmed by the terminal, one card with three lines: `cd` to the course folder, `mysql -u root -p`, `SOURCE labs/...`.

Stretch: `accounts_check_starter.sql` and `debug_old_advice.sql`; the lesson's Exercise 4 (a second server side by side in Docker, MySQL 8.4 on port 3307, confirmed with `SELECT VERSION();`) and the Bonus Challenge (find `@@datadir` and the error log).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note and leave it on the way out. Sort the notes into "got it" and "not yet" before Session 2: question 1's pile tells you how much of the Session 2 warm-up goes to the server, client and database picture.

1. In one sentence each: what is the MySQL server, what is a client, and what is a database? **Answer:** the server (`mysqld`) is the program that stores data and answers SQL; a client (the `mysql` command, phpMyAdmin) is what you type into and it sends SQL to the server; a database (such as `preview_shop`, later `bookstore`) is a named collection of tables inside the server.
2. You type `mysql -u root -p` and get `ERROR 2002`. What do you check first? And if it were `ERROR 1045`? **Answer:** 2002: is the server running (and am I asking the right place)? 1045: the user name and password.
3. `orders(id, customer_id, order_date)`: name the primary key and the foreign key. Why not store the customer's name in orders? **Answer:** `id` is the primary key, `customer_id` the foreign key. The name lives once in customers; copies in every order would drift apart.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** The bookstore sketch (Steps 3 to 5: keys, linking tables, compare with the course design) and the health check (Step 5: stop, see the error, start, write the commands).

- **Lesson exercises.** 1.1: Exercises 1 to 3 (parts of a table, spreadsheet problems, database types); Exercise 4 is a stretch. 1.2: Exercises 1 to 3 (client versus server version, read two settings, break it on purpose); Exercise 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages (five questions each; answers are in the Answer Key).
- **Journal:** both entries. The 1.2 prompt asks for their installation story, including the exact error they saw: it will save them an hour next time.
- **Before the Next Lesson (from 1.2):** log in once more from a fresh terminal; install a code editor for `.sql` files if they have none (VS Code); optionally set up phpMyAdmin for the Lesson 1.3 tour.
- **If the environment check was not READY:** follow the numbered plan from the clinic, run the check again and email you its last table.
- **Optional:** `labs/module-01/l1_0-orientation/diagnostic_starter.sql` for anyone who missed two or more diagnostic questions; `crud_tour_starter.sql` and `accounts_check_starter.sql` for anyone who did not reach them.

BRING LIST FOR SESSION 2

- A laptop on which the environment check ends with READY (or READY WITH NOTES), and the root password (in a password manager, not on a sticky note).
- The course folder (`mysql-course` in your home folder) with the `labs/` folder in it. Lesson 1.3's `hello.sql` and Lesson 2.1's `create_bookstore.sql` will be saved there too. Session 2's starters are in `labs/module-02/`.
- The bookstore sketch from the 1.1 project: Lesson 2.1 builds the first three of its tables.
- The workbook and the journal with the health-check answers. No downloads needed.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the check result for every learner; the install clinic; 1.1 demo Steps 1 to 8; the order-sheet lab through TODO 3.
- **Cut first (saves 3m):** 1.2 demo Steps 9 and 10. Say: "two pieces of old advice fail on 9.7; the handout `debug_old_advice.sql` shows you why".
- **Cut second (saves 10m):** the 1.1 project start. Read the store's description aloud, send all five steps home.
- **Cut third (saves 10m):** the 1.2 project start: the guided lab already ran the same kind of queries; Steps 3 to 5 go home.

RUNNING SHORT: WHAT TO ADD

- Run `debug_rules.sql` for everyone and fix the four statements together: one error number per volunteer.

- The lesson's 1.2 Exercise 3 live: stop your server, show 2002, start it, show 1045 with a wrong password. Then the triage line once more.
- Start the 1.1 Bonus Challenge together: the five tables of a food-delivery app, and one value it should copy on purpose (the price paid, like `order_items.price_at_time`).

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 2: The MySQL Client and Your First Tables

How to teach the second session of MySQL Foundations. Learners get at home in the mysql client (finding their way around a server, the prompts, client commands, running .sql files, a look at phpMyAdmin), then stop exploring and build: the bookstore database and its first three tables from a script they can rerun, the habits of reading DESCRIBE after every change and of naming things well.

`2 lessons` `3 hrs session` `Lessons 1.3-2.1` `Session 2 of 14 · Part 1`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice) → the lesson's own project. The lesson exercises, quiz and journal are homework.
- Type, do not paste. Every SQL block below was run to produce the output printed under it, on MySQL 9.7.2 with the default settings. Ask for a prediction before every statement that prints something, and read error messages aloud from the left: the number, then the reason.
- Keep `lesson_03.html` and `lesson_04.html` open in browser tabs for the diagrams and the phpMyAdmin tour. Labs live in `labs/module-02/`: `_starter.sql` files are handouts, `_solution.sql` files are yours.
- Today is the day learners stop exploring and build: Lesson 2.1's project, `create_bookstore.sql`, is done **in the room** (all five steps), because Session 3 starts from its three empty tables. Lesson 1.3's project starts in the room and finishes at home.
- Part 1 (Sessions 1 to 7) can run on its own. Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (1.3, 2.1) match the course site.

> **THE #1 RISK OF SESSION 2: LEAVING WITHOUT A BOOKSTORE**
>
> Session 3 begins with `USE bookstore;` and three empty tables. A learner who leaves today without them starts Session 3 by debugging a build script instead of learning data types. **Protect the last 20 minutes before the wrap-up** for `create_bookstore.sql`: every learner runs it twice and shows you the self-check table (books, categories, customers, all `utf8mb4_unicode_ci`). If the 2.1 demo runs long, cut demo Steps 10 to 12, never the project. The second risk is the machine that was NOT READY in Session 1: the warm-up checks it first.

BEFORE THE SESSION

- ☐ **Sort the Session 1 exit tickets.** The "not yet" pile on question 1 (server, client, database) tells you how long the warm-up spends on the three-box picture.
- ☐ **Files to hand out:** the `labs/module-02/` starters and the two debug files (no `*_solution.sql`). Learners keep `labs/` in their course folder. The course folder is `mysql-course` in each learner's home folder (the setup guide and Session 1's bring list): Lesson 1.3's `hello.sql` and `explore.sql` and Lesson 2.1's `create_bookstore.sql` go there. Anyone without it creates it now and moves `labs/` into it.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-02` from this kit's folder. Every line should say PASS (10 files, a few seconds).
- ☐ **Projector machine:** MySQL 9.7 running and logged out; a terminal open in the course folder at 18 pt or larger; a plain-text editor (VS Code) with `~/mysql-course/hello.sql` already saved (the lesson's three lines) for demo Step 11; phpMyAdmin if you have it (Docker `phpmyadmin:5.2` on port 8080, set up as in Lesson 1.2). Browser tabs: `lesson_03.html`, `lesson_04.html`.
- ☐ **Make sure `bookstore` does NOT exist on the projector machine** (`DROP DATABASE IF EXISTS bookstore;`): the 1.3 demo shows error 1049 for it, and the 2.1 demo creates it.
- ☐ **Board, before learners arrive:** the four prompts (`mysql>` ready · `->` waiting for ; · `'>` open quote · `\c` gives up); two columns "SQL (any client)" and "mysql client only"; and the cabinet picture: server = cabinet, database = drawer, table = folder.
- ☐ **Print or share:** the workbook (warm-up, guided labs, self-checks, exit ticket).

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`Failed to open file 'labs/...'` after SOURCE	The client was started in another folder	`exit`, `cd` into the course folder, log in again (or give the full path).
Windows: SOURCE with backslashes fails	The path was typed with `\` and spaces	Use forward slashes: `SOURCE C:/Users/you/mysql-course/hello.sql`; quote nothing.
`mysql -u root -p < file.sql` in PowerShell: "The '<' operator is reserved"	PowerShell has no input redirect	`cmd /c "mysql -u root -p < file.sql"`, or SOURCE from the prompt.
`ERROR at line 1: Unknown command '\G'.` from the shell	The 9.x client ignores client commands in batch mode	End statements with `;` in scripts, or `mysql --commands ...`.
A script saved from Word or TextEdit fails with 1064 on the first line	Rich text or smart quotes (`’` instead of `'`)	Save as plain text from VS Code (TextEdit: Format > Make Plain Text).

SYMPTOM	CAUSE	FIX
phpMyAdmin login fails with the right password	The container was started without `PMA_HOST`, or links to a stopped server	Lesson 1.2's docker run line for phpMyAdmin; skip the tour if it is not quick to fix.
`ERROR 1007` when running `create_bookstore.sql` a second time	The learner's script lacks `DROP DATABASE IF EXISTS bookstore;`	That line goes first: Step 1 of the project.

SESSION 2 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 1, the environment check for anyone not READY last time, homework check.
0:10	10	**1.3** Frame and mental model	One server, many clients; SQL versus client commands; the four prompts.
0:20	25	**1.3** Live demo	Find your way around a fresh server, read a wide row with \G, two errors on purpose, SOURCE and the shell, a two-minute phpMyAdmin tour.
0:45	20	**1.3** Guided lab	`labs/module-02/l1_3-mysql-cli-phpmyadmin/find_your_way_starter.sql` in pairs: explore a database you have never seen.
1:05	15	**1.3** Start the lesson project	explore.sql, Steps 1 to 3 (where am I, count tables, save the script); Steps 4 and 5 go home.
1:20	10	Break	Leave the server, database, table picture (cabinet, drawer, folder) on the board.
1:30	10	**2.1** Frame and mental model	Cabinet, drawer, folder; utf8mb4 and a named collation; the naming rules.
1:40	30	**2.1** Live demo	Build bookstore and books live, inspect it, IF NOT EXISTS, ALTER TABLE and the MODIFY trap, three errors on purpose, then drop it all.
2:10	20	**2.1** Guided lab	`labs/module-02/l2_1-creating-databases-tables/events_build_starter.sql`: a build script from a written spec.
2:30	20	**2.1** Build the lesson project	create_bookstore.sql, all five steps in the room, run twice: Session 3 starts from it.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 3.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 85 + 85 = 170 minutes for a solo learner; after the warm-up, the break and the wrap-up the room has 150. Lesson 1.3 gets 70 minutes (model, demo, lab, project start) and Lesson 2.1 gets 80, because its whole project is done here.

Goes home (about 2 to 3 hours). explore.sql Steps 4 and 5 (run it three ways; phpMyAdmin or DESCRIBE); each lesson's Hands-on Exercises; both Quick Quizzes; both journal entries; Lesson 2.1's "Before the Next Lesson" list (run `create_bookstore.sql` once more from a fresh session).

Protect, in this order: `create_bookstore.sql` for every learner (Session 3 needs it); the 2.1 demo through the MODIFY trap (Step 9); the find-your-way lab through TODO 5; the 1.3 demo's error steps (8 and 9). **Cut first:** the phpMyAdmin tour (send learners to the lesson's tour section), then 2.1 demo Steps 10 to 12 (hand out `debug_build_script.sql`), then the 1.3 project start (all five steps go home).

OPENING SEGMENT

0:00 Warm-Up: Session 1 Retrieval — 10 min

TIMING

4m	Five questions W1 to W5 in the workbook, no notes, then fingers up for each answer
3m	Machines that were NOT READY in Session 1: the environment check again, last table on the projector
3m	Homework check: hands up for the bookstore sketch and the health check; one learner reads their 1.2 journal error and its fix

WARM-UP ANSWERS

Answers: **W1** B) No: MySQL is the server (a DBMS); a database such as bookstore lives inside it; **W2** C) `ERROR 1045 (28000): Access denied`; **W3** A) Rows 1 Rex and 3 Fido; **W4** C) `DELETE FROM customers WHERE id = 3;`; **W5** D) Nothing: it describes the client; ask the server with SELECT VERSION();.

- **W1 and W5** are Session 1's two big ideas (server, client, database; ask the server). If more than a few miss W1, draw the three boxes again before the 1.3 frame; it is the frame anyway.
- **W2** is the triage line. Misses: write `command not found = PATH · 1045 = answered and refused · 2002/2003 = nothing answered` on the board again.
- **W3 and W4** check CRUD from 1.1. A wrong W4 (DROP TABLE or DELETE without WHERE) is the most dangerous misconception in the course: say "WHERE picks the rows" once more.
- **Still NOT READY?** Pair the learner with a neighbor for today and book ten minutes at the break. Do not fix installs during the demo.

LESSON PLANS

1.3 MySQL CLI & phpMyAdmin

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Log in to and out of the mysql client, and read its prompts (mysql>, ->, '>)
- Explore a server with SHOW DATABASES, USE, SHOW TABLES, DESCRIBE and SELECT DATABASE()
- Tell SQL statements apart from client commands such as \G, \c, SOURCE and help
- Save SQL in a .sql file and run it with SOURCE and with mysql < file.sql, knowing what MySQL 9.x's client does differently in scripts
- Find the Structure, SQL, Import and Export tabs in phpMyAdmin and choose the right tool for a task

TIMING IN THE ROOM

10m	Frame: one server, many clients; SQL statements versus client commands; the prompts
25m	Live demo: find your way around, \G, two errors on purpose, SOURCE and the shell, phpMyAdmin
20m	Guided lab: `find_your_way_starter.sql` in pairs
15m	Start the lesson project: explore.sql, Steps 1 to 3

MENTAL MODEL TO SAY

Point at the three boxes from Session 1: "A **client** is any program that sends SQL to the server and shows you the answer: the mysql command, phpMyAdmin in a browser, Workbench. The server does not care which one you use: same statement, same result."

Then the two columns on the board: "Most of what you type is **SQL**, sent to the server: SHOW, USE, DESCRIBE, SELECT. A few things only the mysql program understands: backslash G, SOURCE, help, exit. Those are **client commands**; phpMyAdmin has never heard of them."

Then the prompts: "Nothing runs until the semicolon. The prompt tells you why the client is waiting: `->` it wants the end of the statement, `'>` a quote is open. Backslash c gives up." Tie it to Session 1's missing-semicolon mistake.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Log in on the projector (`mysql -u root -p`); learners log in too and type along. A fresh session has no current database. Ask for a prediction before every statement that prints something.

1. **A fresh server (2m).** "Logged in, no database chosen yet. What is on this server?" Point at the four system databases: look, never change.

```
SHOW DATABASES;
```

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
```

2 **A database that is not there yet (1m).** "Which database will we build today? Watch what happens if I go there now." Read the error from the left.

```
USE bookstore;
```

```
ERROR 1049 (42000): Unknown database 'bookstore'
```

1049 = no database by that name on this server: a typo, the wrong server, or (here) one you have not created. Lesson 2.1 fixes this at 1:40.

3 **USE, and where am I? (2m).** "USE picks the drawer. SELECT DATABASE() answers 'where am I' at any time; before the first USE it says NULL."

```
USE mysql;
SELECT DATABASE();
```

```
Database changed
+------------+
| DATABASE() |
+------------+
| mysql      |
+------------+
```

4 **What is in this drawer? (2m).** "SHOW TABLES on its own lists 38 tables here; LIKE filters the names, % means anything."

```
SHOW TABLES LIKE 'help%';
```

```
+-------------------------+
| Tables_in_mysql (help%) |
+-------------------------+
| help_category           |
| help_keyword            |
| help_relation           |
| help_topic              |
+-------------------------+
```

5 **What does one table look like? (2m).** "DESCRIBE: one row per column. Read the Null, Key and Default columns aloud: NO means required, PRI is the primary key, UNI unique."

```
DESCRIBE help_category;
```

```
+--------------------+-------------------+------+-----+---------+-------+
| Field              | Type              | Null | Key | Default | Extra |
+--------------------+-------------------+------+-----+---------+-------+
| help_category_id   | smallint unsigned | NO   | PRI | NULL    |       |
| name               | char(64)          | NO   | UNI | NULL    |       |
| parent_category_id | smallint unsigned | YES  |     | NULL    |       |
| url                | text              | NO   |     | NULL    |       |
+--------------------+-------------------+------+-----+---------+-------+
```

6 **Read some rows (2m).** "Type it over several lines on purpose: nothing runs until the semicolon. Watch the prompt change to ->."

```
SELECT User, Host
FROM user
ORDER BY User, Host;
```

```
+------------------+-----------+
| User             | Host      |
+------------------+-----------+
| mysql.infoschema | localhost |
| mysql.session    | localhost |
| mysql.sys        | localhost |
| root             | localhost |
+------------------+-----------+
```

The three `mysql.*` accounts are internal and locked. Docker shows a second root with host `%`.

7 **A wide row, vertically (2m).** "End with backslash G instead of the semicolon: one line per column. That is a client command, not SQL."

```
SELECT User, Host, plugin, account_locked
FROM user
WHERE User = 'root' AND Host = 'localhost'\G
```

```
*************************** 1. row ***************************
          User: root
          Host: localhost
        plugin: caching_sha2_password
account_locked: N
```

8 **Looking in the wrong drawer (2m).** "The error names the database it looked in. That is the hint: where am I?"

```
DESCRIBE customers;
```

```
ERROR 1146 (42S02): Table 'mysql.customers' doesn't exist
```

9 **A reserved word as a name (2m).** "1064 is the general 'I could not read this'; near '...' points at the spot. CURRENT_TIME is a built-in function."

```
SELECT NOW() AS current_time;
```

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual tha
```

10 **The fix: another name (1m).** "Pick a name that is not a keyword (or use backticks)."

```
SELECT NOW() AS now_time, CURRENT_USER() AS logged_in_as;
```

```
+---------------------+----------------+
| now_time            | logged_in_as   |
+---------------------+----------------+
| 2026-10-16 01:26:20 | root@localhost |
+---------------------+----------------+
```

Your time differs, of course.

11 **Run a file: SOURCE and the shell (4m).** Open `hello.sql` (the lesson's three-line script) in your editor next to the terminal. "At the prompt, SOURCE runs every statement in the file. From the shell, the

file goes into mysql with <, in batch mode: plain tab-separated text, -t brings the boxes back." Show `SOURCE ~/mysql-course/hello.sql`, then `exit` and the two shell lines.

```
SOURCE ~/mysql-course/hello.sql
exit
mysql -u root -p < ~/mysql-course/hello.sql
mysql -t -u root -p < ~/mysql-course/hello.sql
```

The lesson shows the outputs. Say the 9.x rule once: in a redirected file the client ignores its own commands (\G, SOURCE) unless started with `--commands`. PowerShell has no `<`: `cmd /c "..."`.

12 **phpMyAdmin in two minutes (2m).** If you have it (Docker `http://localhost:8080`): log in with the same root account, expand mysql > user in the navigation panel, show Browse (and the SELECT it ran above the grid), Structure, the SQL tab with the demo's SELECT, Import and Export. Point at the red Drop and Empty buttons. No phpMyAdmin: show the lesson's tour section instead.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Typing the password after `-p` with a space.** `mysql -u root -p mypassword` asks for the password and then fails with 1049 Unknown database 'mypassword'. Coach: "-p means 'ask me'. The next word is read as a database." (Self-check Q4.)
- **Looking for a table in the wrong database.** 1146 names the database it looked in. Coach: "Read the name in the message. Where are you? SELECT DATABASE()."
- **SOURCE with the wrong path.** `Failed to open file` almost always means the client was started in another folder, or Windows backslashes. Coach: forward slashes, and start the client in the course folder (or type the full path).

GUIDED LAB

File: learners open `labs/module-02/l1_3-mysql-cli-phpmyadmin/find_your_way_starter.sql`; your reference is `labs/module-02/l1_3-mysql-cli-phpmyadmin/find_your_way_solution.sql`. The setup part builds `kit_s2_cafe`, the bookstore café's database, the way a coworker might leave it. Learners do not read the setup: they find their way with the statements from the demo.

- **SOURCE it as shipped (2m):** `SOURCE labs/module-02/l1_3-mysql-cli-phpmyadmin/find_your_way_starter.sql`. Everyone sees `current_db kit_s2_cafe` and six TODO lines.
- **TODO 1 together (2m), TODOs 2 to 6 in pairs (14m).** Each TODO replaces one placeholder SELECT with one statement; save, SOURCE again. One learner types, the other writes the answers in the workbook; swap at TODO 4.
- **Expected:** three tables (cafe_orders, menu_items, staff_notes); `id` is the primary key of menu_items and `name`, `price` are required (Null NO); `ordered_at` and `paid_at`; five time_zone tables;

item 3 is Oat Latte, 4.10, vegan 1; staff_notes is InnoDB with utf8mb4_unicode_ci.

- **Circulate for:** TODO 3 written as `LIKE '%_at'` (it still works here, but `_` means "any one character": say why `\_` is exact); TODO 5 ended with `;` instead of `\G` (one wide horizontal row: ask which is easier to read); TODO 4 answered with `USE mysql;` (it works, but the file then runs the rest in the wrong database: SHOW TABLES FROM avoids moving).
- **Debrief (2m):** "You learned four things about a database you had never seen without reading a line of its code: that is what DESCRIBE and SHOW are for."
- **Fast finishers:** `labs/module-02/l1_3-mysql-cli-phpmyadmin/system_tour_starter.sql` (information_schema questions about the mysql database), then `labs/module-02/l1_3-mysql-cli-phpmyadmin/debug_lost.sql` (errors 1049, 1146 and two 1064s).

START THE LESSON PROJECT

The lesson project is **A Server Report Script**: `explore.sql`, a short script that describes the server, run three ways. In the room: Steps 1 to 3 (15 minutes).

- Step 1: the orientation query (USER, CURRENT_USER, DATABASE) at the prompt.
- Step 2: the information_schema count per system database. Read GROUP BY as "count the rows for each database" (Lesson 3.3 teaches it).
- Step 3: save the file in `mysql-course` with a header comment, every statement ending in `;` and no `\G`.
- **Done in the room** = `explore.sql` saved and run once with SOURCE without errors.
- **Goes home:** Step 4 (run it from the shell, plain and with -t, and explain the difference) and Step 5 (phpMyAdmin's SQL tab, or DESCRIBE in the client).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Is `SHOW TABLES;` SQL or a client command? **Answer:** SQL: phpMyAdmin can send it too.
- The prompt shows `->`. What do you type? **Answer:** `;` and Enter (or `\c` to give up).
- `ERROR 1146 (42S02): Table 'mysql.customers' doesn't exist`: what is the first thing you check? **Answer:** which database you are in (`SELECT DATABASE();`): the message says mysql.
- Why does `\G` fail in `mysql -u root -p < report.sql` on 9.7 but not with SOURCE? **Answer:** a redirected file runs in batch mode, where the 9.x client ignores its own commands unless started with `--commands`.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** C) `SOURCE ~/mysql-course/hello.sql`; **Q2** B) A single quote is still open: close it, then type ;; **Q3** 5 (`time_zone`, `time_zone_leap_second`, `time_zone_name`, `time_zone_transition`,

`time_zone_transition_type`); **Q4** D) It asks for the password, then fails with `ERROR 1049 (42000): Unknown database 'secret123'`; **Q5** A) `mysql -u root -p < report.sql`; **Q6** B) `SHOW COLUMNS FROM mysql.user LIKE '%lock%';`. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: a card with the five exploring statements (SHOW DATABASES, USE, SELECT DATABASE(), SHOW TABLES, DESCRIBE) and the four prompts. Learners new to terminals keep `SOURCE labs/module-02/...` on the board and use tab completion.

Stretch: `system_tour_starter.sql` and `debug_lost.sql`; the lesson's Exercise 4 (a script that uses \G, run with --commands) and the Bonus Challenge (a custom prompt in an option file).

2.1 Creating Databases & Tables

80 min

OBJECTIVES (FROM THE LESSON PAGE)

- Create, select and drop a database, with the utf8mb4 character set and an explicit collation
- Write a CREATE TABLE statement with column names, data types and basic constraints
- Inspect a table with SHOW TABLES, DESCRIBE and SHOW CREATE TABLE
- Add, rename, change and remove columns with ALTER TABLE, without losing a constraint by accident
- Rename, empty and drop tables, and use IF EXISTS / IF NOT EXISTS to write scripts you can rerun
- Read and fix errors 1007, 1046, 1050, 1064 and 1146

TIMING IN THE ROOM

10m	Frame: cabinet, drawer, folder; utf8mb4 and a named collation; naming conventions
30m	Live demo: build bookstore and books, inspect, IF NOT EXISTS, ALTER and the MODIFY trap, three errors, drop it all
20m	Guided lab: `events_build_starter.sql`
20m	Build the lesson project: create_bookstore.sql, Steps 1 to 5, run twice

MENTAL MODEL TO SAY

Draw the filing cabinet: "The **server** is the cabinet. Each **database** is a drawer: mostly a name and a few settings. Each **table** is a folder in the drawer, and the data lives in the tables." Point at the four system drawers from the 1.3 demo: "Today we add our own: bookstore."

Then the two settings: "The **character set** decides which characters can be stored: utf8mb4 is all of Unicode, emoji included. Never write utf8: in MySQL that means a three-byte subset. The **collation** decides comparing and sorting: is dune equal to Dune? We name utf8mb4_unicode_ci so MySQL 9.7, 8.4 and MariaDB sort the same way."

Then the habit that the whole lesson builds to: "Real databases are built from **script files** that start clean (`DROP DATABASE IF EXISTS`) and can run any number of times. If an experiment goes wrong in the next two sessions, your script puts everything back in a second." And the naming rules on one line: lowercase, snake_case, plural tables, `id`, `customer_id`, no reserved words.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Type the lesson's statements live; learners type along. This builds a throwaway `bookstore` on the projector and drops it at the end (Step 13), so the project script after the lab starts from nothing for everyone.

1. **Create the database, named settings (3m).** "A database is a drawer: a name and a few settings. utf8mb4 stores every character; the collation decides comparing and sorting. We name it so every server gives the same answers."

```
CREATE DATABASE bookstore
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
SHOW CREATE DATABASE bookstore;
```

```
+-----------+-----------------------------------------------------------------
| Database  | Create Database
+-----------+-----------------------------------------------------------------
| bookstore | CREATE DATABASE `bookstore` /*!40100 DEFAULT CHARACTER SET utf8r
+-----------+-----------------------------------------------------------------
```

Point at `utf8mb4_unicode_ci` inside the versioned comment `/*!40100 ... */`.

2. **The same name again (1m).** "Run it again."

```
CREATE DATABASE bookstore;
```

```
ERROR 1007 (HY000): Can't create database 'bookstore'; database exists
```

Fix: IF NOT EXISTS, or drop it first if you meant to start over.

3. **Go there (1m).** "Where am I now?"

```
USE bookstore;
SELECT DATABASE();
```

```
Database changed
+------------+
| DATABASE() |
+------------+
| bookstore  |
+------------+
```

4. **The first table (4m).** Type it slowly; read each line as "name, type, rule". "Commas between the columns, none after the last. Types today on trust; Session 3 explains each one."

```
CREATE TABLE books (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    title       VARCHAR(200) NOT NULL,
    author      VARCHAR(100) NOT NULL,
    price       DECIMAL(8, 2) NOT NULL,
    pages       INT,
    published   DATE,
    in_stock    BOOLEAN DEFAULT TRUE
);
```

Success prints only Query OK (0 rows affected): the table exists and is empty.

5 **Look at what MySQL stored (3m).** "DESCRIBE is how you check your own work. Where did BOOLEAN go?" (tinyint(1), default 1.)

```
DESCRIBE books;
```

```
+-----------+--------------+------+-----+---------+----------------+
| Field     | Type         | Null | Key | Default | Extra          |
+-----------+--------------+------+-----+---------+----------------+
| id        | int          | NO   | PRI | NULL    | auto_increment |
| title     | varchar(200) | NO   |     | NULL    |                |
| author    | varchar(100) | NO   |     | NULL    |                |
| price     | decimal(8,2) | NO   |     | NULL    |                |
| pages     | int          | YES  |     | NULL    |                |
| published | date         | YES  |     | NULL    |                |
| in_stock  | tinyint(1)   | YES  |     | 1       |                |
+-----------+--------------+------+-----+---------+----------------+
```

6 **IF NOT EXISTS keeps the old table (2m).** "A smaller definition, IF NOT EXISTS. Did books lose five columns?" Ask for a prediction.

```
CREATE TABLE IF NOT EXISTS books (
    id INT PRIMARY KEY
);
SHOW WARNINGS;
```

```
+-------+------+-----------------------------+
| Level | Code | Message                     |
+-------+------+-----------------------------+
| Note  | 1050 | Table 'books' already exists |
+-------+------+-----------------------------+
```

No: a Note, and books is untouched. IF NOT EXISTS never updates a table; ALTER TABLE does.

7 **ALTER TABLE: two new columns (3m).** "ALTER changes the structure and keeps the rows. AFTER places a column; a DEFAULT fills existing rows."

```
ALTER TABLE books
    ADD COLUMN isbn CHAR(13) AFTER id,
    ADD COLUMN language VARCHAR(30) DEFAULT 'English';
DESCRIBE books;
```

```
+-----------+--------------+------+-----+---------+----------------+
| Field     | Type         | Null | Key | Default | Extra          |
+-----------+--------------+------+-----+---------+----------------+
| id        | int          | NO   | PRI | NULL    | auto_increment |
| isbn      | char(13)     | YES  |     | NULL    |                |
| title     | varchar(200) | NO   |     | NULL    |                |
| author    | varchar(100) | NO   |     | NULL    |                |
| price     | decimal(8,2) | NO   |     | NULL    |                |
| pages     | int          | YES  |     | NULL    |                |
| published | date         | YES  |     | NULL    |                |
| in_stock  | tinyint(1)   | YES  |     | 1       |                |
| language  | varchar(30)  | YES  |     | English |                |
+-----------+--------------+------+-----+---------+----------------+
```

8 **The silent trap (3m).** "I only want longer titles." Type the MODIFY without NOT NULL; ask the room what DESCRIBE will say before you run the check.

```
ALTER TABLE books MODIFY COLUMN title VARCHAR(250);
SHOW COLUMNS FROM books LIKE 'title';
```

```
+-------+--------------+------+-----+---------+-------+
| Field | Type         | Null | Key | Default | Extra |
+-------+--------------+------+-----+---------+-------+
| title | varchar(250) | YES  |     | NULL    |       |
+-------+--------------+------+-----+---------+-------+
```

No error, no warning, and title now allows NULL. MODIFY (and CHANGE) replace the WHOLE definition: repeat every rule.

9 **Put it back, in one statement (2m).** "Several changes, commas between them; MySQL rebuilds the table once."

```
ALTER TABLE books
    MODIFY COLUMN title VARCHAR(200) NOT NULL,
    DROP COLUMN isbn,
    DROP COLUMN language;
SHOW COLUMNS FROM books LIKE 'title';
```

```
+-------+--------------+------+-----+---------+-------+
| Field | Type         | Null | Key | Default | Extra |
+-------+--------------+------+-----+---------+-------+
| title | varchar(200) | NO   |     | NULL    |       |
+-------+--------------+------+-----+---------+-------+
```

10 **A reserved word as a table name (2m).** "A table for orders, named order."

```
CREATE TABLE order (
    id INT AUTO_INCREMENT PRIMARY KEY
);
```

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual tha
    id INT AUTO_INCREMENT PRIMARY KEY
)' at line 1
```

ORDER is a keyword (ORDER BY). The naming rule: tables are plural nouns, so `orders`.

11 **A misspelled table (1m).** "The table is books, plural."

```
DESCRIBE book;
```

```
ERROR 1146 (42S02): Table 'bookstore.book' doesn't exist
```

The message names the database too: `bookstore.book`.

12 **Copy, rename, look (2m).** "LIKE copies the structure and none of the rows: the safe way to practice destructive commands."

```
CREATE TABLE books_copy LIKE books;
RENAME TABLE books_copy TO old_books;
SHOW TABLES;
```

```
+---------------------+
| Tables_in_bookstore |
+---------------------+
| books               |
| old_books           |
+---------------------+
```

13 **Drop it all (3m).** "DROP has no undo. Read the name twice. And after dropping the database you were in, where are you?"

```sql
DROP TABLE old_books;
DROP DATABASE bookstore;
SELECT DATABASE();
```

```
+------------+
| DATABASE() |
+------------+
| NULL       |
+------------+
```

NULL: no current database, so the next CREATE TABLE would fail with 1046. Now everybody builds the real bookstore from a script.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **A comma after the last column, or a missing one between two.** Both give 1064, near `)` or near the next column's name. Coach: "Read the `near '...'` part: the mistake is just before it."
- **MODIFY (or CHANGE) without the old rules.** No error; NOT NULL or a DEFAULT silently disappears. Coach: "After every ALTER, DESCRIBE. Is Null still NO?" (Self-check Q11, exit ticket 3.)
- **1046 after DROP DATABASE.** Learners drop the database they were in and the next CREATE TABLE fails. Coach: "Where are you? SELECT DATABASE() says NULL: USE one, or write database.table."

GUIDED LAB

File: `labs/module-02/l2_1-creating-databases-tables/events_build_starter.sql`; reference `labs/module-02/l2_1-creating-databases-tables/events_build_solution.sql`. The bookstore's events team wants a small database for author readings: learners write the build script from a written spec (two tables), then make it safe to rerun.

- **Step 1 is given** (DROP IF EXISTS, CREATE DATABASE with utf8mb4 and the collation, USE). TODO 1 is a skeleton with three gaps; TODO 2 is written from the spec alone; TODO 3 turns on `CREATE TABLE IF NOT EXISTS` + `SHOW WARNINGS`.

- **Expected:** TODO 3 prints `Note 1050 Table 'rooms' already exists`; the self-check lists readings and rooms, both `utf8mb4_unicode_ci`, with 7 and 3 columns (the solution also DESCRIBEs readings). Running the file twice gives the same result: that is the test of a build script.
- **Circulate for:** a trailing comma before `)` (1064 near `)`); `starts_at DATE` instead of `DATETIME` (no error: ask "where does the 7:30 pm go?"); `BOOLEAN DEFAULT 'false'` (an error 1067 Invalid default value: FALSE without quotes).
- **Fast finishers:** `labs/module-02/l2_1-creating-databases-tables/change_request_starter.sql` (ALTER TABLE on a table with rows, the MODIFY trap, TRUNCATE resetting the id), then `labs/module-02/l2_1-creating-databases-tables/debug_build_script.sql` (errors 1007, 1046, 1064, 1050, 1146).
- **Debrief (1m):** "Your spec became a script anyone can rerun. The project is the same thing for the bookstore."

BUILD THE LESSON PROJECT

The lesson project is **The Bookstore Build Script**: `create_bookstore.sql`, which rebuilds the empty bookstore from nothing. All five steps happen in the room (20 minutes): Session 3 starts from these three tables.

- Steps 1 to 3 (12m): the header, `DROP DATABASE IF EXISTS bookstore;`, the database with utf8mb4 and utf8mb4_unicode_ci, `USE`, then books, customers and categories from the lesson page, each with a comment. Run each CREATE at the prompt as it is added, so a typo shows up now.
- Step 4 (3m): the information_schema self-check at the end of the file.
- Step 5 (5m): `SOURCE ~/mysql-course/create_bookstore.sql`, then once more. **Done** = the same self-check table twice: books, categories, customers, all `utf8mb4_unicode_ci`. Walk the room and look at every screen.
- Not done by 2:50? The learner finishes at home and emails you the self-check table before Session 3. The kit's `labs/_states/schema.sql` builds exactly the same three tables, as a fallback at the start of Session 3.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Why name the collation if utf8mb4 is already the default? **Answer:** the default collation differs between servers (9.7/8.4 `utf8mb4_0900_ai_ci`, MariaDB `utf8mb4_uca1400_ai_ci`); naming one makes every server sort the same way.
- `CREATE TABLE IF NOT EXISTS books (id INT PRIMARY KEY);` on the existing seven-column books: what happens? **Answer:** nothing changes; a Note 1050.
- TRUNCATE or DELETE: which one starts the ids at 1 again? **Answer:** TRUNCATE.

- Why does `create_bookstore.sql` start with `DROP DATABASE IF EXISTS bookstore;`?
  Answer: so it can run any number of times from any state; only on a practice database.

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** D) The database is created with utf8mb3 (no emoji), and warning 3719 says to use utf8mb4; **Q8** `email` (then `id` and `name`); **Q9** A) There is no current database: run USE first, or write database.table; **Q10** C) `book_reviews`; **Q11** B) The column is now book_title, VARCHAR(250), and allows NULL; **Q12** `TRUNCATE TABLE customers;`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners who fall behind in the project copy the three CREATE TABLE statements from the lesson page (typing them is the goal, but having them matters more today). Pair a fast typist with them for Step 5.

Stretch: `change_request_starter.sql` and `debug_build_script.sql`; the lesson's Exercise 4 (work without USE, with database.table names) and the Bonus Challenge (table and column comments that survive a rebuild).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide; learners answer in the workbook's Exit Ticket box or on a sticky note. Sort them before Session 3: question 3's "not yet" pile gets a minute of DESCRIBE in the Session 3 warm-up.

1. Name two statements that work in phpMyAdmin's SQL tab and one command that only the mysql client understands. **Answer:** SQL (any client): `SHOW TABLES;`, `DESCRIBE books;`, `USE bookstore;`, `SELECT DATABASE();`, `CREATE TABLE ...`. Client only: `\G`, `SOURCE file.sql`, `\c`, `help`, `exit`, `status`.

2. A script stops with `ERROR 1046 (3D000): No database selected`. Give two ways to fix it. **Answer:** Put `USE bookstore;` (the right database) before the statement, or write the full name `bookstore.books`. It happens in a fresh session and after dropping the database you were in.

3. title is `VARCHAR(200) NOT NULL`. You run `ALTER TABLE books MODIFY COLUMN title VARCHAR(250);`. What did you lose, and which statement shows it? **Answer:** The NOT NULL rule: MODIFY replaces the whole definition. `DESCRIBE books;` (or `SHOW COLUMNS FROM books LIKE 'title';`) shows YES under Null. Fix: repeat it, `VARCHAR(250) NOT NULL`.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish explore.sql** (Lesson 1.3 project Steps 4 and 5: run it with SOURCE, from the shell plain and with -t; phpMyAdmin's SQL tab or DESCRIBE in the client).

- **create_bookstore.sql:** if it was not finished in class, finish it and run it twice. Everyone: run it once more from a fresh session (2.1's "Before the Next Lesson") so bookstore holds exactly three empty tables.
- **Lesson exercises.** 1.3: Exercises 1 to 3 (privilege tables, one column's details, vertical status); Exercise 4 is a stretch. 2.1: Exercises 1 to 3 (ALTER round trip, a publishers table, fix the broken table); Exercise 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages (answers are in the Answer Key).
- **Journal:** both entries. The 2.1 prompt (a sketch of tables for something they know) comes back in Lesson 2.2's journal, so keep it.
- **Optional:** `labs/module-02/l1_3-mysql-cli-phpmyadmin/system_tour_starter.sql` and `labs/module-02/l2_1-creating-databases-tables/change_request_starter.sql` for anyone who did not reach them; the two debug handouts.

BRING LIST FOR SESSION 3

- A laptop on which `USE bookstore; SHOW TABLES;` lists books, categories and customers (empty), and the root password.
- `create_bookstore.sql` in the `mysql-course` folder (Session 3 reruns it whenever an experiment goes wrong).
- The 2.1 journal sketch (tables for something you know): Lesson 2.2 asks you to revise its types.
- The course folder with `labs/` (Session 3's starters are in `labs/module-03/`), the workbook and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** `create_bookstore.sql` run twice by everyone; 2.1 demo Steps 1 to 9; the find-your-way lab through TODO 5.
- **Cut first (saves 2m):** the phpMyAdmin tour: "the lesson's tour section shows every tab; try it at home".
- **Cut second (saves 6m):** 2.1 demo Steps 10 to 12: hand out `debug_build_script.sql` instead.
- **Cut third (saves 15m):** the 1.3 project start: all five steps of explore.sql go home.

RUNNING SHORT: WHAT TO ADD

- Run `debug_build_script.sql` for everyone and fix the five errors together, one volunteer per error number.
- Lesson 2.1 Exercise 3 live: the coworker's broken `CREATE TABLE Order Items (...)` with three problems.
- The 1.3 Bonus Challenge together: `prompt \u@\d>\_` and watch the prompt follow USE.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 3: Data Types, Constraints and CRUD

How to teach the third session of MySQL Foundations. Learners see why every column has a type and why rules belong in the database (MySQL refusing bad numbers, money, text, dates and duplicates, each with its own error number), then fill the bookstore with data and change it safely: INSERT, SELECT, UPDATE and DELETE with the preview habit, upserts, safe-updates mode, and a reload script for the data Module 3 queries.

`2 lessons` `3 hrs session` `Lessons 2.2-2.3` `Session 3 of 14 · Part 1`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame → live demo (you type, they follow) → guided lab (a kit lab, new practice) → the lesson's own project. The lesson exercises, quiz and journal are homework.
- Every SQL block below was run to produce the output under it, on MySQL 9.7.2, inside `bookstore` as Lesson 2.1's build script leaves it. Ask for a prediction before every statement, especially the ones that fail: predicting the error number is the skill of the day.
- Keep `lesson_05.html` and `lesson_06.html` open for the type tables and the DELETE/TRUNCATE/DROP table. Labs live in `labs/module-03/` and run inside `bookstore` (learners `USE bookstore;` once).
- This session has many error numbers. Do not ask learners to memorize them: put the lesson's error table on the board at the start and point at it each time.
- Session 4 rebuilds `bookstore` with Lesson 3.1's setup script, so nothing here can leave a learner stuck; the lesson projects (employees.sql, load_bookstore_data.sql) are still worth finishing.

> **THE #1 RISK OF SESSION 3: A LEARNER'S BOOKSTORE IS NOT WHAT THE LESSONS EXPECT**
>
> Everything today assumes `bookstore` with exactly three empty tables. Learners who did not finish `create_bookstore.sql`, or who left demo tables behind, get different outputs from the first statement on. **The warm-up checks it on every laptop** (`USE bookstore; SHOW TABLES;`), and the fix takes ten seconds: rerun `create_bookstore.sql`, or `SOURCE labs/_states/schema.sql` (the same three empty tables). The second risk is the 2.2 demo's length: twelve small steps; keep each to its minutes and cut Step 5 (text) first.

BEFORE THE SESSION

- ☐ **Sort the Session 2 exit tickets.** Question 3's "not yet" pile (the MODIFY trap) gets W4 in today's warm-up.

- ☐ **Files to hand out:** the `labs/module-03/` starters and the two debug files (no `*_solution.sql`). Remind learners to bring `create_bookstore.sql`.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-03`. Every line should say PASS (10 files).
- ☐ **Projector machine:** `SOURCE labs/_states/schema.sql` (bookstore with three empty tables, exactly what learners have), then `USE bookstore;`. A terminal at 18 pt or larger; browser tabs `lesson_05.html`, `lesson_06.html`.
- ☐ **Board, before learners arrive:** the error table (1048 NULL given · 1364 left out · 1062 duplicate · 1264 out of range · 1265 not on the list / rounded · 1292 bad date · 1406 too long · 3819 CHECK); and, for 2.3, "SELECT first, same WHERE, read Rows matched".
- ☐ **Print or share:** the workbook (warm-up, guided labs, self-checks, exit ticket).

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`ERROR 1046 No database selected` in a lab	The learner did not `USE bookstore;` first	`USE bookstore;` once per session; the labs never say USE themselves.
`ERROR 1146 Table 'bookstore.books' doesn't exist`	bookstore was dropped and not rebuilt	Rerun `create_bookstore.sql` or `SOURCE labs/_states/schema.sql`.
Bad values are stored with only a warning	Strict mode is off (some hosts, old XAMPP configs)	`SELECT @@sql_mode;` should list STRICT_TRANS_TABLES; for the session: the lesson's `SET SESSION sql_mode = CONCAT(...)` line.
`AS new` gives ERROR 1064	MariaDB (XAMPP)	`ON DUPLICATE KEY UPDATE description = VALUES(description)` (the lesson's version note).
CHECK errors read `ERROR 4025 ... CONSTRAINT ... failed`	MariaDB	Same rule, different number: say so and move on.
An UPDATE in the 2.3 demo is refused with 1175	The learner started the client with `--safe-updates` (homework from Lesson 2.3) or forgot `SET SQL_SAFE_UPDATES = 0;`	`SET SQL_SAFE_UPDATES = 0;` for the session.

SESSION 3 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 2; every laptop shows bookstore with three empty tables; homework check.
0:10	10	**2.2** Frame and mental model	A type is a promise, a constraint is a rule; strict mode refuses instead of guessing.
0:20	25	**2.2** Live demo	Numbers, money, text, dates and ENUM refusing bad values; the coupons table and its four errors; the BOOLEAN trap.
0:45	20	**2.2** Guided lab	`labs/module-03/l2_2-data-types-constraints/types_matter_starter.sql` in pairs: wrong types, wrong answers, no errors.
1:05	15	**2.2** Start the lesson project	employees.sql, Steps 1 to 4, then Step 6's cleanup; Step 5 goes home.
1:20	10	Break	Leave the error table (1048, 1062, 1264, 1265, 1292, 1364, 1406, 3819) on the board.
1:30	10	**2.3** Frame and mental model	CRUD, and WHERE picks the rows; the preview habit.
1:40	25	**2.3** Live demo	Fill books, read with aliases and DISTINCT, update and delete with previews, an upsert, safe-updates, the AND trap.
2:05	20	**2.3** Guided lab	`labs/module-03/l2_3-crud-operations/new_arrivals_starter.sql`: insert, preview, change, count.
2:25	25	**2.3** Build the lesson project	load_bookstore_data.sql, all five steps, run twice: the data Module 3 queries.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 4.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 100 + 105 = 205 minutes for a solo learner; the room has 150. Lesson 2.2 gets 70 minutes: its long reference tables are read at home, the room demos one example of each refusal. Lesson 2.3 gets 80, including its whole project.

Goes home (about 2 to 3 hours). employees.sql Step 5 and a rerun from a fresh session; each lesson's Hands-on Exercises; both Quick Quizzes; both journal entries; 2.3's "Before the Next Lesson" list (run load_bookstore_data.sql once more; try --safe-updates for a week).

Protect, in this order: the warm-up's bookstore check; 2.2 demo Steps 4 (money) and 8 to 9 (constraints); 2.3 demo Steps 8, 9 and 13 (preview, then the AND trap); the types lab through TODO 2; the load_bookstore_data project. **Cut first:** 2.2 demo Step 5 (text), then 2.3 demo Step 11 (the upsert: the fast finishers' lab covers it), then the 2.2 project start (all six steps go home).

OPENING SEGMENT

0:00 Warm-Up: Session 2 Retrieval — 10 min

TIMING

4m	Five questions W1 to W5 in the workbook, no notes, then fingers up
3m	Every laptop: `USE bookstore; SHOW TABLES;` lists books, categories, customers. Anyone else reruns create_bookstore.sql (or `SOURCE labs/_states/schema.sql`) now
3m	Homework check: hands up for explore.sql run from the shell; one learner reads a 2.1 journal sketch (it comes back in the 2.2 journal)

WARM-UP ANSWERS

Answers: **W1** B) `;` and Enter; **W2** C) Which database you are in: the message says mysql, not bookstore; **W3** A) `DESCRIBE books;`; **W4** B) `YES`: MODIFY replaced the whole definition; **W5** D) No database selected (1046): USE one, or write database.table.

- **W1 and W3** are the prompt and the SQL-versus-client split from 1.3: quick, and nearly everyone gets them.
- **W2** is reading an error message: the database it names. If it is missed, point at the message and ask "where did MySQL look?"
- **W4** is the MODIFY trap, and it is today's theme in miniature: the database keeps exactly the rules you write.
- **W5** (1046) catches the cabinet picture: the session is not inside any drawer.

2.2 Data Types & Constraints

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Pick an integer type and decide when UNSIGNED makes sense; explain why money is DECIMAL, never FLOAT
- Choose between CHAR, VARCHAR and TEXT, and between DATE, DATETIME and TIMESTAMP
- Use BOOLEAN and ENUM correctly, knowing what MySQL really stores
- Apply PRIMARY KEY, AUTO_INCREMENT, NOT NULL, UNIQUE, DEFAULT and named CHECK constraints
- Read the errors strict mode raises for bad values (1048, 1062, 1264, 1265, 1292, 1364, 1406, 3819)

TIMING IN THE ROOM

10m	Frame: a type is a promise, a constraint a rule; strict mode refuses instead of guessing
25m	Live demo: numbers, money, text, dates, ENUM; the coupons table; the BOOLEAN trap
20m	Guided lab: `types_matter_starter.sql` in pairs
15m	Start the lesson project: employees.sql Steps 1 to 4, then Step 6's cleanup

MENTAL MODEL TO SAY

Draw the lesson's flowchart: a value arrives → does it fit the column's type? → does it pass the constraints? → stored, or the row is rejected with a number. "Every program and every person that writes to this table goes through this door. That is far safer than hoping each one checks."

Then strict mode in one breath: "MySQL 9.7 starts in strict mode: a value that does not fit is an error. Without it MySQL would quietly cut, round or clamp and only warn. Old tutorials show that; some hosts still do it. Keep it on."

Then the rule of thumb on the board: **choose the type that says what the value is, then the smallest size that comfortably fits the biggest value you can imagine, then NOT NULL on every column a row cannot do without.** Money is DECIMAL; phone numbers and ZIP codes are text.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Inside `bookstore` (`USE bookstore;`). The demo creates six small demo tables next to the three real ones and drops them in the last step. Ask for the error number before every refused statement.

1. **One column of each kind of number (3m).** "A type is a promise about the values; MySQL keeps it for you." Read the four types aloud.

```
CREATE TABLE number_demo (
    stars     TINYINT,
    copies    SMALLINT UNSIGNED,
    price     DECIMAL(8, 2),
    weight_kg FLOAT
);
INSERT INTO number_demo (stars, copies, price, weight_kg)
VALUES (5, 120, 14.99, 0.1);
SELECT * FROM number_demo;
```

```
+-------+--------+-------+-----------+
| stars | copies | price | weight_kg |
+-------+--------+-------+-----------+
|     5 |    120 | 14.99 |       0.1 |
+-------+--------+-------+-----------+
```

2. **A number that does not fit (1m).** "A TINYINT stops at 127. Predict: what happens to 200?"

```
INSERT INTO number_demo (stars) VALUES (200);
```

```
ERROR 1264 (22003): Out of range value for column 'stars' at row 1
```

Strict mode (the default) refuses instead of quietly storing 127.

3. **Too many decimals is different (2m).** "Three decimals into a two-decimal column."

```
INSERT INTO number_demo (price) VALUES (12.345);
SHOW WARNINGS;
SELECT price FROM number_demo WHERE stars IS NULL;
```

```
+-------+------+-------------------------------------------+
| Level | Code | Message                                   |
+-------+------+-------------------------------------------+
| Note  | 1265 | Data truncated for column 'price' at row 1 |
+-------+------+-------------------------------------------+
+-------+
| price |
+-------+
| 12.35 |
+-------+
```

Rounded, with a Note. Rounding to the column's precision is not an error.

4 **Why money is never FLOAT (3m).** "Multiply both by 3. And is the stored 0.1 even equal to 0.1?"

```
SELECT
    price * 3        AS decimal_times_3,
    weight_kg * 3    AS float_times_3,
    weight_kg = 0.1  AS float_equals_0_1
FROM number_demo
WHERE stars = 5;
```

```
+-----------------+---------------------+------------------+
| decimal_times_3 | float_times_3       | float_equals_0_1 |
+-----------------+---------------------+------------------+
|           44.97 | 0.30000000447034836 |                0 |
+-----------------+---------------------+------------------+
```

The DECIMAL result is exact; FLOAT stores binary fractions. Over thousands of orders the crumbs become real money.

5 **Text that is too long (1m).** "Fourteen characters into VARCHAR(5)."

```
CREATE TABLE text_demo (
    code  CHAR(5),
    label VARCHAR(5)
);
INSERT INTO text_demo (label) VALUES ('Fahrenheit 451');
```

```
ERROR 1406 (22001): Data too long for column 'label' at row 1
```

Strict mode refuses instead of storing Fahre.

6 **Dates are checked (2m).** "February 30th. Then the date the way an American writes it."

```
CREATE TABLE date_demo (d DATE, dt DATETIME);
INSERT INTO date_demo (d) VALUES ('2026-02-30');
INSERT INTO date_demo (d) VALUES ('03/10/2026');
```

```
ERROR 1292 (22007): Incorrect date value: '2026-02-30' for column 'd' at row 1
ERROR 1292 (22007): Incorrect date value: '03/10/2026' for column 'd' at row 1
```

Dates are year first: '2026-03-10'. Spreadsheet dates go in with STR_TO_DATE (the lab uses it).

7 **ENUM: one value from a list (2m).** "A default, and a value that is not on the list."

```
CREATE TABLE format_demo (
    title  VARCHAR(50) NOT NULL,
    format ENUM('paperback', 'hardcover', 'ebook') NOT NULL DEFAULT 'paperback
);
INSERT INTO format_demo (title, format) VALUES ('Dune', 'hardcover');
INSERT INTO format_demo (title) VALUES ('Emma');
INSERT INTO format_demo (title, format) VALUES ('1984', 'audiobook');
SELECT title, format FROM format_demo;
```

```
ERROR 1265 (01000): Data truncated for column 'format' at row 1
+-------+-----------+
| title | format    |
+-------+-----------+
| Dune  | hardcover |
| Emma  | paperback |
+-------+-----------+
```

1265 "Data truncated" is how MySQL says the value is not one of the options.

8 **A table with every kind of rule (3m).** Read the CREATE line by line: "required, never repeated, a default, and a named CHECK."

```
CREATE TABLE coupons (
    id           INT AUTO_INCREMENT PRIMARY KEY,
    code         VARCHAR(20) NOT NULL UNIQUE,
    percent_off  TINYINT UNSIGNED NOT NULL,
    min_order    DECIMAL(8, 2) NOT NULL DEFAULT 0.00,
    expires      DATE,
    CONSTRAINT chk_coupon_percent CHECK (percent_off BETWEEN 5 AND 50)
);
INSERT INTO coupons (code, percent_off, expires) VALUES ('WELCOME10', 10, '202
INSERT INTO coupons (code, percent_off, min_order) VALUES ('BIGBASKET', 20, 50
SELECT * FROM coupons;
```

```
+----+-----------+-------------+-----------+------------+
| id | code      | percent_off | min_order | expires    |
+----+-----------+-------------+-----------+------------+
|  1 | WELCOME10 |          10 |      0.00 | 2026-12-31 |
|  2 | BIGBASKET |          20 |     50.00 | NULL       |
+----+-----------+-------------+-----------+------------+
```

9 **Break every rule (3m).** Ask for the four error numbers BEFORE you run it; write predictions on the board.

```
INSERT INTO coupons (code, percent_off) VALUES (NULL, 15);
INSERT INTO coupons (code) VALUES ('SPRING15');
INSERT INTO coupons (code, percent_off) VALUES ('WELCOME10', 15);
INSERT INTO coupons (code, percent_off) VALUES ('HALFPRICE', 75);
```

```
ERROR 1048 (23000): Column 'code' cannot be null
ERROR 1364 (HY000): Field 'percent_off' doesn't have a default value
ERROR 1062 (23000): Duplicate entry 'WELCOME10' for key 'coupons.code'
ERROR 3819 (HY000): Check constraint 'chk_coupon_percent' is violated.
```

Explicit NULL 1048, left out 1364, duplicate 1062, CHECK 3819 (named, so you know which rule).

10 **The id after failed inserts (2m).** "Which id does SUMMER25 get?"

```
INSERT INTO coupons (code, percent_off) VALUES ('SUMMER25', 25);
SELECT LAST_INSERT_ID();
SELECT id, code FROM coupons ORDER BY id;
```

```
+------------------+
| LAST_INSERT_ID() |
+------------------+
|                4 |
+------------------+
+----+-----------+
| id | code      |
+----+-----------+
|  1 | WELCOME10 |
|  2 | BIGBASKET |
|  4 | SUMMER25  |
+----+-----------+
```

4, not 3: a failed insert used up an id. Gaps are normal; an id is a label, not a count.

11 **The silent BOOLEAN trap (2m).** "BOOLEAN is a TINYINT, so 5 goes in. Which books are in stock?"

```
CREATE TABLE flag_demo (title VARCHAR(50), in_stock BOOLEAN);
INSERT INTO flag_demo VALUES ('Dune', TRUE), ('Emma', FALSE), ('1984', 5);
SELECT title, in_stock FROM flag_demo WHERE in_stock = TRUE;
SELECT title, in_stock FROM flag_demo WHERE in_stock;
```

```
+-------+----------+
| title | in_stock |
+-------+----------+
| Dune  |        1 |
+-------+----------+
+-------+----------+
| title | in_stock |
+-------+----------+
| Dune  |        1 |
| 1984  |        5 |
+-------+----------+
```

`= TRUE` means `= 1`; 1984 disappears. Test truth with `WHERE in_stock`.

12 **Clean up (1m).** "Back to the three tables Lesson 2.3 expects."

```
DROP TABLE IF EXISTS number_demo, text_demo, date_demo, format_demo, coupons,
SHOW TABLES;
```

```
+---------------------+
| Tables_in_bookstore |
+---------------------+
| books               |
| categories          |
| customers           |
+---------------------+
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Numbers that are really identifiers stored as numbers.** A ZIP code or phone as INT loses leading zeros. Coach: "Will you ever add two of these together? No? Then it is text."
- **Constraints forgotten for the obvious columns.** Learners write `email VARCHAR(100)` and stop. Coach with the question: "Does a row make sense without this value? Can two rows share it?"
- **Reading 1265 as a rounding message only.** It also means "not on the ENUM list". Coach: "Read the column name in the message: price (rounded) or format (not an option)?"

GUIDED LAB

File: `labs/module-03/l2_2-data-types-constraints/types_matter_starter.sql`; reference `labs/module-03/l2_2-data-types-constraints/types_matter_solution.sql`. A delivery list pasted from a spreadsheet with copies as text, cost as FLOAT and dates as MM/DD/YYYY text. As shipped, three of its four answers are wrong and MySQL never complains.

- **Run it as shipped first (3m)** and ask pairs which answers are wrong and why: copies sort 9, 15, 120 (text order); the total is 556.9499866962433; the dates sort by month text; the "last" delivery is 12/01/2026.
- **TODOs 1 to 3 in pairs (12m):** INT UNSIGNED, DECIMAL(6, 2), DATE, and the dates converted with `STR_TO_DATE('03/10/2026', '%m/%d/%Y')` (or retyped). A DATE column refuses the US format with 1292: let them hit it once.
- **Expected:** Dune 120, Emma 15, 1984 9; total exactly 556.95; 2026-03-10, 2026-12-01, 2027-01-15; last delivery 2027-01-15.
- **Debrief (3m):** "Nothing in the first run was an error. Wrong types give wrong answers silently; the right types make MySQL do the right thing."
- **Fast finishers:** `labs/module-03/l2_2-data-types-constraints/membership_rules_starter.sql` (five written rules become constraints), then `labs/module-03/l2_2-data-types-constraints/debug_bad_rows.sql` (six sign-ups refused with 1048, 1062, 1265, 1292, 1364 and 3819).

START THE LESSON PROJECT

The lesson project is **The Staff Table**: `employees.sql`, designed on paper, then proven by trying to break every rule. In the room: Steps 1 to 4 (13 minutes), then Step 6's DROP TABLE (2 minutes).

- Step 1 on paper in pairs (3m): a type and the rules for each requirement. Compare with Step 2 before typing.
- Steps 2 and 3: the table and the three staff members; nobody types id or is_active.
- Step 4: the four INSERTs that must fail; predict each error number first (3819, 1062, 1265, 1364).
- **Before the break:** Step 6's `DROP TABLE IF EXISTS employees, ...;` so bookstore is back to three tables for Lesson 2.3. `employees.sql` is saved, so nothing is lost.
- **Goes home:** Step 5 (COUNT and SHOW CREATE TABLE) and a rerun of the whole file from a fresh session (then Step 6 again).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Why is a phone number VARCHAR and not INT? **Answer:** it can start with 0 or +, has dashes, can be longer than INT allows, and nobody adds phone numbers.
- 1048 or 1364: which one is an explicit NULL? **Answer:** 1048; 1364 is a required column left out.
- Why name a CHECK constraint? **Answer:** the error then says which rule failed (`chk_salary`), instead of `employees_chk_1`.
- `WHERE in_stock = TRUE` misses a row holding 5. What should you write? **Answer:** `WHERE in_stock` (any non-zero), and a CHECK if only 0/1 is allowed.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** A) `TINYINT UNSIGNED`; **Q2** `7.01`; **Q3** C) `ERROR 1292 (22007): Incorrect date value: '2026-04-31' for column 'due' at row 1`; **Q4** D) `DATETIME`; **Q5** B) `ERROR 1048`: Column 'email' cannot be null; **Q6** `small`, `medium`, `large`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: the lesson's "Choosing the Right Type" table printed as a card; learners pick from it instead of from memory.

Stretch: `membership_rules_starter.sql` and `debug_bad_rows.sql`; the lesson's Exercise 4 (created/updated timestamps) and the Bonus Challenge (a CHECK comparing two columns).

2.3 CRUD Operations

80 min

OBJECTIVES (FROM THE LESSON PAGE)

- Create rows with INSERT: one row, many rows, with defaults, and read back the new id
- Read rows with SELECT: chosen columns, aliases, calculated columns and DISTINCT
- Update and delete exactly the rows you mean, using the "SELECT first" habit and safe-updates mode
- Write an upsert with INSERT ... AS new ON DUPLICATE KEY UPDATE, and explain how it differs from REPLACE and INSERT IGNORE
- Read and fix errors 1054, 1062, 1136 and 1175

TIMING IN THE ROOM

10m	Frame: CRUD; WHERE picks the rows; the preview habit
25m	Live demo: INSERT, SELECT, UPDATE and DELETE with previews, an upsert, safe-updates, the AND trap
20m	Guided lab: `new_arrivals_starter.sql`
25m	Build the lesson project: load_bookstore_data.sql, Steps 1 to 5, run twice

MENTAL MODEL TO SAY

Write CRUD on the board with its four statements. Then the lesson's key sentence: "INSERT only ever adds. UPDATE and DELETE act on **every row their WHERE matches**, and on every row in the table if there is no WHERE at all. There is no undo."

Then the habit, as a three-step recipe: **1 SELECT with the WHERE you mean · 2 the UPDATE or DELETE with exactly the same WHERE · 3 compare the counts** (the preview's rows and Rows matched). "Safe-updates mode is the seat belt; the preview is looking before you pull out."

And the upsert in one line: "Insert it, or update it if it is already there": the nightly-import problem every shop has.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Inside `bookstore` with three empty tables. These blocks keep MySQL's `Query OK` and `Rows matched` lines on purpose: read them aloud every time. The demo empties books and categories at the end, so the project starts clean.

1. **Create: one row (2m).** "List the columns, then the values in the same order. id and in_stock are left out on purpose."

```
INSERT INTO books (title, author, price, pages, published)
VALUES ('Dune', 'Frank Herbert', 12.99, 412, '1965-08-01');
```

```
Query OK, 1 row affected
```

2 **Read it back, and the new id (1m).** "LAST_INSERT_ID: what a program uses to show the new book's page."

```
SELECT * FROM books;
SELECT LAST_INSERT_ID();
```

```
+----+-------+---------------+-------+-------+------------+----------+
| id | title | author        | price | pages | published  | in_stock |
+----+-------+---------------+-------+-------+------------+----------+
|  1 | Dune  | Frank Herbert | 12.99 |   412 | 1965-08-01 |        1 |
+----+-------+---------------+-------+-------+------------+----------+
+------------------+
| LAST_INSERT_ID() |
+------------------+
|                1 |
+------------------+
```

3 **Many rows in one statement (2m).** "Commas between the value lists. Read the Records line."

```
INSERT INTO books (title, author, price, pages, published, in_stock) VALUES
    ('1984', 'George Orwell', 9.99, 328, '1949-06-08', TRUE),
    ('To Kill a Mockingbird', 'Harper Lee', 11.99, 281, '1960-07-11', TRUE),
    ('The Great Gatsby', 'F. Scott Fitzgerald', 8.99, 180, '1925-04-10', TRUE)
    ('Pride and Prejudice', 'Jane Austen', 7.99, 432, '1813-01-28', TRUE),
    ('The Hobbit', 'J.R.R. Tolkien', 10.99, 310, '1937-09-21', TRUE),
    ('Brave New World', 'Aldous Huxley', 9.49, 311, '1932-01-01', TRUE),
    ('The Catcher in the Rye', 'J.D. Salinger', 8.99, 234, '1951-07-16', FALSE
    ('Lord of the Rings', 'J.R.R. Tolkien', 15.99, 1178, '1954-07-29', TRUE),
    ('Fahrenheit 451', 'Ray Bradbury', 10.49, 158, '1953-10-19', TRUE);
```

```
Query OK, 9 rows affected
Records: 9  Duplicates: 0  Warnings: 0
```

4 **Aliases and a calculated column (2m).** "A calculation never changes the table."

```
SELECT
    title,
    price,
    ROUND(price * 1.08, 2) AS price_with_tax
FROM books
WHERE price > 11;
```

```
+-----------------------+-------+----------------+
| title                 | price | price_with_tax |
+-----------------------+-------+----------------+
| Dune                  | 12.99 |          14.03 |
| To Kill a Mockingbird | 11.99 |          12.95 |
| Lord of the Rings     | 15.99 |          17.27 |
+-----------------------+-------+----------------+
```

5 **DISTINCT (1m).** "Ten books, how many authors?"

```
SELECT DISTINCT author
FROM books
ORDER BY author;
```

```
+---------------------+
| author              |
+---------------------+
| Aldous Huxley       |
| F. Scott Fitzgerald |
| Frank Herbert       |
| George Orwell       |
| Harper Lee          |
| J.D. Salinger       |
| J.R.R. Tolkien      |
| Jane Austen         |
| Ray Bradbury        |
+---------------------+
```

6 **Update one row by its id (2m).** "Read the second line every time: matched, then changed."

```
UPDATE books
SET price = 14.99
WHERE id = 1;
```

```
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0
```

7 **The same update again (1m).** "Predict both numbers."

```
UPDATE books
SET price = 14.99
WHERE id = 1;
```

```
Query OK, 0 rows affected
Rows matched: 1  Changed: 0  Warnings: 0
```

Matched 1, changed 0: the price was already 14.99. Matched 0 would mean a typo in the WHERE.

8 **Preview first (2m).** "A sale on everything over $10. Before changing anything: which rows?"

```
SELECT id, title, price
FROM books
WHERE price > 10
ORDER BY id;
```

```
+----+-----------------------+-------+
| id | title                 | price |
+----+-----------------------+-------+
|  1 | Dune                  | 14.99 |
|  3 | To Kill a Mockingbird | 11.99 |
|  6 | The Hobbit            | 10.99 |
|  9 | Lord of the Rings     | 15.99 |
| 10 | Fahrenheit 451        | 10.49 |
+----+-----------------------+-------+
```

9 **Then the same WHERE (1m).** "Five previewed, five matched: that is the habit."

```
UPDATE books
SET price = ROUND(price * 0.90, 2)
WHERE price > 10;
```

```
Query OK, 5 rows affected
Rows matched: 5  Changed: 5  Warnings: 0
```

10 **Delete, and the id is not reused (2m).** "Fahrenheit 451 goes and comes back. Which id?"

```sql
DELETE FROM books WHERE id = 10;
INSERT INTO books (title, author, price, pages, published)
VALUES ('Fahrenheit 451', 'Ray Bradbury', 10.49, 158, '1953-10-19');
SELECT id, title FROM books WHERE title = 'Fahrenheit 451';
```

```
+----+---------------+
| id | title         |
+----+---------------+
| 11 | Fahrenheit 451 |
+----+---------------+
```

11, never 10 again: another table, an email or a receipt may still mention old id 10.

11 **Categories, then an upsert (3m).** Insert the five categories, then: "Fiction exists. Insert it, or update it if it is there."

```sql
INSERT INTO categories (name, description) VALUES
    ('Fiction', 'Novels, short stories, and other imaginative works'),
    ('Science Fiction', 'Stories exploring futuristic or scientific concepts'),
    ('Classic', 'Timeless literary works recognized across generations'),
    ('Fantasy', 'Stories set in imaginary worlds with magical elements'),
    ('Dystopian', 'Stories about oppressive societies and dark futures');

INSERT INTO categories (name, description)
VALUES ('Fiction', 'Imaginative prose of every kind') AS new
ON DUPLICATE KEY UPDATE description = new.description;
```

```
Query OK, 2 rows affected
```

2 rows affected = the update branch ran (1 = inserted, 0 = unchanged). MariaDB has no `AS new`: `VALUES(description)` there.

12 **The safety switch (2m).** "The classic slip, with safe-updates on."

```sql
SET SQL_SAFE_UPDATES = 1;
UPDATE books SET price = 0;
```

```
ERROR 1175 (HY000): You are using safe update mode and you tried to update a t
```

Then `SET SQL_SAFE_UPDATES = 0;` again for the rest of the demo (its WHEREs use non-key columns).

13 **AND instead of a comma (2m).** "Dune: 15.99 and out of stock. Predict."

```
UPDATE books
SET price = 15.99 AND in_stock = FALSE
WHERE id = 1;
SELECT title, price, in_stock FROM books WHERE id = 1;
```

```
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0
+-------+-------+----------+
| title | price | in_stock |
+-------+-------+----------+
| Dune  |  0.00 |        1 |
+-------+-------+----------+
```

No error, one row changed, price 0.00 and still in stock: `price = (15.99 AND (in_stock = FALSE))`. With TEXT values the same slip is refused (1292), as the debug handout shows.

14 **Empty the tables (1m).** "The project loads the official data from nothing."

```
TRUNCATE TABLE books;
TRUNCATE TABLE categories;
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Text without quotes** (`VALUES (Emma, ...)` → 1054 Unknown column 'Emma'). Coach: "Without quotes MySQL thinks Emma is a column." A quote inside text is doubled: `'Charlotte''s Web'`.
- **Not reading the second line.** Learners see Query OK and move on. Coach: "How many rows did the WHERE match? Is that the number your preview showed?"
- **AND in SET.** No error with numbers (price becomes 0.00), ERROR 1292 with text. Coach: "Assignments are separated by commas." Then look at the changed row.

GUIDED LAB

File: `labs/module-03/l2_3-crud-operations/new_arrivals_starter.sql`; reference `labs/module-03/l2_3-crud-operations/new_arrivals_solution.sql`. Six new books arrive; then a price rise, a sold-out book, a clear-out and one more arrival, each change previewed and counted with `ROW_COUNT()`. The file empties books at the end (TRUNCATE), so it can run any number of times.

- **Steps 1 and 2 run as shipped:** predict `LAST_INSERT_ID()` after a six-row INSERT first (it is 1: the first row of the batch, not 6).
- **TODOs 3 to 6 (15m):** each turned-off statement has one gap. TODO 4's gap is the comma between two assignments: the AND trap in disguise.

- **Expected:** preview Moby Dick 12.40 and Little Women 10.20; rows_changed 2; Frankenstein 6.99, in_stock 0; Emma and Frankenstein previewed, rows_deleted 2; Middlemarch gets id 7; final prices 13.02 and 10.71.
- **Circulate for:** TODO 3's WHERE different from the preview's (the counts disagree: good, ask why); TODO 5 run before its preview; learners running the lab after their project (its TRUNCATE empties books: rerun load_bookstore_data.sql).
- **Fast finishers:** `labs/module-03/l2_3-crud-operations/category_feed_starter.sql` (a multi-row upsert with ROW_COUNT 3, INSERT IGNORE, REPLACE), then `labs/module-03/l2_3-crud-operations/debug_crud_slips.sql` (1136, 1054, 1062, 1175, 1292).

BUILD THE LESSON PROJECT

The lesson project is **Stock the Bookstore**: `load_bookstore_data.sql`, which empties the three tables and loads the official 10 books, 8 customers and 5 categories. All five steps in the room (25 minutes).

- Step 1: header, `USE bookstore;`, three TRUNCATEs.
- Steps 2 and 3: the three INSERTs from the lesson page (typing them is fine practice; copying is acceptable if time is short, then check each row).
- Step 4: the check queries. **Done** = books ends with id 10 Fahrenheit 451, Gatsby and Catcher show in_stock 0, 8 customers, 5 categories.
- Step 5: SOURCE it twice; the same result both times. Walk the room and look at every screen.
- Session 4 starts with Lesson 3.1's setup script, which rebuilds the same data from nothing: a learner who does not finish here is not stuck, but the habit of a reload script is the point.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- `UPDATE books SET price = 0;`: what happens in a normal session? **Answer:** every price becomes 0.00, no undo.
- You deleted id 10 and inserted a book. Which id? **Answer:** 11: AUTO_INCREMENT never reuses a number.
- The upsert says 2 rows affected. Which branch ran? **Answer:** the update (1 = inserted, 0 = unchanged).
- Why avoid REPLACE on a row other tables point to? **Answer:** it deletes and re-inserts: a new id, and the unlisted columns get their defaults.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** D) It depends on the table's exact column order: an added column breaks it (1136) or shifts values silently; **Q8** `2`; **Q9** A) The WHERE found no rows: often a typo in the value or the wrong column; **Q10** C) `UPDATE books SET price = 9.99 WHERE id = 2;`; **Q11** B) `INSERT ... AS new ON DUPLICATE KEY UPDATE description = new.description;`; **Q12** `3`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: the three-step recipe (preview, same WHERE, compare counts) on a card next to the keyboard.

Stretch: `category_feed_starter.sql` and `debug_crud_slips.sql`; the lesson's Exercise 5 (an idempotent upsert) and the Bonus Challenge (an upsert that adds to a count; note the Answer Key's warning about qualifying the column).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide; learners answer in the workbook or on a sticky note. Question 3's answers tell you whether the preview habit stuck.

1. Why is money stored as DECIMAL and never as FLOAT? One sentence and one example. **Answer:** DECIMAL is exact; FLOAT stores binary fractions, so most decimal values are slightly off and the crumbs add up: 0.1 stored as FLOAT times 3 printed 0.30000000447034836 in the demo, and the lab's FLOAT total was 556.9499866962433 instead of 556.95.
2. An UPDATE says `Rows matched: 3 Changed: 2`. What happened to the third row? **Answer:** The WHERE found it, but it already had the new value, so nothing changed in it. (Matched 0 would mean the WHERE found nothing.)
3. What do you run before every UPDATE or DELETE that touches more than one row, and why? **Answer:** A SELECT with exactly the same WHERE: it shows which rows will be touched, and its row count must match the UPDATE's Rows matched. Safe-updates mode (`SET SQL_SAFE_UPDATES = 1;` or `mysql --safe-updates`) is the second net.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish employees.sql** (Lesson 2.2 project Step 5, then rerun the whole file and Step 6's cleanup so bookstore is back to three tables).
- **load_bookstore_data.sql:** if it was not finished in class, finish it; everyone runs it once more (2.3's "Before the Next Lesson").
- **Lesson exercises.** 2.2: Exercises 1 to 3 (pick the type, prove DECIMAL rounds, fix the bad table); Exercise 4 is a stretch. 2.3: Exercises 1 to 4 (add, read, update and remove, each putting the data back); Exercise 5 is a stretch.
- **Both Quick Quizzes** on the lesson pages (answers in the Answer Key).
- **Journal:** both entries; the 2.2 prompt revises the 2.1 sketch.
- **Try** starting the client with `--safe-updates` for a week.
- **Optional:** `labs/module-03/l2_2-data-types-constraints/membership_rules_starter.sql`, `labs/module-03/l2_3-crud-operations/category_feed_starter.sql` and the two debug handouts.

BRING LIST FOR SESSION 4

- A laptop with bookstore loaded (10 books, 8 customers, 5 categories), the root password.
- `create_bookstore.sql` and `load_bookstore_data.sql` in the mysql-course folder.
- The course folder with `labs/` (Session 4's starters are in `labs/module-04/`), the workbook and the journal.
- Your notes from Sessions 1 to 3: Session 4 ends with **Checkpoint 1** on Modules 1 and 2 (30 minutes, open notes).

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the warm-up's bookstore check; 2.2 demo Steps 4, 8 and 9; 2.3 demo Steps 8, 9 and 14; the load_bookstore_data project.
- **Cut first (saves 1m):** 2.2 demo Step 5 (text too long): the lesson page shows it.
- **Cut second (saves 3m):** 2.3 demo Step 11 (the upsert): "the lesson's upsert section, and the fast finishers' lab".
- **Cut third (saves 15m):** the 2.2 project start: all six steps of employees.sql go home.

RUNNING SHORT: WHAT TO ADD

- Run `debug_bad_rows.sql` for everyone: match each error to one of the club's rules R1 to R5.
- Lesson 2.2 Exercise 3 live: the coworker's Orders table with almost every column wrong.
- Lesson 2.3 Exercise 5 live: run the Horror upsert three times and read the three counts.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 4: Filtering, Sorting and Limiting

How to teach the fourth session of MySQL Foundations. Learners filter rows precisely (comparisons, AND/OR with parentheses, IN, BETWEEN, LIKE and the rules for NULL), sort and page results so they come out the same every time, and close Modules 1 and 2 with Checkpoint 1, a scored 30-minute task: build, fill and read a small database from nothing.

`2 lessons` `3 hrs session` `Lessons 3.1–3.2` `Checkpoint 1` `Session 4 of 14 · Part 1`

HOW TO USE THIS GUIDE

- Two lessons in shorter blocks (65 and 55 minutes) because the 30 minutes before the wrap-up are **Checkpoint 1**, a scored open-notes task on Modules 1 and 2 (Lessons 1.1 to 2.3).
- Every SQL block below was run to produce the output under it, on MySQL 9.7.2, on the Module 3 data (Lesson 3.1's setup script = `labs/_states/flat.sql`). Ask for a row count before every query: counting first is how silent filter mistakes get caught.
- Keep `lesson_07.html` and `lesson_08.html` open. Labs live in `labs/module-04/`; the lesson labs are read-only and run inside `bookstore`.
- Part 1 (Sessions 1 to 7) can run on its own. Teal blocks are hands-on, amber blocks are watch-outs.

> **THE #1 RISK OF SESSION 4: RUNNING OUT OF TIME FOR THE CHECKPOINT**
>
> Both lessons are dense, and a room of laptops is slower than your demo. **Protect the 2:20 start of Checkpoint 1.** If 3.1 runs over, cut its project start (send all seven steps home); if 3.2 runs over, cut the lab to TODOs 1 to 4 and the project start. The second risk is data: anyone whose `bookstore` is not the Module 3 data gets different rows all session. The warm-up loads it on every laptop.

BEFORE THE SESSION

- ☐ **Sort the Session 3 exit tickets.** Question 3 (the preview habit) tells you who needs Part B's preview in the checkpoint pointed out.
- ☐ **Files to hand out:** the `labs/module-04/` starters and debug files; **hold back** `checkpoint-1/checkpoint_starter.sql` until 2:20 (print the workbook's Checkpoint 1 page or share it at that moment). Keep `checkpoint_solution.sql` and the rubric (Answer Key) to yourself.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-04`. Every line should say PASS (12

files).

- ☐ **Projector machine:** `SOURCE labs/_states/flat.sql` (it ends in bookstore). Browser tabs `lesson_07.html`, `lesson_08.html`.
- ☐ **Board, before learners arrive:** the evaluation order (comparisons → NOT → AND → OR), "NULL: IS NULL, never = NULL", and the clause order SELECT · FROM · WHERE · ORDER BY · LIMIT.
- ☐ **Checkpoint logistics:** decide how learners hand in (their `checkpoint_starter.sql` by email or USB, plus the workbook page for Part D). Open notes means workbook, scripts and lesson pages; no neighbors, no AI assistants.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
Results do not match the guide	bookstore is not the Module 3 data (Session 3's labs or experiments changed it)	`SOURCE labs/_states/flat.sql` (or Lesson 3.1's setup block); it rebuilds bookstore.
`ERROR 1046 No database selected`	A fresh session	`USE bookstore;` (flat.sql ends with it).
Sorted results differ on MariaDB	Ties without a tie-breaker are ordered differently	That is the lesson: add a tie-breaker.
`NULLS LAST` is a syntax error	MySQL does not support it	`ORDER BY col IS NULL, col` (the lesson's version note).
Checkpoint: `ERROR 1064` at `condition`	CONDITION is a reserved word	Call the column `book_condition` (the task says so).

SESSION 4 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Load the Module 3 data on every laptop, five retrieval questions on Session 3, homework check.
0:10	10	**3.1** Frame and mental model	WHERE tests every row: true, false or unknown; only true is kept.
0:20	25	**3.1** Live demo	Comparisons, AND/OR/NOT, IN, BETWEEN, a half-open date range, LIKE, NULL, precedence, two mistakes.
0:45	20	**3.1** Guided lab	`labs/module-04/l3_1-where-clauses-operators-filtering/staff_requests_starter.sql` in pairs: seven requests,

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
			seven WHERE clauses.
1:05	10	**3.1** Start the lesson project	saved_searches.sql, Steps 1 to 3; Steps 4 to 7 go home.
1:15	10	Break	Leave "AND before OR: add parentheses" and "NULL: IS NULL" on the board.
1:25	10	**3.2** Frame and mental model	A table has no order; ORDER BY, tie-breakers, then LIMIT cuts.
1:35	20	**3.2** Live demo	Several keys, an alias, CASE, where NULLs go, top N, OFFSET pages, keyset, three mistakes.
1:55	15	**3.2** Guided lab	`labs/module-04/l3_2-sorting-limiting-results/bestseller_board_starter.sql`: six orders for a display board.
2:10	10	**3.2** Start the lesson project	storefront.sql, Steps 1 and 2; Steps 3 to 6 go home.
2:20	30	Checkpoint 1 (Modules 1 and 2)	`labs/module-04/checkpoint-1/checkpoint_starter.sql`: open notes, 30 minutes, scored with the 5-row rubric.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 5.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 95 + 85 = 180 minutes for a solo learner; with the warm-up, the break, the checkpoint and the wrap-up the room has 120. Each lesson keeps its full shape in shorter blocks; both projects start here and finish at home.

Goes home (about 2 to 3 hours). saved_searches.sql Steps 4 to 7 and storefront.sql Steps 3 to 6; each lesson's Hands-on Exercises; both Quick Quizzes; both journal entries; anything unfinished in the checkpoint is NOT homework (it is scored as handed in).

Protect, in this order: the checkpoint's 30 minutes; the warm-up's data load; 3.1 demo Steps 4, 11 to 14 (NULL and precedence); 3.2 demo Steps 1, 2 and 11 (ties and the silent quoted name); the staff-requests lab through request 5. **Cut first:** the 3.1 project start, then 3.2 demo Steps 4 and 5 (alias, CASE), then the 3.2 project start.

OPENING SEGMENT

0:00 Warm-Up: Session 3 Retrieval and the Module 3 Data — 10 min

TIMING

3m	Everyone: `SOURCE labs/_states/flat.sql` (or Lesson 3.1's setup block), then `SELECT COUNT(*) FROM books;` says 10
4m	Five questions W1 to W5 in the workbook, no notes, then fingers up
3m	Homework check: load_bookstore_data.sql run twice; who tried --safe-updates, and what did it stop?

WARM-UP ANSWERS

Answers: **W1** B) `DECIMAL(8, 2)`; **W2** C) `1048`; **W3** D) `Rows matched: 1 Changed: 0`; **W4** A) A SELECT with exactly the same WHERE; **W5** C) `REPLACE INTO ...`.

- **W1 to W3** are Lesson 2.2 and 2.3 facts the checkpoint uses: DECIMAL, 1048, and the second line of an UPDATE.
- **W4** is the preview habit; the checkpoint's B4 asks for it.
- **W5:** REPLACE versus the upsert, the checkpoint's B6.
- Tell the room now: "Checkpoint 1 at 2:20, 30 minutes, open notes, on everything up to Lesson 2.3."

LESSON PLANS

3.1 WHERE Clauses, Operators & Filtering

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Filter rows by numbers, text and dates with the comparison operators (=, !=, <, >, <=, >=)
- Combine conditions with AND, OR and NOT, and use parentheses to say exactly what you mean
- Match lists, ranges and text patterns with IN, BETWEEN and LIKE
- Find missing values with IS NULL and explain why = NULL finds nothing
- Spot the silent filtering mistakes (missing NULL rows, AND/OR mix-ups) and fix error 1054
- Save a set of reusable searches in a .sql script

TIMING IN THE ROOM

10m	Frame: a test on every row; true, false, unknown
25m	Live demo: comparisons to precedence, two mistakes
20m	Guided lab: `staff_requests_starter.sql` in pairs
10m	Start the lesson project: saved_searches.sql, Steps 1 to 3

MENTAL MODEL TO SAY

Draw the lesson's diagram: each row of books → the test → kept (true) or dropped (false, **or unknown**). "WHERE is a test MySQL runs on every row. The third answer, unknown, appears whenever a NULL is involved, and unknown is dropped like false."

Then the counting habit: "Before you run a filter, say how many rows you expect. Three of today's mistakes give no error at all; the count is how you catch them."

Then precedence as arithmetic: "Multiplication before addition; AND before OR. When you mix them, type the parentheses."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Inside `bookstore` with the Module 3 data. Ask for the row count before each query.

1. **WHERE as a test on every row (2m).** "Five of ten pass. The ORDER BY makes the result read the same everywhere: Lesson 3.2."

```
SELECT title, price
FROM books
WHERE price < 10
ORDER BY price, title;
```

```
+------------------------+-------+
| title                  | price |
+------------------------+-------+
| Pride and Prejudice    |  7.99 |
| The Catcher in the Rye |  8.99 |
| The Great Gatsby       |  8.99 |
| Brave New World        |  9.49 |
| 1984                   |  9.99 |
+------------------------+-------+
```

2 **See the test itself (2m).** "A condition is a value: 1, 0 or NULL. Put it in the SELECT list to debug a WHERE."

```
SELECT title, price, price < 10 AS passes
FROM books
ORDER BY id
LIMIT 4;
```

```
+-----------------------+-------+--------+
| title                 | price | passes |
+-----------------------+-------+--------+
| Dune                  | 14.99 |      0 |
| 1984                  |  9.99 |      1 |
| To Kill a Mockingbird | 11.99 |      0 |
| The Great Gatsby      |  8.99 |      1 |
+-----------------------+-------+--------+
```

3 **Text ignores case (1m).** "The collation is _ci: case-insensitive."

```
SELECT title
FROM books
WHERE author = 'j.r.r. tolkien'
ORDER BY title;
```

```
+-------------------+
| title             |
+-------------------+
| Lord of the Rings |
| The Hobbit        |
+-------------------+
```

4 **Not equal, and two missing customers (2m).** "Eight customers, two in New York. Count the rows."

```sql
SELECT first_name, city
FROM customers
WHERE city != 'New York'
ORDER BY first_name;
```

```
+------------+-------------+
| first_name | city        |
+------------+-------------+
| Bob        | Chicago     |
| Carol      | Los Angeles |
| Dave       | Houston     |
| Grace      | Chicago     |
+------------+-------------+
```

Four. Eve and Hank are gone: an unknown city is not "different from New York". Leave the question open until Step 12.

5 **AND narrows (1m).** "Cheap AND long."

```sql
SELECT title, price, pages
FROM books
WHERE price < 10 AND pages > 300
ORDER BY price;
```

```
+---------------------+-------+-------+
| title               | price | pages |
+---------------------+-------+-------+
| Pride and Prejudice |  7.99 |   432 |
| Brave New World     |  9.49 |   311 |
| 1984                |  9.99 |   328 |
+---------------------+-------+-------+
```

6 **OR widens (1m).** "Cheap OR short. Gatsby passes both: does it appear twice?"

```sql
SELECT title, price, pages
FROM books
WHERE price < 9 OR pages < 200
ORDER BY title;
```

```
+------------------------+-------+-------+
| title                  | price | pages |
+------------------------+-------+-------+
| Fahrenheit 451         | 10.49 |   158 |
| Pride and Prejudice    |  7.99 |   432 |
| The Catcher in the Rye |  8.99 |   234 |
| The Great Gatsby       |  8.99 |   180 |
+------------------------+-------+-------+
```

7 **IN: a list (1m).** "Three ORs on one column, said once."

```sql
SELECT title, author
FROM books
WHERE author IN ('George Orwell', 'Aldous Huxley', 'Ray Bradbury')
ORDER BY author;
```

```
+-----------------+---------------+
| title           | author        |
+-----------------+---------------+
| Brave New World | Aldous Huxley |
| 1984            | George Orwell |
| Fahrenheit 451  | Ray Bradbury  |
+-----------------+---------------+
```

8 **BETWEEN includes both ends (1m).** "Are 9.49 and 10.99 in?"

```sql
SELECT title, price
FROM books
WHERE price BETWEEN 9.49 AND 10.99
ORDER BY price;
```

```
+-----------------+-------+
| title           | price |
+-----------------+-------+
| Brave New World |  9.49 |
| 1984            |  9.99 |
| Fahrenheit 451  | 10.49 |
| The Hobbit      | 10.99 |
+-----------------+-------+
```

9 **A month of timestamps (2m).** "created_at has a time. Half-open: on or after the first, before the first of next month."

```sql
SELECT first_name, created_at
FROM customers
WHERE created_at >= '2026-01-01'
  AND created_at <  '2026-02-01'
ORDER BY created_at;
```

```
+------------+---------------------+
| first_name | created_at          |
+------------+---------------------+
| Alice      | 2026-01-04 10:15:00 |
| Bob        | 2026-01-06 14:02:00 |
| Carol      | 2026-01-09 09:40:00 |
| Dave       | 2026-01-12 16:25:00 |
| Eve        | 2026-01-20 11:08:00 |
| Frank      | 2026-01-27 13:30:00 |
+------------+---------------------+
```

10 **LIKE: contains (2m).** "% is any number of characters. Lowercase 'the' finds The too."

```sql
SELECT title
FROM books
WHERE title LIKE '%the%'
ORDER BY title;
```

```
+------------------------+
| title                  |
+------------------------+
| Lord of the Rings      |
| The Catcher in the Rye |
| The Great Gatsby       |
| The Hobbit             |
+------------------------+
```

11 **NULL is unknown (2m).** "Is NULL equal to NULL?" Let them guess before you run it.

```sql
SELECT
    NULL = NULL   AS equals,
    NULL != 'x'   AS not_equals,
    NULL IS NULL  AS is_null;
```

```
+--------+------------+---------+
| equals | not_equals | is_null |
+--------+------------+---------+
|   NULL |       NULL |       1 |
+--------+------------+---------+
```

12 **Keep the NULL rows on purpose (2m).** "Back to Step 4: say what the empty rows should do."

```sql
SELECT first_name, city
FROM customers
WHERE city != 'New York' OR city IS NULL
ORDER BY first_name;
```

```
+------------+-------------+
| first_name | city        |
+------------+-------------+
| Bob        | Chicago     |
| Carol      | Los Angeles |
| Dave       | Houston     |
| Eve        | NULL        |
| Grace      | Chicago     |
| Hank       | NULL        |
+------------+-------------+
```

13 **AND before OR (2m).** "Tolkien or Orwell books under $12, written the way we say it."

```
SELECT title, author, price
FROM books
WHERE author = 'J.R.R. Tolkien' OR author = 'George Orwell' AND price < 12
ORDER BY price;
```

```
+-------------------+----------------+-------+
| title             | author         | price |
+-------------------+----------------+-------+
| 1984              | George Orwell  |  9.99 |
| The Hobbit        | J.R.R. Tolkien | 10.99 |
| Lord of the Rings | J.R.R. Tolkien | 15.99 |
+-------------------+----------------+-------+
```

Lord of the Rings at 15.99 is in it: MySQL read A OR (B AND C).

14 **Parentheses say what you meant (1m).**

```
SELECT title, author, price
FROM books
WHERE (author = 'J.R.R. Tolkien' OR author = 'George Orwell')
  AND price < 12
ORDER BY price;
```

```
+------------+----------------+-------+
| title      | author         | price |
+------------+----------------+-------+
| 1984       | George Orwell  |  9.99 |
| The Hobbit | J.R.R. Tolkien | 10.99 |
+------------+----------------+-------+
```

15 **Two mistakes (2m).** "Text without quotes; and = NULL."

```
SELECT title FROM books WHERE author = Tolkien;
SELECT first_name, city
FROM customers
WHERE city = NULL;
```

```
ERROR 1054 (42S22): Unknown column 'Tolkien' in 'where clause'
Empty set
```

The second is silent: no error, no rows. IS NULL.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- `= NULL` **or** `!=` **that drops the NULL rows.** Coach: "What happens to Eve and Hank in this test?" Show the condition as a column.
- **AND/OR without parentheses.** Coach: "Read it the way MySQL does: AND first. Where would you put the brackets?"
- **Text without quotes (1054) and wildcards with = (silent).** Coach: "Quotes for text; % only means something to LIKE."

GUIDED LAB

File: `labs/module-04/l3_1-where-clauses-operators-filtering/staff_requests_starter.sql`; reference `labs/module-04/l3_1-where-clauses-operators-filtering/staff_requests_solution.sql`. Seven requests from the staff, each a ready SELECT whose WHERE says FALSE (Empty set).

- **Request 1 together (3m), 2 to 7 in pairs (14m):** count the expected rows from the lesson's two tables first, then write the WHERE.
- **Expected:** 3 · 3 · 2 (Johnson, Wilson) · 2 (1984, Dune) · 6 · 3 · 2 (Grace, Hank). Request 5 is the NULL trap (4 without `OR city IS NULL`); request 7 the half-open range.
- **Circulate for:** request 2 with `'1950'` (works by accident: compare with a full date); request 4 with `LIKE '%____%'` (every title); request 6 with `OR` instead of `AND` before the Tolkien test.
- **Debrief (3m):** which request would have been wrong without an error? (5, and 6 if mis-bracketed.)
- **Fast finishers:** `labs/module-04/l3_1-where-clauses-operators-filtering/null_detective_starter.sql`, then `labs/module-04/l3_1-where-clauses-operators-filtering/debug_filters.sql`.

START THE LESSON PROJECT

The lesson project is **Saved Searches**: `saved_searches.sql`, six commented searches the staff can rerun. In the room: Steps 1 to 3 (10 minutes).

- Steps 1 to 3: gift ideas, long reads, mid-century shelf, each run as it is added.
- **Done in the room** = three searches saved and run with SOURCE.
- **Goes home:** Steps 4 to 7 (title search, follow-up list, flyer, run the whole file).

CHECKS FOR UNDERSTANDING

Ask these out loud:

- `NULL = NULL`: true, false or unknown? **Answer:** unknown (NULL).
- BETWEEN 9.49 AND 10.99: are the ends in? **Answer:** yes, both.
- Why a half-open range for created_at? **Answer:** BETWEEN '...-01' AND '...-31' stops at midnight of the 31st

and misses that day.

- `'The%'` or `'The %'`? **Answer:** the space keeps out words like Theory.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** B) 4; **Q2** `4`; **Q3** A) `WHERE author NOT IN ('J.R.R. Tolkien', 'George Orwell')`; **Q4** C) `WHERE created_at >= '2026-01-27' AND created_at < '2026-01-28'`; **Q5** `Alice`, `Frank`, `Grace`; **Q6** D) `WHERE city NOT IN ('Reno', NULL)`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a card with the operator table and "NULL → IS NULL".

Stretch: `null_detective_starter.sql` and `debug_filters.sql`; the lesson's Exercise 4 (the picky customer) and the REGEXP Bonus Challenge.

3.2 Sorting & Limiting Results

55 min

OBJECTIVES (FROM THE LESSON PAGE)

- Sort results with ORDER BY on one or more columns or expressions, each ascending or descending
- Predict where NULL values land in a sort and move them to the end
- Take the top N rows with LIMIT, and add a tie-breaker so the result is always the same
- Split a result into pages with LIMIT ... OFFSET, and explain when keyset pagination is better
- Read and fix the errors MySQL raises for misplaced or malformed ORDER BY and LIMIT clauses

TIMING IN THE ROOM

10m	Frame: a table has no order; sort keys and tie-breakers; LIMIT runs last
20m	Live demo: ties, directions, alias, CASE, NULLs, top N, pages, keyset, three mistakes
15m	Guided lab: `bestseller_board_starter.sql`
10m	Start the lesson project: storefront.sql, Steps 1 and 2

MENTAL MODEL TO SAY

"A table is a set of rows, not a list. Without ORDER BY you get whatever is cheapest for the server, which usually looks right until one day it is not." Then the phone book: sorted by last name, and by first name only among people with the same last name.

Then the run order on the board: FROM → WHERE → SELECT → ORDER BY → LIMIT. "That is why an alias works in ORDER BY, and why LIMIT cuts the sorted result." And the rule: **every ORDER BY that feeds a LIMIT ends with a unique column.**

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Same data; read-only.

1. **Ties without a tie-breaker (2m).** "Cheapest first. Two books cost 8.99: which comes first, and will it always?"

```sql
SELECT title, price
FROM books
ORDER BY price;
```

```
+------------------------+-------+
| title                  | price |
+------------------------+-------+
| Pride and Prejudice    |  7.99 |
| The Great Gatsby       |  8.99 |
| The Catcher in the Rye |  8.99 |
| Brave New World        |  9.49 |
| 1984                   |  9.99 |
| Fahrenheit 451         | 10.49 |
| The Hobbit             | 10.99 |
| To Kill a Mockingbird  | 11.99 |
| Dune                   | 14.99 |
| Lord of the Rings      | 15.99 |
+------------------------+-------+
```

Here Gatsby came first; nothing promises it. Lesson 3.2's card says the same.

2 **A tie-breaker (1m).**

```sql
SELECT title, price
FROM books
ORDER BY price, title;
```

```
+------------------------+-------+
| title                  | price |
+------------------------+-------+
| Pride and Prejudice    |  7.99 |
| The Catcher in the Rye |  8.99 |
| The Great Gatsby       |  8.99 |
| Brave New World        |  9.49 |
| 1984                   |  9.99 |
| Fahrenheit 451         | 10.49 |
| The Hobbit             | 10.99 |
| To Kill a Mockingbird  | 11.99 |
| Dune                   | 14.99 |
| Lord of the Rings      | 15.99 |
+------------------------+-------+
```

3 **A direction per column (2m).** "In stock first (1 before 0, so DESC), then cheapest."

```sql
SELECT title, in_stock, price
FROM books
ORDER BY in_stock DESC, price, title;
```

```
+------------------------+----------+-------+
| title                  | in_stock | price |
+------------------------+----------+-------+
| Pride and Prejudice    |        1 |  7.99 |
| Brave New World        |        1 |  9.49 |
| 1984                   |        1 |  9.99 |
| Fahrenheit 451         |        1 | 10.49 |
| The Hobbit             |        1 | 10.99 |
| To Kill a Mockingbird  |        1 | 11.99 |
| Dune                   |        1 | 14.99 |
| Lord of the Rings      |        1 | 15.99 |
| The Catcher in the Rye |        0 |  8.99 |
| The Great Gatsby       |        0 |  8.99 |
+------------------------+----------+-------+
```

4 **Sort by an alias (2m).** "Best value per page; the alias works because ORDER BY runs after SELECT."

```sql
SELECT title, price, pages,
       ROUND(pages / price, 1) AS pages_per_dollar
FROM books
ORDER BY pages_per_dollar DESC, title
LIMIT 5;
```

```
+---------------------+-------+-------+------------------+
| title               | price | pages | pages_per_dollar |
+---------------------+-------+-------+------------------+
| Lord of the Rings   | 15.99 |  1178 |             73.7 |
| Pride and Prejudice |  7.99 |   432 |             54.1 |
| 1984                |  9.99 |   328 |             32.8 |
| Brave New World     |  9.49 |   311 |             32.8 |
| The Hobbit          | 10.99 |   310 |             28.2 |
+---------------------+-------+-------+------------------+
```

5 **A custom order with CASE (2m).** "The Chicago team wants their customers first."

```sql
SELECT first_name, city
FROM customers
ORDER BY
    CASE city
        WHEN 'Chicago'  THEN 1
        WHEN 'New York' THEN 2
        ELSE 3
    END,
    first_name;
```

```
+------------+-------------+
| first_name | city        |
+------------+-------------+
| Bob        | Chicago     |
| Grace      | Chicago     |
| Alice      | New York    |
| Frank      | New York    |
| Carol      | Los Angeles |
| Dave       | Houston     |
| Eve        | NULL        |
| Hank       | NULL        |
+------------+-------------+
```

6 **Where NULLs go (2m).** "Ascending: NULL first." Then push them down.

```sql
SELECT first_name, city
FROM customers
ORDER BY city, first_name;

SELECT first_name, city
FROM customers
ORDER BY city IS NULL, city, first_name;
```

```
+------------+-------------+
| first_name | city        |
+------------+-------------+
| Eve        | NULL        |
| Hank       | NULL        |
| Bob        | Chicago     |
| Grace      | Chicago     |
| Dave       | Houston     |
| Carol      | Los Angeles |
| Alice      | New York    |
| Frank      | New York    |
+------------+-------------+
+------------+-------------+
| first_name | city        |
+------------+-------------+
| Bob        | Chicago     |
| Grace      | Chicago     |
| Dave       | Houston     |
| Carol      | Los Angeles |
| Alice      | New York    |
| Frank      | New York    |
| Eve        | NULL        |
| Hank       | NULL        |
+------------+-------------+
```

7 **Top N (1m).**

```sql
SELECT title, pages
FROM books
ORDER BY pages DESC
LIMIT 3;
```

```
+---------------------+-------+
| title               | pages |
+---------------------+-------+
| Lord of the Rings   |  1178 |
| Pride and Prejudice |   432 |
| Dune                |   412 |
+---------------------+-------+
```

8 **OFFSET (1m).** "Rows 4 to 6 by price."

```sql
SELECT title, price
FROM books
ORDER BY price, title
LIMIT 3 OFFSET 3;
```

```
+-----------------+-------+
| title           | price |
+-----------------+-------+
| Brave New World |  9.49 |
| 1984            |  9.99 |
| Fahrenheit 451  | 10.49 |
+-----------------+-------+
```

9 **A page and the page count (2m).** "Page 2 of the catalog, four per page, and how many pages."

```sql
SELECT title, price
FROM books
ORDER BY title
LIMIT 4 OFFSET 4;

SELECT COUNT(*)              AS total_books,
       CEILING(COUNT(*) / 4) AS pages_needed
FROM books;
```

```
+------------------------+-------+
| title                  | price |
+------------------------+-------+
| Lord of the Rings      | 15.99 |
| Pride and Prejudice    |  7.99 |
| The Catcher in the Rye |  8.99 |
| The Great Gatsby       |  8.99 |
+------------------------+-------+
+-------------+--------------+
| total_books | pages_needed |
+-------------+--------------+
|          10 |            3 |
+-------------+--------------+
```

10 **Keyset: the page after id 4 (1m).**

```
SELECT id, title
FROM books
WHERE id > 4
ORDER BY id
LIMIT 4;
```

```
+----+------------------------+
| id | title                  |
+----+------------------------+
|  5 | Pride and Prejudice    |
|  6 | The Hobbit             |
|  7 | Brave New World        |
|  8 | The Catcher in the Rye |
+----+------------------------+
```

11 **Three mistakes (3m).** "Clauses in the wrong order; a formula in LIMIT; a column name in quotes."

```
SELECT title, price
FROM books
ORDER BY price
WHERE price > 10;

SELECT title
FROM books
ORDER BY title
LIMIT 4 OFFSET (3 - 1) * 4;

SELECT title, price
FROM books
ORDER BY 'price' DESC
LIMIT 3;
```

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual tha
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual tha
+-----------------------+-------+
| title                 | price |
+-----------------------+-------+
| Dune                  | 14.99 |
| 1984                  |  9.99 |
| To Kill a Mockingbird | 11.99 |
+-----------------------+-------+
```

The third is silent: 'price' is text, the same for every row. Not the three most expensive.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- `ORDER BY a, b DESC` **expecting both descending.** Coach: "DESC belongs to the column before it."
- **A top N with ties at the cut.** Coach: "Could two rows have the same value here? Then add a unique column."
- **Clauses out of order or a formula in LIMIT (1064).** Coach with the clause order on the board; compute the offset first.

GUIDED LAB

File: `labs/module-04/l3_2-sorting-limiting-results/bestseller_board_starter.sql`; reference `labs/module-04/l3_2-sorting-limiting-results/bestseller_board_solution.sql`. Six lists for the shop's display board; each starts as ORDER BY id.

- **Lists 1 to 4 in pairs (10m), 5 and 6 for the quick (3m), debrief (2m).**
- **Expected:** Hank first; The Catcher in the Rye (22 characters) first, 1984 before Dune; authors A to Z, Tolkien's Lord of the Rings before The Hobbit; Fahrenheit 451, To Kill a Mockingbird, The Hobbit; Classic, Fiction, Dystopian, Fantasy, Science Fiction; Eve and Hank on top.
- **Circulate for:** list 4 without WHERE (out-of-stock Gatsby appears); list 6 with `city IS NULL` (NULLs at the bottom: which way does 1 sort?).
- **Fast finishers:** `labs/module-04/l3_2-sorting-limiting-results/paging_starter.sql`, then `labs/module-04/l3_2-sorting-limiting-results/debug_sorting.sql`.

START THE LESSON PROJECT

The lesson project is **The Storefront Queries**: `storefront.sql`. In the room: Steps 1 and 2 (10 minutes).

- Step 1: three newest in-stock books (add `, title` as the tie-breaker the lesson suggests). Step 2: the bargain shelf.
- **Goes home:** Steps 3 to 6 (catalog pages, page count, customer directory, review).

CHECKS FOR UNDERSTANDING

Ask these out loud:

- Where do NULLs go in an ascending sort? **Answer:** first.
- LIMIT 3 OFFSET 6 returns which rows? **Answer:** rows 7 to 9.
- Why does the alias work in ORDER BY but not in WHERE? **Answer:** WHERE runs before SELECT computes it; ORDER BY after.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** A) in_stock ascending (0 first), then price descending within each group; **Q8** `Dune` (412 pages: the third longest); **Q9** C) No error, but the rows come in no particular order: 'price' is a fixed piece of text; **Q10** D) Last; **Q11** B) `... WHERE id > 57 ORDER BY id LIMIT 20`; **Q12** `4`. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: the clause order as a card.

Stretch: `paging_starter.sql` and `debug_sorting.sql`; the lesson's Exercise 4 (Tolkien week) and the keyset Bonus Challenge.

CHECKPOINT 1

2:20 Checkpoint 1: Modules 1 and 2

30 min

TIMING

2m	Hand out `labs/module-04/checkpoint-1/checkpoint_starter.sql` and the workbook's Checkpoint 1 page; read the four parts aloud
24m	Learners work alone: Parts A to C in the file, Part D on the page; SOURCE after each part
4m	Hand in the file and the page; learners mark their own confidence (1 to 3) next to each part

WHAT IT CHECKS

Everything up to Lesson 2.3: a rerunnable build script with a named collation (2.1), types and constraints (2.2), INSERT, UPDATE and DELETE with a preview and an upsert (2.3), reading MySQL's results and error numbers (all of Module 1 and 2). The domain is new (a used-book trade-in desk), so nobody can copy a lesson project. Filtering and sorting (Module 3) are not scored; a learner who uses them is welcome to.

RUNNING IT

- **Open notes:** the workbook, their own scripts (`create_bookstore.sql`, `employees.sql`, `load_bookstore_data.sql`) and the lesson pages. Alone; no AI assistants.
- **Answer questions about the task, not about SQL.** "Is paid required?" (yes, default no) is fine; "How do I write the CHECK?" gets "look at Lesson 2.2's coupons table".
- **Stuck on Part A for more than 10 minutes?** Hand them the model answer's two CREATE TABLE statements (from the Answer Key) so Parts B and C can still be scored; record that Part A is 0.
- **Scoring:** the 5-row rubric in the Answer Key, 0 to 2 per row (10 points). 8 to 10: on track. 5 to 7: review the weakest row before Session 6. Under 5: a one-to-one before Lesson 4.1 (relationships build on all of it). Return the scores at the start of Session 5.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

After the checkpoint, keep it short: three questions on the slide, sticky notes on the way out.

1. Why does `WHERE city != 'Chicago'` skip Eve and Hank, and how do you include them? **Answer:** Their city is NULL, so the comparison is unknown, not true, and only true rows are kept. Add `OR city IS NULL`.

2. Write the WHERE clause for: books from $9 to $11, both ends included, that are not by Tolkien. **Answer:** `WHERE price BETWEEN 9 AND 11 AND author != 'J.R.R. Tolkien'` (on the course data: Brave New World, 1984, Fahrenheit 451).

3. Why should every ORDER BY that feeds a LIMIT end with a unique column? **Answer:** If rows tie at the cut-off, the server may return either one, and the choice can change between runs: page 1 and page 2 can repeat or skip a row. A unique column (id, title here) makes the order repeatable.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both projects:** saved_searches.sql (Steps 4 to 7) and storefront.sql (Steps 3 to 6).
- **Lesson exercises.** 3.1: Exercises 1 to 3; Exercise 4 is a stretch. 3.2: Exercises 1 to 3; Exercise 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages.
- **Journal:** both entries.
- **Optional:** `labs/module-04/l3_1-where-clauses-operators-filtering/null_detective_starter.sql`, `labs/module-04/l3_2-sorting-limiting-results/paging_starter.sql` and the two debug handouts.

BRING LIST FOR SESSION 5

- A laptop with the Module 3 data in bookstore (rerun `labs/_states/flat.sql` if in doubt), the root password.
- saved_searches.sql and storefront.sql: Lesson 3.3's project builds on SOURCE-ing script files.
- The course folder with `labs/` (Session 5's starters are in `labs/module-05/`), the workbook and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the checkpoint's 30 minutes, starting at 2:20; the data load; 3.1 demo Steps 4 and 11 to 14; 3.2 demo Steps 1, 2 and 11.
- **Cut first (saves 10m):** the 3.1 project start.
- **Cut second (saves 4m):** 3.2 demo Steps 4 and 5.
- **Cut third (saves 10m):** the 3.2 project start; then shorten the 3.2 lab to lists 1 to 4.

RUNNING SHORT: WHAT TO ADD

- Run `debug_filters.sql` together: which three bugs give no error?
- Lesson 3.2's keyset Bonus Challenge with the row-comparison form `(price, id) > (8.99, 8)`.
- Give back the checkpoint model answer immediately and let learners compare their Part D.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 5: Aggregates and Subqueries

How to teach the fifth session of MySQL Foundations: from rows to numbers. Aggregates and GROUP BY turn the books and customers into counts, totals and averages per group (with WHERE before the piles and HAVING after), then subqueries answer two-step questions in one statement: above average, the row behind the maximum, a list for IN, the NOT IN / NULL trap, correlated lookups, NOT EXISTS and derived tables.

`2 lessons` `3 hrs session` `Lessons 3.3-3.4` `Session 5 of 14 · Part 1`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (one mental model) → live demo (you type, they follow) → guided lab (a kit lab: new practice on the lesson's ideas, never the lesson project) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Type, do not paste. Every SQL block below was run on MySQL 9.7.2 in the flat bookstore of Module 3 (Lesson 3.1's setup script, `labs/_states/flat.sql`) to produce the output printed under it. Ask for a prediction before every statement that prints something; read every error number aloud.
- The lessons are the reference: keep `lesson_09.html` and `lesson_10.html` open for the diagrams (piles, the run order, inner and outer query). Labs live in `labs/module-05/`: `_starter.sql` files are handouts, `_solution.sql` files are yours.
- This is the session where SQL turns from "show me rows" into "tell me something about the rows". Two ideas carry it: **the piles** (WHERE before, HAVING after) and **inside out** (the inner query answers first). Draw both and leave them on the board all session.
- Part 1 (Sessions 1 to 7) can run on its own as a short course; nothing here depends on Part 2. Teal blocks are hands-on, amber blocks are watch-outs.

THE #1 RISK OF SESSION 5: CORRELATED SUBQUERIES EAT THE SECOND HALF

Lesson 3.4 is 100 minutes for a solo learner and the most abstract hour of Part 1. Learners who follow the scalar subquery and the IN list often stall at the correlated one, and a room can lose twenty minutes on why `author = author` is always true. **Protect** the scalar subquery, the NOT IN trap and the walk-through of one correlated row (Steps 2, 5 to 8): those are what Session 6 and 7 lean on (Lesson 4.1 uses correlated lookups before joins exist). ANY/ALL is deliberately not in the demo (it is in the lesson and the lab); keep the derived table to one example, and send the price review's Steps 3 to 6 home. If the room is still lost at 2:00, do the correlated step of the project (Step 2) together on the projector, inner query first, instead of the lab's TODO 2.

BEFORE THE SESSION

- ☐ **Checkpoint 1:** scored with the 5-row rubric in Session 4's Answer Key, one sentence of advice per learner; you return it in the warm-up. Learners under 5 points get a one-to-one before Lesson 4.1 (Session 6).
- ☐ **Data:** every machine has the flat bookstore of Module 3 (10 books with a text author column, 8 customers). A learner whose data drifted runs `SOURCE labs/_states/flat.sql` (it drops and rebuilds `bookstore`). Do NOT let anyone load `bookstore.sql` yet: that is the relational database Lesson 4.1 builds in Session 6, and every count today would change (11 books, an authors table).
- ☐ **Files to hand out:** `labs/module-05/` without the `*_solution.sql` files. Check the folder once: `python3 labs/check_labs.py module-05` (10 files, a few seconds, every line PASS).
- ☐ **Projector machine:** MySQL 9.7 with the flat bookstore loaded, a terminal in the course folder at 18 pt or larger, already in the `mysql>` client with `USE bookstore;`. Browser tabs: `lesson_09.html`, `lesson_10.html`.
- ☐ **Board, before learners arrive:** the run order as a column of seven boxes (FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT) with "rows in" beside WHERE and "piles out" beside HAVING; a second area headed "inside out" with `WHERE price > ( ___ )` and an arrow from a small box labeled "inner query" into the blank.
- ☐ **Print or share:** the workbook (warm-up, worked examples, predict-then-run, self-check), the two debug handouts for the error clinic, sticky notes for the exit ticket.

KNOWN TRAPS TODAY

SYMPTOM	CAUSE	FIX
Counts differ from the slides (11 books, `author` column missing)	The learner loaded `bookstore.sql` (the Module 4 database) early	`SOURCE labs/_states/flat.sql`, then rerun.

SYMPTOM	CAUSE	FIX
`ERROR 1055` on a query that "works on my friend's laptop"	The friend runs MariaDB (XAMPP), which has no ONLY_FULL_GROUP_BY	Fix the query; never `SET sql_mode = ''`. The lesson's Common Mistake 2 shows the MariaDB output.
Groups come out in a different order than the lesson	No ORDER BY (MySQL 8.0+ does not sort groups); or MariaDB, which does	Add ORDER BY. Every grouped query in a report ends with one.
`ERROR 1064 ... near ''` with a subquery in FROM	MariaDB's wording for a derived table without an alias (MySQL says 1248)	Add `AS name` after the closing parenthesis.
A correlated query returns the same number on every row	No aliases, so `author = author` compares the inner row with itself	Alias both copies and qualify every column: `b2.author = b.author`.
`Failed to open file` after SOURCE	The client was started in another folder	`exit`, `cd` into the course folder (the one with `labs/`), log in again.

SESSION 5 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 3.1 and 3.2 (AND before OR, IS NULL, LIMIT and OFFSET, a NULL inside NOT IN); homework check.
0:10	10	**3.3** Frame and mental model	From rows to one number: the cashier's end-of-day slip; piles, then one answer per pile.
0:20	25	**3.3** Live demo	COUNT three ways, SUM and AVG, MIN and MAX, GROUP BY, HAVING, then errors 1140, 1055 and 1111 on purpose.
0:45	15	**3.3** Guided lab	`labs/module-05/l3_3-aggregate-functions/customer_summary_starter.sql` in pairs: summarize the customers.
1:00	10	**3.3** Start the lesson project	The inventory report, Steps 1 and 2; Steps 3 to 6 go home.
1:10	10	Break	Leave the run-order diagram (FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT) on the board.
1:20	10	**3.4** Frame and mental model	Fill in the blank: the inner query works out the value, the outer query uses it; three shapes of result.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:30	30	**3.4** Live demo	Above average, the row behind MAX, IN with GROUP BY, the NOT IN trap, correlated, NOT EXISTS, a derived table, errors 1242 and 1248.
2:00	25	**3.4** Guided lab	`labs/module-05/l3_4-subqueries/page_checks_starter.sql`, then `city_neighbors_starter.sql` in pairs.
2:25	15	**3.4** Start the lesson project	The price review, Steps 1 and 2 (the correlated subquery built inner query first).
2:40	10	Error clinic	One debug handout per table: eight numbered errors, one fix each, said aloud.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 6.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 90 + 100 = 190 minutes for a solo learner; after the warm-up, the break, the error clinic and the wrap-up the room has 140. Both lessons keep their full shape (model, demo, lab, project start). Lesson 3.4 gets the larger share because correlated subqueries are where learners need a person in the room.

Goes home (about 2 to 3 hours). The inventory report Steps 3 to 6 and the price review Steps 3 to 6; each lesson's Hands-on Exercises; both Quick Quizzes; both journal entries; the labs not finished in class (`page_report_starter.sql`, `city_neighbors_starter.sql`).

Protect, in this order: the 3.4 scalar subquery and the NOT IN trap (Steps 2, 5 to 7); one correlated row walked through aloud (Step 8); the 3.3 WHERE-versus-HAVING pair (Steps 7 and 8); the 3.3 lab TODOs 1 and 3. **Cut first:** the error clinic (hand out both debug files as homework), then the 3.4 lab's second file, then 3.3 demo Step 4 (aggregates over no rows: it is in the workbook's self-check, Q3).

OPENING SEGMENT

0:00 Warm-up: Filtering, Sorting and Limiting — 10 min

TIMING

6m	Five retrieval questions on Lessons 3.1 and 3.2, answered in the workbook, then on the projector
4m	Checkpoint 1 scores returned (the 5-row rubric, one sentence of advice each); homework check: the storefront queries (3.2 project)

RETRIEVAL QUESTIONS AND ANSWERS

Put the five questions on the slide; learners answer in the workbook's Warm-up box without running anything, then you run the SQL ones. The real output is below each.

1. **Who comes back, and why?** (AND binds before OR.)

```
SELECT first_name, city, id
FROM customers
WHERE city = 'Chicago' OR city = 'Houston' AND id > 5
ORDER BY id;
```

```
+------------+---------+----+
| first_name | city    | id |
+------------+---------+----+
| Bob        | Chicago |  2 |
| Grace      | Chicago |  7 |
+------------+---------+----+
```

Answer: Bob and Grace. AND is evaluated first, so the condition reads `city = 'Chicago' OR (city = 'Houston' AND id > 5)`; Dave (Houston) has id 4. Parentheses say what you mean.

2. **Why does this find nobody, although two customers have no city?**

```
SELECT first_name FROM customers WHERE city = NULL;
```

```
Empty set
```

Answer: nothing is equal to NULL, not even NULL. Write `WHERE city IS NULL` (Eve and Hank).

3. **In which order must you write WHERE, ORDER BY and LIMIT? Answer:** WHERE, then ORDER BY, then LIMIT (LIMIT always last). Today adds GROUP BY and HAVING between WHERE and ORDER BY.

4 **Which three books does this show?**

```
SELECT title, price
FROM books
ORDER BY price DESC, title
LIMIT 3 OFFSET 2;
```

```
+-----------------------+-------+
| title                 | price |
+-----------------------+-------+
| To Kill a Mockingbird | 11.99 |
| The Hobbit            | 10.99 |
| Fahrenheit 451        | 10.49 |
+-----------------------+-------+
```

Answer: the 3rd to 5th most expensive: OFFSET 2 skips Lord of the Rings and Dune. The tie-breaker `title` makes the order repeatable.

5 **Predict: how many customers outside Chicago?**

```
SELECT first_name
FROM customers
WHERE city NOT IN ('Chicago', NULL);
```

```
Empty set
```

Answer: none. Lesson 3.1 warned: a NULL inside a NOT IN list makes every row unknown. Keep this one in mind: after the break, the same trap hides inside a subquery.

HOW TO READ THE WARM-UP

- **Q1 wrong** (Dave included, or Bob only): precedence is shaky. Today's HAVING conditions combine AND and OR (the lesson project's Step 4); say "parentheses" every time you see both.
- **Q2 wrong:** spend one minute on NULL now: it is the main character of both lessons today (COUNT(column), AVG skipping NULL, the NOT IN trap).
- **Q5 right** from more than half the room: say "you already know the trap; after the break you will meet it where it is harder to see".

3.3 Aggregate Functions & GROUP BY

60 min

OBJECTIVES (FROM THE LESSON PAGE)

- Summarize a table with COUNT, SUM, AVG, MIN and MAX, and predict how each one treats NULL
- Produce one summary row per group with GROUP BY, sorted with ORDER BY
- Keep or drop whole groups with HAVING, and explain when to use WHERE instead
- Read and fix the errors MySQL raises for broken aggregate queries (1140, 1055, 1111)
- Build a small inventory report as a reusable .sql script

TIMING IN THE ROOM

10m	Frame: the cashier's slip; rows into piles, one answer per pile; the run order
25m	Live demo: COUNT three ways, SUM, AVG, MIN, MAX, GROUP BY, HAVING, errors 1140, 1055, 1111
15m	Guided lab: `customer_summary_starter.sql` in pairs
10m	Start the lesson project: the inventory report, Steps 1 and 2

MENTAL MODEL TO SAY

Start with the slip: "At the end of the day the cashier does not read every receipt. She reads the totals at the bottom. That is an aggregate: many rows in, one value out. COUNT, SUM, AVG, MIN, MAX."

Then the piles, drawn on the board: "GROUP BY sorts the rows into piles that share a value, then runs the aggregate once per pile. One pile, one result row. Every column you show must be either the pile's label (the GROUP BY column) or a summary of the pile (an aggregate). Anything else, like one title from a pile of two, is a guess, and MySQL refuses to guess."

Then the run order, pointing at the seven boxes: "You WRITE SELECT first, but the server RUNS it fifth. WHERE runs before the piles exist, so it cannot count them. HAVING runs after, so it can."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

The lesson's own examples, typed live in the flat bookstore. Learners type along. Ask for a prediction before each statement and read every error number aloud.

1. **COUNT three ways (3m).** "A plain SELECT gives one result row per table row. An aggregate reads many rows and gives back ONE value." Ask first: eight customers, two without a city, New York and Chicago twice each: what are the three numbers?

```
SELECT
    COUNT(*)             AS customers,
    COUNT(city)          AS with_a_city,
    COUNT(DISTINCT city) AS cities
FROM customers;
```

```
+-----------+-------------+--------+
| customers | with_a_city | cities |
+-----------+-------------+--------+
|         8 |           6 |      4 |
+-----------+-------------+--------+
```

Write the rule on the board: **every aggregate skips NULL, except** `COUNT(*)`.

2 **SUM and AVG (2m).** "SUM adds, AVG divides by the number of values. Why six decimals?" Let them guess before you explain: price is DECIMAL(8,2), and MySQL keeps the average exact by adding four places.

```
SELECT
    SUM(price)           AS one_of_each,
    AVG(price)           AS avg_price,
    ROUND(AVG(price), 2) AS avg_rounded
FROM books;
```

```
+-------------+-----------+-------------+
| one_of_each | avg_price | avg_rounded |
+-------------+-----------+-------------+
|      109.90 | 10.990000 |       10.99 |
+-------------+-----------+-------------+
```

Habit for the rest of the course: `ROUND(AVG(...), 2)` whenever a person reads money.

3 **MIN tells the value, not the row (2m).** "MIN says the cheapest price is 7.99. Which book is that?" It cannot say. Sort and take one row, as in Lesson 3.2.

```
SELECT MIN(price) AS cheapest, MAX(price) AS priciest
FROM books;
SELECT title, price
FROM books
ORDER BY price
LIMIT 1;
```

```
+----------+----------+
| cheapest | priciest |
+----------+----------+
|     7.99 |    15.99 |
+----------+----------+
+----------------------+-------+
| title                | price |
+----------------------+-------+
| Pride and Prejudice |  7.99 |
+----------------------+-------+
```

Leave this open: "what if two books shared the lowest price? LIMIT 1 would hide one." Lesson 3.4 closes the gap after the break.

4 **Aggregates over no rows (2m).** "Nothing costs over $100. COUNT says 0. What does SUM say?"

```sql
SELECT
    COUNT(*)   AS books_over_100,
    SUM(price) AS their_total
FROM books
WHERE price > 100;
```

```
+----------------+-------------+
| books_over_100 | their_total |
+----------------+-------------+
|              0 |        NULL |
+----------------+-------------+
```

COUNT answers 0, the others NULL. A report that adds a NULL to something gets NULL: `COALESCE(SUM(price), 0)`.

5 **GROUP BY: one answer per pile (4m).** Draw ten books falling into piles by author, then COUNT on each pile. "Nine authors, so nine piles, so nine rows. I show four."

```sql
SELECT author, COUNT(*) AS book_count
FROM books
GROUP BY author
ORDER BY book_count DESC, author
LIMIT 4;
```

```
+---------------------+------------+
| author              | book_count |
+---------------------+------------+
| J.R.R. Tolkien      |          2 |
| Aldous Huxley       |          1 |
| F. Scott Fitzgerald |          1 |
| Frank Herbert       |          1 |
+---------------------+------------+
```

The ORDER BY is not decoration: on MySQL 8.0 and later groups come back in no promised order (the lesson's Common Mistake 5).

6 **Several aggregates per group (2m).** "One pile for in stock, one for out of stock, four numbers each."

```sql
SELECT
    in_stock,
    COUNT(*)             AS books,
    ROUND(AVG(price), 2) AS avg_price,
    MAX(pages)           AS longest
FROM books
GROUP BY in_stock
ORDER BY in_stock DESC;
```

```
+----------+-------+-----------+---------+
| in_stock | books | avg_price | longest |
+----------+-------+-----------+---------+
|        1 |     8 |     11.49 |    1178 |
|        0 |     2 |      8.99 |     234 |
+----------+-------+-----------+---------+
```

in_stock is a BOOLEAN stored as 1 and 0. The two out-of-stock books both cost 8.99.

7 **HAVING keeps whole piles (3m).** "WHERE picks rows going INTO the piles. HAVING picks piles coming OUT." Point at the run-order diagram on the board.

```sql
SELECT city, COUNT(*) AS customers
FROM customers
WHERE city IS NOT NULL
GROUP BY city
HAVING customers > 1
ORDER BY city;
```

```
+----------+-----------+
| city     | customers |
+----------+-----------+
| Chicago  |         2 |
| New York |         2 |
+----------+-----------+
```

8 **WHERE and HAVING together (3m).** Read each line's job aloud: drop out-of-stock books, pile by author, keep piles averaging over $10, most expensive first.

```
SELECT author, ROUND(AVG(price), 2) AS avg_price
FROM books
WHERE in_stock = TRUE
GROUP BY author
HAVING AVG(price) > 10
ORDER BY avg_price DESC;
```

```
+----------------+-----------+
| author         | avg_price |
+----------------+-----------+
| Frank Herbert  |     14.99 |
| J.R.R. Tolkien |     13.49 |
| Harper Lee     |     11.99 |
| Ray Bradbury   |     10.49 |
+----------------+-----------+
```

Rule of thumb on the board: a condition on one row's column goes in WHERE; a condition on a group's aggregate goes in HAVING.

9 **Error 1140 on purpose (1m).** "Ten titles squashed into one row. Which title goes in it?"

```
SELECT title, COUNT(*) FROM books;
```

```
ERROR 1140 (42000): In aggregated query without GROUP BY, expression #1 of SEI
```

10 **Error 1055 on purpose (2m).** "Tolkien's pile has two titles. Which one?" Read the message from the left: number, then 'not in GROUP BY clause'.

```
SELECT author, title, COUNT(*) AS books
FROM books
GROUP BY author;
```

```
ERROR 1055 (42000): Expression #2 of SELECT list is not in GROUP BY clause and
```

Fix: aggregate it (`MIN(title)`), group by it, or drop it. Never switch off ONLY_FULL_GROUP_BY to make it go away: MariaDB already does, and silently picks a title.

11 **Error 1111 on purpose (1m).** "WHERE runs before the piles exist. There is nothing to count yet."

```sql
SELECT author, COUNT(*) AS books
FROM books
WHERE COUNT(*) > 1
GROUP BY author;
```

```
ERROR 1111 (HY000): Invalid use of group function
```

Fix: `HAVING COUNT(*) > 1` after GROUP BY.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **A plain column next to an aggregate.** `SELECT title, COUNT(*)` (1140) or an extra column with GROUP BY (1055). Coach with the piles: "Point at the pile. Which ONE title belongs in its row?" Then the three fixes: aggregate it, group by it, drop it.
- **The aggregate in WHERE.** `WHERE COUNT(*) > 1` (1111). Coach with the run order: "When WHERE runs, do the piles exist yet?" Then move it to HAVING.
- **Trusting the group order, or forgetting ORDER BY.** The result looks sorted on one run and not on the next server. Coach: "Who promised you that order?" Every grouped query in the project ends with ORDER BY (the project's checklist says so).

GUIDED LAB

File: learners open `labs/module-05/l3_3-aggregate-functions/customer_summary_starter.sql`; your reference is `labs/module-05/l3_3-aggregate-functions/customer_summary_solution.sql`. The lesson summarized the books; this lab summarizes the customers: four TODOs, each a small change to a query that already runs.

- **SOURCE it as shipped (1m):** `SOURCE labs/module-05/l3_3-aggregate-functions/customer_summary_starter.sql`. They see 8 customers with zero placeholders, a TODO line, every known city, and the cities with a NULL group.
- **TODO 1 together (4m), TODOs 2 to 4 in pairs (8m).** TODO 1 replaces five placeholders with COUNT(city), `COUNT(*) - COUNT(city)`, COUNT(DISTINCT city), MIN and MAX of created_at. TODO 2 is written from scratch (GROUP BY a DATE_FORMAT label). TODO 3 adds HAVING. TODO 4 combines WHERE, COALESCE and an ORDER BY on the count.

- **Expected:** TODO 1: 8 / 6 / 2 / 4 cities, first signup `2026-01-04 10:15:00`, latest `2026-02-10 19:12:00`; TODO 2: `2026-01` 6, `2026-02` 2; TODO 3: New York and Chicago, 2 each; TODO 4: New York 2, then `(no city)`, Chicago, Houston, Los Angeles with 1 each.
- **Circulate for:** COALESCE in the SELECT list but not in GROUP BY (it still works on MySQL, since the alias is allowed in GROUP BY; ask them to say why), `HAVING customers > 1` written before GROUP BY (1064), and a missing comma between two aggregates.
- **Debrief (2m):** "In TODO 4, which line picks the rows, and which line makes the piles?" Then: "why is `(no city)` a pile at all?" (NULLs land together in one group).
- **Fast finishers:** `labs/module-05/l3_3-aggregate-functions/page_report_starter.sql` (length bands with CASE, pages per author, books per century, WHERE plus HAVING), then `labs/module-05/l3_3-aggregate-functions/debug_group_by.sql` (four broken summaries: 1140, 1055, 1111, 1064).

START THE LESSON PROJECT

The lesson project is **The Inventory Report**: `inventory_report.sql`, a five-part summary the owner reruns every Monday. In the room: Steps 1 and 2 (10 minutes).

- Step 1: create the file in the course folder with `USE bookstore;` and the headline query (titles, in stock, total pages, average price). The `SUM(in_stock)` trick deserves one sentence: true is 1, so adding them counts the true rows.
- Step 2: stock value by status with `IF(in_stock, 'In stock', 'Out of stock')` grouped by that label.
- **Done in the room** = the file runs with SOURCE and prints the 10 / 8 / 3824 / 10.99 headline and the two status rows.
- **Goes home:** Steps 3 to 6 (price bands, display-table authors, customers by city, running the whole file).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- `COUNT(*)` or COUNT(city): which one counts Eve? **Answer:** `COUNT(*)`: it counts rows; COUNT(city) skips her NULL.
- Where does `price > 10` go, and where does `AVG(price) > 10` go? **Answer:** WHERE (one row's column) and HAVING (a group's aggregate).
- Why does `SELECT author, title, COUNT(*) ... GROUP BY author` fail? **Answer:** Tolkien's pile holds two titles; MySQL will not pick one (1055).
- What does SUM return when no row matches? **Answer:** NULL (COUNT returns 0); COALESCE turns it into 0.

Workbook self-check, items Q1 to Q6:

Answers: **Q1** B) `2` and `1`; **Q2** C) In `HAVING`, after `GROUP BY`; **Q3** One row: `n` = 0 and `top` = NULL; **Q4** A) `ERROR 1055 (42000)`: first_name is not in GROUP BY and not aggregated; **Q5** D) No: since MySQL 8.0 the order of groups is not guaranteed; add ORDER BY; **Q6** Two rows: `0 | 2` and `1 | 3`. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: learners who freeze at an empty editor get the starter's TODO 3 first (only one line to add). Pair a learner who missed warm-up Q2 (NULL) with one who got it: today's TODO 1 is all about NULL.

Stretch: `page_report_starter.sql` and `debug_group_by.sql`; the lesson's Exercise 4 (books by decade) and the WITH ROLLUP Bonus Challenge (ask: why is the grand total's average not the average of the two group averages?).

3.4 Subqueries & Nested Queries

80 min

OBJECTIVES (FROM THE LESSON PAGE)

- Compare every row with an aggregate using a scalar subquery ("above average", "equal to the maximum")
- Filter with IN, NOT IN, ANY and ALL subqueries, and keep NULLs from emptying a NOT IN
- Write correlated subqueries and EXISTS / NOT EXISTS checks that compare a row with related rows
- Build derived tables in FROM and computed columns in SELECT
- Read and fix the subquery errors 1242, 1241, 1248 and 1235

TIMING IN THE ROOM

10m	Frame: fill in the blank; inside out; the three shapes of result (one value, a list, a table)
30m	Live demo: scalar, the MAX row, IN, the NOT IN trap, correlated, NOT EXISTS, a derived table, errors 1242 and 1248
25m	Guided lab: `page_checks_starter.sql`, then `city_neighbors_starter.sql`
15m	Start the lesson project: the price review, Steps 1 and 2

MENTAL MODEL TO SAY

Point at the blank on the board: "`WHERE price > ( ___ )`. Before the break you would have typed 10.99 there. A subquery is a whole SELECT in that blank: the inner query works out the value, the outer query uses it. Read nested queries from the inside out."

Then the shapes: "What the blank expects depends on where it is. After `=` or `>`: ONE value (a scalar). After IN: one column, any number of rows (a list). In FROM: a whole table, and it needs a name. Most subquery errors are a shape mismatch: 1242 is too many rows, 1241 too many columns, 1248 a table without a name."

Then correlation, slowly: "Every subquery so far could run on its own. A correlated one cannot: it mentions the OUTER row. Picture MySQL walking down the books one at a time and asking the inner question again for each one: what is the average of THIS book's author?"

LIVE DEMO (YOU TYPE, THEY FOLLOW)

The lesson's examples, typed live. Run every inner query on its own first (select it and run it), then the whole statement: it is the habit that makes nested queries readable.

1. **Two steps, by hand (2m).** "Which books cost more than average? First the average."

```
SELECT AVG(price) FROM books;
```

```
+------------+
| AVG(price) |
+------------+
|  10.990000 |
+------------+
```

Type `WHERE price > 10.99` once and show it works, then ask: "what happens to that 10.99 next week when a price changes?"

2. **Put the first query where the number was (3m).** "Parentheses, a whole SELECT inside. MySQL works out the inner one first, then uses its value."

```
SELECT title, price
FROM books
WHERE price > (SELECT AVG(price) FROM books)
ORDER BY price;
```

```
+-----------------------+-------+
| title                 | price |
+-----------------------+-------+
| To Kill a Mockingbird | 11.99 |
| Dune                  | 14.99 |
| Lord of the Rings     | 15.99 |
+-----------------------+-------+
```

Read it inside out, aloud. This is the whole idea of the lesson; everything after is a variation.

3. **The row behind the maximum (2m).** "Lesson 3.3 could not say which book costs the most. Now:"

```
SELECT title, author, price
FROM books
WHERE price = (SELECT MAX(price) FROM books);
```

```
+-------------------+----------------+-------+
| title             | author         | price |
+-------------------+----------------+-------+
| Lord of the Rings | J.R.R. Tolkien | 15.99 |
+-------------------+----------------+-------+
```

Ask: what would `ORDER BY price DESC LIMIT 1` do if two books tied? The subquery returns all of them.

4 **A list: IN with a GROUP BY inside (4m).** "One column, several rows: that is a list, and lists go with IN." Run the inner query alone first, highlighted.

```
SELECT first_name, city
FROM customers
WHERE city IN (
    SELECT city FROM customers
    GROUP BY city
    HAVING COUNT(*) > 1
)
ORDER BY city, first_name;
```

```
+------------+----------+
| first_name | city     |
+------------+----------+
| Bob        | Chicago  |
| Grace      | Chicago  |
| Alice      | New York |
| Frank      | New York |
+------------+----------+
```

The inner query also returns NULL (Eve and Hank form a group of two). In a plain IN that does no harm.

5 **NOT IN, and the NULL trap (5m).** "Which January customers live in a city where no February customer lives? First, the February cities."

```
SELECT city FROM customers WHERE created_at >= '2026-02-01';
```

```
+---------+
| city    |
+---------+
| Chicago |
| NULL    |
+---------+
```

6 **The trap springs.** "Predict how many rows." Then run it.

```
SELECT first_name, city
FROM customers
WHERE created_at < '2026-02-01'
  AND city NOT IN (
      SELECT city FROM customers
      WHERE created_at >= '2026-02-01'
  );
```

```
Empty set
```

No error, no rows. city != 'Chicago' AND city != NULL: the second half is unknown for every row. Write the rule: **every NOT IN subquery gets an IS NOT NULL**.

7 **The fix.** Add one line inside the subquery.

```
SELECT first_name, city
FROM customers
WHERE created_at < '2026-02-01'
  AND city NOT IN (
      SELECT city FROM customers
      WHERE created_at >= '2026-02-01'
        AND city IS NOT NULL
  )
ORDER BY first_name;
```

```
+------------+-------------+
| first_name | city        |
+------------+-------------+
| Alice      | New York    |
| Carol      | Los Angeles |
| Dave       | Houston     |
| Frank      | New York    |
+------------+-------------+
```

Eve is missing too: her own city is NULL, so NOT IN cannot decide about her. NOT EXISTS treats her differently (Step 9).

8 **Correlated: above their OWN author's average (5m).** "Now the inner query depends on the outer row. b is the book being tested, b2 the scan of the same table." Walk the Hobbit row, then the Lord of the Rings row, out loud.

```
SELECT b.title, b.author, b.price
FROM books AS b
WHERE b.price > (
    SELECT AVG(b2.price)
    FROM books AS b2
    WHERE b2.author = b.author
);
```

```
+------------------+----------------+-------+
| title            | author         | price |
+------------------+----------------+-------+
| Lord of the Rings | J.R.R. Tolkien | 15.99 |
+------------------+----------------+-------+
```

Every single-book author's average is that book's price, and a price is never above itself: only one row survives.

9 **NOT EXISTS (3m).** "Is there ANOTHER customer in my city? NOT EXISTS keeps the ones where there is not." SELECT 1 means: the columns do not matter.

```
SELECT c.first_name, c.city
FROM customers AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM customers AS other
    WHERE other.city = c.city
      AND other.id != c.id
)
ORDER BY c.first_name;
```

```
+------------+-------------+
| first_name | city        |
+------------+-------------+
| Carol      | Los Angeles |
| Dave       | Houston     |
| Eve        | NULL        |
| Hank       | NULL        |
+------------+-------------+
```

Eve and Hank appear: nobody can be found in an unknown city. No NULL trap.

10 **A derived table: a summary of a summary (3m).** "Count per author first, then average the counts. You cannot write AVG(COUNT(*)) in one level."

```
SELECT ROUND(AVG(book_count), 2) AS avg_books_per_author,
       MAX(book_count)           AS most_books
FROM (
    SELECT author, COUNT(*) AS book_count
    FROM books
    GROUP BY author
) AS author_counts;
```

```
+----------------------+------------+
| avg_books_per_author | most_books |
+----------------------+------------+
|                 1.11 |          2 |
+----------------------+------------+
```

11 **A whole-table number next to every row (1m).** "A subquery in the SELECT list: per-row and whole-table side by side."

```
SELECT
    title,
    price,
    ROUND(price - (SELECT AVG(price) FROM books), 2) AS diff_from_avg
FROM books
ORDER BY diff_from_avg DESC, title
LIMIT 4;
```

```
+-----------------------+-------+---------------+
| title                 | price | diff_from_avg |
+-----------------------+-------+---------------+
| Lord of the Rings     | 15.99 |          5.00 |
| Dune                  | 14.99 |          4.00 |
| To Kill a Mockingbird | 11.99 |          1.00 |
| The Hobbit            | 10.99 |          0.00 |
+-----------------------+-------+---------------+
```

12 **Error 1242 on purpose (1m).** "= needs ONE value. Two books are out of stock."

```
SELECT title, price
FROM books
WHERE price = (SELECT price FROM books WHERE in_stock = FALSE);
```

```
ERROR 1242 (21000): Subquery returns more than 1 row
```

Fix: IN, or MIN/MAX. This one works today and breaks the day a second row arrives.

13 **Error 1248 on purpose.** "A table in FROM must have a name."

```
SELECT AVG(book_count)
FROM (
    SELECT author, COUNT(*) AS book_count
    FROM books
    GROUP BY author
);
```

```
ERROR 1248 (42000): Every derived table must have its own alias
```

Fix: `AS author_counts` after the closing parenthesis.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **A shape mismatch.** `=` with a list (1242), two columns in an IN list (1241), a derived table without a name (1248). Coach: "Run the inner query alone. How many rows and columns came back? What does the blank expect?"
- **NOT IN with a NULL in the list (silent).** An empty result and no error. Coach: "Run the inner query alone. Do you see a NULL?" Then the rule: every NOT IN subquery gets `IS NOT NULL`, or use NOT EXISTS.
- **A correlated subquery without aliases (silent).** `WHERE author = author` returns the store average. Coach: "Inside the parentheses, which table does `author` mean?" Then alias both copies and qualify every column inside the subquery.

GUIDED LAB

Files: `labs/module-05/l3_4-subqueries/page_checks_starter.sql` (scalar, IN and ALL subqueries on dates and page counts), then `labs/module-05/l3_4-subqueries/city_neighbors_starter.sql` (correlated, a derived table, the NOT IN trap on new data). References: the matching `_solution.sql` files.

- **page_checks (10m, pairs):** each TODO replaces a hand-typed placeholder with a subquery, so the file ends with no typed numbers. Expected: Pride and Prejudice (1813-01-28); five quick reads from Fahrenheit 451 (158 pages) to 1984 (328); The Great Gatsby and The Catcher in the Rye; four books after every out-of-stock book, and `same_with_max` 4.
- **city_neighbors (15m, pairs):** TODO 1 and 2 are correlated (count the neighbors; the first signup per city), TODO 3 is a derived table, TODO 4 is the NOT IN trap as shipped: 4a prints nothing until the IS NOT NULL is added. Expected: neighbors 1, 1, 0, 0, 0, 1, 1, 0; first signups Alice, Bob, Carol, Dave (Eve and Hank never match: NULL = NULL is not true); `authors_over_300` 5, `longest_total` 1488; 4a and 4b both Carol and Dave.

- **Coach the build order:** inner query first, for ONE example value (`WHERE city = 'Chicago'`), check it, then replace the example with the outer column (`= c.city`). Say it every time someone is stuck.
- **Fast finishers:** `labs/module-05/l3_4-subqueries/debug_subqueries.sql`: four broken subqueries (1242, 1241, 1248, 1235), one per shape.
- **Debrief (2m):** "In 4a, why did nothing come back, with no error?" Someone should say "a NULL in the list". Then: "which version would you trust in a monthly report?"

START THE LESSON PROJECT

The lesson project is **The Price Review**: `price_review.sql`, five queries that recompute themselves after every price change. In the room: Steps 1 and 2 (15 minutes).

- Step 1 (5m): the books above the store average, with the average and the difference beside each (three subqueries, none correlated).
- Step 2 (10m): each author's entry-level book, the correlated subquery. Make everyone write the inner query first for `WHERE author = 'J.R.R. Tolkien'`, run it, then correlate it. This is the exact skill the journal prompt asks them to explain.
- **Done in the room** = Step 1 prints three books (5.00, 4.00, 1.00 above the 10.99 average) and Step 2 prints nine authors, The Hobbit for Tolkien.
- **Goes home:** Steps 3 to 6 (NOT EXISTS for fully-in-stock authors, the top price tier via a derived table, the headline numbers, running it inside a transaction).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- What shape does `WHERE price = ( ... )` expect? And `IN ( ... )`? **Answer:** one value; one column with any number of rows.
- Why did the NOT IN query return nothing? **Answer:** the list held a NULL, and `city != NULL` is unknown for every row.
- What makes a subquery correlated? **Answer:** it refers to a column of the outer query's current row (`b2.author = b.author`).
- Why does `FROM (SELECT ...)` need `AS name`? **Answer:** every table in FROM must have a name (1248).

Workbook self-check, items Q7 to Q12:

Answers: **Q7** B) `6`; **Q8** A) `ERROR 1242 (21000): Subquery returns more than 1 row`; **Q9** `0` (no customer passes the test); **Q10** C) `NOT EXISTS (SELECT 1 FROM customers AS other WHERE other.city = c.city AND other.id != c.id)`; **Q11** `2`; **Q12** D) Both author names refer to the inner table, so author = author is always true and the subquery returns the store average. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: for learners lost at the correlated step, the workbook's worked example builds one in two runs (one city by hand, then correlated). Let them do page_checks only and leave city_neighbors for home.

Stretch: `debug_subqueries.sql`; the lesson's Exercise 4 (cheaper than their author) and the Bonus Challenge (the second-highest price without LIMIT, then the third: watch the nesting grow, a preview of Lesson 5.2's DENSE_RANK).

2:40 Error Clinic: Eight Numbered Errors — 10 min

◆ HOW IT RUNS

Half the room runs `labs/module-05/l3_3-aggregate-functions/debug_group_by.sql`, the other half `labs/module-05/l3_4-subqueries/debug_subqueries.sql`. Each pair takes one ERROR line, writes the number, what MySQL could not decide, and the fix; then one sentence per error, aloud, in error-number order.

ERROR	WHAT MYSQL COULD NOT DECIDE	FIX
1140	which first name goes in the one row of a whole-table COUNT	aggregate everything, or GROUP BY
1055	which last name goes in each city's row	`MIN(last_name)`, group by it, or drop it
1111	how to count piles that do not exist yet (aggregate in WHERE)	`HAVING COUNT(*) > 1` after GROUP BY
1064	how to read HAVING before GROUP BY	GROUP BY, then HAVING
1242	which of two dates to compare with `=`	`IN` (or an aggregate)
1241	how to compare one author with a two-column row	select only the author; the SUM stays in HAVING
1248	what to call a table in FROM	`AS city_counts` after the parenthesis
1235	how to apply LIMIT inside an IN list	wrap the LIMIT query in a derived table

The fixed versions and their real outputs are in the Answer Key.

CLOSING

◆ EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Learners answer in the workbook's Exit Ticket box or on a sticky note. Sort the notes before Session 6: question 2's pile tells you whether to reteach NOT IN in the Session 6 warm-up; question 3's pile tells you who to sit with during the 4.1 migration.

1. Write one query condition that belongs in WHERE and one that belongs in HAVING, for the books table. **Answer:** for example `WHERE in_stock = TRUE` (one row's column) and `HAVING COUNT(*) > 1` or `HAVING AVG(price) > 10` (a group's aggregate).
2. A NOT IN subquery returns `('Chicago', NULL)`. What does the outer query return, and what are two fixes? **Answer:** no rows at all. Add `AND city IS NOT NULL` inside the subquery, or rewrite with NOT EXISTS.
3. In your own words: what makes a subquery correlated, and why does it need aliases? **Answer:** it uses a column of the outer query's current row, so it is re-evaluated per row; with the same table twice, aliases (b, b2) are the only way to say which copy each column belongs to.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects:** the inventory report (Steps 3 to 6) and the price review (Steps 3 to 6). Both are scripts: each must run start to finish with SOURCE and no error.
- **Lesson exercises.** 3.3: Exercises 1 to 3 (Exercise 4 is a stretch). 3.4: Exercises 1 to 3 (Exercise 4 is a stretch).
- **Both Quick Quizzes** on the lesson pages (answers in the Answer Key).
- **Journal:** both entries. The 3.4 prompt (explain the correlated subquery as the database walking through the books) is the best preparation for Session 6.
- **Labs not finished in class:** `page_report_starter.sql`, `city_neighbors_starter.sql`; the two debug handouts if the clinic was cut.
- **Before the Next Lesson (from 3.4):** keep the flat bookstore and Lesson 3.1's setup script: Lesson 4.1 starts from this same flat bookstore and turns it into related tables. Download `bookstore.sql` into the course folder too (do not run it yet).

BRING LIST FOR SESSION 6

- A laptop with the flat bookstore loaded (`SELECT COUNT(*) FROM books;` says 10 and books still has an `author` column). If not: `SOURCE labs/_states/flat.sql` before you arrive.
- `bookstore.sql` saved in the course folder (not run yet): Lesson 4.1 ends by comparing with it, and Lesson 4.2 starts by loading it.
- `inventory_report.sql` and `price_review.sql`, finished, and the course folder with `labs/`.
- The workbook and the journal; paper for an ER sketch.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** 3.4 demo Steps 2 and 5 to 8 (inside out, the NOT IN trap, one correlated row aloud); 3.3 demo Steps 5, 7 and 8 (piles, HAVING, WHERE plus HAVING); 3.3 lab TODOs 1 and 3.
- **Cut first (saves 10m):** the error clinic. Hand out both debug files as homework; their fixes are in the Answer Key.
- **Cut second (saves 10m):** the 3.4 lab's second file (`city_neighbors`) goes home; keep TODO 4a (the trap) as a 2-minute demo instead.
- **Cut third (saves 5m):** 3.4 demo Steps 10 and 11 (derived table, SELECT-list subquery): show the slides only.

RUNNING SHORT: WHAT TO ADD

- Run `debug_subqueries.sql` with the whole room and fix Bug D together: the derived-table workaround for LIMIT inside IN.
- The 3.3 Bonus Challenge live: `GROUP BY in_stock WITH ROLLUP`, then label the total row with `GROUPING()`.
- The 3.4 Bonus Challenge on the board: the second-highest price without LIMIT. Collect two different correct answers.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 6: Relationships and Foreign Keys

How to teach the sixth session of MySQL Foundations: from one flat table to a relational database. Learners watch an update anomaly happen, migrate the books' author text into an authors table with a checked, reversible sequence, build a junction table, then put the foreign keys to the test: 1452 and 1451, CASCADE, SET NULL and RESTRICT inside transactions, and keys added to data that already holds an orphan.

`2 lessons` `3 hrs session` `Lessons 4.1-4.2` `Session 6 of 14 · Part 1`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (one mental model) → live demo → guided lab (new practice, never the lesson project) → start the lesson's own project. Exercises, quiz and journal are homework.
- Type the short statements; paste the long INSERTs (the 25 links, the authors). Every SQL block below was run on MySQL 9.7.2: Lesson 4.1's demo from the flat bookstore, in one session, through the migration; Lesson 4.2's demo from a fresh `bookstore.sql`.
- **This session changes the database.** Lesson 4.1 turns the flat bookstore into the relational one, and from Lesson 4.2 on every lesson starts by loading `bookstore.sql` (8 tables: authors 10, books 11, categories 5, links 25, customers 8, orders 12, order items 18, employees 10). After the 4.1 project start, every machine loads it; nobody needs to finish the migration by hand to follow 4.2.
- The lessons are the reference: keep `lesson_11.html` (the ER diagrams) and `lesson_12.html` (the rules diagram) open. Labs live in `labs/module-06/`.
- Part 1 (Sessions 1 to 7) can run on its own; Session 7 is the Part 1 assessment. Tell the room today: next session ends with it, and today's tables are in it.

THE #1 RISK OF SESSION 6: HALF THE ROOM IN THE FLAT DATABASE, HALF IN THE RELATIONAL ONE

4.1 migrates live data. Learners who mistype a step (a missing comma in the ALTER, the UPDATE run twice, a DROP COLUMN before the check) end up between the two shapes, and from then on nothing on the projector matches their screen. **Prevent it:** in the 4.1 demo, ready learners type along in their flat bookstore, but say clearly that the outcome does not matter: at 1:05 EVERYONE loads `bookstore.sql` (`SOURCE bookstore.sql` from the course folder, or `labs/_states/bookstore.sql`), and the counts query proves it (10, 11, 5, 25, 8, 12, 18, 10). **Recover:** anyone confused before then reloads the flat state (`SOURCE labs/_states/flat.sql`) and watches. The 4.1 lab works on two practice tables of its own, so it cannot damage either shape.

BEFORE THE SESSION

- ☐ **Data:** learners arrive with the flat bookstore (Module 3) loaded and `bookstore.sql` saved in the course folder (Session 5's bring list). Have `labs/_states/flat.sql` and `labs/_states/bookstore.sql` on a USB stick for anyone without them.
- ☐ **Files to hand out:** `labs/module-06/` without the `*_solution.sql` files. Check: `python3 labs/check_labs.py module-06` (10 files, every line PASS).
- ☐ **Projector machine:** MySQL 9.7 with the FLAT bookstore loaded (for the 4.1 demo) and `bookstore.sql` ready for 1:05; terminal at 18 pt; browser tabs `lesson_11.html`, `lesson_12.html`. Keep the long INSERTs (authors, the 25 links) in a text file to paste.
- ☐ **Board, before learners arrive:** "Can one A have many Bs? Can one B have many As?" with the three answers (1:M key on the many side · 1:1 key + UNIQUE · M:M junction table); and the rules diagram: "Can the child exist without its parent? no → CASCADE · yes → must it keep naming the parent? yes → RESTRICT, no → SET NULL".
- ☐ **Print or share:** the workbook (warm-up, the club worked example, predict-then-run, self-check, the relationship map page), sticky notes for the exit ticket, paper for the crow's foot sketch.

KNOWN TRAPS TODAY

SYMPTOM	CAUSE	FIX
`ERROR 1054 Unknown column 'author'` in Lesson 3 queries	The learner is already in the relational bookstore (the author text column is gone)	Expected from 4.1 on; joins (4.3) bring the name back. For 3.x homework: `SOURCE labs/_states/flat.sql`.
`ERROR 1138 Invalid use of NULL value` in Step 4 of the migration	A book's author_id is still NULL (a typo in the authors INSERT)	Run the IS NULL check, fix the author name or the id, then the ALTER again.

SYMPTOM	CAUSE	FIX
`ERROR 1824 Failed to open the referenced table`	A child created before its parent, or a misspelled parent (`book`)	Create parents first; check the table name.
`ERROR 1452` with a table called `#sql-...`	Adding a key to a table that already holds an orphan	Find orphans with `NOT IN (SELECT id FROM parent)`, fix them, add the key again.
MariaDB: `ERROR 1005 ... errno: 150`	MariaDB's single message for a badly formed key (MySQL: 3780, 6125, 1830, 1824)	Walk the requirements: same type and sign, parent key exists and is unique, SET NULL needs a nullable column.
A delete "does nothing" after a test	A transaction left open (START TRANSACTION without ROLLBACK or COMMIT)	Type `ROLLBACK;` and look again. Every test in the labs ends with ROLLBACK.

SESSION 6 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 3.3 and 3.4 (COUNT and NULL, WHERE or HAVING, the row behind MAX, the NOT IN trap, 1055); homework check.
0:10	10	**4.1** Frame and mental model	Store each fact once and point to it; the anomalies of one big table; where the foreign key goes.
0:20	30	**4.1** Live demo	The update anomaly in the flat bookstore, then the migration live: authors, author_id, UPDATE ... JOIN, the NULL check, lock-in, a junction table.
0:50	15	**4.1** Guided lab	`labs/module-06/l4_1-table-relationships/publisher_split_starter.sql` in pairs: the same migration on new data, with a typo to catch.
1:05	10	**4.1** Start the lesson project	Load bookstore.sql, then read the project's design (orders, order_items, employees) together; the typing goes home.
1:15	10	Break	Everyone's bookstore is now the relational one: 8 tables (`SHOW TABLES;`).
1:25	10	**4.2** Frame and mental model	Referential integrity: no orphans; who is checked when (child insert 1452, parent delete 1451); three rules.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:35	25	**4.2** Live demo	1452 and 1451, CASCADE and SET NULL inside transactions, ON UPDATE, every rule from information_schema, adding a key to data with an orphan.
2:00	20	**4.2** Guided lab	`labs/module-06/l4_2-foreign-keys/delete_rules_starter.sql`: predict, then watch each rule act.
2:20	20	**4.2** Start the lesson project	Book reviews with two delete rules: Steps 1 to 4 (decide, create, refuse an orphan, test CASCADE).
2:40	10	Relationship map	Sketch the eight tables with crow's foot lines and label each key's delete rule; check against information_schema.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 7 (the Part 1 assessment).
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 105 + 100 = 205 minutes for a solo learner; the room has 140. Lesson 4.1's live migration is the heart of the session (it is the one time learners watch a table change shape under its data); the 4.1 project is mostly pasting CREATE TABLE and INSERT statements, so the room reads its design and the typing goes home. Lesson 4.2 keeps its full shape.

Goes home (about 2 to 3 hours). The 4.1 project (orders, order_items, employees; or simply compare with `bookstore.sql` after loading it) and the 4.2 project Steps 5 and 6; each lesson's Hands-on Exercises; both Quick Quizzes; both journal entries; the labs not finished in class (`navigate_links_starter.sql`, `orphan_audit_starter.sql`, the two debug handouts). **Plus preparation for the Part 1 assessment** (see the bring list).

Protect, in this order: the 4.1 anomaly and the four migration steps with the NULL check (demo Steps 2 to 8); everyone loading `bookstore.sql` at 1:05; the 4.2 pair 1452/1451 and the CASCADE and SET NULL tests (Steps 1 to 4); the 4.2 lab. **Cut first:** the relationship map (it becomes homework), then 4.1 demo Steps 11 and 12 (one-to-one: show the slide), then 4.2 demo Steps 8 to 10 (adding a key to dirty data: the `orphan_audit` lab does it).

OPENING SEGMENT

0:00 Warm-up: Aggregates and Subqueries — 10 min

TIMING

6m	Five retrieval questions on Lessons 3.3 and 3.4, answered in the workbook, then on the projector
4m	Homework check: the inventory report and the price review run end to end with SOURCE?

RETRIEVAL QUESTIONS AND ANSWERS

Questions on the slide; learners answer in the workbook's Warm-up box, then you run the SQL ones in the flat bookstore (still loaded from Session 5).

1. **Three numbers: predict them.** (Customers 1 to 5 signed up before 25 January; Eve has no city.)

```
SELECT COUNT(*), COUNT(city), COUNT(DISTINCT city)
FROM customers
WHERE created_at < '2026-01-25';
```

```
+----------+-------------+----------------------+
| COUNT(*) | COUNT(city) | COUNT(DISTINCT city) |
+----------+-------------+----------------------+
|        5 |           4 |                    4 |
+----------+-------------+----------------------+
```

   Answer: 5, 4, 4. `COUNT(*)` counts rows; COUNT(city) skips Eve's NULL; the four filled-in cities are all different.

2. `price > 10` **and** `COUNT(*) > 1` **: which goes in WHERE and which in HAVING? Answer:** `price > 10` in WHERE (one row's column), `COUNT(*) > 1` in HAVING (a group's aggregate).

3. **Which book is it?**

```
SELECT title
FROM books
WHERE pages = (SELECT MAX(pages) FROM books);
```

```
+-------------------+
| title             |
+-------------------+
| Lord of the Rings |
+-------------------+
```

Answer: Lord of the Rings: the row behind MAX(pages). If two books tied, both would come back.

4 **How many customers?** (Customer 7 is Grace in Chicago, 8 is Hank with no city.)

```sql
SELECT COUNT(*)
FROM customers
WHERE city NOT IN (SELECT city FROM customers WHERE id > 6);
```

```
+----------+
| COUNT(*) |
+----------+
|        0 |
+----------+
```

Answer: 0. The list is ('Chicago', NULL), and one NULL empties a NOT IN. Fix: `AND city IS NOT NULL` inside, or NOT EXISTS.

5 **What is wrong, and what is the error number?**

```sql
SELECT author, title, MAX(price)
FROM books
GROUP BY author;
```

```
ERROR 1055 (42000): Expression #2 of SELECT list is not in GROUP BY clause and
```

Answer: 1055: title is neither grouped nor aggregated (Tolkien's pile has two titles). Fix: drop title, or `MIN(title)`.

HOW TO READ THE WARM-UP

- **Q4 wrong for more than a third of the room:** the NOT IN trap comes back today (the lesson's orphan query uses `NOT IN (SELECT id FROM parent)`, safe only because a primary key is never NULL). Say so when you reach it.
- **Q1 or Q5 wrong:** pair these learners with someone confident for the 4.1 lab; the migration's IS NULL check leans on COUNT.

- **Projects not running end to end:** a 2-minute fix at the break, not now. Both scripts are good revision for the Part 1 assessment next session.

LESSON PLANS

4.1 Table Relationships

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain the update, insert and delete problems of a flat table, using the bookstore's own data
- Tell one-to-many, one-to-one and many-to-many relationships apart, and say where the foreign key goes in each
- Move a repeated text column into its own table with CREATE TABLE, ALTER TABLE and UPDATE ... JOIN
- Build a junction table with a composite primary key, with and without extra columns
- Read a crow's foot ER diagram of the finished bookstore database

TIMING IN THE ROOM

10m	Frame: one big sales table and its four anomalies; primary and foreign keys; where the key goes
30m	Live demo: the update anomaly, then the migration (parent, column, UPDATE ... JOIN, NULL check, lock-in), one-to-one, a junction table
15m	Guided lab: `publisher_split_starter.sql` in pairs
10m	Start the lesson project: load bookstore.sql, read the design of orders, order_items and employees

MENTAL MODEL TO SAY

Draw the one-big-table sales sheet from the lesson (Alice's name and email on every row she bought) and name the four anomalies as you point: "duplication; change her email and miss a row (update); delete Bob's only sale and Bob is gone (delete); a new customer who bought nothing has nowhere to go (insert)."

Then the relational idea: "Store each fact once, in the table it describes, and let other tables point at it by its id. The table with the id is the parent; the table holding the pointer is the child."

Then the one question that places every foreign key, written on the board: **"Can one A have many Bs? Can one B have many As?"** One yes: the key goes on the many side. Two yeses: a junction table. One-to-one is a one-to-many whose key is UNIQUE.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

The lesson's own migration, in the flat bookstore, in one session. Ready learners type along; the outcome on their machines does not matter, because everyone loads `bookstore.sql` at 1:05.

1. **The repeated text (2m).** "Lesson 3.3's report counted books per author. Look how the author is stored."

```
SELECT id, title, author
FROM books
WHERE author = 'J.R.R. Tolkien';
```

```
+----+-------------------+----------------+
| id | title             | author         |
+----+-------------------+----------------+
|  6 | The Hobbit        | J.R.R. Tolkien |
|  9 | Lord of the Rings | J.R.R. Tolkien |
+----+-------------------+----------------+
```

The same 14 characters typed twice.

2. **The update anomaly, on purpose (3m).** "A staff member adds spaces to the initials on one row. No rule stops them." Ask: how many Tolkiens does the database believe in now?

```
UPDATE books SET author = 'J. R. R. Tolkien' WHERE id = 9;
SELECT author, COUNT(*) AS books
FROM books
WHERE author LIKE '%Tolkien%'
GROUP BY author
ORDER BY author;
```

```
+------------------+-------+
| author           | books |
+------------------+-------+
| J. R. R. Tolkien |     1 |
| J.R.R. Tolkien   |     1 |
+------------------+-------+
```

No error, and every report that groups by author is quietly wrong.

3. **Put it back (1m).**

```
-- @status
UPDATE books SET author = 'J.R.R. Tolkien' WHERE id = 9;
```

```
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0
```

Then the relational idea on the board: **store each fact once, in the table it describes; point at it by id.**

4 **Step 1: the parent table (3m).** "Each author once, with an id. Asimov has no books yet: allowed on the 'one' side." Paste the INSERT (the point is the structure, not the typing).

```
CREATE TABLE authors (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    birth_year  INT
);
INSERT INTO authors (id, name, birth_year) VALUES
    (1, 'Frank Herbert', 1920), (2, 'George Orwell', 1903),
    (3, 'Harper Lee', 1926), (4, 'F. Scott Fitzgerald', 1896),
    (5, 'Jane Austen', 1775), (6, 'J.R.R. Tolkien', 1892),
    (7, 'Aldous Huxley', 1894), (8, 'J.D. Salinger', 1919),
    (9, 'Ray Bradbury', 1920), (10, 'Isaac Asimov', 1920);
SELECT COUNT(*) AS authors FROM authors;
```

```
+---------+
| authors |
+---------+
|      10 |
+---------+
```

5 **Step 2: the foreign key column (2m).** "Can one author have many books? Yes, so the key goes in books."

```
ALTER TABLE books ADD COLUMN author_id INT AFTER title;
SELECT id, title, author, author_id
FROM books
WHERE id <= 3;
```

```
+----+-----------------------+---------------+-----------+
| id | title                 | author        | author_id |
+----+-----------------------+---------------+-----------+
|  1 | Dune                  | Frank Herbert |      NULL |
|  2 | 1984                  | George Orwell |      NULL |
|  3 | To Kill a Mockingbird | Harper Lee    |      NULL |
+----+-----------------------+---------------+-----------+
```

Empty (NULL) on every row: nothing is filled in yet.

6 **Step 3: fill it by matching names (4m).** "For every pair of a book and an author whose name equals the book's author text, copy the id." JOIN ... ON pairs rows of two tables; Lesson 4.3 is all about it, today only this one pattern.

```
UPDATE books
JOIN authors ON authors.name = books.author
SET books.author_id = authors.id;
SELECT id, title, author, author_id
FROM books
WHERE id IN (6, 9, 10)
ORDER BY id;
```

```
+----+------------------+----------------+-----------+
| id | title            | author         | author_id |
+----+------------------+----------------+-----------+
|  6 | The Hobbit       | J.R.R. Tolkien |         6 |
|  9 | Lord of the Rings | J.R.R. Tolkien |         6 |
| 10 | Fahrenheit 451   | Ray Bradbury   |         9 |
+----+------------------+----------------+-----------+
```

Both Tolkien books carry 6. (The lesson shows all ten rows; three are enough on the projector.)

7 **The check before anything is dropped (2m).** "A misspelled name would have matched nothing and left a NULL. Count them BEFORE you drop the text column: DROP COLUMN has no undo."

```
SELECT COUNT(*) AS books_without_author_id
FROM books
WHERE author_id IS NULL;
```

```
+-------------------------+
| books_without_author_id |
+-------------------------+
|                       0 |
+-------------------------+
```

8 **Step 4: lock it in (4m).** "Three changes in one ALTER: required, the old column gone, the key declared and named." Point at MUL in the Key column: MySQL indexed the foreign key.

```
ALTER TABLE books
    MODIFY COLUMN author_id INT NOT NULL,
    DROP COLUMN author,
    ADD CONSTRAINT fk_books_author
        FOREIGN KEY (author_id) REFERENCES authors(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE;
DESCRIBE books;
```

```
+-----------+--------------+------+-----+---------+----------------+
| Field     | Type         | Null | Key | Default | Extra          |
+-----------+--------------+------+-----+---------+----------------+
| id        | int          | NO   | PRI | NULL    | auto_increment |
| title     | varchar(200) | NO   |     | NULL    |                |
| author_id | int          | NO   | MUL | NULL    |                |
| price     | decimal(8,2) | NO   |     | NULL    |                |
| pages     | int          | YES  |     | NULL    |                |
| published | date         | YES  |     | NULL    |                |
| in_stock  | tinyint(1)   | YES  |     | 1       |                |
+-----------+--------------+------+-----+---------+----------------+
```

9 **Use the relationship (2m).** "A book's author is one lookup away: a correlated subquery from Lesson 3.4."

```sql
SELECT
    title,
    (SELECT name FROM authors WHERE authors.id = books.author_id) AS author
FROM books
WHERE author_id IN (2, 6)
ORDER BY title;
```

```
+-------------------+----------------+
| title             | author         |
+-------------------+----------------+
| 1984              | George Orwell  |
| Lord of the Rings | J.R.R. Tolkien |
| The Hobbit        | J.R.R. Tolkien |
+-------------------+----------------+
```

10 **Add a book the relational way (2m).** "Animal Farm: Orwell's second book, by id. His name can never be split again."

```sql
INSERT INTO books (id, title, author_id, price, pages, published, in_stock) VA
    (11, 'Animal Farm', 2, 8.49, 112, '1945-08-17', TRUE);
SELECT author_id, COUNT(*) AS books
FROM books
GROUP BY author_id
HAVING COUNT(*) > 1
ORDER BY author_id;
```

```
+-----------+-------+
| author_id | books |
+-----------+-------+
|         2 |     2 |
|         6 |     2 |
+-----------+-------+
```

11 **One-to-one: a UNIQUE foreign key (2m).** "A profile per customer, at most one. The UNIQUE on customer_id is the 'one'." The second INSERT is refused.

```
CREATE TABLE customer_profiles (
    id             INT AUTO_INCREMENT PRIMARY KEY,
    customer_id    INT NOT NULL UNIQUE,
    favorite_genre VARCHAR(50),
    CONSTRAINT fk_profiles_customer
        FOREIGN KEY (customer_id) REFERENCES customers(id)
);
INSERT INTO customer_profiles (customer_id, favorite_genre) VALUES (1, 'Scienc
INSERT INTO customer_profiles (customer_id, favorite_genre) VALUES (1, 'Fantas
```

```
ERROR 1062 (23000): Duplicate entry '1' for key 'customer_profiles.customer_ic
```

Without UNIQUE it would succeed, and the relationship would silently become one-to-many. Drop the table: the course database has no profiles.

12 **Clean up the profile table.**

```
DROP TABLE customer_profiles;
```

13 **Many-to-many: a junction table (3m).** "1984 is Fiction, Classic and Dystopian. One column cannot hold that; a third table can: one row per link, a composite primary key." Paste the 25 links.

```
CREATE TABLE book_categories (
    book_id     INT NOT NULL,
    category_id INT NOT NULL,
    PRIMARY KEY (book_id, category_id),
    CONSTRAINT fk_bc_book
        FOREIGN KEY (book_id) REFERENCES books(id) ON DELETE CASCADE,
    CONSTRAINT fk_bc_category
        FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE CASCADE
);
INSERT INTO book_categories (book_id, category_id) VALUES
    (1, 1), (1, 2), (2, 1), (2, 3), (2, 5), (3, 1), (3, 3), (4, 1), (4, 3),
    (5, 1), (5, 3), (6, 1), (6, 4), (7, 1), (7, 2), (7, 5), (8, 1), (8, 3),
    (9, 1), (9, 4), (10, 1), (10, 2), (10, 5), (11, 1), (11, 3);
SELECT COUNT(*) AS links FROM book_categories;
```

```
+-------+
| links |
+-------+
|    25 |
+-------+
```

14 **Follow the links (1m).** "Which categories is 1984 in?"

```
SELECT name
FROM categories
WHERE id IN (SELECT category_id FROM book_categories WHERE book_id = 2)
ORDER BY name;
```

```
+-----------+
| name      |
+-----------+
| Classic   |
| Dystopian |
| Fiction   |
+-----------+
```

15 **The same link twice (1m).** "1984 is already Dystopian."

```
INSERT INTO book_categories (book_id, category_id) VALUES (2, 5);
```

```
ERROR 1062 (23000): Duplicate entry '2-5' for key 'book_categories.PRIMARY'
```

The composite key doing its job: each pair once. Every learner's database now matches the lesson's catalog half; the project adds orders, order_items and employees, which is why everyone loads bookstore.sql next.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **The key on the wrong side.** `customers.order_id` "so each customer points at their order". Coach with the question: "Can one customer have many orders? Then where can all those pointers live?"
- **Dropping before checking.** DROP COLUMN, then discovering a NULL author_id (and the 1138 at NOT NULL). Coach the order: add, fill, COUNT the NULLs, then lock in. The lab has a typo planted to make the check matter.
- **A list in one column.** `categories = 'Fiction,Fantasy'`. Coach with the lesson's Mistake 4: "`= 'Fiction'` finds nothing; `LIKE '%Fiction%'` finds Science Fiction too." Many values = many rows in a junction table.

GUIDED LAB

File: `labs/module-06/l4_1-table-relationships/publisher_split_starter.sql`; reference `labs/module-06/l4_1-table-relationships/publisher_split_solution.sql`. The same migration on new data: six (fictional) editions that type each publisher's name and city into every row. It works on two practice tables (`kit_editions`, `kit_publishers`) and drops them at the end, so it is safe in either shape of the bookstore.

- **As shipped (1m):** Step 1 prints the anomalies: Harbor Lane Press in Boston AND Cambridge, and North Star split in two by a typo (`North Star Book`).
- **Steps 2 to 6 in pairs (12m):** each step is turned off with `--`; learners turn on one at a time and SOURCE again. Step 3 has one gap (`kit_publishers.id`), Step 4 one gap (row 5).
- **Expected:** Step 4 finds exactly one unmatched row (5, Fahrenheit 451, `North Star Book`) and 0 after the fix; Step 6 shows each publisher with 2 editions. The planted typo is the point: without the check, Step 5's NOT NULL fails with 1138 (or worse, on a real migration, a book loses its publisher).
- **Circulate for:** a step turned on with its comment lines too (harmless) or with the `--` left on one line of a multi-line statement (1064 near the next line); Step 5 run before the fix (1138: "good, now you know why the check exists").
- **Debrief (2m):** "Which city did you give Harbor Lane Press, and how did you decide?" A migration forces a decision about which copy was right; that is the update anomaly's cost.
- **Fast finishers:** `labs/module-06/l4_1-table-relationships/navigate_links_starter.sql` (needs the relational bookstore: after 1:05), then `labs/module-06/l4_1-table-relationships/debug_relationships.sql` (1824, 1062, 1138).

START THE LESSON PROJECT

The lesson project is **Orders, Order Items and Staff**: three more tables that finish the bookstore every later lesson uses. Its value is the design; its typing is long INSERTs. In the room (10 minutes):

- **Everyone loads the finished database:** `SOURCE bookstore.sql` (from the course folder; or `SOURCE labs/_states/bookstore.sql`). It drops and rebuilds `bookstore`, so the flat shape is gone for good.
- **Run Step 5's counts query together** (the lesson's eight-subquery SELECT plus SHOW TABLES): 10, 11, 5, 25, 8, 12, 18, 10 and eight tables on every machine.
- **Read the design aloud** with `SHOW CREATE TABLE orders\G`: no total column on purpose (Lesson 6.3), order_items as a junction table with extra facts (quantity, price_at_time: history, not duplication), employees pointing at itself.
- **Goes home:** the project's Steps 1 to 4 typed into a practice copy if they want the practice (the lesson's "I made a mistake" FAQ explains how to start over), Step 6 (compare their CREATE TABLEs with `bookstore.sql`), and the sketch from the lesson's "Before the Next Lesson".

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Where does the key go for authors and books, and why? **Answer:** books.author_id: one author, many books.
- Why count the NULL author_ids before DROP COLUMN? **Answer:** a NULL means a name did not match; dropping the text would lose that book's author for good.
- What turns a one-to-many into a one-to-one? **Answer:** UNIQUE on the foreign key column.
- Why does order_items store price_at_time when books has a price? **Answer:** it records what the customer paid then; the book's price may change later.

Workbook self-check, items Q1 to Q6:

Answers: **Q1** B) `addresses.customer_id`; **Q2** C) Students and the courses they take; **Q3** Three rows: books `2`, `7` and `10`, each with `3`; **Q4** A) An update anomaly: one fact stored twice now disagrees with itself; **Q5** D) One-to-many: a customer can now have several profiles; **Q6** `ERROR 1824 (HY000): Failed to open the referenced table 'book'`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners overwhelmed by the live ALTERs watch and do the lab only (it is the same four steps on six rows); the workbook's club example is a complete, small many-to-many to reread at home.

Stretch: `navigate_links_starter.sql` and `debug_relationships.sql`; the lesson's Exercise 4 (a music library schema) and the Bonus Challenge (build the authors list with INSERT ... SELECT: do the ids match?).

4.2 Foreign Keys & Referential Integrity

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain referential integrity and predict when MySQL raises errors 1452 and 1451
- Choose RESTRICT, CASCADE or SET NULL for ON DELETE and ON UPDATE, and justify the choice
- Add, drop and change a named foreign key on an existing table, cleaning up orphan rows first
- List a database's foreign keys and their rules from information_schema
- Diagnose the errors raised by a badly formed key (3780, 6125, 1830)

TIMING IN THE ROOM

10m	Frame: referential integrity; four moments MySQL checks; three rules and the two questions
25m	Live demo: 1452, 1451, CASCADE and SET NULL in transactions, ON UPDATE, the rules from information_schema, a key on dirty data
20m	Guided lab: `delete_rules_starter.sql`
20m	Start the lesson project: book reviews, Steps 1 to 4

MENTAL MODEL TO SAY

"Referential integrity means every foreign key value points at a row that really exists. A child without its parent is an orphan, and a declared key makes orphans impossible." Then the four moments, as a two-by-two on the board: **child INSERT or UPDATE of the key** → does the parent exist? (1452) · **parent DELETE or key UPDATE** → do children point at it? then apply the rule (1451 under RESTRICT).

Then the rules, with the two questions: "Can the child exist without its parent? No: CASCADE (an order line without its order). Yes, and it must keep naming the parent: RESTRICT (an order must keep its customer: deactivate the customer instead). Yes, and the link is optional: SET NULL (staff stay when a manager leaves)." And the safety net: **when in doubt, RESTRICT**: a refused delete costs one step, a wrong cascade costs data.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

From a freshly loaded `bookstore.sql`. Every destructive test runs inside START TRANSACTION ... ROLLBACK; say so every time, and end every block with ROLLBACK before anything else.

1. **An order for nobody (2m).** "Customer 999 does not exist." Read the message from the END: the constraint, the table, the column, the parent.

```
INSERT INTO orders (customer_id, order_date) VALUES (999, '2026-06-15');
```

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constrain
```

"add or update a CHILD row": 1452 is always about the child.

2. **Delete a customer with orders (2m).** "Alice has three orders, and fk_orders_customer says RESTRICT."

```
DELETE FROM customers WHERE id = 1;
```

```
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constr
```

"delete or update a PARENT row": 1451. Alice and her history stay; RESTRICT makes you decide.

3. **CASCADE, safely (4m).** "Order lines mean nothing without their order. I wrap the test in a transaction: START TRANSACTION begins, ROLLBACK undoes everything (Lesson 7.2)."

```
START TRANSACTION;
SELECT id, order_id, book_id FROM order_items WHERE order_id = 1;
DELETE FROM orders WHERE id = 1;
SELECT id, order_id, book_id FROM order_items WHERE order_id = 1;
ROLLBACK;
```

```
+----+----------+---------+
| id | order_id | book_id |
+----+----------+---------+
|  1 |        1 |       1 |
|  2 |        1 |       2 |
+----+----------+---------+
Empty set
```

One DELETE on orders removed two rows of order_items. After ROLLBACK they are back.

4. **SET NULL (3m).** "Priya manages Tom, Lena and Ben. Priya leaves."

```
START TRANSACTION;
DELETE FROM employees WHERE id = 3;
SELECT id, first_name, manager_id
FROM employees
WHERE id IN (4, 5, 10)
ORDER BY id;
ROLLBACK;
```

```
+----+------------+------------+
| id | first_name | manager_id |
+----+------------+------------+
|  4 | Tom        |       NULL |
|  5 | Lena       |       NULL |
| 10 | Ben        |       NULL |
+----+------------+------------+
```

The reports stay, with no manager. The column must allow NULL (error 1830 otherwise; Step 11).

5 **ON UPDATE CASCADE (2m).** "Renumber Orwell from 2 to 20: his books follow."

```
START TRANSACTION;
UPDATE authors SET id = 20 WHERE id = 2;
SELECT id, title, author_id
FROM books
WHERE author_id = 20
ORDER BY id;
ROLLBACK;
```

```
+----+-------------+-----------+
| id | title       | author_id |
+----+-------------+-----------+
|  2 | 1984        |        20 |
| 11 | Animal Farm |        20 |
+----+-------------+-----------+
```

6 **No ON UPDATE rule = NO ACTION (1m).** "The employees self-reference has no ON UPDATE."

```
UPDATE employees SET id = 100 WHERE id = 1;
```

```
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constr
```

NO ACTION behaves exactly like RESTRICT in MySQL.

7 **Every rule in the bookstore (3m).** "information_schema describes all the other databases. Read each row as a decision." Walk the two-question diagram: can the child exist without its parent? Must it keep naming it?

```
SELECT TABLE_NAME, CONSTRAINT_NAME, REFERENCED_TABLE_NAME,
       DELETE_RULE, UPDATE_RULE
FROM information_schema.REFERENTIAL_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = 'bookstore'
ORDER BY TABLE_NAME, CONSTRAINT_NAME;
```

```
+-----------------+----------------------+-----------------------+-----------
| TABLE_NAME      | CONSTRAINT_NAME      | REFERENCED_TABLE_NAME | DELETE_RULE
+-----------------+----------------------+-----------------------+-----------
| book_categories | fk_bc_book           | books                 | CASCADE
| book_categories | fk_bc_category       | categories            | CASCADE
| books           | fk_books_author      | authors               | RESTRICT
| employees       | fk_employees_manager | employees             | SET NULL
| order_items     | fk_items_book        | books                 | RESTRICT
| order_items     | fk_items_order       | orders                | CASCADE
| orders          | fk_orders_customer   | customers             | RESTRICT
+-----------------+----------------------+-----------------------+-----------
```

8 **A table that started without a key (2m).** "Most real tables get their keys late. Kim's department 7 does not exist."

```
CREATE TABLE demo_departments (
    id   INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL
);
CREATE TABLE demo_staff (
    id            INT PRIMARY KEY,
    name          VARCHAR(50) NOT NULL,
    department_id INT
);
INSERT INTO demo_departments (id, name) VALUES (1, 'Front desk'), (2, 'Warehou
INSERT INTO demo_staff (id, name, department_id) VALUES
    (1, 'Ana', 1), (2, 'Raj', 2), (3, 'Kim', 7);
```

9 **Adding the key fails (2m).** "MySQL copies the table to add the key and finds the orphan." The #sql name differs on every server.

```
ALTER TABLE demo_staff
    ADD CONSTRAINT fk_staff_department
        FOREIGN KEY (department_id) REFERENCES demo_departments(id)
        ON DELETE SET NULL;
```

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint
```

10 **Find the orphan, fix it, add the key (3m).** "Find with a subquery, fix with the truth (Kim is not assigned yet: NULL), then the key goes on."

```
SELECT id, name, department_id
FROM demo_staff
WHERE department_id NOT IN (SELECT id FROM demo_departments);
UPDATE demo_staff SET department_id = NULL WHERE id = 3;
ALTER TABLE demo_staff
    ADD CONSTRAINT fk_staff_department
        FOREIGN KEY (department_id) REFERENCES demo_departments(id)
        ON DELETE SET NULL;
```

```
+----+------+---------------+
| id | name | department_id |
+----+------+---------------+
|  3 | Kim  |             7 |
+----+------+---------------+
```

Choose the fix that tells the truth about the data; deleting Kim would be the most drastic.

11 **A badly formed key: 1830 (1m).** "SET NULL on a column that forbids NULL."

```
CREATE TABLE demo_notes (
    id          INT PRIMARY KEY,
    customer_id INT NOT NULL,
    CONSTRAINT fk_notes_customer FOREIGN KEY (customer_id)
        REFERENCES customers(id) ON DELETE SET NULL
);
```

```
ERROR 1830 (HY000): Column 'customer_id' cannot be NOT NULL: needed in a fore
```

12 **Clean up.** Children first.

```
DROP TABLE demo_staff;
DROP TABLE demo_departments;
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Reading 1451 and 1452 the wrong way round.** Coach: "Read the first words: 'add or update a CHILD

row' (1452: the parent is missing) or 'delete or update a PARENT row' (1451: children still point at it)."

- **Cascading everything.** "CASCADE makes the error go away." Coach with the lesson's warning: if orders cascaded from customers, one DELETE would erase years of sales without a warning. Ask the two questions out loud for that key.
- **Switching the checks off.** `SET FOREIGN_KEY_CHECKS = 0` to get past an error. Coach: "Turning it back on does not re-check. The error was telling you about bad data." Never in class; the lesson's Mistake 5 shows the orphan that slips in.

GUIDED LAB

File: `labs/module-06/l4_2-foreign-keys/delete_rules_starter.sql`; reference `labs/module-06/l4_2-foreign-keys/delete_rules_solution.sql`. Five tests of the bookstore's rules on rows the lesson did not use; each in a transaction, each ending in ROLLBACK. As shipped, the changing line is turned off, so learners see the "before" picture, predict in the workbook, then turn the line on.

- **Predict first (3m):** for each test, which rule fires and what the SELECT will show. Learners look rules up with the demo's information_schema query.
- **Run them (12m), one TODO at a time.** Expected: links 25 → 22 (Dystopian's three links cascade); Sofia's manager_id NULL (SET NULL); 7 customers left (Hank has no orders: RESTRICT has nothing to protect); orders 6 and 11 now say customer 70 (ON UPDATE CASCADE); order lines 18 → 16 (order 9 had two). The last row is unchanged after every ROLLBACK: 5, 25, 8, 12, 18 and Sofia's manager 6.
- **Circulate for:** a DELETE run at the prompt outside a transaction (if it happened: `SOURCE labs/_states/bookstore.sql`), and a forgotten ROLLBACK when someone experiments by hand.
- **Debrief (3m):** the file's bonus question: "why is 'delete Alice' not in this file?" (1451: her orders RESTRICT it; an error would stop a lab file).
- **Fast finishers:** `labs/module-06/l4_2-foreign-keys/orphan_audit_starter.sql` (find two orphan gift cards, fix them, add the key with SET NULL, prove it), then `labs/module-06/l4_2-foreign-keys/debug_fk_errors.sql` (1452, 1451, 3780, 1830, 6125).

START THE LESSON PROJECT

The lesson project is **Book Reviews with the Right Rules**: one table, two foreign keys, two different delete rules, each proved with a test that is undone. In the room: Steps 1 to 4 (20 minutes).

- Step 1 (5m): decide the rules out loud with the two questions. Review → book: CASCADE (a review of nothing is meaningless). Review → customer: SET NULL (keep the review, "by a former customer"), so customer_id must allow NULL.
- Step 2 (5m): create the table and insert four reviews (paste).
- Step 3 (3m): the insert for book 42 is refused with 1452.

- Step 4 (7m): delete The Great Gatsby (4) inside a transaction; its two reviews go with it; ROLLBACK.
- **Done in the room** = the four reviews exist, the 1452 was seen, the CASCADE test was run and rolled back.
- **Goes home:** Step 5 (the SET NULL test with Carol) and Step 6 (list the rules from information_schema, then drop the table: the course database has no reviews).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- 1452 or 1451: which one is about a missing parent? **Answer:** 1452 ("add or update a child row").
- Which rule does order_items → orders use, and why? **Answer:** CASCADE: a line has no meaning without its order.
- What happens when you leave ON DELETE out? **Answer:** NO ACTION, which MySQL enforces exactly like RESTRICT.
- Why must a SET NULL column allow NULL? **Answer:** the rule writes NULL into it; NOT NULL makes the key impossible (1830).

Workbook self-check, items Q7 to Q12:

Answers: **Q7** B) `ERROR 1452 (23000): Cannot add or update a child row`; **Q8** A) `16`; **Q9** `NULL` (Nina stays, with no manager); **Q10** C) go with it: ON DELETE CASCADE; **Q11** D) `DROP FOREIGN KEY fk_items_order`, then `ADD CONSTRAINT fk_items_order ... ON DELETE RESTRICT`; **Q12** `SET NULL`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: the workbook's worked example (deleting Eve: refused, then her order, then her) shows RESTRICT and CASCADE working together in one transaction. Learners who struggle do tests 1 to 3 of the lab only.

Stretch: `orphan_audit_starter.sql` and `debug_fk_errors.sql`; the lesson's Exercise 4 (tighten a rule, then restore it) and the Bonus Challenge (a three-level cascade: how many rows disappeared in total?).

2:40 Relationship Map

10 min

HOW IT RUNS

A consolidation of both lessons, on paper. Learners draw the eight tables as boxes and every foreign key as a crow's foot line (the lesson's ER diagram is the model, but they draw from memory first), then label each line with its delete rule. Then they check against the database:

```
SELECT TABLE_NAME, CONSTRAINT_NAME, DELETE_RULE, UPDATE_RULE
FROM information_schema.REFERENTIAL_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = 'bookstore'
ORDER BY TABLE_NAME, CONSTRAINT_NAME;
```

```
+-----------------+----------------------+-------------+-------------+
| TABLE_NAME      | CONSTRAINT_NAME      | DELETE_RULE | UPDATE_RULE |
+-----------------+----------------------+-------------+-------------+
| book_categories | fk_bc_book           | CASCADE     | NO ACTION   |
| book_categories | fk_bc_category       | CASCADE     | NO ACTION   |
| books           | fk_books_author      | RESTRICT    | CASCADE     |
| employees       | fk_employees_manager | SET NULL    | NO ACTION   |
| order_items     | fk_items_book        | RESTRICT    | CASCADE     |
| order_items     | fk_items_order       | CASCADE     | CASCADE     |
| orders          | fk_orders_customer   | RESTRICT    | CASCADE     |
+-----------------+----------------------+-------------+-------------+
```

- Seven keys: one 1:M per line, plus the employees self-reference. Ask for the two junction tables (book_categories, order_items) and what extra facts order_items carries.
- For each rule, one learner says the reason in one sentence (the lesson's "Choosing the Rules" table). Every UPDATE_RULE that says NO ACTION was simply not written: it behaves like RESTRICT.
- This map is the best single revision sheet for the Part 1 assessment: tell them to bring it.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Learners answer in the workbook's Exit Ticket box or on a sticky note. Sort before Session 7: question 1's misses predict trouble with the assessment's table-design task; question 2's with reading its error messages.

1. For one-to-many, one-to-one and many-to-many: where does the foreign key go? **Answer:** on the many side; on either side with UNIQUE (usually the optional table); in a junction table with two foreign keys and a

composite primary key.

2. You get `ERROR 1452`. Which table were you changing (parent or child), and what is missing? And `ERROR 1451`? **Answer:** 1452: inserting or updating a CHILD whose parent does not exist. 1451: deleting or changing a PARENT that children still point at (under RESTRICT or NO ACTION).

3. Choose an ON DELETE rule for `order_items.book_id → books` and justify it. **Answer:** RESTRICT: a book that was sold must stay on file for the sales history (mark it out of stock instead). CASCADE would delete sales lines; SET NULL would forget what was sold.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Lesson projects:** 4.1 (compare your design with `bookstore.sql`; type the three tables into a practice database if you want the practice); 4.2 Steps 5 and 6.
- **Lesson exercises.** 4.1: Exercises 1 to 3 (Exercise 4 is a stretch). 4.2: Exercises 1 to 3 (Exercise 4 is a stretch).
- **Both Quick Quizzes** on the lesson pages; **both journal entries**.
- **Labs not finished in class:** `navigate_links_starter.sql`, `orphan_audit_starter.sql`, the two debug handouts; the relationship map if it was cut.
- **Prepare for the Part 1 assessment (Session 7, second half, 60 minutes, open book):** reread your project scripts from Sessions 3 to 6 and your two debug-handout notes; redo one exercise per lesson from Module 2 to 4 without looking; make sure `SOURCE` works from your course folder. It covers creating tables with keys and constraints, CRUD, filtering and sorting, aggregates, subqueries, foreign keys, and the basic joins taught in the first half of Session 7.
- **Before the Next Lesson (from 4.2):** reload `bookstore.sql`, run the "every rule" query once more, explain each rule aloud.

BRING LIST FOR SESSION 7

- A laptop with `bookstore.sql` loaded (8 tables, the counts 10, 11, 5, 25, 8, 12, 18, 10), charged: the assessment needs your own machine for 60 minutes.
- The course folder with `labs/` and every project script so far: the assessment is open book (lesson pages, your notes, your own scripts; no other people and no AI assistants).
- The workbook with today's relationship map, and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** 4.1 demo Steps 2 to 8 (anomaly, the four migration steps, the NULL check); everyone loading `bookstore.sql`; 4.2 demo Steps 1 to 4; the 4.2 lab tests 1 to 3.
- **Cut first (saves 10m):** the relationship map; it becomes homework with the query in the workbook.

- **Cut second (saves 4m):** 4.1 demo Steps 11 and 12 (one-to-one): show the slide; the self-check Q5 covers it.
- **Cut third (saves 7m):** 4.2 demo Steps 8 to 10 (a key added to dirty data): `orphan_audit_starter.sql` does it at home.

RUNNING SHORT: WHAT TO ADD

- Run `debug_fk_errors.sql` together: five errors, five sentences (child or parent, insert or delete or create).
- The 4.2 Exercise 4 live: tighten fk_items_order to RESTRICT, show the 1451 on order 12, restore CASCADE, check with information_schema.
- A mini-design on the board: a library (books, members, loans). Where does each key go, and which delete rule for loans → members?

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 7: Joins and the Part 1 Assessment

How to teach the seventh session of MySQL Foundations, the last of Part 1: Lesson 4.3 (INNER and LEFT joins, the anti-join, self-joins, joins with GROUP BY and the three silent join bugs), then the 60-minute Part 1 assessment on a new database (the Riverside Cooking School), a short review, and a close that works whether the course ends here or continues with Part 2.

`1 lesson + assessment` `3 hrs session` `Lesson 4.3 · Part 1 assessment` `Session 7 of 14 · Part 1`

HOW TO USE THIS GUIDE

- **Two halves.** The first half teaches Lesson 4.3 in the usual rhythm (frame, live demo, guided lab, project start); the second half is the **Part 1 assessment** (60 minutes) and a short review, then a close that works both when Part 1 runs on its own and when Part 2 follows.
- **The assessment lives in two places.** Learners work in `labs/module-07/part1-assessment/part1_assessment_starter.sql`: the file IS the brief (rules, data, tasks, hand-in). You score with the rubric in `final-assessment.html` at the kit root, Part 1 assessment section, whose instructor answer notes hold the reference output (from `labs/module-07/part1-assessment/part1_assessment_solution.sql`) and the common wrong answers with their real numbers. The Answer Key repeats the reference output.
- Every SQL block below was run on MySQL 9.7.2 on a fresh `bookstore.sql` (Lesson 4.3) or in the assessment's own practice database `kit_riverside`. Type the short joins; the long ones can be pasted.
- Keep `lesson_13.html` open for the join diagrams. Labs live in `labs/module-07/`.
- Joins are the most used and most interviewed-about skill in SQL, and learners meet them one hour before an assessment that includes them. The assessment is designed for that: every join task also accepts a subquery answer (Lessons 3.4 and 4.1), and the rubric scores the result and the reasoning, not the technique.

> **THE #1 RISK OF SESSION 7: THE ASSESSMENT STARTS LATE**
>
> Sixty minutes is the minimum for the assessment; every minute lost before 1:35 comes out of it, and learners who rush Part B make the mistakes the rubric is built to catch. **Protect the 1:25 start:** the 4.3 lab is cut first (it continues at home), then the project start. Have the starter file on every laptop BEFORE the break (put it on the shared folder or USB stick at 0:00; the break slide says "copy it now"). **Protect the 60 minutes:** no new content after 1:25; questions about the environment only (a client that will not start, a file that will not open), never about the SQL. If the room starts late anyway, keep the 60 minutes and shorten the review to five minutes: the review is the only segment that can go home (the Answer Key and the reference solution can be shared after scoring).

BEFORE THE SESSION

- ☐ **Data:** every laptop has `bookstore.sql` loaded (Session 6's bring list). The assessment builds its own database (`kit_riverside`) and drops it at the end; it needs nothing else.
- ☐ **Files:** `labs/module-07/` WITHOUT the two `*_solution.sql` files and WITHOUT `part1-assessment/part1_assessment_solution.sql`. Run `python3 labs/check_labs.py module-07` once (7 files, every line PASS). Print or open `final-assessment.html` (Part 1 assessment section): the rubric, the scoring sheet and the answer notes.
- ☐ **Hand-in channel:** decide now how files come back (a shared folder, a USB stick passed around, email, the LMS). Learners rename the file `part1_assessment_<name>.sql`. Test it with your own file.
- ☐ **Projector machine:** `bookstore.sql` loaded; terminal at 18 pt; `lesson_13.html` open. For the review, the reference solution's output ready to show (the Answer Key has it).
- ☐ **Board:** "What does one row of this join stand for?" in large letters (it stays up during the assessment); next to it the join decision: "must every row of one table appear? → put it first, LEFT JOIN; only the unmatched? → WHERE right.id IS NULL".
- ☐ **Room for the assessment:** power for every laptop, seats spread out if possible, a visible clock. Learners with extended time agreed in advance (see the accommodations in the assessment section).
- ☐ **If Part 1 runs on its own:** certificates or completion letters to fill in after scoring, and the course evaluation link for the close. If Part 2 follows: Session 8's bring list on the last slide.

KNOWN TRAPS TODAY

SYMPTOM	CAUSE	FIX
`ERROR 1052 ... ambiguous`	A column (usually `id`) that exists in both joined tables	Qualify it: `o.id`, `c.id`.
`ERROR 1054 Unknown column 'books.title'`	The table name used after an alias was given	Use the alias everywhere in that query.

SYMPTOM	CAUSE	FIX
`ERROR 1066 Not unique table/alias`	The same table joined twice without aliases (a self-join)	Give each copy its own alias: `employees AS e`, `employees AS m`.
A join returns 110 or 96 rows	A missing ON (a silent cross join)	Add the ON condition.
Counts too high after a join	Fan-out: one row per order line, not per order	`COUNT(DISTINCT o.id)`, or count before you join.
Assessment: `Failed to open file`	The client was started outside the course folder	`exit`, `cd` into the folder that holds `labs/`, log in, SOURCE again.
Assessment: everything after A1 fails	The CREATE TABLE has an error, so the enrollments INSERT fails too	Read the FIRST error; fix A1; SOURCE again (every run rebuilds `kit_riverside`).

SESSION 7 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 4.1 and 4.2 (where the key goes, CASCADE, 1451, NO ACTION, following a link); homework check.
0:10	10	**4.3** Frame and mental model	A join pairs rows: INNER keeps pairs, LEFT keeps every left row; what one joined row stands for.
0:20	25	**4.3** Live demo	INNER JOIN and aliases, LEFT JOIN and the anti-join, a join without ON, a self-join, four tables, joins with GROUP BY, three traps.
0:45	20	**4.3** Guided lab	`labs/module-07/l4_3-joins/catalog_joins_starter.sql` in pairs.
1:05	10	**4.3** Start the lesson project	The sales report, Step 1 (one line per order); the rest goes home.
1:15	10	Break	Laptops charged and plugged in; the assessment file copied into everyone's course folder.
1:25	10	Part 1 assessment: brief and setup	Rules (open book, alone, no AI), the Riverside Cooking School data, how to hand in; everyone SOURCEs the starter once.
1:35	60	Part 1 assessment	`labs/module-07/part1-assessment/part1_assessment_starter.sql`: Part A build (about 20 min), Part B eight queries (about 35), a final SOURCE (5).

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
2:35	15	Part 1 review	How each task is solved (approaches, not scores), then the map of Part 1: what each module gave you.
2:50	10	Wrap-up and close	Exit ticket; then the course close (Part 1 on its own) or the bridge to Part 2 (bring list for Session 8).
Total	180		Ends at 3:00.

In the room. Lesson 4.3 estimates 110 minutes for a solo learner; the room has 65 for it, so the lesson keeps its model, demo, a guided lab and the first project step, and the project's other five steps go home. The second half belongs to the assessment: 10 minutes of setup, 60 of work, 15 of review.

Goes home (about 2 to 3 hours). The sales report Steps 2 to 6; the 4.3 Hands-on Exercises 1 to 3; the 4.3 Quick Quiz and journal; the labs not finished (`catalog_joins` , `city_sales` , `debug_joins`); the workbook's Part 1 review self-check (Q7 to Q12). Programs that end with Part 1 make all of it optional practice.

Protect, in this order: the 60 assessment minutes; the 1:25 start; the 4.3 demo Steps 1 to 4 and 12 to 14 (INNER, LEFT, the anti-join, the WHERE-versus-ON trap, `COUNT(*)`); the review's walk-through of B6 to B8. **Cut first:** the 4.3 lab (homework with its README expectations), then the project start, then demo Steps 5 to 8 (show the slides).

OPENING SEGMENT

0:00 Warm-up: Relationships and Foreign Keys — 10 min

TIMING

6m	Five retrieval questions on Lessons 4.1 and 4.2, in the workbook, then on the projector
4m	Homework check: book_reviews Steps 5 and 6; is `bookstore.sql` loaded on every laptop (8 tables)?

RETRIEVAL QUESTIONS AND ANSWERS

1. **One order has many lines. Where does the foreign key go? Answer:** `order_items.order_id` (the many side).

2. **Predict the count.** (Order 3 has one line; fk_items_order is ON DELETE CASCADE.)

```
START TRANSACTION;
DELETE FROM orders WHERE id = 3;
SELECT COUNT(*) AS items FROM order_items;
ROLLBACK;
```

```
+-------+
| items |
+-------+
|    17 |
+-------+
```

Answer: 17: the order's one line went with it; ROLLBACK put both back.

3. **What happens, and why?**

```
DELETE FROM authors WHERE id = 6;
```

```
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constr
```

Answer: 1451: Tolkien has books and fk_books_author is RESTRICT. "delete or update a PARENT row".

4. **A key is declared with no ON DELETE. What does MySQL do when a parent with children is deleted?**
Answer: it refuses: NO ACTION behaves exactly like RESTRICT.

5. **How many links?**

```sql
SELECT COUNT(*) AS links
FROM book_categories
WHERE book_id IN (SELECT id FROM books WHERE author_id = 6);
```

```
+-------+
| links |
+-------+
|     4 |
+-------+
```

Answer: 4: Tolkien's two books are each Fiction and Fantasy. Today the same question becomes one join instead of a nested IN.

HOW TO READ THE WARM-UP

- **Q5 slow or wrong:** following keys through two tables is exactly what a join does in one step; say "after this hour, you will write that as a join".
- **Q2 or Q3 wrong:** the assessment's Part A asks for delete rules with a reason; point those learners to their Session 6 relationship map before the break.

LESSON PLAN

4.3 Joins

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Combine matching rows from two or more tables with INNER JOIN ... ON, using table aliases
- Keep unmatched rows with LEFT JOIN, and find rows with no match using the anti-join pattern
- Explain what RIGHT JOIN and CROSS JOIN return, and when (rarely) to use them
- Join a table to itself to read a hierarchy such as employees and managers
- Combine joins with GROUP BY without the two silent traps: COUNT(*) on outer joins and row fan-out

TIMING IN THE ROOM

10m	Frame: a join pairs rows; INNER keeps pairs, LEFT keeps every left row; one joined row stands for what?
25m	Live demo: INNER, aliases, LEFT, the anti-join, a join without ON, a self-join, four tables, GROUP BY, three traps
20m	Guided lab: `catalog_joins_starter.sql` in pairs
10m	Start the lesson project: the sales report, Step 1

MENTAL MODEL TO SAY

Two short lists on the board, a guest list and the RSVPs (the lesson's journal prompt): "A JOIN pairs each guest with their RSVPs and keeps the pairs: a guest who never replied is not in the result. A LEFT JOIN keeps every guest, and where there is no RSVP it writes NULL. Same lists, same condition, one more word."

Then the question that prevents the silent bugs, pointing at the board: **"what does one row of this join stand for?"** "Join customers to orders and one row is an order. Add order_items and one row is an order LINE: Alice's first order now appears twice. Count after that and you count lines."

Then where conditions go: "ON says how the tables connect. WHERE filters the joined rows. On a LEFT JOIN, a condition on the right table goes in ON, or the NULL rows you wanted to keep are filtered away."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

The lesson's own examples on a fresh `bookstore.sql`. Ask for a row count before every join: it is the habit that catches fan-out and missing ON clauses.

1. **INNER JOIN: only the matches (3m).** "An order row says customer_id 1; the name lives in customers. A join pairs each row with the rows of another table that meet a condition, usually foreign key equals primary key." Point at the ON line.

```
SELECT books.title, authors.name AS author
FROM books
INNER JOIN authors ON books.author_id = authors.id
ORDER BY books.title;
```

```
+------------------------+---------------------+
| title                  | author              |
+------------------------+---------------------+
| 1984                   | George Orwell       |
| Animal Farm            | George Orwell       |
| Brave New World        | Aldous Huxley       |
| Dune                   | Frank Herbert       |
| Fahrenheit 451         | Ray Bradbury        |
| Lord of the Rings      | J.R.R. Tolkien      |
| Pride and Prejudice    | Jane Austen         |
| The Catcher in the Rye | J.D. Salinger       |
| The Great Gatsby       | F. Scott Fitzgerald |
| The Hobbit             | J.R.R. Tolkien      |
| To Kill a Mockingbird  | Harper Lee          |
+------------------------+---------------------+
```

Eleven books, eleven rows. Asimov is absent: an inner join keeps only pairs.

2 **Aliases and WHERE (2m).** "b and a are short names. ON says how the tables connect; WHERE says which results you want."

```
SELECT b.title, b.price, a.name AS author
FROM books AS b
JOIN authors AS a ON b.author_id = a.id
WHERE a.name = 'George Orwell'
ORDER BY b.title;
```

```
+-------------+-------+---------------+
| title       | price | author        |
+-------------+-------+---------------+
| 1984        |  9.99 | George Orwell |
| Animal Farm |  8.49 | George Orwell |
+-------------+-------+---------------+
```

JOIN on its own means INNER JOIN. Once a table has an alias, use only the alias (Step 11).

3 **LEFT JOIN: keep every left row (3m).** "Same tables, same ON, one more word." Ask: how many rows now? Then run it.

```
SELECT a.name AS author, b.title
FROM authors AS a
LEFT JOIN books AS b ON b.author_id = a.id
ORDER BY a.id, b.title;
```

```
+---------------------+------------------------+
| author              | title                  |
+---------------------+------------------------+
| Frank Herbert       | Dune                   |
| George Orwell       | 1984                   |
| George Orwell       | Animal Farm            |
| Harper Lee          | To Kill a Mockingbird  |
| F. Scott Fitzgerald | The Great Gatsby       |
| Jane Austen         | Pride and Prejudice    |
| J.R.R. Tolkien      | Lord of the Rings      |
| J.R.R. Tolkien      | The Hobbit             |
| Aldous Huxley       | Brave New World        |
| J.D. Salinger       | The Catcher in the Rye |
| Ray Bradbury        | Fahrenheit 451         |
| Isaac Asimov        | NULL                   |
+---------------------+------------------------+
```

Twelve: the eleven pairs plus Asimov with NULL. The left table is the one before the word JOIN.

4 **The anti-join (3m).** "The NULLs are a tool. Keep only the rows where the right side is missing."

```
SELECT c.first_name, c.last_name
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id
WHERE o.id IS NULL
ORDER BY c.id;
```

```
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| Carol      | Williams  |
| Hank       | Taylor    |
+------------+-----------+
```

Test the right table's primary key: it can only be NULL when there was no match. NOT EXISTS (Lesson 3.4) answers the same question.

5 **Books never sold (1m).**

```sql
SELECT b.title
FROM books AS b
LEFT JOIN order_items AS oi ON oi.book_id = b.id
WHERE oi.id IS NULL
ORDER BY b.title;
```

```
+------------------------+
| title                  |
+------------------------+
| The Catcher in the Rye |
| The Great Gatsby       |
+------------------------+
```

6 **A join without ON (1m).** "Forget the ON and MySQL does not complain."

```sql
SELECT COUNT(*) AS rows_returned
FROM books
JOIN authors;
```

```
+---------------+
| rows_returned |
+---------------+
|           110 |
+---------------+
```

11 × 10 = 110: a cross join. If a join returns far more rows than expected, check the ON first.

7 **Self-join: two roles, one table (3m).** "employees.manager_id points at another employee. e plays the employee, m the manager."

```sql
SELECT
    CONCAT(e.first_name, ' ', e.last_name) AS employee,
    CONCAT(m.first_name, ' ', m.last_name) AS manager
FROM employees AS e
LEFT JOIN employees AS m ON e.manager_id = m.id
ORDER BY e.id;
```

```
+---------------+--------------+
| employee      | manager      |
+---------------+--------------+
| Maria Santos  | NULL         |
| Daniel Reyes  | Maria Santos |
| Priya Nair    | Daniel Reyes |
| Tom Becker    | Priya Nair   |
| Lena Park     | Priya Nair   |
| Marcus Hill   | Daniel Reyes |
| Sofia Alvarez | Marcus Hill  |
| James Okafor  | Maria Santos |
| Nina Kowalski | James Okafor |
| Ben Carter    | Priya Nair   |
+---------------+--------------+
```

Maria survives only because this is a LEFT JOIN; an inner join would drop the head of the company. To walk the whole tree: Lesson 5.1 (recursive CTE).

8 **Four tables: who bought what (2m).** "Each JOIN adds one table to the row you are building. Walk the keys from orders outward."

```sql
SELECT o.id AS order_id, o.order_date, b.title, oi.quantity
FROM orders AS o
JOIN customers AS c    ON o.customer_id = c.id
JOIN order_items AS oi ON oi.order_id = o.id
JOIN books AS b        ON oi.book_id = b.id
WHERE c.first_name = 'Alice'
ORDER BY o.order_date, b.title;
```

```
+----------+------------+---------------------+----------+
| order_id | order_date | title               | quantity |
+----------+------------+---------------------+----------+
|        1 | 2026-01-15 | 1984                |        2 |
|        1 | 2026-01-15 | Dune                |        1 |
|        2 | 2026-02-20 | Pride and Prejudice |        1 |
|        8 | 2026-04-05 | Lord of the Rings   |        1 |
+----------+------------+---------------------+----------+
```

Order 1 appears twice: one row per order LINE. Remember that before adding things up.

9 **Joins with GROUP BY: zeros included (2m).** "Books per author, Asimov included. COUNT the right table's id."

```
SELECT a.name AS author, COUNT(b.id) AS books
FROM authors AS a
LEFT JOIN books AS b ON b.author_id = a.id
GROUP BY a.id, a.name
ORDER BY books DESC, author;
```

```
+----------------------+-------+
| author               | books |
+----------------------+-------+
| George Orwell        |     2 |
| J.R.R. Tolkien       |     2 |
| Aldous Huxley        |     1 |
| F. Scott Fitzgerald  |     1 |
| Frank Herbert        |     1 |
| Harper Lee           |     1 |
| J.D. Salinger        |     1 |
| Jane Austen          |     1 |
| Ray Bradbury         |     1 |
| Isaac Asimov         |     0 |
+----------------------+-------+
```

10 **What each customer spent (2m).** "Both joins LEFT so Carol and Hank stay; COUNT(DISTINCT o.id) counts orders, not lines; COALESCE turns their NULL sums into 0."

```
SELECT
    CONCAT(c.first_name, ' ', c.last_name)          AS customer,
    COUNT(DISTINCT o.id)                            AS orders,
    COALESCE(SUM(oi.quantity * oi.price_at_time), 0) AS spent
FROM customers AS c
LEFT JOIN orders AS o       ON o.customer_id = c.id
LEFT JOIN order_items AS oi ON oi.order_id = o.id
GROUP BY c.id, c.first_name, c.last_name
ORDER BY spent DESC, customer;
```

```
+----------------+--------+-------+
| customer       | orders | spent |
+----------------+--------+-------+
| Alice Johnson  |      3 | 58.95 |
| Dave Brown     |      2 | 45.96 |
| Frank Miller   |      2 | 41.96 |
| Grace Wilson   |      2 | 33.97 |
| Bob Smith      |      2 | 33.47 |
| Eve Davis      |      1 | 22.98 |
| Carol Williams |      0 |  0.00 |
| Hank Taylor    |      0 |  0.00 |
+----------------+--------+-------+
```

11 **Trap 1: an ambiguous column.**

```sql
SELECT id, title, name
FROM books
JOIN authors ON books.author_id = authors.id;
```

```
ERROR 1052 (23000): Column 'id' in field list is ambiguous
```

Qualify it: `books.id` or `b.id`.

12 **Trap 2: WHERE on the right table (silent) (2m).** "Every customer, with their orders from May on." Predict how many customers come back.

```sql
SELECT c.first_name, o.order_date
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id
WHERE o.order_date >= '2026-05-01'
ORDER BY c.first_name;
```

```
+------------+------------+
| first_name | order_date |
+------------+------------+
| Dave       | 2026-06-07 |
| Frank      | 2026-05-02 |
| Grace      | 2026-05-19 |
+------------+------------+
```

Three, not eight: for customers without a May order the date is NULL and WHERE drops them. The LEFT JOIN has quietly become an inner join.

13 **The fix: the condition goes in ON (1m).**

```sql
SELECT c.first_name, o.order_date
FROM customers AS c
LEFT JOIN orders AS o
       ON o.customer_id = c.id
      AND o.order_date >= '2026-05-01'
ORDER BY c.first_name;
```

```
+------------+------------+
| first_name | order_date |
+------------+------------+
| Alice      | NULL       |
| Bob        | NULL       |
| Carol      | NULL       |
| Dave       | 2026-06-07 |
| Eve        | NULL       |
| Frank      | 2026-05-02 |
| Grace      | 2026-05-19 |
| Hank       | NULL       |
+------------+------------+
```

Eight customers; the condition decides which orders match without removing anyone.

14 **Trap 3: COUNT(*) after a LEFT JOIN (silent).** "Asimov has no books."

```sql
SELECT a.name, COUNT(*) AS books
FROM authors AS a
LEFT JOIN books AS b ON b.author_id = a.id
WHERE a.id IN (2, 10)
GROUP BY a.id, a.name
ORDER BY a.id;
```

```
+---------------+-------+
| name          | books |
+---------------+-------+
| George Orwell |     2 |
| Isaac Asimov  |     1 |
+---------------+-------+
```

`COUNT(*)` counted his padded row. Count the right table's column: `COUNT(b.id)`.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Ambiguous or alias-mismatched columns** (1052, 1054). Coach: "Which table does this column come from? Write it in front, with the alias." Many developers qualify every column in a join.

- **A LEFT JOIN that silently became an inner join** (a right-table condition in WHERE). Coach: "Who disappeared? Carol and Hank? Then your WHERE removed the NULL rows: move the condition into ON."
- **Counting after a fan-out.** Coach with the board question: "What does one row stand for here?" Then `COUNT(DISTINCT ...)`, or count the right table's id after a LEFT JOIN.

GUIDED LAB

File: `labs/module-07/l4_3-joins/catalog_joins_starter.sql`; reference `labs/module-07/l4_3-joins/catalog_joins_solution.sql`. Four questions, each missing its join: an INNER JOIN through three tables, a LEFT JOIN with GROUP BY (Asimov's 0), an anti-join on the self-reference (Session 6 did it with NOT EXISTS), and a self-join with a computed column.

- **TODO 1 together (5m), TODOs 2 to 4 in pairs (13m).** Expected: order 7 = 1984 and Animal Farm (Orwell), order 12 = Animal Farm and Fahrenheit 451; authors Orwell 2 / 8.49, Tolkien 2 / 10.99 ... Asimov 0 / NULL; Tom, Lena, Sofia, Nina, Ben manage nobody; Sofia was hired 5 years after Marcus, Tom 3 after Priya.
- **Circulate for:** `COUNT(*)` in TODO 2 (Asimov gets 1: ask them to read his row), the anti-join testing `e.id IS NULL` instead of `report.id IS NULL` (nothing comes back), and a self-join without aliases (1066).
- **Debrief (2m):** "TODO 3 answered a Session 6 question. Which version, NOT EXISTS or the anti-join, would you rather read next month?" Either is right.
- **Fast finishers:** `labs/module-07/l4_3-joins/city_sales_starter.sql` (revenue per city with the NULL group, DISTINCT after a fan-out, the WHERE-versus-ON trap on a new question), then `labs/module-07/l4_3-joins/debug_joins.sql` (1052, 1054, 1066 and one silent count).

START THE LESSON PROJECT

The lesson project is **The Sales Report**: `sales_report.sql`, six joined queries for the owner. In the room: Step 1 only (10 minutes).

- Step 1: one row per order with the customer's name, the number of books and the total, grouped by the order. Twelve rows; order 1 totals 34.97.
- **Done in the room** = Step 1 runs from the file with SOURCE.
- **Goes home:** Steps 2 to 6 (revenue by month, best sellers, customers to welcome with the anti-join, sales by category, the staff directory with a self-join). The checklist's "the six monthly revenues add up to the sum of all order totals" is a good self-test: 237.29 both ways.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Customers JOIN orders returns 12 rows; LEFT JOIN returns 14. Who are the two extra rows? **Answer:** Carol and Hank, with NULL order columns.

- How do you find the customers who never ordered? **Answer:** LEFT JOIN orders, WHERE o.id IS NULL (or NOT EXISTS).
- Why `COUNT(b.id)` and not `COUNT(*)` after a LEFT JOIN? **Answer:** `COUNT(*)` counts the NULL-padded row; COUNT(b.id) skips it.
- Why does a self-join need aliases? **Answer:** the same table plays two roles; without aliases every column is ambiguous (1066).

Workbook self-check, items Q1 to Q6 (4.3):

Answers: **Q1** C) `14`; **Q2** A) `FROM books b LEFT JOIN order_items oi ON oi.book_id = b.id WHERE oi.id IS NULL`; **Q3** `4` (one row per order line, not per order); **Q4** B) `ERROR 1052 (23000): Column 'id' in field list is ambiguous`; **Q5** `Carol | 0` and `Dave | 2`; **Q6** D) The WHERE runs after the join: for the padded rows b.in_stock is NULL, so the LEFT JOIN behaves like an inner join. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: the workbook's worked example (guests and RSVPs, four guests, three replies) shows JOIN, LEFT JOIN, the anti-join and `COUNT(*)` against `COUNT(r.id)` on data small enough to check by eye.

Stretch: `city_sales_starter.sql` and `debug_joins.sql`; the lesson's Exercise 4 (skip-level managers) and the Bonus Challenge (a full outer join with UNION).

PART 1 ASSESSMENT

1:25 Part 1 Assessment: The Riverside Cooking School

70 min

TIMING

10m	Brief and setup: the rules, the data, the hand-in; everyone SOURCEs the starter once and sees its placeholder output
20m	Part A (suggested): build the enrollments table, load it, two refused statements, one UPDATE, one DELETE
35m	Part B (suggested): eight queries on the data after Part A
5m	Final run: SOURCE the whole file, fix the first error, hand in

WHAT TO SAY AT 1:25 (BRIEF AND SETUP)

- "For the next hour you show what you can do with Part 1, alone. It is **open book**: the lesson pages, your notes, your workbook and your own scripts are all allowed. Other people and AI assistants are not. Nobody can finish everything perfectly in an hour; do what you can do well."
- "Open `labs/module-07/part1-assessment/part1_assessment_starter.sql` in your editor. The file is the brief: read its first block with me." Walk through the header: the cooking school, the three given tables, Part A, Part B, the hand-in. Everyone runs it once: `SOURCE labs/module-07/part1-assessment/part1_assessment_starter.sql`. They should see the three tables built, TODO lines and no ERROR.
- "Write every answer in the file, under its TODO. After each answer, save and SOURCE the file again: it rebuilds the school from scratch every time, so you cannot break anything. Part B runs on the data AFTER Part A: if you skip a Part A step, later numbers change, and that is fine; just keep going."
- "You may answer with joins or with subqueries: either is fine if the result is right and the query reads clearly. Every query that returns several rows needs an ORDER BY."
- "At 2:30 I say 'five minutes': SOURCE the whole file once more, fix the FIRST error if there is one, rename the file `part1_assessment_<yourname>.sql` and hand it in." Write the hand-in channel and 2:35 on the board.

DURING THE ASSESSMENT

- **Answer environment questions only:** a client that will not start, `Failed to open file`, a forgotten password. For SQL questions: "Look at the error number, then the lesson page for it."
- **Time calls on the board:** 1:55 "Part A should be done", 2:15 "halfway through Part B", 2:30 "five minutes: run the whole file, hand in".

- **Watch, do not grade yet:** note learners who are stuck on A1 for more than 15 minutes; at 1:50 tell them privately to paste the A2 INSERT under a minimal table (two INT columns and a date) so Part B can run (they lose points on A1 only).
- **A learner finishes early:** ask them to reread every ORDER BY and every typed number, then the stretch: "what would change in B8 if class prices changed later?" (a price_at_time column).

SCORING (AFTER THE SESSION)

Score each file with the five-row rubric in `final-assessment.html` (Part 1 assessment section): table design and constraints; data changes and integrity; filtering, sorting and aggregates; subqueries and joins; script quality. Run each file first (`SOURCE` it, or `python3 labs/check_labs.py --file <file>` after adding a `-- state: none` line if it is missing); a file that stops with an error is scored on what it printed and the code you can read. The answer notes list the reference numbers and what each common wrong answer prints.

The reference output (instructor copy, never handed out before scoring):

```
+---------------+-------------+---------+
| name          | enrollments | revenue |
+---------------+-------------+---------+
| Marco Rossi   |           5 |  330.00 |
| Aiko Tanaka   |           4 |  260.00 |
| Lena Fischer  |           4 |  240.00 |
| Diego Morales |           2 |  100.00 |
| Nadia Haddad  |           0 |    0.00 |
+---------------+-------------+---------+
```

Above: B8, the hardest task (five instructors, Nadia Haddad with 0 enrollments and 0.00). The Answer Key prints the whole reference run.

2:35 Part 1 Review

15 min

HOW IT RUNS

Files are in; scores come later. This is about how to think, not about marks. On the projector, the reference solution's approaches (not its numbers, if your program returns scored work before sharing answers):

- **A1 (3m):** the composite primary key (`PRIMARY KEY (student_id, class_id)`) is the "only once" rule; the CHECK is the 1 to 5 rule; two named keys. Ask two learners for their delete rules and reasons: different choices with good reasons both score.
- **B2 and B4 (3m):** WHERE before the piles, HAVING after; `COUNT(rating)` and `AVG(rating)` skip the blanks, `COALESCE(rating, 0)` would turn "not yet rated" into a zero star and drag the averages down.
- **B5 (1m):** a typed 61.43 gives the right rows today and the wrong ones after the next price change: a subquery computes it fresh.
- **B6 to B8 (5m):** one row of B6's join is one enrollment; B7 is the anti-join (or NOT EXISTS); B8 needs every instructor (LEFT JOIN from instructors) and a sum that ignores the padded rows.
- **The map of Part 1 (3m):** the workbook's Part 1 map, one line per module: Module 1 server, client, database; Module 2 tables, types, constraints, CRUD; Module 3 filter, sort, summarize, nest; Module 4 relate, protect, join. "Every task in the assessment came from one of these lines."

The workbook's Part 1 review self-check (Q7 to Q12, one question per lesson group; homework or a Session 8 refresher):

Answers: **Q7** A) `DECIMAL(8, 2)`; **Q8** `ERROR 1062 (23000): Duplicate entry 'alice@example.com' for key 'customers.email'`; **Q9** B) `3`; **Q10** D) SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT; **Q11** `Pride and Prejudice` (one row); **Q12** C) A junction table linking orders and books (many-to-many), carrying facts about each link: quantity and price_at_time. Full set with explanations in the Answer Key.

CLOSING

EXIT TICKET (3 QUESTIONS, 4 MINUTES)

A reflective exit ticket after the assessment; collect it with the files. Question 2 tells you what to reteach first if Part 2 follows.

1. In one sentence each, with the customers and orders tables: what does JOIN return, and what does LEFT JOIN add? **Answer:** JOIN returns one row per customer-order pair (12 rows, no Carol or Hank); LEFT JOIN also keeps every customer without an order, with NULLs (14 rows).

2. Which assessment task was hardest for you, and what would you look up first? **Answer:** a reflection; group the answers by task (A1, B4, B7 and B8 are the usual ones).
3. Name one thing from Part 1 you will use in a project or at work, and one you want to practice. **Answer:** a reflection.

VERSION A: PART 1 RUNS ON ITS OWN (THE COURSE ENDS TODAY)

- **Close the course (5m).** Thank the room. Show the map of Part 1 once more: "Seven sessions ago you installed a server. Today you designed a table with keys and rules, changed data safely, and answered eight questions about a database you had never seen."
- **Completion:** say how and when results come back (within a week is a good promise). A learner completes Part 1 when the assessment is met on the rubric in `final-assessment.html` (or met after one revision) and the sessions were attended or made up.
- **Next steps:** Part 2 if your program offers it (CTEs and window functions, indexes and views, design and normalization, stored programs, transactions and JSON, users and backups, a final project); otherwise the course site's Lessons 5.1 to 8.3 are a self-study path in the same style. Practice suggestion: one real question a week about data they care about, answered in SQL.
- **Course evaluation** before they leave (link or paper).

VERSION B: PART 2 FOLLOWS (SESSION 8 IS NEXT)

- **Homework (about 2 to 3 hours):** the sales report Steps 2 to 6; 4.3 Hands-on Exercises 1 to 3; the 4.3 Quick Quiz and journal; the workbook's Part 1 review self-check (Q7 to Q12); the 4.3 labs not finished.
- **Bring list for Session 8:** a laptop with `bookstore.sql` loaded (reload it: today's labs change nothing, but the 4.3 lesson's "Before the Next Lesson" asks for a fresh copy); `sales_report.sql`, finished; the course folder with `labs/`; the workbook and the journal.
- **Bridge (1m):** "Your sales report reads inside out in places, and the staff hierarchy needed one join per level. Session 8 fixes both: CTEs name the steps of a long query, and window functions rank and total without collapsing rows."

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish the lesson project:** `sales_report.sql` Steps 2 to 6 (optional practice if Part 1 ends today).
- **Lesson exercises:** 4.3 Exercises 1 to 3 (Exercise 4 is a stretch).
- **The 4.3 Quick Quiz** on the lesson page and **the 4.3 journal entry** (the guest list and RSVPs).
- **Part 1 review:** the workbook's self-check Q7 to Q12, then the matching lesson sections for any you missed.
- **Labs not finished in class:** `catalog_joins_starter.sql`, `city_sales_starter.sql`, `debug_joins.sql`.

- **Before the Next Lesson (from 4.3, if Part 2 follows):** run `sales_report.sql` from a fresh session; rewrite one anti-join with NOT EXISTS; reload `bookstore.sql`.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the 60 assessment minutes and the 1:25 start; 4.3 demo Steps 1 to 4 and 12 to 14.
- **Cut first (saves up to 20m):** the 4.3 guided lab. Learners take `catalog_joins_starter.sql` home; its expected output is in the file.
- **Cut second (saves 10m):** the project start: Step 1 goes home with the rest.
- **Cut third (saves 7m):** demo Steps 5 to 8 (books never sold, the cross join, the self-join, four tables): show the slides only.
- **If the assessment starts late anyway:** keep the full 60 minutes and shorten the review to five minutes (A1 and B8 only).

RUNNING SHORT: WHAT TO ADD

- Before the break: `debug_joins.sql` with the whole room (three errors and the silent fan-out).
- After the assessment: let learners compare their B8 with a neighbor's and explain any difference in one sentence (the most instructive task to discuss).
- For Version A: ten minutes of "ask me anything" about where SQL goes next (the jobs, the tools, the other databases).

MYSQL FOUNDATIONS · FACILITATOR GUIDE

Part 2: Beyond the Basics

Sessions 8 to 14. Modules 5 to 8: CTEs and window functions, indexes, views and design, stored programs, transactions, JSON, users and backups, ending with the final project.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 8: CTEs and Window Functions

How to teach the first session of Part 2. Check that every machine has the full course database, refresh Part 1's joins and grouping in five questions, then turn inside-out queries into named steps, follow the bookstore's chain of managers with recursive CTEs, and put every row next to a total, a rank or its neighbor with window functions.

2 lessons | 3 hrs session | Lessons 5.1–5.2 | Session 8 of 14 · Part 2

HOW TO USE THIS GUIDE

- Same rhythm as Part 1: frame (one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice) → start the lesson's own project. Exercises, quizzes, journals and the rest of each project go home.
- Every SQL block below was run to produce the output printed under it, on a freshly loaded bookstore on MySQL 9.7 with the default settings. Ask for a prediction before every statement that prints something; read error numbers aloud.
- Keep `lesson_ctes.html` and `lesson_window_functions.html` open for the round-by-round diagrams and the version notes. Labs live in `labs/module-08/`: `_starter.sql` files are handouts, `_solution.sql` files are yours.
- Session 8 opens Part 2. The warm-up is 20 minutes instead of 10: an environment check that requires the full bookstore, a five-question Part 1 refresher, and the map of Part 2. Both lessons start from a freshly loaded `bookstore.sql` and only read, so nobody needs to reload between the two lessons.
- Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (5.1, 5.2) match the course site.

STARTING WITH PART 2? (PROGRAMS THAT SKIP PART 1)

Part 2 can be sold and run on its own for learners who already write SQL. It assumes everything Part 1 teaches: a working MySQL 9.7 (or 8.4) and the `mysql` client, SOURCE, CREATE TABLE with keys and constraints, CRUD, WHERE/ORDER BY/LIMIT, GROUP BY with HAVING, subqueries and derived tables, foreign keys, and all four joins including self-joins. **Before:** send Part 1's setup guide (Lesson 1.2's install section for their platform), the course download `bookstore.sql`, and two files from this session: `labs/module-08/l5_0-part2-start/check_environment.sh` (or `.ps1`) and `labs/module-08/l5_0-part2-start/refresher_starter.sql`. Ask for the check's verdict and the refresher's four answers by email. **In the room:** the warm-up below runs unchanged; learners who struggled with the refresher sit with someone who did not. **Gaps to expect:** self-joins and LEFT JOIN counting (W2 to W4); give those learners Lesson 4.3 as reading before Session 9. The Session 1 environment check is not needed: this session's version checks the same things and also requires the bookstore.

For a cohort coming straight from Part 1 (Session 7), the note does not apply: they ran the Part 1 assessment last time, and the refresher doubles as a quick look at what stuck.

BEFORE THE SESSION

- ☐ **Files to hand out:** `labs/module-08/` without the `*_solution.sql` files; the workbook. Learners keep `labs/` in their course folder, next to `bookstore.sql`, and open their terminal there.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-08` from this kit's folder (13 files, a few seconds, all PASS). Run the environment check on the projector machine so you can show READY for Part 2.
- ☐ **Projector machine:** MySQL 9.7 with a freshly loaded bookstore (`SOURCE bookstore.sql`), the `mysql>` client at 18 pt or larger, and the course folder as the current folder. Browser tabs: `lesson_ctes.html`, `lesson_window_functions.html`.
- ☐ **Board, before learners arrive:** the Part 2 map (Sessions 8 to 14: queries → design → programs → operations → final project); a three-box "recipe" (customer_totals → average → keep above); and two columns headed "GROUP BY: one row per group" and "OVER: every row, plus the group's number".
- ☐ **Print or share:** the workbook (its Start Here page has the Part 2 check and the refresher), sticky notes for the exit ticket.

KNOWN TRAPS TODAY

SYMPTOM	CAUSE	FIX
`ERROR 1146 ... Table 'bookstore.org' doesn't exist`	WITH without RECURSIVE (or a CTE used in the next statement)	WITH RECURSIVE; repeat the WITH in each statement that needs it.
`ERROR 3636 ... Recursive query aborted after 1001`	No stop condition, or one that never becomes false	Add or fix the WHERE in the recursive member. Do not raise the limit to "fix" it.

SYMPTOM	CAUSE	FIX
`iterations`		
`ERROR 1406 ... Data too long for column 'path'`	The anchor fixed a narrow column (a literal like `'Maria'`)	Widen it in the anchor: `CAST(... AS CHAR(200))`.
`ERROR 3593 ... cannot use the window function`	A window function in WHERE, HAVING or GROUP BY	Compute it in a CTE, filter in the outer query.
`ERROR 1064 ... near 'rank`	`rank`, `row_number`, `groups`, `window` are reserved words	Name columns `rank_num`, `pos`; or use backticks.
A learner on MariaDB sees no error for a runaway recursion	MariaDB stops at `max_recursive_iterations` with a warning, not an error	`SHOW WARNINGS;` and always write the stop condition. `LAG(x, 2, 0)` is a 1064 there: wrap in COALESCE.
The check says `NOT READY ... bookstore has 9 tables`	A leftover table from Part 1 work (or a lesson in progress)	`SOURCE bookstore.sql` rebuilds the database from scratch.

SESSION 8 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	20	Welcome to Part 2: environment check and Part 1 refresher	Everyone reloads `bookstore.sql` and runs `check_environment` (READY for Part 2); five retrieval questions on joins and Part 1; the map of Part 2.
0:20	10	**5.1** Frame and mental model	One question, three steps; a step with a name; recursion in rounds.
0:30	20	**5.1** Live demo	Inside-out nested query → WITH; two CTEs; a CTE joined to itself; recursion: a series, a calendar, the org chart, a chain of command; two errors on purpose.
0:50	15	**5.1** Guided lab	`labs/module-08/l5_1-ctes/staff_tree_starter.sql` in pairs: twelve months, then a division for every employee.
1:05	10	**5.1** Start the lesson project	`org_report.sql`: Steps 1 and 3 (org chart, chain of command); Steps 2, 4 and 5 go home.
1:15	10	Break	Leave the GROUP BY versus OVER picture on the board.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:25	10	**5.2** Frame and mental model	GROUP BY squashes, OVER keeps every row; the three parts of OVER; where windows run in the clause order.
1:35	25	**5.2** Live demo	OVER (), PARTITION BY, three ranking functions, running total and moving average, LAG and LEAD, top N per group, three mistakes.
2:00	20	**5.2** Guided lab	`labs/module-08/l5_2-window-functions/leaderboards_starter.sql`: ties, top 2 per category, share of department payroll.
2:20	15	**5.2** Start the lesson project	`sales_dashboard.sql`: Steps 1 and 2 (revenue so far, month over month); Steps 3 to 5 go home.
2:35	15	Debug clinic: both lessons	`debug_ctes.sql` and `debug_windows.sql` in pairs: six errors, one fix each, then the CTE rewrite.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 9.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 100 + 100 = 200 minutes for a solo learner; after the 20-minute Part 2 opening, the break, the debug clinic and the wrap-up, the room has 135. Lesson 5.1 gets 55 minutes and 5.2 gets 70: window functions have more moving parts (partition, order, frame), and recursion is easier to finish at home once the rounds are clear.

Goes home (about 2 to 3 hours). `org_report.sql` Steps 2, 4 and 5; `sales_dashboard.sql` Steps 3 to 5; each lesson's Exercises 1 to 3 (4 is a stretch); both Quick Quizzes; both journal entries; the lab files not finished in class (`category_steps_starter.sql`, `order_trends_starter.sql`).

Protect, in this order: the environment check (Part 2 needs the full bookstore every session); 5.1 demo steps 6 to 8 (recursion in rounds: the idea people cannot get from reading alone); the 5.2 ranking step with a prediction; top N per group with the CTE pattern; the staff-tree lab through TODO 2. **Cut first:** the debug clinic (hand out both debug files as homework), then the 5.1 project start (send Steps 1 to 5 home), then 5.2 demo step 9 (the RANGE trap is in the self-check).

OPENING SEGMENT

0:00 Welcome to Part 2: Environment Check and Part 1 Refresher — 20 min

TIMING

5m	Reload `bookstore.sql`; run the Part 2 environment check on every machine
12m	Part 1 refresher: five retrieval questions (W1 to W5), answered from memory, then shown
3m	The map of Part 2 and today's two lessons

SEQUENCE AND TALKING POINTS

- **Environment check (5m).** Say: "Part 2 starts from the full course database, every session." On the projector, then theirs: `SOURCE bookstore.sql` (it prints the eight row counts), then `SOURCE labs/module-08/l5_0-part2-start/check_environment.sql`. Done = `READY for Part 2` (`READY WITH NOTES` on 8.4 or MariaDB). The check fails on purpose when bookstore is missing or has a ninth table left from Part 1 work: reload and rerun. Machines that are not ready pair up; you fix them during the 5.1 lab, not now. The `.sh` / `.ps1` helpers in the same folder also check the client version.
- **Refresher (12m).** Say: "Five questions from Part 1, no notes. Today's lessons stand on joins, grouping and derived tables." Learners write answers to W1 to W5 on the workbook's Start Here page (5 minutes), then you take them one by one: a volunteer answers, you show the query or the fix. Tally misses on a sticky note: W3 and W4 (counting after joins) predict trouble with today's CTEs that total orders.
- **Part 2 map (3m).** Point at the board: "Sessions 8 to 14: better queries today (CTEs and window functions), then design for speed and safety (indexes, views, normalization), programs inside the database (procedures, transactions, JSON), operations (users, backups), and your own database as the final project." Today: "5.1 names the steps of a long query and follows chains of rows; 5.2 puts each row next to a total, a rank or its neighbor."

REFRESHER ANSWERS (W1 TO W5)

1. **W1** Which customers have never placed an order? Say the query shape, then the names. **Answer:** A LEFT JOIN from customers to orders, keeping the customers whose orders side is NULL (or HAVING COUNT(o.id) = 0): Carol and Hank. Lesson 4.3, the anti-join.
2. **W2** employees has a manager_id. How do you show each employee next to their manager's name? Who is Sofia's manager? **Answer:** A self-join: the table twice with two aliases, ON m.id = e.manager_id. Sofia (id 7) reports to Marcus.
3. **W3** SELECT a.name, COUNT(*) FROM authors a LEFT JOIN books b ON b.author_id = a.id GROUP BY a.id, a.name: what does it say for Isaac Asimov, who has no books, and how do you fix it? **Answer:** 1,

because COUNT(*) counts the NULL-padded row the LEFT JOIN made. Count a column of the right table, COUNT(b.id), which skips NULL: 0. (Lesson 4.3, Common Mistake 4.)

4. **W4** Books per author and copies sold in one query: Orwell shows 5 books, but he wrote 2. What happened? **Answer:** Fan-out: joining order_items repeats each book once per order line. COUNT(DISTINCT b.id) fixes it. Ask what one row of the join stands for. (Lesson 4.3, Common Mistake 5.)

5. **W5** A subquery in FROM (a derived table) fails with error 1248. What is missing? **Answer:** An alias: every derived table needs its own name, (...) AS t (Lesson 3.4). Today's CTEs give such steps a name up front, and read top to bottom.

The key shows W1 and W2's real output. `labs/module-08/l5_0-part2-start/refresher_starter.sql` practices the same four moves (an inner join, LEFT JOIN with COUNT, a derived table, a self-join): hand it to anyone who missed two or more.

WATCH-OUTS

- **Do not reteach Part 1.** Five minutes of answers, not a lecture: Lesson 4.3 and the refresher lab are the place for gaps.
- **A machine with no bookstore.** `SOURCE` needs the path to `bookstore.sql`; `Failed to open file` means the client was started in another folder (exit, `cd`, log in again), or give the full path with forward slashes on Windows.
- **New learners in a Part 2-only program.** Pair them with each other only if both passed the refresher; otherwise with someone from the strongest pile.

LESSON PLANS

5.1 CTEs & Recursive Queries

55 min

OBJECTIVES (FROM THE LESSON PAGE)

- Rewrite a nested subquery as a WITH query and name each step
- Chain several CTEs in one statement, letting later steps read earlier ones
- Choose between a CTE, a derived table and a view for a given job
- Write a recursive CTE with an anchor, a recursive member and a stop condition, and generate number and month series with it
- Walk the employee hierarchy down (an org chart) and up (a chain of command)
- Read and fix errors 1146, 3636 and 1406 from broken CTEs

TIMING IN THE ROOM

10m	Frame: one question, three steps; WITH names a step; recursion in rounds
20m	Live demo: nested query → CTE; two CTEs; self-joined CTE; series, calendar, org chart, chain of command; 1146 and 3636
15m	Guided lab: `staff_tree_starter.sql` in pairs
10m	Start the lesson project: `org_report.sql` Steps 1 and 3

MENTAL MODEL TO SAY

Point at the three boxes on the board: "A real question takes steps. Total each customer's orders. Average those totals. Keep the customers above it. Until now you nested the steps inside each other and read them inside out. A **CTE** writes each step once, gives it a name, and lists the steps top to bottom, like a recipe. The name lives for one statement."

Then recursion, with the number series drawn as rounds: "A **recursive CTE** has an **anchor** that runs once and makes the starting rows, and a **recursive member** that runs again and again, each time reading only the rows the last round made. When a round makes no rows, it stops: that is the **stop condition**. It is how you follow a chain whose length you do not know: managers above managers, months until July."

The two directions of a hierarchy, as two arrows on the board: down = `e.manager_id = cte.id` ("whose manager is this?"), up = `e.id = cte.manager_id` ("who is this person's manager?").

LIVE DEMO (YOU TYPE, THEY FOLLOW)

The lesson's own examples, typed live on the freshly loaded bookstore. Learners type along on their own machines; the long queries can be pasted from the lesson page by anyone who falls behind.

1. **The question, inside out (2m).** "Which customers spent more than the average customer? Three steps: total each customer, average the totals, keep the ones above. With Lesson 3.4's derived tables it looks like this." Scroll it slowly; ask where to start reading (the middle) and point at the total-per-customer step written twice.

```sql
SELECT t.customer, t.total
FROM (
    SELECT c.first_name AS customer,
           SUM(oi.quantity * oi.price_at_time) AS total
    FROM customers c
    JOIN orders o       ON o.customer_id = c.id
    JOIN order_items oi ON oi.order_id = o.id
    GROUP BY c.id, c.first_name
) AS t
WHERE t.total > (
    SELECT AVG(t2.total)
    FROM (
        SELECT SUM(oi.quantity * oi.price_at_time) AS total
        FROM orders o
        JOIN order_items oi ON oi.order_id = o.id
        GROUP BY o.customer_id
    ) AS t2
)
ORDER BY t.total DESC;
```

```
+----------+-------+
| customer | total |
+----------+-------+
| Alice    | 58.95 |
| Dave     | 45.96 |
| Frank    | 41.96 |
+----------+-------+
```

2. **The same question, top to bottom (3m).** "Name the step once, up front, with WITH. Now it reads like a recipe: here is customer_totals; show the ones above its average." Same three rows: Alice, Dave, Frank.

```
WITH customer_totals AS (
    SELECT c.first_name AS customer,
           SUM(oi.quantity * oi.price_at_time) AS total
    FROM customers c
    JOIN orders o       ON o.customer_id = c.id
    JOIN order_items oi ON oi.order_id = o.id
    GROUP BY c.id, c.first_name
)
SELECT customer, total
FROM customer_totals
WHERE total > (SELECT AVG(total) FROM customer_totals)
ORDER BY total DESC;
```

```
+----------+-------+
| customer | total |
+----------+-------+
| Alice    | 58.95 |
| Dave     | 45.96 |
| Frank    | 41.96 |
+----------+-------+
```

Average of the six customers who ordered: about 39.55 (Carol and Hank never ordered, so the inner joins leave them out).

3 **Two steps, one WITH (2m).** "One WITH, several CTEs, separated by commas. The second reads the first. average has one row, so CROSS JOIN attaches it to every customer."

```
WITH customer_totals AS (
    SELECT o.customer_id,
           SUM(oi.quantity * oi.price_at_time) AS total
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.id
    GROUP BY o.customer_id
),
average AS (
    SELECT AVG(total) AS avg_total
    FROM customer_totals
)
SELECT c.first_name, c.last_name, t.total,
       ROUND(t.total - a.avg_total, 2) AS above_average
FROM customer_totals t
JOIN customers c ON c.id = t.customer_id
CROSS JOIN average a
WHERE t.total > a.avg_total
ORDER BY t.total DESC;
```

```
+------------+-----------+-------+---------------+
| first_name | last_name | total | above_average |
+------------+-----------+-------+---------------+
| Alice      | Johnson   | 58.95 |         19.40 |
| Dave       | Brown     | 45.96 |          6.41 |
| Frank      | Miller    | 41.96 |          2.41 |
+------------+-----------+-------+---------------+
```

4 **A CTE joined to itself (2m).** "A CTE is read like a table, so it can be read twice, under two aliases: this month next to last month." Point at January's NULLs: the LEFT JOIN keeps a month with no previous month. Say: "after the break, LAG does this in one line".

```sql
WITH monthly AS (
    SELECT MONTH(o.order_date)                 AS month_num,
           SUM(oi.quantity * oi.price_at_time) AS revenue
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.id
    GROUP BY MONTH(o.order_date)
)
SELECT cur.month_num,
       cur.revenue,
       prev.revenue                AS previous_month,
       cur.revenue - prev.revenue  AS change_amount
FROM monthly cur
LEFT JOIN monthly prev ON prev.month_num = cur.month_num - 1
ORDER BY cur.month_num;
```

```
+-----------+---------+----------------+---------------+
| month_num | revenue | previous_month | change_amount |
+-----------+---------+----------------+---------------+
|         1 |   61.95 |           NULL |          NULL |
|         2 |   51.95 |          61.95 |        -10.00 |
|         3 |   33.47 |          51.95 |        -18.48 |
|         4 |   38.97 |          33.47 |          5.50 |
|         5 |   31.97 |          38.97 |         -7.00 |
|         6 |   18.98 |          31.97 |        -12.99 |
+-----------+---------+----------------+---------------+
```

5 **Gone after the semicolon (1m).** "The CTE belongs to its statement. The next statement cannot see it."

```
WITH monthly AS (
    SELECT MONTH(order_date) AS month_num, COUNT(*) AS orders
    FROM orders
    GROUP BY MONTH(order_date)
)
SELECT MAX(orders) AS busiest FROM monthly;
SELECT * FROM monthly;
```

```
+---------+
| busiest |
+---------+
|       3 |
+---------+
ERROR 1146 (42S02): Table 'bookstore.monthly' doesn't exist
```

Fix: repeat the WITH, or, if many scripts need it, make it a view (Session 9).

6 **Recursion in rounds (3m).** Draw the rounds on the board as you type: anchor 1; round 1 reads 1, makes 2; ... round 5 reads 5, the stop condition fails, nothing comes out, done. "Each round sees only the rows the previous round made."

```
WITH RECURSIVE numbers AS (
    SELECT 1 AS n            -- anchor: the first row
    UNION ALL
    SELECT n + 1             -- recursive member: next row from the last one
    FROM numbers
    WHERE n < 5              -- stop condition
)
SELECT n FROM numbers;
```

```
+------+
| n    |
+------+
|    1 |
|    2 |
|    3 |
|    4 |
|    5 |
+------+
```

7 **A calendar that fills the gaps (2m).** "Nobody ordered in July, so GROUP BY over orders cannot show July. Generate the months first, then hang the orders on them with a LEFT JOIN."

```
WITH RECURSIVE months AS (
    SELECT DATE('2026-01-01') AS month_start
    UNION ALL
    SELECT month_start + INTERVAL 1 MONTH
    FROM months
    WHERE month_start < '2026-07-01'
)
SELECT DATE_FORMAT(m.month_start, '%Y-%m') AS month,
       COUNT(o.id)                         AS orders
FROM months m
LEFT JOIN orders o
       ON o.order_date >= m.month_start
      AND o.order_date <  m.month_start + INTERVAL 1 MONTH
GROUP BY m.month_start
ORDER BY m.month_start;
```

```
+---------+--------+
| month   | orders |
+---------+--------+
| 2026-01 |      2 |
| 2026-02 |      3 |
| 2026-03 |      2 |
| 2026-04 |      2 |
| 2026-05 |      2 |
| 2026-06 |      1 |
| 2026-07 |      0 |
+---------+--------+
```

COUNT(o.id) counts matched orders only, so July is 0. The date range in ON is the safe way to say "in this month".

8 **The org chart, top down (3m).** "Anchor: the person with no manager. Each round: the people whose manager is someone from the round before. The data itself is the stop condition." Point at the CAST in the anchor: the anchor fixes every column's width.

```
WITH RECURSIVE org AS (
    SELECT id, first_name, last_name, department, 1 AS level,
           CAST(first_name AS CHAR(200)) AS path
    FROM employees
    WHERE manager_id IS NULL
    UNION ALL
    SELECT e.id, e.first_name, e.last_name, e.department, o.level + 1,
           CONCAT(o.path, ' > ', e.first_name)
    FROM employees e
    JOIN org o ON e.manager_id = o.id
)
SELECT CONCAT(REPEAT('    ', level - 1), first_name, ' ', last_name) AS org_ch
       department
FROM org
ORDER BY path;
```

```
+---------------------------+------------+
| org_chart                 | department |
+---------------------------+------------+
| Maria Santos              | management |
|     Daniel Reyes          | management |
|         Marcus Hill       | inventory  |
|             Sofia Alvarez | inventory  |
|         Priya Nair        | sales      |
|             Ben Carter    | sales      |
|             Lena Park     | sales      |
|             Tom Becker    | sales      |
|     James Okafor          | support    |
|         Nina Kowalski     | support    |
+---------------------------+------------+
```

9 **A chain of command, bottom up (1m).** "Swap the join and it climbs: who is this person's manager?"

```
WITH RECURSIVE chain AS (
    SELECT id, first_name, last_name, manager_id, 0 AS steps_up
    FROM employees
    WHERE id = 7
    UNION ALL
    SELECT e.id, e.first_name, e.last_name, e.manager_id, c.steps_up + 1
    FROM employees e
    JOIN chain c ON e.id = c.manager_id
)
SELECT steps_up, CONCAT(first_name, ' ', last_name) AS person
FROM chain
ORDER BY steps_up;
```

```
+----------+---------------+
| steps_up | person        |
+----------+---------------+
|        0 | Sofia Alvarez |
|        1 | Marcus Hill   |
|        2 | Daniel Reyes  |
|        3 | Maria Santos  |
+----------+---------------+
```

10 **Two errors on purpose (1m).** "First I forget RECURSIVE, then I forget the stop condition." Read both numbers aloud.

```
WITH org AS (
    SELECT id, 1 AS level FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.id, o.level + 1 FROM employees e JOIN org o ON e.manager_id = o.i
)
SELECT * FROM org;
WITH RECURSIVE numbers AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM numbers
)
SELECT COUNT(*) FROM numbers;
```

```
ERROR 1146 (42S02): Table 'bookstore.org' doesn't exist
ERROR 3636 (HY000): Recursive query aborted after 1001 iterations. Try increas
```

Raising `cte_max_recursion_depth` is the fix only for a series that really needs more than 1000 rounds; here the fix is a WHERE in the recursive member.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Expecting a CTE to persist.** "I made monthly and now it is gone." Coach: "Which statement was it written in? A CTE is part of one statement; for something everyone reuses, that is a view (Session 9)."
- **Reading the stop condition as a row filter.** Learners expect `WHERE n < 5` to stop before 5. Coach with the rounds on the board: "Which row does the round read? 4 passes and makes 5; 5 fails and makes nothing." (Self-check Q2.)
- **An anchor that starts everywhere.** No WHERE in the anchor: 30 rows for 10 employees, no error. Coach: "How many starting points did you give it? Compare COUNT(*) with COUNT(DISTINCT id)." (Self-check Q5, the lesson's Common Mistake 5.)

GUIDED LAB

File: learners open `labs/module-08/l5_1-ctes/staff_tree_starter.sql`; your reference is `labs/module-08/l5_1-ctes/staff_tree_solution.sql`. Recursion on new questions: the twelve months of the year with zero months, then every employee's division (the level-2 manager they work under), carried down the tree.

- **SOURCE it as shipped (2m):** `SOURCE labs/module-08/l5_1-ctes/staff_tree_starter.sql`. Everyone should see one month, a tree where every division says `(owner)`, and a TODO line.
- **TODO 1 together (3m):** the stop condition. Ask "which round makes month 12, and which round stops?" before anyone edits.
- **TODO 2 in pairs (7m):** each round copies the division from the manager's row, but the owner has none, so nobody ever gets one. The fix is `COALESCE(o.division, e.first_name)`: level-2 people start a division, everyone below inherits it. Circulate for edits to the anchor instead of the recursive member.
- **Expected:** TODO 1 twelve rows, 5, 5, 3, 3, 3, 2, then six zeros; TODO 2 Daniel's division of 7 (Daniel, Marcus, Priya, Ben, Lena, Sofia, Tom) and James's of 2; TODO 3 (home or fast finishers) Daniel 7 people 310500.00, James 2 people 88500.00.
- **Debrief (1m):** "The anchor decided where to start and how wide the division column is; the recursive member decided how to get from one round to the next. Nothing in the query says how deep the company is."
- **Fast finishers:** TODO 3, then `labs/module-08/l5_1-ctes/category_steps_starter.sql` (non-recursive: two CTEs, and one CTE read three times).

START THE LESSON PROJECT

The lesson project is **The Staff and Sales Briefing**, `org_report.sql`: four commented queries in one script. In the room: Steps 1 and 3 (10 minutes), the two you just saw in the demo.

- Step 1: create `org_report.sql` in the course folder with `USE bookstore;` and the indented org chart, with the level as a number.
- Step 3: the chain of command for Nina (id 9); change the id to 7 and see Sofia's chain from the demo.
- **Done in the room** = the file runs with SOURCE and prints the chart (10 people, 4 levels) and one chain.
- **Goes home:** Step 2 (team size and payroll for every manager: an anchor that starts everywhere, on purpose), Step 4 (the sales calendar: a recursive CTE plus two ordinary ones), Step 5 (run the whole file).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Can the main statement read a CTE twice? **Answer:** yes: the demo's average step reads customer_totals, and the month comparison joins monthly to itself.
- Which part of a recursive CTE runs exactly once? **Answer:** the anchor.

- What stops the org chart's recursion? **Answer:** the data: in the last round nobody has Tom, Lena, Ben or Sofia as a manager, the join finds nothing, no rows come out.
- When would you raise `cte_max_recursion_depth`? **Answer:** only for a series that really needs more than 1000 rounds, for your session only, and back to DEFAULT afterward; never to silence a missing stop condition.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** B) Only while that one statement runs; **Q2** `2,4,8,16,32` (one row, column `series`); **Q3** A) `ERROR 1146 (42S02): Table 'bookstore.org' doesn't exist`; **Q4** C) `JOIN chain c ON e.id = c.manager_id`; **Q5** D) The anchor has no WHERE, so every employee starts a walk of their own; **Q6** GROUP BY over orders can only show months that have orders; the recursive calendar has every month, and the LEFT JOIN keeps the empty ones (July shows 0). Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: draw each round as a row on paper before running anything; for the lab, TODO 1 only, then read the solution's TODO 2 line by line with a partner. For learners shaky on joins (W2 to W4 missed), the refresher lab first.

Stretch: the lesson's Exercise 4 (the shape of the company) and the Bonus Challenge (a cycle on purpose inside a transaction, then a FIND_IN_SET guard). Ask them what MariaDB would do with the same runaway query.

5.2 Window Functions

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain how OVER() differs from GROUP BY, and split a window with PARTITION BY
- Number and rank rows with ROW_NUMBER, RANK and DENSE_RANK, and pick the right one when there are ties
- Build running totals and moving averages, and control them with a frame (ROWS BETWEEN ...)
- Compare each row with the one before or after it using LAG and LEAD
- Return the top N rows per group by filtering a window result in a CTE, and explain why WHERE cannot do it (error 3593)

TIMING IN THE ROOM

10m	Frame: GROUP BY squashes, OVER keeps; partition, order, frame; windows run late
25m	Live demo: OVER (), PARTITION BY, three ranks, running total and frame, LAG/LEAD, top N per group, 3593, 1064, the RANGE trap
20m	Guided lab: `leaderboards_starter.sql`
15m	Start the lesson project: `sales_dashboard.sql` Steps 1 and 2

MENTAL MODEL TO SAY

The two columns on the board: "GROUP BY answers 'what is the total?' by squashing each group into one row. A **window function** computes the same kind of number across a set of rows, the **window**, and hands it back to **every** row. Eleven books in, eleven books out, each next to the average."

Then the three parts of OVER, written one under the other: **PARTITION BY** (one window per group), **ORDER BY** (an order inside each window: needed for ranks, running totals, LAG), the **frame** (which neighbors count: ROWS BETWEEN 2 PRECEDING AND CURRENT ROW).

Last, the clause order as a vertical list: FROM/JOIN → WHERE → GROUP BY/HAVING → **window functions** → SELECT list → ORDER BY/LIMIT. "Windows are computed late. That is why WHERE cannot filter on them, and why the fix is one more step: a CTE."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

The lesson's own examples on the freshly loaded bookstore. Predictions first on steps 3 and 9.

1. **One number, or every row (2m).** "AVG gives one number and the eleven books are gone. Add OVER (), and it hands the same average back to every row."

```sql
SELECT ROUND(AVG(price), 2) AS avg_price FROM books;
SELECT title,
       price,
       ROUND(AVG(price) OVER (), 2)         AS avg_price,
       ROUND(price - AVG(price) OVER (), 2) AS vs_average
FROM books
ORDER BY price DESC, title;
```

```
+-----------+
| avg_price |
+-----------+
|     10.76 |
+-----------+
+------------------------+-------+-----------+------------+
| title                  | price | avg_price | vs_average |
+------------------------+-------+-----------+------------+
| Lord of the Rings      | 15.99 |     10.76 |       5.23 |
| Dune                   | 14.99 |     10.76 |       4.23 |
| To Kill a Mockingbird  | 11.99 |     10.76 |       1.23 |
| The Hobbit             | 10.99 |     10.76 |       0.23 |
| Fahrenheit 451         | 10.49 |     10.76 |      -0.27 |
| 1984                   |  9.99 |     10.76 |      -0.77 |
| Brave New World        |  9.49 |     10.76 |      -1.27 |
| The Catcher in the Rye |  8.99 |     10.76 |      -1.77 |
| The Great Gatsby       |  8.99 |     10.76 |      -1.77 |
| Animal Farm            |  8.49 |     10.76 |      -2.27 |
| Pride and Prejudice    |  7.99 |     10.76 |      -2.77 |
+------------------------+-------+-----------+------------+
```

2 **A window per department (3m).** "PARTITION BY splits like GROUP BY, without collapsing. Every sales row sees the sales average." Ask why sales comes first: department is an ENUM, sorted in declared order.

```sql
SELECT first_name,
       department,
       salary,
       ROUND(AVG(salary) OVER (PARTITION BY department), 2) AS dept_avg,
       COUNT(*) OVER (PARTITION BY department)              AS dept_size
FROM employees
ORDER BY department, salary DESC;
```

```
+------------+------------+----------+----------+-----------+
| first_name | department | salary   | dept_avg | dept_size |
+------------+------------+----------+----------+-----------+
| Priya      | sales      | 46000.00 | 39625.00 |         4 |
| Lena       | sales      | 39500.00 | 39625.00 |         4 |
| Tom        | sales      | 38000.00 | 39625.00 |         4 |
| Ben        | sales      | 35000.00 | 39625.00 |         4 |
| Marcus     | inventory  | 47000.00 | 42000.00 |         2 |
| Sofia      | inventory  | 37000.00 | 42000.00 |         2 |
| Maria      | management | 92000.00 | 80000.00 |         2 |
| Daniel     | management | 68000.00 | 80000.00 |         2 |
| James      | support    | 52000.00 | 44250.00 |         2 |
| Nina       | support    | 36500.00 | 44250.00 |         2 |
+------------+------------+----------+----------+-----------+
```

3 **Three ways to number (4m).** Predict first: "Dune and The Hobbit both sold 3. What does each function give them, and the next book?" Then run. Write 1, 2, 3, 4 / 1, 2, 2, 4 / 1, 2, 2, 3 on the board.

```sql
WITH copies AS (
    SELECT b.title, SUM(oi.quantity) AS sold
    FROM books b
    JOIN order_items oi ON oi.book_id = b.id
    GROUP BY b.id, b.title
)
SELECT title,
       sold,
       ROW_NUMBER() OVER (ORDER BY sold DESC, title) AS row_num,
       RANK()       OVER (ORDER BY sold DESC)        AS rank_num,
       DENSE_RANK() OVER (ORDER BY sold DESC)        AS dense_num
FROM copies
ORDER BY sold DESC, title;
```

```
+----------------------+------+---------+----------+-----------+
| title                | sold | row_num | rank_num | dense_num |
+----------------------+------+---------+----------+-----------+
| 1984                 |    4 |       1 |        1 |         1 |
| Dune                 |    3 |       2 |        2 |         2 |
| The Hobbit           |    3 |       3 |        2 |         2 |
| Animal Farm          |    2 |       4 |        4 |         3 |
| Fahrenheit 451       |    2 |       5 |        4 |         3 |
| Lord of the Rings    |    2 |       6 |        4 |         3 |
| Pride and Prejudice  |    2 |       7 |        4 |         3 |
| To Kill a Mockingbird |    2 |       8 |        4 |         3 |
| Brave New World      |    1 |       9 |        9 |         4 |
+----------------------+------+---------+----------+-----------+
```

The aliases end in `_num` because `rank` is a reserved word (step 9).

4 **Running total and moving average (4m).** "ORDER BY inside OVER turns SUM into a running total. A frame, ROWS BETWEEN 2 PRECEDING AND CURRENT ROW, makes a three-month moving average." January's average is January alone.

```
WITH monthly AS (
    SELECT MONTH(o.order_date)                 AS month_num,
           SUM(oi.quantity * oi.price_at_time) AS revenue
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.id
    GROUP BY MONTH(o.order_date)
)
SELECT month_num,
       revenue,
       SUM(revenue) OVER (ORDER BY month_num) AS running_total,
       ROUND(AVG(revenue) OVER (ORDER BY month_num
                                ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2)
FROM monthly
ORDER BY month_num;
```

```
+-----------+---------+---------------+--------------+
| month_num | revenue | running_total | moving_avg_3 |
+-----------+---------+---------------+--------------+
|         1 |   61.95 |         61.95 |        61.95 |
|         2 |   51.95 |        113.90 |        56.95 |
|         3 |   33.47 |        147.37 |        49.12 |
|         4 |   38.97 |        186.34 |        41.46 |
|         5 |   31.97 |        218.31 |        34.80 |
|         6 |   18.98 |        237.29 |        29.97 |
+-----------+---------+---------------+--------------+
```

5 **LAG and LEAD (3m).** "Before the break this took a self-join. LAG reads the previous row, LEAD the next." Compare the change column with the demo's step 4: identical.

```
WITH monthly AS (
    SELECT MONTH(o.order_date)                AS month_num,
           SUM(oi.quantity * oi.price_at_time) AS revenue
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.id
    GROUP BY MONTH(o.order_date)
)
SELECT month_num,
       revenue,
       LAG(revenue)  OVER (ORDER BY month_num)           AS prev_month,
       revenue - LAG(revenue) OVER (ORDER BY month_num)  AS change_amount,
       LEAD(revenue) OVER (ORDER BY month_num)           AS next_month
FROM monthly
ORDER BY month_num;
```

```
+-----------+---------+------------+---------------+------------+
| month_num | revenue | prev_month | change_amount | next_month |
+-----------+---------+------------+---------------+------------+
|         1 |   61.95 |       NULL |          NULL |      51.95 |
|         2 |   51.95 |      61.95 |        -10.00 |      33.47 |
|         3 |   33.47 |      51.95 |        -18.48 |      38.97 |
|         4 |   38.97 |      33.47 |          5.50 |      31.97 |
|         5 |   31.97 |      38.97 |         -7.00 |      18.98 |
|         6 |   18.98 |      31.97 |        -12.99 |       NULL |
+-----------+---------+------------+---------------+------------+
```

6 **Top 1 per department (3m).** "Number the rows inside each department in a CTE, keep number 1 outside."

```
WITH ranked AS (
    SELECT first_name,
           last_name,
           department,
           salary,
           ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS
    FROM employees
)
SELECT department, first_name, last_name, salary
FROM ranked
WHERE pos = 1
ORDER BY salary DESC;
```

```
+------------+------------+-----------+----------+
| department | first_name | last_name | salary   |
+------------+------------+-----------+----------+
| management | Maria      | Santos    | 92000.00 |
| support    | James      | Okafor    | 52000.00 |
| inventory  | Marcus     | Hill      | 47000.00 |
| sales      | Priya      | Nair      | 46000.00 |
+------------+------------+-----------+----------+
```

7 **Why not WHERE? (2m).** "Windows are computed after WHERE, GROUP BY and HAVING. WHERE cannot see them." Point at the stray quote at the end of the message: MySQL's typo, not ours.

```sql
SELECT first_name, department, salary
FROM employees
WHERE ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) = 1;
```

```
ERROR 3593 (HY000): You cannot use the window function 'row_number' in this c
```

8 **A reserved word (1m).** "Old tutorials name the column rank."

```sql
SELECT title, price,
       RANK() OVER (ORDER BY price DESC) AS rank
FROM books;
```

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual tha
FROM books' at line 2
```

Fix: `rank_num`, or backticks. MariaDB still accepts `AS rank`, which is why the habit spreads.

9 **A running total that jumps (silent) (3m).** "No error, but look at the two 8.99 books: same running total. The default frame is RANGE, which adds tied rows in one step. Say ROWS and make the order unique."

```
SELECT title, price,
       SUM(price) OVER (ORDER BY price) AS running_total
FROM books
ORDER BY price, title
LIMIT 5;
SELECT title, price,
       SUM(price) OVER (ORDER BY price, title
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS r
FROM books
ORDER BY price, title
LIMIT 5;
```

```
+------------------------+-------+---------------+
| title                  | price | running_total |
+------------------------+-------+---------------+
| Pride and Prejudice    |  7.99 |          7.99 |
| Animal Farm            |  8.49 |         16.48 |
| The Catcher in the Rye |  8.99 |         34.46 |
| The Great Gatsby       |  8.99 |         34.46 |
| Brave New World        |  9.49 |         43.95 |
+------------------------+-------+---------------+
+------------------------+-------+---------------+
| title                  | price | running_total |
+------------------------+-------+---------------+
| Pride and Prejudice    |  7.99 |          7.99 |
| Animal Farm            |  8.49 |         16.48 |
| The Catcher in the Rye |  8.99 |         25.47 |
| The Great Gatsby       |  8.99 |         34.46 |
| Brave New World        |  9.49 |         43.95 |
+------------------------+-------+---------------+
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Filtering a window in WHERE (3593, or 1054 with the alias).** Coach with the clause list on the board: "Where in this list does WHERE run? Where do windows run?" Then the two-step CTE pattern.
- **Forgetting PARTITION BY.** "Top 2 per category" returns two books in total. Coach: "Where does the numbering restart? Nowhere: tell it to restart per category." (The lab's TODO 2 is built on this.)
- **Trusting a running total with ties.** No error, two rows with the same total. Coach: "Which frame did you get by default? RANGE. Say ROWS, and make the order unique."

GUIDED LAB

File: `labs/module-08/l5_2-window-functions/leaderboards_starter.sql`; reference `labs/module-08/l5_2-window-functions/leaderboards_solution.sql`. New leaderboards on

the bookstore: customers by copies bought (the data has ties), the two cheapest books per category, and each salary's share of its department's payroll.

- **TODO 1 with a prediction (6m):** before adding RANK and DENSE_RANK, every pair writes what Dave and Frank (both 4 copies) and Bob and Grace (both 3) will get from each function.
- **Expected TODO 1:** Dave 4 2 2 2 and Frank 4 3 2 2 (copies, row, rank, dense); Eve last with rank 6 and dense rank 4.
- **TODO 2 (7m):** without PARTITION BY only two books come back in total; with it, two per category (ten rows). Ask why Pride and Prejudice appears twice (it is in Classic and Fiction).
- **TODO 3 (5m):** `OVER ()` gives the company payroll on every row; `PARTITION BY department` gives sales 158500.00 and inventory 84000.00, then the percentage.
- **Debrief (2m):** "One query, three kinds of window: an order (ranks), a partition plus an order (top N per group), a partition alone (a group total on every row)."
- **Fast finishers:** `labs/module-08/l5_2-window-functions/order_trends_starter.sql` (a running total over orders, LAG per customer, every customer's first order).

START THE LESSON PROJECT

The lesson project is **The Sales Dashboard**, `sales_dashboard.sql`. In the room: Steps 1 and 2 (15 minutes).

- Step 1: revenue so far: the monthly CTE, a running total with an explicit ROWS frame, and `pct_done` (running total over `SUM(revenue) OVER ()`).
- Step 2: month over month: `LAG` and a three-month moving average sharing one named window (`WINDOW w AS (ORDER BY month_num)`, then `OVER (w ROWS BETWEEN ...)`).
- **Done in the room** = both queries print, the running total ends at 237.29 and `pct_done` at 100.0.
- **Goes home:** Step 3 (best-seller leaderboard with DENSE_RANK), Step 4 (top customer per city with ROW_NUMBER), Step 5 (run the whole file).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Does the ORDER BY inside OVER sort the result? **Answer:** no, it orders the rows for the function; always end with the query's own ORDER BY.
- Why does January's three-month average equal January? **Answer:** there are no earlier rows in its frame.
- Which function do you want for "every book at the top two sales levels"? **Answer:** DENSE_RANK (ties included, no gaps).
- A WHERE that filters a plain column, like `WHERE is_active`: does it break a window? **Answer:** no: it runs first and decides which rows enter the windows (the lesson's Exercise 1).

Workbook self-check, items Q7 to Q12.

Answers: **Q7** C) 10; **Q8** `RANK` 4, `DENSE_RANK` 3; **Q9** B) Compute the rank in a CTE, then filter on it in the outer query; **Q10** D) The default frame is RANGE: rows with the same ORDER BY value are added in one step; **Q11** Two rows: order 3, 2026-03-10, next_order 2026-03-28; order 7, 2026-03-28, next_order `NULL`; **Q12** A) `ROW_NUMBER()` with a tie-breaker, e.g. `ORDER BY salary DESC, id`. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: keep the three-function table (1, 2, 3, 4 / 1, 2, 2, 4 / 1, 2, 2, 3) on the board. For the project, Step 1 only, typed from the lesson page, then change one thing (the frame) and watch what moves.

Stretch: the lesson's Exercise 4 (cheapest book per author with two windows) and the NTILE bonus challenge; ask them to rewrite the 5.1 self-join with LAG and compare the plans with EXPLAIN (a preview of Session 9).

2:35 Debug Clinic: Both Lessons

15 min

HOW IT RUNS

- **Pairs (8m):** SOURCE `labs/module-08/l5_1-ctes/debug_ctes.sql` and `labs/module-08/l5_2-window-functions/debug_windows.sql`. For each ERROR line: the number, the cause in one sentence, the fix in one line, written in the workbook.
- **Whole room (5m):** one volunteer per error reads the line and the fix. Then everyone rewrites `debug_windows.sql` query A with the CTE pattern.
- **Close (2m):** "Six errors, three rules: RECURSIVE when a CTE names itself; a stop condition that becomes false; windows are filtered one step later."

WHAT THE HANDOUTS PRINT, AND THE FIXES

```
ERROR 1146 (42S02): Table 'bookstore.countdown' doesn't exist
ERROR 3636 (HY000): Recursive query aborted after 1001 iterations. Try increasing
ERROR 1406 (22001): Data too long for column 'path' at row 1
```

```
ERROR 3593 (HY000): You cannot use the window function 'row_number' in this conte
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that c
FROM books
ORDER BY price DESC
LIMIT 3' at line 1
ERROR 6037 (HY000): 'QUALIFY clause' can be used only if the hypergraph optimizer
```

- **A, 1146:** `WITH countdown` refers to itself: add RECURSIVE.
- **B, 3636:** no stop condition: `WHERE d < '2026-12-31'` in the recursive member (365 rows).
- **C, 1406:** the anchor's `'Maria'` made a five-character column: `CAST('Maria' AS CHAR(200))` (or `CAST(first_name AS CHAR(200))`).
- **3593:** a window in WHERE: number the rows in a CTE, keep `pos <= 3` outside.
- **1064:** `rank` is reserved: `AS rank_num`.
- **6037:** QUALIFY needs the hypergraph optimizer on MySQL 9.7; use the CTE pattern (it works on every server).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Learners answer in the workbook's Exit Ticket box or on a sticky note. Sort into "got it" and "not yet" before Session 9: question 3's pile tells you whether to open Session 9's warm-up with the CTE-filter pattern.

1. Write the skeleton of a recursive CTE that counts from 1 to 10, and label the anchor, the recursive member and the stop condition. **Answer:** `WITH RECURSIVE s AS (SELECT 1 AS n /* anchor */ UNION ALL SELECT n + 1 FROM s /* recursive member */ WHERE n < 10 /* stop */) SELECT n FROM s;`
2. `GROUP BY department` versus `PARTITION BY department`: what is the difference in the rows you get back? **Answer:** GROUP BY returns one row per department; PARTITION BY keeps every employee row and adds the department's figure to each.
3. Why can't WHERE filter on `ROW_NUMBER()`, and what do you do instead? **Answer:** window functions are computed after WHERE (error 3593); compute the number in a CTE and filter it in the outer query.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** `org_report.sql` Steps 2, 4 and 5; `sales_dashboard.sql` Steps 3 to 5. Each ends with SOURCE running the whole file without an error.
- **Lesson exercises.** 5.1: Exercises 1 to 3 (above-average sellers, a price ladder, Daniel's team level by level); 4 is a stretch. 5.2: Exercises 1 to 3 (pay rank by department, each order's share, who comes back and when); 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages (answers in the Answer Key).
- **Journal:** both entries (a chain in your own life as a recursive CTE; a ranking you know and its tie rule).
- **Labs not finished in class:** `labs/module-08/l5_1-ctes/category_steps_starter.sql`, `labs/module-08/l5_2-window-functions/order_trends_starter.sql`, and the debug handouts if the clinic was cut.
- **Before the Next Lesson (from 5.2):** run `sales_dashboard.sql` from a fresh session; rewrite one query with a named WINDOW.

BRING LIST FOR SESSION 9

- A laptop that printed `READY for Part 2`, and `bookstore.sql` in the course folder: Lesson 6.1 reloads it and then generates a 100,000-row table (a few seconds; nothing to download).
- The course folder with `labs/` (Session 9's starters are in `labs/module-09/`), `org_report.sql` and `sales_dashboard.sql`.
- The workbook and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the environment check; 5.1 demo steps 6 to 8; the 5.2 ranking prediction and top N per group; the staff-tree lab through TODO 2; the leaderboards lab through TODO 2.
- **Cut first (saves 15m):** the debug clinic: hand out both debug files with the README's expected errors as homework.
- **Cut second (saves 10m):** the 5.1 project start: read Step 1's goal aloud and send all five steps home.
- **Cut third (saves 3m):** 5.2 demo step 9 (the RANGE trap): self-check Q10 and the lesson's Common Mistake 4 cover it.

RUNNING SHORT: WHAT TO ADD

- The lesson 5.1 Bonus Challenge live: inside a transaction, make Maria report to Nina, run the org chart anchored on id 1 (error 3636), then ROLLBACK.
- Rewrite the 5.1 month comparison with LAG and put both versions side by side.
- The NTILE bonus: three price tiers, and which tier gets the extra books.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 9: Indexes and Views

How to teach indexes and views in one session: measure a slow query on 100,000 rows, add the index the plan points at and measure again, learn the habits that hide a column from its index, then save the best queries as views, make some of them safe to update, and use one as a security boundary. A clinic ties them together: a view is exactly as fast as its query.

2 lessons | 3 hrs session | Lessons 6.1–6.2 | Session 9 of 14 · Part 2

HOW TO USE THIS GUIDE

- Same rhythm as every session: frame → live demo (you type, they follow) → guided lab → start the lesson's own project. Exercises, quizzes, journals and the rest of each project go home.
- Every SQL block below was run on a freshly loaded bookstore on MySQL 9.7 to produce the output printed under it. EXPLAIN shows ESTIMATES (cost, rows) and EXPLAIN ANALYZE real times: learners' numbers differ; the plan's shape (Table scan, Index lookup, range scan) must not.
- Keep `lesson_14.html` (6.1) and `lesson_15.html` (6.2) open for the plan-word table and the updatable-view rules. Labs live in `labs/module-09/`.
- **Reloads today.** Lesson 6.1 starts from a fresh bookstore and adds `sales_log`; Lesson 6.2 starts by reloading `bookstore.sql`, which removes it. Say so at the break: everyone types `SOURCE bookstore.sql` before the 6.2 demo. The labs build and drop their own practice tables.
- Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (6.1, 6.2) match the course site.

BEFORE THE SESSION

- ☐ **Files to hand out:** `labs/module-09/` without the `*_solution.sql` files; the workbook.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-09` (10 files, a few seconds; two lines show a WARN for warning 1739, which is the point of that lab).
- ☐ **Projector machine:** a freshly loaded bookstore, the client at 18 pt, `lesson_14.html` open at the sales_log setup (you paste the generator from there). Run the generator once beforehand to see it takes about a second on your machine.
- ☐ **Board, before learners arrive:** the loop "measure → index → measure again"; a three-level B-tree sketch (root, branches, leaf pages); the plan words "Table scan · Index lookup · Index range scan · Covering · Rows

fetched before execution"; for 6.2 a window in a wall.

- ☐ **Print or share:** the workbook; sticky notes for the exit ticket.

KNOWN TRAPS TODAY

SYMPTOM	CAUSE	FIX
EXPLAIN prints a wide box instead of the tree	The statement ended with `;` instead of `\G`	End with `\G`; or `EXPLAIN FORMAT=TRADITIONAL` for the table.
`ERROR 1064` near `ANALYZE` on MariaDB	MariaDB has no EXPLAIN ANALYZE (and no FORMAT=TREE)	Use `ANALYZE SELECT ...` there; read the traditional table.
Learners' costs and times differ from the projector	Estimates come from sampled statistics; times from the machine	Compare the shape and the actual row counts, not the numbers.
The index exists, the plan still says Table scan	A function or a type conversion around the column, a leading %, or a tiny table	The lesson's "What stops an index" list, in order; then ANALYZE TABLE.
`ERROR 1050 ... already exists` when rerunning a view script	CREATE VIEW instead of CREATE OR REPLACE VIEW	CREATE OR REPLACE makes scripts rerunnable.
Views vanish	`SOURCE bookstore.sql` drops the whole database, views included	Keep them in `reporting_views.sql` and rerun it after a reload.
On MariaDB `ALTER INDEX ... INVISIBLE` is a 1064	MariaDB says `IGNORED`	`ALTER TABLE t ALTER INDEX i IGNORED;` there.

SESSION 9 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on 5.1 and 5.2; who finished org_report.sql and sales_dashboard.sql.
0:10	10	**6.1** Frame and mental model	The textbook index; a B-tree; what an index costs; measure, index, measure again.
0:20	25	**6.1** Live demo	Generate sales_log; EXPLAIN ANALYZE before and after; composite and unique indexes; three index killers; invisible indexes.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:45	15	**6.1** Guided lab	`labs/module-09/l6_1-indexes/plan_reading_starter.sql`: read a plan, add the index, read it again.
1:00	10	**6.1** Start the lesson project	`tune_sales.sql`: Steps 1 to 3 (reports A and B); Steps 4 to 6 go home.
1:10	10	Break	Leave the "measure → index → measure" loop on the board.
1:20	10	**6.2** Frame and mental model	A window in a wall: a saved SELECT, recomputed every time; three jobs for views.
1:30	20	**6.2** Live demo	Your first view; what MySQL saves; views over joins and aggregates; updatable views and WITH CHECK OPTION; a view as a security boundary.
1:50	20	**6.2** Guided lab	`labs/module-09/l6_2-views/report_views_starter.sql`: sales per author and orders by city as views.
2:10	15	**6.2** Start the lesson project	`reporting_views.sql`: Steps 1 to 3; Steps 4 to 6 go home.
2:25	25	Clinic: a view's plan, and the debug handouts	EXPLAIN on a merged view and on a grouped view; `debug_indexes.sql` and `debug_views.sql` in pairs.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 10.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 105 + 95 = 200 minutes for a solo learner; after the warm-up, the break, the clinic and the wrap-up the room has 135. Lesson 6.1 gets 60 minutes (reading plans is the hard, visual part, best done together); 6.2 gets 65 including the project start. The 25-minute clinic joins the two lessons: a view is exactly as fast as its query and the indexes under it.

Goes home (about 2 to 3 hours). `tune_sales.sql` Steps 4 to 6 (report C's composite index, the redundancy check, the index list); `reporting_views.sql` Steps 4 to 6; each lesson's Exercises 1 to 3 (4 is a stretch); both Quick Quizzes; both journal entries; `index_killers_starter.sql` and `guarded_views_starter.sql` if not reached.

Protect, in this order: the before-and-after EXPLAIN ANALYZE (the session's one unforgettable moment); the composite index and its column order; the plan_reading lab through TODO 2; WITH CHECK OPTION; the report_views lab through TODO 1. **Cut first:** the clinic's EXPLAIN part (keep the debug handouts as homework), then 6.1 demo steps 10 to 12, then the 6.2 security step (Lesson 7.4 covers accounts).

OPENING SEGMENT

0:00 Warm-up: CTEs and Window Functions — 10 min

SEQUENCE

- **Retrieval (7m).** Five questions on the workbook's Warm-up box, no notes, three minutes; then take them one by one. Run W1 and W2 on the projector after the answers.
- **Homework check (3m).** Hands up: who ran `org_report.sql` and `sales_dashboard.sql` from a fresh session without an error? One volunteer reads the dashboard's leaderboard: 1984 alone in first place. Collect questions for the break.

WARM-UP ANSWERS (W1 TO W5)

1. **W1** What does this print? **Answer:** 1,3,5,7: the round that reads 7 fails n < 7 and stops.
2. **W2** Which price_level do The Catcher in the Rye and The Great Gatsby (both 8.99) get, and Animal Farm after them? **Answer:** Both 2, Animal Farm 3: DENSE_RANK ties without a gap (RANK would give Animal Farm 4).
3. **W3** Why can't WHERE filter on ROW_NUMBER(), and what do you write instead? **Answer:** Windows are computed after WHERE (3593): number the rows in a CTE, filter in the outer query.
4. **W4** In a recursive CTE over employees, which join walks UP the hierarchy? **Answer:** e.id = cte.manager_id ("who is this person's manager?"); e.manager_id = cte.id walks down.
5. **W5** AVG(salary) with GROUP BY department versus AVG(salary) OVER (PARTITION BY department): how many rows from the 10 employees? **Answer:** 4 versus 10: GROUP BY collapses, OVER keeps every row.

W1 and W2's real output is in the Answer Key. W3 misses mean the CTE-filter pattern did not stick: it comes back today as a view that filters a window (the 6.2 worked example).

LESSON PLANS

6.1 Indexes & Query Optimization

60 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain what an index is, what it costs, and which indexes every bookstore table already has
- Read a query plan from EXPLAIN and measure a query with EXPLAIN ANALYZE
- Create single-column, composite and unique indexes, and choose the column order of a composite index
- Spot and rewrite the patterns that stop an index from being used: functions on the column, leading wildcards, type mismatches
- List, rename, hide and drop indexes, and fix the errors MySQL raises while you do it

TIMING IN THE ROOM

10m	Frame: the textbook index; B-tree; costs; the measure → index → measure loop
25m	Live demo: sales_log; before/after; composite; unique; two killers; invisible; a silent duplicate; 1553
15m	Guided lab: `plan_reading_starter.sql`
10m	Start the lesson project: `tune_sales.sql` Steps 1 to 3

MENTAL MODEL TO SAY

Hold up any textbook: "To find 'foreign keys' you do not read the whole book. You look the term up in a sorted list at the back and jump to the pages. An **index** is that list: a sorted copy of one or more columns, each value with a pointer to its row. Without it, MySQL has one way to answer WHERE: read every row. That is a **full table scan**."

Point at the B-tree sketch: "Sorted pages in a shallow tree: three or four levels for a million values. The primary key is special: the rows themselves are stored in its order."

Then the price, ticked off on fingers: space, slower writes (every INSERT updates every index), more choices for the optimizer. "The goal is not 'index everything'. It is: measure the slow query, index what the plan points at, measure again."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

The lesson's own examples on a freshly loaded bookstore; learners generate the same `sales_log` and follow. EXPLAIN ANALYZE times are from the run that produced this guide.

1. **A table big enough to notice (3m).** "Eleven books never make a query slow. This builds the lesson's 100,000-row sales log: a CTE and a cross join, every value computed from a running number, so

everyone gets the same rows." Paste it from the lesson page; while it runs (about a second), explain ANALYZE TABLE: fresh statistics for the optimizer.

```
CREATE TABLE sales_log (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    receipt_no  CHAR(7)     NOT NULL,
    customer_id INT         NOT NULL,
    book_id     INT         NOT NULL,
    sold_on     DATE        NOT NULL,
    quantity    INT         NOT NULL,
    channel     VARCHAR(10) NOT NULL
);
INSERT INTO sales_log (receipt_no, customer_id, book_id, sold_on, quantity, ch
WITH digits AS (
    SELECT 0 AS d UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL SELECT 3 UNI
    UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT
),
seq AS (
    SELECT 1 + a.d + 10 * b.d + 100 * c.d + 1000 * e.d + 10000 * f.d AS n
    FROM digits a
    CROSS JOIN digits b CROSS JOIN digits c CROSS JOIN digits e CROSS JOIN dig
)
SELECT
    LPAD(n, 7, '0'),                                -- receipt 0000001, 0000002
    1 + MOD(n * 7, 8),                              -- customers 1 to 8
    1 + MOD(n * 13, 11),                            -- books 1 to 11
    DATE '2020-01-01' + INTERVAL MOD(n * 37, 2400) DAY,
    1 + MOD(n, 3),                                  -- 1 to 3 copies
    ELT(1 + MOD(n, 4), 'web', 'store', 'phone', 'web')
FROM seq
ORDER BY n;
ANALYZE TABLE sales_log;
```

```
+---------------------+---------+----------+----------+
| Table               | Op      | Msg_type | Msg_text |
+---------------------+---------+----------+----------+
| bookstore.sales_log | analyze | status   | OK       |
+---------------------+---------+----------+----------+
```

2 **How big? (1m).**

```
SELECT COUNT(*) AS sales, MIN(sold_on) AS first_sale, MAX(sold_on) AS last_sal
FROM sales_log;
```

```
+--------+------------+------------+
| sales  | first_sale | last_sale  |
+--------+------------+------------+
| 100000 | 2020-01-01 | 2026-07-27 |
+--------+------------+------------+
```

3 **The indexes you already have (2m).** "Three automatic sources: every primary key, every UNIQUE, every foreign key." Point at fk_orders_customer: that is why Lesson 4.3's joins were fast.

```
SELECT TABLE_NAME, INDEX_NAME, COLUMN_NAME, NON_UNIQUE
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = 'bookstore'
  AND TABLE_NAME IN ('books', 'customers', 'orders', 'order_items')
ORDER BY TABLE_NAME, INDEX_NAME, SEQ_IN_INDEX;
```

```
+-------------+--------------------+-------------+------------+
| TABLE_NAME  | INDEX_NAME         | COLUMN_NAME | NON_UNIQUE |
+-------------+--------------------+-------------+------------+
| books       | fk_books_author    | author_id   |          1 |
| books       | PRIMARY            | id          |          0 |
| customers   | email              | email       |          0 |
| customers   | PRIMARY            | id          |          0 |
| order_items | fk_items_book      | book_id     |          1 |
| order_items | fk_items_order     | order_id    |          1 |
| order_items | PRIMARY            | id          |          0 |
| orders      | fk_orders_customer | customer_id |          1 |
| orders      | PRIMARY            | id          |          0 |
+-------------+--------------------+-------------+------------+
```

4 **Measure (3m).** "EXPLAIN ANALYZE runs the query and times every step. Read from the most indented line up: a table scan of 100,000 rows feeds a filter that keeps 42." Write the total time on the board.

```
EXPLAIN ANALYZE SELECT * FROM sales_log WHERE sold_on = '2025-03-15'\G
```

```
*************************** 1. row ***************************
EXPLAIN: -> Filter: (sales_log.sold_on = DATE'2025-03-15')  (cost=10078 rows=9
    -> Table scan on sales_log  (cost=10078 rows=99900) (actual time=0.134..13
```

On the run that produced this guide the total was 16.5 ms; yours will differ.

5 **Index, measure again (3m).** "One line instead of two: an Index lookup straight to the 42 rows." Compare with the number on the board.

```
CREATE INDEX idx_sales_sold_on ON sales_log (sold_on);
EXPLAIN ANALYZE SELECT * FROM sales_log WHERE sold_on = '2025-03-15'\G
```

```
*************************** 1. row ***************************
EXPLAIN: -> Index lookup on sales_log using idx_sales_sold_on (sold_on = DATE
```

Here 0.0779 ms. A drop of a hundred times or more is typical; the estimate (rows=42) is now exact.

6 **A composite index (3m).** "Sorted by customer, then by date within each customer, like a phone book. Customer 3 in March is one slice." Covering: COUNT(*) needs nothing but the indexed columns.

```
CREATE INDEX idx_sales_customer_date ON sales_log (customer_id, sold_on);
EXPLAIN SELECT COUNT(*) FROM sales_log
WHERE customer_id = 3
  AND sold_on BETWEEN '2025-03-01' AND '2025-03-31'\G
```

```
*************************** 1. row ***************************
EXPLAIN: -> Aggregate: count(0)  (cost=73.1 rows=1)
    -> Filter: ((sales_log.customer_id = 3) and (sales_log.sold_on between '2(
        -> Covering index range scan on sales_log using idx_sales_customer_dat
```

7 **A unique index (2m).** "An index plus a rule. A constant on a unique index is found while planning." Then the rule in action.

```
CREATE UNIQUE INDEX uq_sales_receipt ON sales_log (receipt_no);
EXPLAIN SELECT * FROM sales_log WHERE receipt_no = '0004242'\G
INSERT INTO sales_log (receipt_no, customer_id, book_id, sold_on, quantity, ch
VALUES ('0004242', 1, 1, '2026-07-28', 1, 'web');
```

```
*************************** 1. row ***************************
EXPLAIN: -> Rows fetched before execution  (cost=0..0 rows=1)
ERROR 1062 (23000): Duplicate entry '0004242' for key 'sales_log.uq_sales_rece
```

8 **Killer 1: a function around the column (3m).** "The index is sorted by sold_on, not by YEAR(sold_on)." Same question as a range on the bare column.

```
EXPLAIN SELECT * FROM sales_log
WHERE YEAR(sold_on) = 2025 AND MONTH(sold_on) = 3\G
EXPLAIN SELECT * FROM sales_log
WHERE sold_on >= '2025-03-01' AND sold_on < '2025-04-01'\G
```

```
*************************** 1. row ***************************
EXPLAIN: -> Filter: ((year(sales_log.sold_on) = 2025) and (month(sales_log.sol
    -> Table scan on sales_log  (cost=10078 rows=99900)
*************************** 1. row ***************************
EXPLAIN: -> Index range scan on sales_log using idx_sales_sold_on over ('2025-
```

9 **Killer 2: a number for a text column (1m).** "receipt_no is CHAR(7). 4242 is a number, so every row is converted." Fix: compare with '0004242'.

```
EXPLAIN SELECT * FROM sales_log WHERE receipt_no = 4242\G
```

```
*************************** 1. row ***************************
EXPLAIN: -> Filter: (sales_log.receipt_no = 4242)  (cost=10078 rows=9990)
    -> Table scan on sales_log  (cost=10078 rows=99900)
```

10 **Test a drop safely (2m).** "Before dropping an index on a busy table, hide it. The optimizer ignores it; writes still maintain it, so VISIBLE is instant."

```
ALTER TABLE sales_log ALTER INDEX idx_sales_sold_on INVISIBLE;
EXPLAIN SELECT * FROM sales_log WHERE sold_on = '2025-03-15'\G
ALTER TABLE sales_log ALTER INDEX idx_sales_sold_on VISIBLE;
```

```
*************************** 1. row ***************************
EXPLAIN: -> Filter: (sales_log.sold_on = DATE'2025-03-15')  (cost=10078 rows=4
    -> Table scan on sales_log  (cost=10078 rows=99900)
```

Back to a table scan: this query does need the index.

11 **Silent: an index you did not need (1m).** "No error, no warning, and pure overhead: the composite index already starts with customer_id."

```
CREATE INDEX idx_sales_customer ON sales_log (customer_id);
SHOW WARNINGS;
DROP INDEX idx_sales_customer ON sales_log;
```

```
Empty set
```

Then the one MySQL refuses: the foreign key's own index (next step).

12 **Refused: the foreign key's index (1m).**

```
DROP INDEX fk_orders_customer ON orders;
```

```
ERROR 1553 (HY000): Cannot drop index 'fk_orders_customer': needed in a foreig
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Reading the plan top down.** Learners report "it filters first". Coach: "Which line is indented most? That one runs first and feeds the line above."
- **Composite index in the wrong order.** `(sold_on, customer_id)` for "customer 3 in March". Coach: "Equality first, range last: which column do you compare with `=`?" (the lab's TODO 2).
- **Comparing numbers instead of shapes.** "My cost is 10083, yours is 10078." Coach: "Same shape? Then same plan. Estimates come from sampled statistics."

GUIDED LAB

File: `labs/module-09/l6_1-indexes/plan_reading_starter.sql`; reference `labs/module-09/l6_1-indexes/plan_reading_solution.sql`. A new 100,000-row `visit_log` (website visits), generated like the lesson's sales_log and dropped at the end of every run.

- **SOURCE it as shipped (2m):** about a second to build; every plan says Table scan, and the index list shows only PRIMARY.
- **TODO 1 with a prediction (4m):** before removing the `--`, each pair writes what the second plan will say. Then run.
- **TODO 2 in pairs (5m):** the composite index. Circulate for `(visited_on, customer_id)`: run it anyway, and ask why the plan is now an index scan of a different shape.
- **Expected:** 200 visits on June 1 through an Index lookup; customer 3 in June: 700 visits through a Covering index range scan on `idx_visits_customer_on`; TODO 3 Rows fetched before execution; four indexes at the end.
- **Debrief (2m):** "Which plan words did you see? Which line ran first?"
- **Fast finishers:** `labs/module-09/l6_1-indexes/index_killers_starter.sql`: four slow queries to rewrite, same answers, better plans.

START THE LESSON PROJECT

The lesson project is **Tune the Sales Reports**, `tune_sales.sql`: three slow reports, measured, indexed and measured again, with notes as comments. In the room: Steps 1 to 3 (10 minutes).

- Step 1: report A (the ten most recent web sales): already fast, through an index made for another query. Write that conclusion as a comment.

- Steps 2 and 3: report B with QUARTER() and YEAR() is a table scan; rewrite the dates as a range and compare the two plans.
- **Done in the room** = the file holds reports A and B, each with its before plan as a comment, and B's fixed plan shows an Index range scan.
- **Goes home:** Step 4 (report C's composite index on customer, book, date), Step 5 (is the old composite index redundant?), Step 6 (the final index list).

CHECKS FOR UNDERSTANDING

Ask these out loud:

- Why does nobody create an index on the primary key? **Answer:** the table is already stored in primary-key order (the clustered index).
- What does `Covering index` mean? **Answer:** the index alone holds every column the query needs; the table is never read.
- When is a table scan the right plan? **Answer:** small tables, or a condition that matches a big share of the rows.
- Why INVISIBLE before DROP? **Answer:** hiding is instant to undo; rebuilding a dropped index on a big table can take a long time.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** B) The primary key: the rows themselves are stored in its order (the clustered index); **Q2** D) `WHERE sold_on BETWEEN '2025-03-01' AND '2025-03-31'`; **Q3** `WHERE sold_on >= '2024-01-01' AND sold_on < '2025-01-01'`; **Q4** A) The text column is converted to a number on every row, so the index order no longer helps; **Q5** C) It succeeds without an error or a warning, and the new index is pure overhead; **Q6** `1553`: the foreign key from orders to customers needs an index on customer_id, and this is the only one. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a printed copy of the lesson's plan-word table next to the keyboard; in the lab, TODO 1 only, then compare plans with a partner.

Stretch: the lesson's Exercise 4 (a functional index on `YEAR(sold_on)`) and the FULLTEXT bonus challenge; ask them why `MATCH ... AGAINST('the')` returns nothing.

6.2 Views

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain what a view stores (a query, not data) and read the definition MySQL saves
- Create, query, replace and drop views, including views over joins and aggregates
- Predict whether a view is updatable, and protect it with WITH CHECK OPTION
- Use a view to hide sensitive columns, and grant access to the view alone
- Recognize and fix the errors views raise (1050, 1288, 1356, 1369, 1393)

TIMING IN THE ROOM

10m	Frame: a window in a wall; stores a query, not rows; three jobs (reuse, updatable slices, security)
20m	Live demo: first view, SHOW CREATE VIEW, join and aggregate views, update through a view, CHECK OPTION, security, frozen SELECT *
20m	Guided lab: `report_views_starter.sql`
15m	Start the lesson project: `reporting_views.sql` Steps 1 to 3

MENTAL MODEL TO SAY

Point at the window drawn on the board: "A **view** is a saved SELECT with a name. It holds no rows. Every time you read it, MySQL runs the saved query against the real tables and hands you the result, like a window that shows part of the room behind it. When the room changes, the view changes."

Then three jobs, one line each: **reuse** (a five-table join written once, read in one line), **a safe slice to edit** (updatable views with WITH CHECK OPTION), **a security boundary** (an account may read the view and not the table).

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Everyone types `SOURCE bookstore.sql` first (it removes `sales_log`). The lesson's own examples follow.

1. **Your first view (2m).** "A view is a named, saved SELECT. Nobody needs to remember WHERE city = 'Chicago' any more."

```
CREATE VIEW v_chicago_customers AS
SELECT id, first_name, last_name, email
FROM customers
WHERE city = 'Chicago';
SELECT * FROM v_chicago_customers ORDER BY id;
```

```
+----+------------+-----------+-------------------+
| id | first_name | last_name | email             |
+----+------------+-----------+-------------------+
|  2 | Bob        | Smith     | bob@example.com   |
|  7 | Grace      | Wilson    | grace@example.com |
+----+------------+-----------+-------------------+
```

2 **What MySQL saved (2m).** "A rewritten copy of the query, plus three settings it filled in: ALGORITHM, DEFINER, SQL SECURITY DEFINER. Remember the last one: it makes the security trick work."

```
SHOW CREATE VIEW v_chicago_customers\G
```

```
*************************** 1. row ***************************
                View: v_chicago_customers
         Create View: CREATE ALGORITHM=UNDEFINED DEFINER=`root`@`localhost` SQ
character_set_client: utf8mb4
collation_connection: utf8mb4_0900_ai_ci
```

3 **A five-table join, written once (3m).** Paste the CREATE VIEW from the lesson; no output on success. Then a question that reads like English.

```
CREATE VIEW v_order_details AS
SELECT
    o.id                                    AS order_id,
    o.order_date,
    CONCAT(c.first_name, ' ', c.last_name) AS customer,
    b.title,
    a.name                                  AS author,
    oi.quantity,
    oi.price_at_time,
    oi.quantity * oi.price_at_time          AS line_total
FROM orders AS o
JOIN customers   AS c  ON c.id = o.customer_id
JOIN order_items AS oi ON oi.order_id = o.id
JOIN books       AS b  ON b.id = oi.book_id
JOIN authors     AS a  ON a.id = b.author_id;
SELECT author, SUM(quantity) AS copies, SUM(line_total) AS revenue
FROM v_order_details
GROUP BY author
ORDER BY revenue DESC, author;
```

```
+----------------+--------+---------+
| author         | copies | revenue |
+----------------+--------+---------+
| J.R.R. Tolkien |      5 |   64.95 |
| George Orwell  |      6 |   56.94 |
| Frank Herbert  |      3 |   44.97 |
| Harper Lee     |      2 |   23.98 |
| Ray Bradbury   |      2 |   20.98 |
| Jane Austen    |      2 |   15.98 |
| Aldous Huxley  |      1 |    9.49 |
+----------------+--------+---------+
```

Money uses price_at_time, the price actually paid, never today's books.price.

4 **A view with aggregates (2m).** "LEFT JOIN, COUNT(DISTINCT), COALESCE: everything a report needs. Now a manager who never wrote a join asks one line."

```
CREATE VIEW v_customer_summary AS
SELECT
    c.id                                        AS customer_id,
    CONCAT(c.first_name, ' ', c.last_name)      AS customer,
    COUNT(DISTINCT o.id)                        AS orders,
    COALESCE(SUM(oi.quantity), 0)               AS items,
    COALESCE(SUM(oi.quantity * oi.price_at_time), 0) AS total_spent
FROM customers AS c
LEFT JOIN orders      AS o  ON o.customer_id = c.id
LEFT JOIN order_items AS oi ON oi.order_id = o.id
GROUP BY c.id, c.first_name, c.last_name;
SELECT customer, total_spent
FROM v_customer_summary
WHERE total_spent > 40
ORDER BY total_spent DESC;
```

```
+---------------+-------------+
| customer      | total_spent |
+---------------+-------------+
| Alice Johnson |       58.95 |
| Dave Brown    |       45.96 |
| Frank Miller  |       41.96 |
+---------------+-------------+
```

5 **Updating through a view (2m).** "Each row of this view is one customer row, so MySQL knows which row to change. The change lands in customers." Inside a transaction, rolled back.

```
START TRANSACTION;
UPDATE v_chicago_customers
SET email = 'grace.wilson@example.com'
WHERE id = 7;
SELECT id, first_name, email FROM customers WHERE id = 7;
ROLLBACK;
```

```
+----+------------+--------------------------+
| id | first_name | email                    |
+----+------------+--------------------------+
|  7 | Grace      | grace.wilson@example.com |
+----+------------+--------------------------+
```

6 **MySQL's verdict (1m).** "Grouping makes a view read-only; a five-table join is still updatable, one table per statement."

```
SELECT TABLE_NAME, IS_UPDATABLE, CHECK_OPTION
FROM information_schema.VIEWS
WHERE TABLE_SCHEMA = 'bookstore'
ORDER BY TABLE_NAME;
```

```
+---------------------+--------------+--------------+
| TABLE_NAME          | IS_UPDATABLE | CHECK_OPTION |
+---------------------+--------------+--------------+
| v_chicago_customers | YES          | NONE         |
| v_customer_summary  | NO           | NONE         |
| v_order_details     | YES          | NONE         |
+---------------------+--------------+--------------+
```

7 **WITH CHECK OPTION (3m).** "Without it, an update could move Bob to Boston and he would vanish from the view that changed him. With it:"

```
CREATE OR REPLACE VIEW v_chicago_customers AS
SELECT id, first_name, last_name, email, city
FROM customers
WHERE city = 'Chicago'
WITH CHECK OPTION;
UPDATE v_chicago_customers SET city = 'Boston' WHERE id = 2;
```

```
ERROR 1369 (HY000): CHECK OPTION failed 'bookstore.v_chicago_customers'
```

8 **A view as a security boundary (3m).** "The support team needs a staff directory without salaries. The view hides columns and rows; the account gets SELECT on the view only." The account is dropped again right away; Lesson 7.4 teaches users properly.

```
CREATE VIEW v_staff_directory AS
SELECT first_name, last_name, email, department
FROM employees
WHERE is_active = TRUE;
CREATE USER 'support_demo'@'localhost' IDENTIFIED BY 'Replace-With-A-Strong-Pa
GRANT SELECT ON bookstore.v_staff_directory TO 'support_demo'@'localhost';
SHOW GRANTS FOR 'support_demo'@'localhost';
DROP USER 'support_demo'@'localhost';
```

```
+-------------------------------------------------------------------------
| Grants for support_demo@localhost
+-------------------------------------------------------------------------
| GRANT USAGE ON *.* TO `support_demo`@`localhost`
| GRANT SELECT ON `bookstore`.`v_staff_directory` TO `support_demo`@`localhost
+-------------------------------------------------------------------------
```

Without a grant on employees, `SELECT salary FROM employees` would fail for that account with 1142.

9 **Silent: SELECT * is frozen (2m).** "The table gains in_stock; the view does not, because * was expanded when it was created."

```
CREATE TABLE demo_products (id INT PRIMARY KEY, name VARCHAR(50), cost DECIMAL
INSERT INTO demo_products VALUES (1, 'Bookmark', 1.50), (2, 'Tote bag', 12.00)
CREATE VIEW v_demo_all AS SELECT * FROM demo_products;
ALTER TABLE demo_products ADD COLUMN in_stock BOOLEAN DEFAULT TRUE;
SELECT * FROM v_demo_all ORDER BY id;
DROP VIEW v_demo_all;
DROP TABLE demo_products;
```

```
+----+----------+-------+
| id | name     | cost  |
+----+----------+-------+
|  1 | Bookmark |  1.50 |
|  2 | Tote bag | 12.00 |
+----+----------+-------+
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Expecting a view to store data.** "Do I need to refresh it?" Coach: "Insert an order, then read the view." (Predict P3 shows it.)
- **Updating an aggregate view (1288).** Coach: "Which customer row would MySQL change to make orders = 5?" There is none.
- **CREATE VIEW in a script that runs twice (1050).** Coach: "What should a rerunnable script say?" CREATE OR REPLACE VIEW.

GUIDED LAB

File: `labs/module-09/l6_2-views/report_views_starter.sql`; reference `labs/module-09/l6_2-views/report_views_solution.sql`. Two new views: sales per author (all ten authors, zero included) and every order with its customer's city.

- **TODO 1 together (6m):** inner joins drop the authors with no sales. LEFT JOIN plus COALESCE. Ask first: "how many authors come back now, and why?" (seven).
- **TODO 2 in pairs (8m):** create `v_city_orders` with no GROUP BY inside; the grouping happens in the query that reads it.
- **Expected:** ten authors from J.R.R. Tolkien 64.95 down to three with 0.00 (F. Scott Fitzgerald, Isaac Asimov with 0 books, J.D. Salinger); New York 5 orders, Chicago 4, Houston 2, (no city) 1; `v_author_sales` NO, `v_city_orders` YES in IS_UPDATABLE.
- **Debrief (2m):** "Why is one view updatable and the other not?" (grouping).
- **Fast finishers:** TODO 3, then `labs/module-09/l6_2-views/guarded_views_starter.sql` (CHECK OPTION, a read-only window, an account that may read only that window).

START THE LESSON PROJECT

The lesson project is **A Reporting Layer**, `reporting_views.sql`: four views the manager reads with plain SELECTs, in a script that runs twice without an error. In the room: Steps 1 to 3 (15 minutes).

- Step 1: `v_book_catalog` with GROUP_CONCAT for the categories.
- Step 2: `v_order_totals` (why the bookstore stores no order total: a preview of Session 10).
- Step 3: `v_book_sales`, with The Great Gatsby and The Catcher in the Rye at 0 copies.
- **Goes home:** Step 4 (a view on a view), Step 5 (IS_UPDATABLE for the layer), Step 6 (run it twice).

CHECKS FOR UNDERSTANDING

Ask these out loud:

- Does a view make a query faster? **Answer:** no; it runs its query. Indexes on the base tables make it fast.
- Where does ORDER BY go? **Answer:** in the query that reads the view.
- What does WITH CHECK OPTION protect against? **Answer:** writes through the view that would push a row out of it (1369).
- What happens to your views when you reload bookstore.sql? **Answer:** they are dropped with the database; keep them in a script.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** A) Only the query: the rows are computed every time the view is read; **Q8** `ERROR 1369 (HY000): CHECK OPTION failed 'bookstore.v_ny'`; **Q9** C) `SELECT id, title, price FROM books WHERE price < 10`; **Q10** D) The old columns only, without color; **Q11** MySQL ignores a view's ORDER BY when the outer query has its own, and the SQL standard promises no order from a view: sort where you read; **Q12** B) Read the directory through the view, but not the employees table itself. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: keep the table-versus-view comparison from the lesson on screen. In the lab, TODO 1 only, then read the solution's v_city_orders aloud with a partner.

Stretch: the lesson's Exercise 4 (a safe editing view with a check option) and the "fake a materialized view" bonus; then the 6.2 worked example (a view that filters a window function).

CLINIC

2:25 Clinic: A View's Plan, and the Debug Handouts

25 min

TIMING

8m	EXPLAIN on a merged view (demo step: v_order_details) and on a grouped view (v_customer_summary)
12m	`debug_indexes.sql` and `debug_views.sql` in pairs: eight errors, cause and fix in the workbook table
5m	Whole room: one volunteer per error

A VIEW IS AS FAST AS ITS QUERY

Run both on the projector (6.2 demo state). The first view is MERGED into the query: no trace of the view, and the base tables' indexes do the work (fk_items_order, PRIMARY). The second groups rows, so MySQL computes it into a temporary table first (Materialize) and filters after all the grouping. Say: "Views do not have indexes. On big tables, filter in the base query, or keep a summary table."

```
EXPLAIN SELECT customer, title FROM v_order_details WHERE order_id = 3\G
```

```
*************************** 1. row ***************************
EXPLAIN: -> Nested loop inner join  (cost=1.05 rows=1)
    -> Nested loop inner join  (cost=0.7 rows=1)
        -> Index lookup on oi using fk_items_order (order_id = 3)  (cost=0.35 row
        -> Single-row index lookup on b using PRIMARY (id = oi.book_id)  (cost=0.
    -> Single-row covering index lookup on a using PRIMARY (id = b.author_id)  (c
```

```
EXPLAIN SELECT * FROM v_customer_summary WHERE total_spent > 40\G
```

```
*************************** 1. row ***************************
EXPLAIN: -> Table scan on v_customer_summary  (cost=2.5..2.5 rows=0)
    -> Materialize  (cost=0..0 rows=0)
        -> Filter: (coalesce(sum(tmp_field),0) > 40)
            -> Group aggregate: sum(tmp_field), sum(order_items.quantity), count(
                -> Sort: c.id, c.first_name, c.last_name
                    -> Stream results  (cost=29.2 rows=288)
                        -> Left hash join (oi.order_id = o.id)  (cost=29.2 rows=2
                            -> Nested loop left join  (cost=4.65 rows=16)
                                -> Table scan on c  (cost=1.05 rows=8)
                                -> Covering index lookup on o using fk_orders_cus
                            -> Hash
                                -> Table scan on oi  (cost=0.129 rows=18)
```

◆ WHAT THE HANDOUTS PRINT, AND THE FIXES

```
ERROR 1061 (42000): Duplicate key name 'fk_books_author'
ERROR 1091 (42000): Can't DROP 'idx_books_price'; check that column/key exists
ERROR 1553 (HY000): Cannot drop index 'fk_orders_customer': needed in a foreign k
ERROR 1062 (23000): Duplicate entry 'New York' for key 'customers.uq_customers_ci
+------------+--------------------+-------------+
| TABLE_NAME | INDEX_NAME         | COLUMN_NAME |
+------------+--------------------+-------------+
| books      | fk_books_author    | author_id   |
| books      | PRIMARY            | id          |
| customers  | email              | email       |
| customers  | PRIMARY            | id          |
| orders     | fk_orders_customer | customer_id |
| orders     | PRIMARY            | id          |
+------------+--------------------+-------------+
```

- **1061:** the name `fk_books_author` is taken (by the foreign key's index). Fix: `idx_books_price`.
- **1091:** there is no index of that name (A failed). Fix: look it up with `SHOW INDEX FROM books` first.
- **1553:** the foreign key needs it. Fix: leave it; create another index starting with customer_id first if you want to replace it.
- **1062:** two customers already share a city (`New York`, the first duplicate found). Fix: a city is not unique; clean data before a UNIQUE index, or drop the idea.

```
ERROR 1050 (42S01): Table 'v_dbg_cheap' already exists
ERROR 1288 (HY000): The target table v_dbg_counts of the UPDATE is not updatable
ERROR 1369 (HY000): CHECK OPTION failed 'bookstore.v_dbg_cheap'
ERROR 1356 (HY000): View 'bookstore.v_dbg_flyer_prices' references invalid table(
+----+-------+-------+
| id | title | price |
+----+-------+-------+
|  2 | 1984  |  9.99 |
+----+-------+-------+
```

- **1050:** the view exists. Fix: CREATE OR REPLACE VIEW.
- **1288:** a grouped view is not updatable. Fix: change the base table.
- **1369:** 12.99 would push 1984 out of the "under 10" view. Fix: change the price in books, if that is really meant.
- **1356:** the view still names `cost`. Fix: CREATE OR REPLACE VIEW with `unit_cost`.

The checker runs these in a scratch database, so its messages name that database instead of `bookstore`.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

1. A query filters `WHERE customer_id = 5 AND book_id = 9 ORDER BY sold_on`. Write the one index that serves the filter and the sort. **Answer:** `CREATE INDEX idx_sales_customer_book_date ON sales_log (customer_id, book_id, sold_on);` (equality columns first, the sort column last: the lesson project's Step 4).
2. Name two habits that stop MySQL from using an index on a column. **Answer:** any two of: a function or arithmetic around the column; a leading % in LIKE; comparing a text column with a number; OR across different columns.
3. Give one reason to create a view and one thing a view does NOT do for you. **Answer:** reasons: reuse a long query by name, give a safe updatable slice, hide columns and rows from an account. Does not: store rows, have indexes, make the query faster.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** `tune_sales.sql` Steps 4 to 6; `reporting_views.sql` Steps 4 to 6 (run it twice in a row).
- **Lesson exercises.** 6.1: Exercises 1 to 3 (predict the plan, rewrite for the index, index the staff directory); 4 is a stretch. 6.2: Exercises 1 to 3 (books over $10, customers who never ordered, find the dependents); 4 is a stretch.

- **Both Quick Quizzes** (answers in the Answer Key) and **both journal entries**.
- **Labs not finished in class:** `labs/module-09/l6_1-indexes/index_killers_starter.sql`, `labs/module-09/l6_2-views/guarded_views_starter.sql`.
- **Before the Next Lesson (from 6.2):** run `reporting_views.sql` on a freshly loaded bookstore; look at a spreadsheet you use and find a value that repeats on many rows: Lesson 6.3 starts there.

BRING LIST FOR SESSION 10

- A laptop with a working bookstore (`bookstore.sql` in the course folder): Lesson 6.3 builds its own practice database (`normalize_lab`) and Lesson 7.1 reloads the bookstore.
- The spreadsheet value from the homework (a sign-up sheet, an expense list): we normalize one in the warm-up.
- The course folder with `labs/` (`labs/module-10/`), both project files, the workbook and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the before-and-after EXPLAIN ANALYZE; the composite index; plan_reading through TODO 2; WITH CHECK OPTION; report_views through TODO 1.
- **Cut first (saves 8m):** the clinic's EXPLAIN part; keep the debug handouts, or send them home with the README's expected errors.
- **Cut second (saves 4m):** 6.1 demo steps 10 to 12 (self-check Q5 and Q6 cover them).
- **Cut third (saves 3m):** the 6.2 security step: say "a view can be the only thing an account may read; Lesson 7.4 shows it".

RUNNING SHORT: WHAT TO ADD

- The lesson 6.1 project's Step 4 live: report C before and after the three-column index.
- The lesson 6.2 bonus: fake a materialized view with CREATE TABLE ... AS SELECT, insert an order, compare table and view.
- The 6.2 worked example: a view that filters a window function (Session 8 meets Session 9).

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 10: Normalization and Stored Programs

How to teach design and stored programs in one session: make a flat sheet corrupt itself, find its dependencies and key, split it into 3NF tables and prove with EXCEPT that nothing was lost; then save logic next to the data as procedures and functions, with parameters, characteristics and input checks that refuse bad calls.

2 lessons | 3 hrs session | Lessons 6.3-7.1 | Session 10 of 14 · Part 2

HOW TO USE THIS GUIDE

- Same rhythm as every session: frame → live demo → guided lab → start the lesson's own project. Exercises, quizzes, journals and the rest of each project go home.
- Every SQL block below was run on MySQL 9.7 to produce the output printed under it: Lesson 6.3 in its own practice database `normalize_lab` (the lesson's choice; bookstore is not touched), Lesson 7.1 on a freshly loaded bookstore.
- Keep `lesson_16.html` (6.3) and `lesson_17.html` (7.1) open: the dependency diagrams, the ER diagrams and the characteristics table are worth showing, not redrawing. Labs live in `labs/module-10/`.
- **Two modes in one session.** Lesson 6.3 is design on paper and in a throwaway database; 7.1 is programming inside the bookstore. The break is the switch: everyone runs `SOURCE bookstore.sql` before the 7.1 demo.
- Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (6.3, 7.1) match the course site.

BEFORE THE SESSION

- ☐ **Files to hand out:** `labs/module-10/` without the `*_solution.sql` files; the workbook.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-10` (10 files, a few seconds). `debug_routines.sql` needs binary logging (the MySQL default): on a server started with `--skip-log-bin` its section B does not fail and the check reports it.
- ☐ **Projector machine:** MySQL 9.7, no `normalize_lab` yet (drop it if a rehearsal left one), the client at 18 pt, both lesson pages open at the setup blocks you will paste.
- ☐ **Board, before learners arrive:** "the key, the whole key, and nothing but the key" with 1NF / 2NF / 3NF under it; for 7.1, two boxes "procedure: CALL, result sets, OUT, changes data" and "function: one value, inside a query".
- ☐ **Print or share:** the workbook; plain paper for the clinic's design sketches; sticky notes for the exit ticket.

KNOWN TRAPS TODAY

SYMPTOM	CAUSE	FIX
The prompt shows `->` after a CREATE PROCEDURE	DELIMITER // is still active, or the body's END has no //	Type `//` and Enter, then `DELIMITER ;`.
Two `ERROR 1064` lines for one procedure	No delimiter change: the client cut the body at the first ;	Wrap it in `DELIMITER //` ... `END //` ... `DELIMITER ;`.
`ERROR 1418` for a function	No characteristic, and binary logging is on	`DETERMINISTIC`, `NO SQL` or `READS SQL DATA`: the true one.
`ERROR 1304 ... already exists` on the second run	No `DROP PROCEDURE IF EXISTS` before the CREATE	Every routine in a script starts with its DROP ... IF EXISTS.
A routine returns every row instead of one	A parameter has a column's name (`WHERE id = id`)	`p_` and `v_` prefixes, or qualify columns with the alias.
`ERROR 3780` on a foreign key	Referencing and referenced columns have different types (BIGINT vs INT)	One id type everywhere; `INT NOT NULL` here.
phpMyAdmin users have no DELIMITER line	The SQL tab has a Delimiter box under the query area	Set it to `//` there (the lesson's tip).

SESSION 10 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on 6.1 and 6.2; two learners share the repeated value they found in a spreadsheet.
0:10	10	**6.3** Frame and mental model	One big sheet corrupts itself; each fact stored once; dependencies and the three normal forms.
0:20	20	**6.3** Live demo	normalize_lab: the order sheet, three anomalies, 1NF, a dependency test, 2NF and 3NF, the four tables, EXCEPT both ways, a stored total that drifts.
0:40	15	**6.3** Guided lab	`labs/module-10/l6_3-normalization/anomalies_starter.sql`: the workshop sheet's anomalies and dependencies.
0:55	10	**6.3** Start the lesson project	`book_club.sql`: Steps 1 to 3 (import, dependencies, the 3NF tables); Steps 4 to 6 go home.
1:05	10	Break	Everyone reloads bookstore.sql after the break: Lesson 7.1 starts from it.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:15	10	**7.1** Frame and mental model	Logic saved next to the data; procedure or function; DELIMITER is a client command.
1:25	25	**7.1** Live demo	DELIMITER; IN, OUT, local variables, IF; a function with its characteristics; 1418; SIGNAL; the silent parameter-name bug; listing routines.
1:50	20	**7.1** Guided lab	`labs/module-10/l7_1-stored-procedures/procedures_starter.sql`: IN, OUT and SIGNAL on three admin-page procedures.
2:10	15	**7.1** Start the lesson project	`bookstore_routines.sql`: Steps 1 and 2 (order_total, customer_tier); Steps 3 to 6 go home.
2:25	25	Clinic: design review and the debug handouts	Two warm-up spreadsheets designed on the board; `debug_design.sql` and `debug_routines.sql` in pairs.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 11.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 105 + 105 = 210 minutes for a solo learner; after the warm-up, the break, the clinic and the wrap-up the room has 135. Lesson 6.3 gets 55 minutes (the ideas need talk; the typing is mostly pasted), 7.1 gets 70 (DELIMITER and SIGNAL are where people get stuck alone).

Goes home (about 2 to 3 hours). `book_club.sql` Steps 4 to 6 (migrate, prove with EXCEPT, the anomaly gone); `bookstore_routines.sql` Steps 3 to 6 (customer_report, add_book, break it on purpose, rerun); each lesson's Exercises 1 to 3 (4 is a stretch); both Quick Quizzes; both journal entries; `normalize_starter.sql` and `functions_starter.sql`.

Protect, in this order: the three anomalies (the reason normalization exists); the EXCEPT proof; DELIMITER with a live mistake; SIGNAL; the procedures lab through TODO 2. **Cut first:** the clinic's design review (keep the debug handouts), then 6.3 demo step 11 (the drifting total is in the lesson), then 7.1 demo step 5 (IF).

OPENING SEGMENT

0:00 Warm-up: Indexes and Views — 10 min

SEQUENCE

- **Retrieval (6m).** W1 to W5 on the workbook's Warm-up box, no notes, three minutes; then the answers, W1 and W2 run on the projector.
- **Spreadsheets (4m).** Session 9's homework: "find a value that repeats on many rows". Two volunteers describe theirs in one sentence (a sign-up sheet with the trainer's phone on every row; an expense list with the project manager's name). Write both on the board: they come back in the clinic as design exercises.

WARM-UP ANSWERS (W1 TO W5)

1. **W1** Which plan line do you expect, and why is there an index at all? **Answer:** Index lookup on orders using fk_orders_customer: InnoDB created that index for the foreign key.
2. **W2** Is this view updatable? **Answer:** YES: each view row is one books row (no grouping, DISTINCT or UNION).
3. **W3** An index on (customer_id, sold_on): can a query on a sold_on range alone seek with it? **Answer:** No: the dates are sorted only within each customer (leftmost prefix).
4. **W4** What does MySQL store when you create a view? **Answer:** Only the query; rows are computed on every read.
5. **W5** What does WITH CHECK OPTION refuse? **Answer:** A write through the view that would produce a row the view cannot show (1369).

6.3 Database Design & Normalization

55 min

OBJECTIVES (FROM THE LESSON PAGE)

- Recognize update, insert and delete anomalies, and show each one with a query
- Find the functional dependencies and the key of a table
- Normalize a flat table to first, second and third normal form (1NF, 2NF, 3NF)
- Build the normalized tables with INSERT ... SELECT DISTINCT and prove with EXCEPT that no data was lost
- Read an ER diagram, and decide when storing derived or duplicated data is worth the risk

TIMING IN THE ROOM

10m	Frame: one big sheet corrupts itself; dependencies; 1NF, 2NF, 3NF
20m	Live demo: anomalies, 1NF, a dependency test, 3NF, four tables, EXCEPT, a drifting total
15m	Guided lab: `anomalies_starter.sql`
10m	Start the lesson project: `book_club.sql` Steps 1 to 3

MENTAL MODEL TO SAY

Start from the warm-up spreadsheets: "Every one of these stores the same fact on many rows. It works until someone changes one copy. A database cannot stop people typing wrong values, but a good design makes sure each fact is typed once, so there is nothing to keep in sync."

Then dependencies, as arrows on the board: "**A → B** means: same A, same B. order_id → order_date, email → name. They come from the rules of the business, not from today's rows." Point at the board line: **1NF** one value per cell and a key; **2NF** nothing depends on part of a composite key; **3NF** nothing depends on a non-key column. "The key, the whole key, and nothing but the key."

Last, the practical recipe: one table per kind of thing, a surrogate id plus UNIQUE on the natural identifier, foreign keys on the many side, a junction table for many-to-many. "This is exactly the bookstore you have used since Module 4."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

The lesson's own sequence in `normalize_lab`. The long setup blocks are pasted from the lesson page; the short ones are typed.

1. **The first-year order sheet (3m).** "Five real orders, the way a spreadsheet holds them: the customer typed on every row, the books squeezed into one cell." Paste the setup from the lesson page (it creates

normalize_lab; bookstore is not touched).

```
CREATE DATABASE normalize_lab CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci
USE normalize_lab;
CREATE TABLE order_sheet (
    order_id     INT PRIMARY KEY,
    order_date   DATE NOT NULL,
    cust_name    VARCHAR(100) NOT NULL,
    cust_email   VARCHAR(100) NOT NULL,
    cust_city    VARCHAR(50),
    book_titles  VARCHAR(255) NOT NULL,
    book_prices  VARCHAR(100) NOT NULL,
    quantities   VARCHAR(50)  NOT NULL,
    order_total  DECIMAL(8, 2) NOT NULL
);
INSERT INTO order_sheet VALUES
    (1, '2026-01-15', 'Alice Johnson', 'alice@example.com', 'New York', 'Dune,
    (2, '2026-02-20', 'Alice Johnson', 'alice@example.com', 'New York', 'Pride
    (3, '2026-03-10', 'Bob Smith', 'bob@example.com', 'Chicago', 'Dune', '14.9
    (4, '2026-01-22', 'Dave Brown', 'dave@example.com', 'Houston', 'The Hobbit
    (7, '2026-03-28', 'Bob Smith', 'bob@example.com', 'Chicago', '1984, Animal
SELECT order_id, cust_name, cust_email, book_titles, quantities, order_total
FROM order_sheet
ORDER BY order_id;
```

```
Database changed
+----------+---------------+-------------------+-----------------------------
| order_id | cust_name     | cust_email        | book_titles
+----------+---------------+-------------------+-----------------------------
|        1 | Alice Johnson | alice@example.com | Dune, 1984
|        2 | Alice Johnson | alice@example.com | Pride and Prejudice
|        3 | Bob Smith     | bob@example.com   | Dune
|        4 | Dave Brown    | dave@example.com  | The Hobbit, Lord of the Rings
|        7 | Bob Smith     | bob@example.com   | 1984, Animal Farm
+----------+---------------+-------------------+-----------------------------
```

2 **A simple question it cannot answer (1m).** "How many copies of 1984? The answer is in there, but no aggregate can add up 'the second number in a list'."

```
SELECT order_id, book_titles, quantities
FROM order_sheet
WHERE book_titles LIKE '%1984%'
ORDER BY order_id;
```

```
+----------+-------------------+------------+
| order_id | book_titles       | quantities |
+----------+-------------------+------------+
|        1 | Dune, 1984        | 1, 2       |
|        7 | 1984, Animal Farm | 1, 1       |
+----------+-------------------+------------+
```

3 **Update anomaly (2m).** "Alice changes her email, and one row gets updated." Inside a transaction, rolled back.

```
START TRANSACTION;
UPDATE order_sheet SET cust_email = 'alice.j@example.com' WHERE order_id = 2;
SELECT DISTINCT cust_name, cust_email
FROM order_sheet
WHERE cust_name = 'Alice Johnson';
ROLLBACK;
```

```
+---------------+---------------------+
| cust_name     | cust_email          |
+---------------+---------------------+
| Alice Johnson | alice@example.com   |
| Alice Johnson | alice.j@example.com |
+---------------+---------------------+
```

4 **Insert anomaly (1m).** "Carol signs up for the newsletter. Where does she go?"

```
INSERT INTO order_sheet (cust_name, cust_email, cust_city)
VALUES ('Carol Williams', 'carol@example.com', 'Los Angeles');
```

```
ERROR 1364 (HY000): Field 'order_id' doesn't have a default value
```

Every row is an order: no order, no row. Inventing a fake order or making columns nullable makes the data lie.

5 **Delete anomaly (1m).** "Cancel Dave's only order, and the store forgets Dave."

```
START TRANSACTION;
DELETE FROM order_sheet WHERE order_id = 4;
SELECT COUNT(*) AS rows_about_dave
FROM order_sheet
WHERE cust_name = 'Dave Brown';
ROLLBACK;
```

```
+-----------------+
| rows_about_dave |
+-----------------+
|               0 |
+-----------------+
```

6 **1NF: one value per cell (3m).** "One row per book per order. The question that defeated the sheet is an ordinary GROUP BY now." Point at the key (order_id, book_title), and at Alice, now on three rows: 1NF fixes querying, not duplication.

```
CREATE TABLE order_lines_1nf (
    order_id    INT NOT NULL,
    order_date  DATE NOT NULL,
    cust_name   VARCHAR(100) NOT NULL,
    cust_email  VARCHAR(100) NOT NULL,
    cust_city   VARCHAR(50),
    book_title  VARCHAR(200) NOT NULL,
    book_price  DECIMAL(8, 2) NOT NULL,
    quantity    INT NOT NULL,
    PRIMARY KEY (order_id, book_title)
);
INSERT INTO order_lines_1nf VALUES
    (1, '2026-01-15', 'Alice Johnson', 'alice@example.com', 'New York', 'Dune'
    (1, '2026-01-15', 'Alice Johnson', 'alice@example.com', 'New York', '1984'
    (2, '2026-02-20', 'Alice Johnson', 'alice@example.com', 'New York', 'Pride
    (3, '2026-03-10', 'Bob Smith', 'bob@example.com', 'Chicago', 'Dune', 14.99
    (4, '2026-01-22', 'Dave Brown', 'dave@example.com', 'Houston', 'The Hobbit
    (4, '2026-01-22', 'Dave Brown', 'dave@example.com', 'Houston', 'Lord of th
    (7, '2026-03-28', 'Bob Smith', 'bob@example.com', 'Chicago', '1984', 9.99,
    (7, '2026-03-28', 'Bob Smith', 'bob@example.com', 'Chicago', 'Animal Farm'
SELECT book_title, SUM(quantity) AS copies
FROM order_lines_1nf
GROUP BY book_title
ORDER BY copies DESC, book_title;
```

```
+---------------------+--------+
| book_title          | copies |
+---------------------+--------+
| 1984                |      3 |
| Dune                |      2 |
| Animal Farm         |      1 |
| Lord of the Rings   |      1 |
| Pride and Prejudice |      1 |
| The Hobbit          |      1 |
+---------------------+--------+
```

7 **Test a dependency (1m).** "book_title → book_price: no title may have two prices. Empty set means no counterexample. Data can disprove a rule, never prove one."

```sql
SELECT book_title, COUNT(DISTINCT book_price) AS prices
FROM order_lines_1nf
GROUP BY book_title
HAVING COUNT(DISTINCT book_price) > 1;
```

```
Empty set
```

8 **3NF: what the customer columns really depend on (2m).** "In the orders the key is order_id, but the name and city are facts about the customer, reached through the email. Three customers, each once." (2NF's two DISTINCT queries are on the lesson page: say them, do not type them.)

```sql
SELECT DISTINCT cust_email, cust_name, cust_city
FROM order_lines_1nf
ORDER BY cust_email;
```

```
+-------------------+---------------+-----------+
| cust_email        | cust_name     | cust_city |
+-------------------+---------------+-----------+
| alice@example.com | Alice Johnson | New York  |
| bob@example.com   | Bob Smith     | Chicago   |
| dave@example.com  | Dave Brown    | Houston   |
+-------------------+---------------+-----------+
```

9 **Build the four tables (3m).** Paste the block from the lesson page. "Parents first: customers and books, with UNIQUE on the natural identifiers. Then orders and order_items, joining back on email and title to find the new ids."

```sql
CREATE TABLE customers (
    id     INT AUTO_INCREMENT PRIMARY KEY,
    name   VARCHAR(100) NOT NULL,
    email  VARCHAR(100) NOT NULL UNIQUE,
    city   VARCHAR(50)
);
INSERT INTO customers (name, email, city)
SELECT DISTINCT cust_name, cust_email, cust_city
FROM order_lines_1nf
ORDER BY cust_email;
CREATE TABLE books (
    id     INT AUTO_INCREMENT PRIMARY KEY,
    title  VARCHAR(200) NOT NULL UNIQUE,
    price  DECIMAL(8, 2) NOT NULL
);
INSERT INTO books (title, price)
SELECT DISTINCT book_title, book_price
FROM order_lines_1nf
ORDER BY book_title;
CREATE TABLE orders (
    id           INT PRIMARY KEY,
    customer_id  INT NOT NULL,
    order_date   DATE NOT NULL,
    CONSTRAINT fk_lab_orders_customer
        FOREIGN KEY (customer_id) REFERENCES customers(id)
);
INSERT INTO orders (id, customer_id, order_date)
SELECT DISTINCT l.order_id, c.id, l.order_date
FROM order_lines_1nf AS l
JOIN customers AS c ON c.email = l.cust_email;
CREATE TABLE order_items (
    order_id       INT NOT NULL,
    book_id        INT NOT NULL,
    quantity       INT NOT NULL,
    price_at_time  DECIMAL(8, 2) NOT NULL,
    PRIMARY KEY (order_id, book_id),
    CONSTRAINT fk_lab_items_order FOREIGN KEY (order_id) REFERENCES orders(id)
    CONSTRAINT fk_lab_items_book  FOREIGN KEY (book_id)  REFERENCES books(id)
);
INSERT INTO order_items (order_id, book_id, quantity, price_at_time)
SELECT l.order_id, b.id, l.quantity, l.book_price
FROM order_lines_1nf AS l
JOIN books AS b ON b.title = l.book_title;
SELECT (SELECT COUNT(*) FROM customers)   AS customers,
       (SELECT COUNT(*) FROM books)       AS books,
```

```
       (SELECT COUNT(*) FROM orders)      AS orders,
       (SELECT COUNT(*) FROM order_items) AS order_items;
```

```
+-----------+-------+--------+-------------+
| customers | books | orders | order_items |
+-----------+-------+--------+-------------+
|         3 |     6 |      5 |           8 |
+-----------+-------+--------+-------------+
```

10 **Prove nothing was lost (2m).** "EXCEPT returns the rows of the first query missing from the second. Both directions empty: same rows on both sides."

```
SELECT order_id, order_date, cust_name, cust_email, cust_city, book_title, boo
FROM order_lines_1nf
EXCEPT
SELECT o.id, o.order_date, c.name, c.email, c.city, b.title, oi.price_at_time,
FROM orders AS o
JOIN customers   AS c  ON c.id = o.customer_id
JOIN order_items AS oi ON oi.order_id = o.id
JOIN books       AS b  ON b.id = oi.book_id;
SELECT o.id, o.order_date, c.name, c.email, c.city, b.title, oi.price_at_time,
FROM orders AS o
JOIN customers   AS c  ON c.id = o.customer_id
JOIN order_items AS oi ON oi.order_id = o.id
JOIN books       AS b  ON b.id = oi.book_id
EXCEPT
SELECT order_id, order_date, cust_name, cust_email, cust_city, book_title, boo
FROM order_lines_1nf;
```

```
Empty set
Empty set
```

11 **Silent: a stored total drifts (1m).** "Add the total back 'for speed', then fix a quantity the ordinary way. No error anywhere, and order 1 has two totals." This is why the bookstore has no stored total.

```
ALTER TABLE orders ADD COLUMN total DECIMAL(8, 2);
UPDATE orders AS o
SET total = (SELECT SUM(quantity * price_at_time) FROM order_items WHERE order
UPDATE order_items SET quantity = 3
WHERE order_id = 1 AND book_id = (SELECT id FROM books WHERE title = '1984');
SELECT o.id, o.total AS stored_total, SUM(oi.quantity * oi.price_at_time) AS r
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.id
GROUP BY o.id, o.total
ORDER BY o.id;
```

```
+----+--------------+------------+
| id | stored_total | real_total |
+----+--------------+------------+
|  1 |        34.97 |      44.96 |
|  2 |         7.99 |       7.99 |
|  3 |        14.99 |      14.99 |
|  4 |        26.98 |      26.98 |
|  7 |        18.48 |      18.48 |
+----+--------------+------------+
```

Undo or not: the lab database is throwaway. `DROP DATABASE normalize_lab;` when the lesson is over.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Confusing 2NF and 3NF.** Coach with one question per column: "Does it need the whole key, part of the key, or another non-key column?" Part = 2NF, another column = 3NF.
- **Treating data as proof.** "The query found no counterexample, so it is a rule." Coach: "Would the business allow two prices for one title next year?" Data can only disprove.
- **Calling price_at_time redundant.** Coach: "What does each column answer?" Today's price versus what this customer paid on this order: two facts (self-check and lesson quiz).

GUIDED LAB

File: `labs/module-10/l6_3-normalization/anomalies_starter.sql`; reference `labs/module-10/l6_3-normalization/anomalies_solution.sql`. A new sheet: the bookstore's workshop sign-ups (attendee, workshop, date, room, host, fee). It builds `kit_s10_sheet` and drops it on every run.

- **TODO 1 together (4m):** the update anomaly. Ask first: "if only Bob's row changes, how many dates does the workshop have?"
- **TODO 2 in pairs (4m):** the delete anomaly: Frank cancels, and the store forgets Speed Reading exists.
- **TODO 3 in pairs (5m):** two dependency tests in the style of the first. Circulate for HAVING written as

WHERE (1111, invalid use of group function).

- **Expected:** two dates for Bookbinding Basics; 0 rows about Speed Reading; one value of each per workshop; `host_email → host_name` no counterexample; `host_email → workshop` broken by Lena, who hosts 2 workshops; key check 6 rows, 6 pairs, 5 attendees.
- **Debrief (2m):** "Write the dependencies as arrows. What is the key? Which tables does that suggest?" (attendees, hosts, workshops, signups).
- **Homework / fast finishers:** `labs/module-10/l6_3-normalization/normalize_starter.sql`: the same sheet migrated to those four tables and proved lossless with EXCEPT.

START THE LESSON PROJECT

The lesson project is **Normalize the Book Club Sheet**, `book_club.sql`. In the room: Steps 1 to 3 (10 minutes).

- Step 1: import the sheet into normalize_lab (paste from the lesson page).
- Step 2: write the dependencies and the key as comments BEFORE reading the lesson's answer; then run the check query.
- Step 3: create members, leaders, clubs and memberships.
- **Goes home:** Step 4 (migrate, parents first), Step 5 (EXCEPT both ways), Step 6 (move the Sci-Fi Circle to Wednesday: one row).

CHECKS FOR UNDERSTANDING

Ask these out loud:

- Which normal form does a comma list in a cell break? **Answer:** 1NF.
- What is the key of the order lines table? **Answer:** (order_id, book_title): neither column alone determines the rest.
- Why does the bookstore store no order total? **Answer:** it can always be computed, and a stored copy can drift (demo step 11).
- When is denormalizing reasonable? **Answer:** for a measured slow query, after indexes and a better query failed, with a plan to keep the copy in sync.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** A) An update anomaly; **Q2** C) First normal form: a repeating group of columns; **Q3** B) 2NF: course_room depends on only part of the key; **Q4** `dept_id → dept_name` (a transitive dependency: id → dept_id → dept_name); move dept_name to `departments(id, name)` and keep dept_id as a foreign key; **Q5** D) The four tables, joined back, hold exactly the original rows: the split lost nothing and added nothing; **Q6** B) The data does not contradict member_email → member_name; whether it is a rule is a business decision. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a printed dependency template ("___ → ___ because ___") for TODO 3 and the project's Step 2. Pair learners who found the warm-up spreadsheets easy with those who did not.

Stretch: the lesson's Exercise 4 (design a recipe site end to end) and the bonus (a foreign key to bookstore.books from another database: what happens on a reload?).

7.1 Stored Procedures & Functions

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain what the DELIMITER client command does, and when you do not need it
- Create and call stored procedures with IN, OUT and INOUT parameters
- Write stored functions with the right characteristic (DETERMINISTIC, READS SQL DATA) and use them inside queries
- Use local variables, IF, CASE and WHILE in a routine, and reject bad input with SIGNAL
- List, inspect, replace and drop routines, and read the errors MySQL raises for broken ones (1064, 1418, 1172, 1304, 1644)

TIMING IN THE ROOM

10m	Frame: logic next to the data; procedure or function; DELIMITER
25m	Live demo: DELIMITER, IN, OUT, local variables, IF, a function, 1418, SIGNAL, WHERE id = id, ROUTINES
20m	Guided lab: `procedures_starter.sql`
15m	Start the lesson project: `bookstore_routines.sql` Steps 1 and 2

MENTAL MODEL TO SAY

"Until now every query lived in your head or in a file. When five programs need the same ten-line query, someone copies it five times and the copies drift: the same problem as this morning's spreadsheet, in code. A **stored routine** is a named block of SQL saved next to the tables, with parameters. Everyone calls the one copy."

Point at the two boxes: a **procedure** is run with CALL, may return result sets, hands values back through OUT parameters and may change data; a **function** returns one value and lives inside a query, like ROUND or CONCAT.

Then the one confusing thing, said plainly: "The mysql client cuts your typing at every semicolon. A body has several. **DELIMITER** // tells the CLIENT to wait for // for a while; the server never sees the word."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Everyone types `SOURCE bookstore.sql` first. The lesson's own examples follow; type steps 1 and 8 (DELIMITER and SIGNAL), paste the rest.

1. **DELIMITER, and a first procedure (3m).** "The client sends your text to the server at every semicolon. A body has several, so for a while we tell the CLIENT to wait for // instead. The server never sees the word

DELIMITER." Point at the two result sets: each SELECT in the body sends one back.

```
DELIMITER //
CREATE PROCEDURE say_hello()
BEGIN
    SELECT 'Hello from the bookstore!' AS greeting;
    SELECT COUNT(*) AS books_in_stock FROM books WHERE in_stock = TRUE;
END //
DELIMITER ;
CALL say_hello();
```

```
+---------------------------+
| greeting                  |
+---------------------------+
| Hello from the bookstore! |
+---------------------------+
+-----------------+
| books_in_stock |
+-----------------+
|              9 |
+-----------------+
```

2 **IN: a parameter (2m).** "p_ marks a parameter. You will see in a minute why the prefix matters."

```
DELIMITER //
CREATE PROCEDURE books_by_author(IN p_author VARCHAR(100))
BEGIN
    SELECT b.title, b.price
    FROM books AS b
    JOIN authors AS a ON a.id = b.author_id
    WHERE a.name = p_author
    ORDER BY b.title;
END //
DELIMITER ;
CALL books_by_author('George Orwell');
```

```
+-------------+-------+
| title       | price |
+-------------+-------+
| 1984        |  9.99 |
| Animal Farm |  8.49 |
+-------------+-------+
```

3 **OUT: an answer through a user variable (2m).** "SELECT ... INTO stores instead of printing, so the CALL prints nothing. The answer lands in @tolkien_books."

```
DELIMITER //
CREATE PROCEDURE count_books_by(IN p_author VARCHAR(100), OUT p_count INT)
BEGIN
    SELECT COUNT(*) INTO p_count
    FROM books AS b
    JOIN authors AS a ON a.id = b.author_id
    WHERE a.name = p_author;
END //
DELIMITER ;
CALL count_books_by('J.R.R. Tolkien', @tolkien_books);
SELECT @tolkien_books AS tolkien_books;
```

```
+---------------+
| tolkien_books |
+---------------+
|             2 |
+---------------+
```

4 **Local variables (3m).** "DECLARE comes first in a block; a local variable lives for one call." 34.97 plus 8% tax.

```
DELIMITER //
CREATE PROCEDURE order_receipt(IN p_order_id INT)
BEGIN
    DECLARE v_subtotal DECIMAL(10, 2) DEFAULT 0;
    DECLARE v_tax_rate DECIMAL(4, 3) DEFAULT 0.080;
    DECLARE v_tax      DECIMAL(10, 2);
    SELECT COALESCE(SUM(quantity * price_at_time), 0) INTO v_subtotal
    FROM order_items
    WHERE order_id = p_order_id;
    SET v_tax = ROUND(v_subtotal * v_tax_rate, 2);
    SELECT p_order_id         AS order_id,
           v_subtotal         AS subtotal,
           v_tax              AS tax,
           v_subtotal + v_tax AS total;
END //
DELIMITER ;
CALL order_receipt(1);
```

```
+----------+----------+------+-------+
| order_id | subtotal | tax  | total |
+----------+----------+------+-------+
|        1 |    34.97 | 2.80 | 37.77 |
+----------+----------+------+-------+
```

5 **IF ... ELSEIF ... ELSE (3m).** "Gold from $50, Silver from $30, Bronze above zero, New otherwise."

```
DELIMITER //
CREATE PROCEDURE classify_customer(IN p_customer_id INT, OUT p_tier VARCHAR(10
BEGIN
    DECLARE v_spent DECIMAL(10, 2);
    SELECT COALESCE(SUM(oi.quantity * oi.price_at_time), 0) INTO v_spent
    FROM orders AS o
    JOIN order_items AS oi ON oi.order_id = o.id
    WHERE o.customer_id = p_customer_id;
    IF v_spent >= 50 THEN
        SET p_tier = 'Gold';
    ELSEIF v_spent >= 30 THEN
        SET p_tier = 'Silver';
    ELSEIF v_spent > 0 THEN
        SET p_tier = 'Bronze';
    ELSE
        SET p_tier = 'New';
    END IF;
END //
DELIMITER ;
CALL classify_customer(1, @alice);
CALL classify_customer(2, @bob);
CALL classify_customer(3, @carol);
SELECT @alice AS alice, @bob AS bob, @carol AS carol;
```

```
+-------+--------+-------+
| alice | bob    | carol |
+-------+--------+-------+
| Gold  | Silver | New   |
+-------+--------+-------+
```

6 **A stored function (3m).** "One value, used inside a query like a built-in function. The characteristics line describes it to the server: same input, same output (DETERMINISTIC), no table read (NO SQL)."

```
DELIMITER //
CREATE FUNCTION with_tax(p_amount DECIMAL(10, 2))
RETURNS DECIMAL(10, 2)
DETERMINISTIC
NO SQL
BEGIN
    RETURN ROUND(p_amount * 1.08, 2);
END //
DELIMITER ;
SELECT title, price, with_tax(price) AS price_with_tax
FROM books
WHERE price >= 11
ORDER BY price DESC;
```

```
+-----------------------+-------+----------------+
| title                 | price | price_with_tax |
+-----------------------+-------+----------------+
| Lord of the Rings     | 15.99 |          17.27 |
| Dune                  | 14.99 |          16.19 |
| To Kill a Mockingbird | 11.99 |          12.95 |
+-----------------------+-------+----------------+
```

7 **Without characteristics (1m).** "Binary logging is on by default since 8.0, so the server wants to know what a function does."

```
CREATE FUNCTION book_count()
RETURNS INT
RETURN (SELECT COUNT(*) FROM books);
```

```
ERROR 1418 (HY000): This function has none of DETERMINISTIC, NO SQL, or READS
```

Fix: `READS SQL DATA` after `RETURNS INT`. Never turn on `log_bin_trust_function_creators` to silence it.

8 **Check first, then change: SIGNAL (4m).** "A procedure that changes data checks its input before the UPDATE. SIGNAL raises your own error, number 1644, with your message."

```
DELIMITER //
CREATE PROCEDURE set_book_price(IN p_book_id INT, IN p_price DECIMAL(8, 2))
BEGIN
    IF p_price IS NULL OR p_price <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Price must be greater than zero';
    END IF;
    IF NOT EXISTS (SELECT 1 FROM books WHERE id = p_book_id) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'No book with that id';
    END IF;
    UPDATE books SET price = p_price WHERE id = p_book_id;
END //
DELIMITER ;
CALL set_book_price(5, -1.00);
CALL set_book_price(99, 9.99);
```

```
ERROR 1644 (45000): Price must be greater than zero
ERROR 1644 (45000): No book with that id
```

Both checks run before the UPDATE, so a refused call changes nothing.

9 **Silent: a parameter named like a column (2m).** "Book 1, please." Run it before explaining. "Inside the routine, a bare id is the parameter, so WHERE id = id is 1 = 1 on every row. Picture this in a DELETE."

```
CREATE PROCEDURE book_by_id(IN id INT)
    SELECT b.id, b.title FROM books AS b WHERE id = id ORDER BY b.id;
CALL book_by_id(1);
```

```
+----+-------------------------+
| id | title                   |
+----+-------------------------+
|  1 | Dune                    |
|  2 | 1984                    |
|  3 | To Kill a Mockingbird   |
|  4 | The Great Gatsby        |
|  5 | Pride and Prejudice     |
|  6 | The Hobbit              |
|  7 | Brave New World         |
|  8 | The Catcher in the Rye  |
|  9 | Lord of the Rings       |
| 10 | Fahrenheit 451          |
| 11 | Animal Farm             |
+----+-------------------------+
```

10 **What the database now holds (2m).** "Procedures say CONTAINS SQL unless declared otherwise. Reloading bookstore.sql wipes all of them: routines live in a script file."

```
SELECT ROUTINE_NAME, ROUTINE_TYPE, IS_DETERMINISTIC, SQL_DATA_ACCESS
FROM information_schema.ROUTINES
WHERE ROUTINE_SCHEMA = 'bookstore'
ORDER BY ROUTINE_TYPE, ROUTINE_NAME;
```

```
+-------------------+--------------+-------------------+-----------------+
| ROUTINE_NAME      | ROUTINE_TYPE | IS_DETERMINISTIC | SQL_DATA_ACCESS |
+-------------------+--------------+-------------------+-----------------+
| with_tax          | FUNCTION     | YES               | NO SQL          |
| books_by_author   | PROCEDURE    | NO                | CONTAINS SQL    |
| book_by_id        | PROCEDURE    | NO                | CONTAINS SQL    |
| classify_customer | PROCEDURE    | NO                | CONTAINS SQL    |
| count_books_by    | PROCEDURE    | NO                | CONTAINS SQL    |
| order_receipt     | PROCEDURE    | NO                | CONTAINS SQL    |
| say_hello         | PROCEDURE    | NO                | CONTAINS SQL    |
| set_book_price    | PROCEDURE    | NO                | CONTAINS SQL    |
+-------------------+--------------+-------------------+-----------------+
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **A stuck `->` prompt.** Coach: "Which delimiter is active? Type // to send, then DELIMITER ;." Every kit example switches back in the same block.
- **Expecting OUT parameters to print.** "CALL printed nothing." Coach: "Where did you ask it to put the answer? SELECT @your_variable."
- **SIGNAL after the change.** A check placed after the UPDATE stops the procedure too late. Coach: "Which statements already ran when SIGNAL fires?" (Lesson 7.2's transactions handle multi-step changes.)

GUIDED LAB

File: `labs/module-10/l7_1-stored-procedures/procedures_starter.sql`; reference `labs/module-10/l7_1-stored-procedures/procedures_solution.sql`. Three new procedures for the store's admin page; every routine has its DROP ... IF EXISTS, so the file reruns after every edit.

- **TODO 1 together (5m):** category_shelf returns the three cheapest books of any category. Add the WHERE on the parameter, delete the LIMIT, SOURCE again.
- **TODO 2 in pairs (7m):** author_stats's second OUT parameter. Ask them to try Isaac Asimov (author 10) with and without COALESCE.
- **TODO 3 in pairs (6m):** the percent guard, placed BEFORE the other one; then `CALL raise_price(6, 80);` at the prompt.

- **Expected:** Dystopian: Brave New World 9.49, 1984 9.99, Fahrenheit 451 10.49; Orwell 2 books 6 copies, Asimov 0 and 0; the guard prints `ERROR 1644 (45000): Percent must be between 1 and 50` and The Hobbit stays 10.99; the good call shows 12.09 inside the transaction, 10.99 after ROLLBACK.
- **Debrief (2m):** "Which of the three changes data? What makes it safe to call with bad input?"
- **Homework / fast finishers:** `labs/module-10/l7_1-stored-procedures/functions_starter.sql`: shipping_fee (DETERMINISTIC, NO SQL) and copies_sold (READS SQL DATA).

START THE LESSON PROJECT

The lesson project is **The Bookstore Routine Library**, `bookstore_routines.sql`: two functions and two procedures in one rerunnable script. In the room: Steps 1 and 2 (15 minutes).

- Step 1: `order_total(p_order_id)`, READS SQL DATA, used for Alice's three orders (34.97, 7.99, 15.99).
- Step 2: `customer_tier(p_customer_id)`, the demo's IF as a function with a CASE in RETURN.
- **Done in the room** = both functions in the file, each after its DROP FUNCTION IF EXISTS, and the file runs twice.
- **Goes home:** Step 3 (customer_report: two result sets), Step 4 (add_book with SIGNAL and LAST_INSERT_ID), Step 5 (break it on purpose), Step 6 (inventory and rerun; test CALLs stay out of the file).

CHECKS FOR UNDERSTANDING

Ask these out loud:

- Who reads `DELIMITER //`? **Answer:** the mysql client (and SOURCE); apps, Workbench and phpMyAdmin's Delimiter box do not need you to type it.
- Why READS SQL DATA and not DETERMINISTIC for order_total? **Answer:** it reads rows that change; DETERMINISTIC would be a false promise.
- What does a procedure give back that a function cannot? **Answer:** result sets, several OUT values, and transactions inside.
- Why the p_ prefix? **Answer:** a parameter named like a column silently shadows it (WHERE id = id).

Workbook self-check, items Q7 to Q12.

Answers: **Q7** A) Two `ERROR 1064` lines: the client sent the text in two pieces at the semicolons; **Q8** `4` (in a column named `cheaper`); **Q9** D) `DETERMINISTIC` and `NO SQL`; **Q10** C) `ERROR 1172 (42000): Result consisted of more than one row`; **Q11** `ERROR 1644 (45000): Quantity must be at least 1`; **Q12** C) Run `COMMIT` or `START TRANSACTION`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a card with the procedure skeleton (DROP, DELIMITER //, CREATE ... BEGIN, END //, DELIMITER ;) taped to the screen. In the lab, TODO 1 and 2 only.

Stretch: the lesson's Exercise 4 (a WHILE loop up the chain of command, compared with Session 8's recursive CTE) and the bonus (a duplicate guard with a message built in a variable).

2:25 Clinic: Design Review and the Debug Handouts

25 min

TIMING

10m	Design review: the two warm-up spreadsheets, as dependencies and tables on the board
12m	`debug_design.sql` and `debug_routines.sql` in pairs: cause and fix in the workbook table
3m	Whole room: one volunteer per error

DESIGN REVIEW (10M)

- Take the first warm-up spreadsheet. The room calls out the dependencies ("trainer_email → trainer_phone"), you write them as arrows, then the key, then the tables. Five minutes, then the second spreadsheet.
- Ask for one junction table and one natural identifier with UNIQUE in each design. These sketches are the warm-up for the final project's design step (Sessions 12 to 13).

WHAT THE HANDOUTS PRINT, AND THE FIXES

```
Database changed
ERROR 1062 (23000): Duplicate entry 'lena.park@bookstore.example' for key 'hosts.
ERROR 3780 (HY000): Referencing column 'workshop_id' and referenced column 'id' i
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constrain
+-------+-----------+
| hosts | workshops |
+-------+-----------+
|     2 |         2 |
+-------+-----------+
```

- **1062:** the lookup table's UNIQUE refused a second copy of every host: the design working. Fix: migration scripts start from a clean state, or insert only missing rows (WHERE NOT EXISTS).
- **3780:** BIGINT cannot reference INT. Fix: `workshop_id INT NOT NULL`; one id type everywhere.
- **1451:** Lena still hosts workshops. Fix: decide what happens to her workshops first (reassign them), or mark hosts inactive instead of deleting.

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that c
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that c
ERROR 1418 (HY000): This function has none of DETERMINISTIC, NO SQL, or READS SQL
ERROR 1172 (42000): Result consisted of more than one row
ERROR 1304 (42000): PROCEDURE dbg_count_books already exists
ERROR 1644 (45000): Pages must be a positive number
+----+----------------+-------+
| id | title          | pages |
+----+----------------+-------+
| 10 | Fahrenheit 451 |   158 |
+----+----------------+-------+
```

- **1064 twice (A):** no delimiter change. Fix: `DELIMITER //` ... `END //` `DELIMITER ;`, or drop BEGIN/END for a one-statement body.
- **1418 (B):** a function with no characteristic. Fix: `READS SQL DATA` after `RETURNS INT`.
- **1172 (C):** Tolkien has two books. Fix: `SELECT MIN(title) INTO ...` or `ORDER BY title LIMIT 1`.
- **1304 (D):** created twice. Fix: `DROP PROCEDURE IF EXISTS` first.
- **1644 (E):** the guard worked; nothing changed (158 pages). Not a bug: the right call is `CALL dbg_set_pages(10, 160);`.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

1. `order_lines(order_id, book_title, order_date, book_price, quantity)`, key `(order_id, book_title)`: name one partial dependency and the table it moves to. **Answer:** `order_id → order_date` (to orders) or `book_title → book_price` (to books).
2. Why does a procedure that changes data check its input before the UPDATE, and how does it refuse? **Answer:** so a refused call changes nothing; `SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '...'` (error 1644).
3. Function or procedure: "show each book's price with tax in a SELECT"? "Add a book and its new author, and return the new id"? **Answer:** a function; a procedure (it changes data and hands back an OUT value).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** `book_club.sql` Steps 4 to 6; `bookstore_routines.sql` Steps 3 to 6 (run it twice; test CALLs at the prompt, not in the file).

- **Lesson exercises.** 6.3: Exercises 1 to 3 (name the violations, test a dependency, normalize the invoices); 4 is a stretch. 7.1: Exercises 1 to 3 (books in a category, sale prices, city statistics); 4 is a stretch.
- **Both Quick Quizzes** and **both journal entries**.
- **Labs:** `labs/module-10/l6_3-normalization/normalize_starter.sql`, `labs/module-10/l7_1-stored-procedures/functions_starter.sql`.
- **Before the Next Lesson (from 7.1):** make sure you can open two mysql client windows at once (two terminals, or two Workbench tabs): Lesson 7.2 uses both.

BRING LIST FOR SESSION 11

- A laptop that can run **two** mysql client sessions side by side (Lesson 7.2's transaction demos), and `bookstore.sql`.
- `book_club.sql` and `bookstore_routines.sql`, both run twice without an error (the Session 11 homework check; Lesson 7.2 adds a procedure with an error handler to a library like `bookstore_routines.sql`).
- Session 11 holds **Checkpoint 2** (Modules 5 and 6, 30 minutes): review your CTE, window, index, view and normalization notes. The course folder with `labs/`, the workbook and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the three anomalies; the EXCEPT proof; DELIMITER and SIGNAL; the procedures lab through TODO 2.
- **Cut first (saves 10m):** the clinic's design review; keep the debug handouts.
- **Cut second (saves 1m):** 6.3 demo step 11 (the drifting total: the lesson's Common Mistake 4).
- **Cut third (saves 3m):** 7.1 demo step 5 (IF): the project's customer_tier does the same with CASE.

RUNNING SHORT: WHAT TO ADD

- The 7.1 project's Step 4 live: add_book creating Ursula K. Le Guin, then Step 5's refused free book.
- Draw the bookstore's full ER diagram with the room, including the employees' manager relationship (a one-to-many on itself).
- Session 8 meets Session 10: the 7.1 Exercise 4 WHILE loop next to the 5.1 recursive chain of command.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 11: Transactions and JSON

How to teach the eleventh session of MySQL Foundations: Checkpoint 2 on Modules 5 and 6 (CTEs, window functions, indexes, views, normalization), then transactions (all or nothing, two silent traps, two windows and a lock timeout, a procedure that rolls back on any error) and JSON columns (validated documents, the two arrows, JSON from rows and back, an indexed path, and when JSON is the wrong tool).

`2 lessons` `3 hrs session` `Lessons 7.2–7.3` `Session 11 of 14 · Part 2` `Checkpoint 2`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they follow) → guided lab (a kit lab, new practice on the lesson's ideas) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Session 11 is Part 2's checkpoint session: **Checkpoint 2** (Modules 5 and 6) takes 30 minutes right after the warm-up, so each lesson gets 60 minutes in the room instead of the usual 65 to 75. The lessons are written for 110 and 100 minutes alone; the room keeps the ideas that are hard alone (two windows, the handler, the arrows) and sends the rest home.
- Every SQL block below was run to produce the output printed under it, on MySQL 9.7 with the default settings, from a fresh `bookstore.sql`. The two-window table in the 7.2 demo comes from two real client sessions on the same server (the test server's lock timeout is 5 seconds; a default server waits 50).
- Keep `lesson_18.html` (7.2) and `lesson_json.html` (7.3) open in browser tabs for the diagrams. Labs live in `labs/module-11/`: `_starter.sql` files are handouts, `_solution.sql` files and `checkpoint-2/checkpoint2_solution.sql` are yours.
- Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (7.2, 7.3) match the course site.

> **THE #1 RISK OF SESSION 11: THE CHECKPOINT EATS THE TRANSACTIONS DEMO**
>
> Thirty minutes of assessment at the start of a session with two dense lessons leaves no slack. **Before:** print the Checkpoint 2 page (it is in the workbook) or make sure every learner can open it, and copy `checkpoint2_starter.sql` into everyone's `labs/` folder; it must not be handed out earlier. **In the room:** start at 0:10 sharp, stop at 0:40 sharp, even mid-sentence: unfinished parts are information, not failure. **After:** mark with the rubric in the Answer Key before Session 12; the warm-up there returns it.

BEFORE THE SESSION

- ☐ **Bring list you can assume (from Session 10):** a laptop with MySQL 9.7 and the `bookstore` database loaded, the root password, the course folder with `labs/`, Lesson 7.1's `bookstore_routines.sql` and Lesson 6.3's `book_club.sql` (homework), the workbook and the journal.
- ☐ **Files:** copy `labs/module-11/` **without** the `*_solution.sql` files and **without** `checkpoint-2/` into the learners' shared folder. Hand out `checkpoint-2/checkpoint2_starter.sql` at 0:10 (USB stick or the shared folder, opened at 0:10).
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-11` from this kit's folder: 12 files, every line PASS.
- ☐ **Projector machine:** MySQL 9.7 with a fresh `bookstore` (`SOURCE labs/_states/bookstore.sql`), **two** terminals side by side at 18 pt or larger, both logged in with `mysql -u root -p bookstore`, labeled A and B. Paste-ready: the lesson's Setup block (accounts) and its `transfer` procedure; the 7.3 setup (the `ALTER TABLE` and three UPDATEs). Browser tabs: `lesson_18.html`, `lesson_json.html`.
- ☐ **Board, before learners arrive:** "START TRANSACTION → … → COMMIT (keep) / ROLLBACK (discard)"; the list "commits by itself: CREATE · ALTER · DROP · TRUNCATE · GRANT"; and two columns headed "column" and "JSON" for the 7.3 frame.
- ☐ **Print or share:** the workbook (the Checkpoint 2 page and its Part 5 table), sticky notes for the exit ticket.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`ERROR 1146 ... Table 'bookstore.accounts' doesn't exist`	The accounts table is the lesson's own setup, not part of bookstore.sql	Run the lesson's Setup block (the CREATE TABLE accounts and its INSERT) first.
The second window says `ERROR 1049 Unknown database`	Logged in without `bookstore`, or into another server (Docker vs local)	`mysql -u root -p bookstore` in both windows; `SELECT VERSION();` in both.
Window B never gives up	A default server waits 50 seconds (`innodb_lock_wait_timeout`)	Wait, or COMMIT in A: B then finishes. Do not change the global setting in class.
`ERROR 1064` at `DELIMITER //` in Workbench or phpMyAdmin	DELIMITER is a mysql-client command	Use the mysql client for procedures (Lesson 7.1), or the tool's own delimiter setting.
`ERROR 1060 Duplicate column name 'attributes'`	The 7.3 setup or catalog_feed.sql ran twice without a reload	`SOURCE labs/_states/bookstore.sql`, then the setup once.

SYMPTOM	CAUSE	FIX
`->>` gives `ERROR 1064` on a laptop	MariaDB (XAMPP) has no arrow operators	`JSON_VALUE(attributes, '$.format')` instead; the lesson's MariaDB section.

SESSION 11 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 6.3 and 7.1; homework check (`book_club.sql`, `bookstore_routines.sql`).
0:10	30	Checkpoint 2 (Modules 5–6)	Five short parts, alone, open notes: a CTE with RANK(), a recursive CTE, a view, an index read with EXPLAIN, a normalization question on paper.
0:40	10	**7.2** Frame and mental model	All or nothing; ACID in one breath; autocommit and the statements that commit by themselves.
0:50	25	**7.2** Live demo	The lesson's store-credit accounts: COMMIT, a rehearsed DELETE, two silent traps, two windows and a lock timeout, a procedure that rolls back on any error.
1:15	15	**7.2** Guided lab	`labs/module-11/l7_2-transactions/loyalty_points_starter.sql` in pairs.
1:30	10	**7.2** Start the lesson project	checkout.sql: the setup and Step 1 (the procedure); Steps 2 to 6 go home.
1:40	10	Break	Learners reload bookstore.sql (Lesson 7.3 starts from a clean copy).
1:50	10	**7.3** Frame and mental model	Columns for what every row has, JSON for the flexible leftovers; paths and the two arrows.
2:00	25	**7.3** Live demo	The lesson's attributes column: validation, -> and ->>, missing keys, JSON_SET, JSON from rows and back, an indexed path, two silent mistakes.
2:25	15	**7.3** Guided lab	`labs/module-11/l7_3-json-columns/reader_prefs_starter.sql` in pairs.
2:40	10	**7.3** Start the lesson project	catalog_feed.sql: the lesson's setup and Step 1; Steps 2 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 12.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 110 + 100 = 210 minutes for a solo learner; after the warm-up, the checkpoint, the break and the wrap-up the room has 120, 60 per lesson. Each lesson keeps its full shape (frame, demo, lab, project start) with a shorter project start.

Goes home (about 2 to 3 hours). checkout.sql Steps 2 to 6 and catalog_feed.sql Steps 2 to 6; each lesson's Hands-on Exercises (7.2 Exercise 3 needs two windows); both Quick Quizzes; both journal entries; the labs not reached in class. Learners who struggled on the checkpoint redo the parts you marked below Proficient.

Protect, in this order: the full 30 minutes of the checkpoint; the two-window step of the 7.2 demo (the idea learners cannot get from the page); the handler (ROLLBACK then RESIGNAL); the two arrows in 7.3. **Cut first:** the 7.3 demo steps 8 and 9 (JSON from rows and back: the guided lab's fast-finisher file does both), then the 7.2 rehearsal step (ROLLBACK is in the lab), then the 7.3 project start (the setup can be done at home from the lesson page).

OPENING SEGMENT

0:00 Warm-up: Normalization and Stored Programs — 10 min

TIMING

6m	Five retrieval questions on Lessons 6.3 and 7.1 (workbook Warm-up box), answers by show of hands
4m	Homework check: who ran book_club.sql and bookstore_routines.sql twice without errors; one question each

THE QUESTIONS AND THEIR ANSWERS

1. **W1** (6.3) A book-club sheet has a cell that reads `Dune, 1984`. Which normal form does that break, and what is the fix? **Answer:** 1NF: one value per cell. Fix: one row per book in a child table (member_id, book_id).

2. **W2** (6.3) "The key, the whole key, and nothing but the key." Which part of the phrase is 2NF, and which is 3NF? **Answer:** "The whole key" is 2NF (no column depends on only part of a composite key); "nothing but the key" is 3NF (no column depends on another non-key column).

3. **W3** (7.1) Which program reads the line `DELIMITER //`: the server, the client or the procedure? **Answer:** The mysql client. It changes where the client cuts statements; the server never sees the word DELIMITER.

4. **W4** (7.1) You want `SELECT title, sale_price(price) FROM books` to work. Do you write a stored procedure or a stored function? **Answer:** A stored function: it returns one value and can sit inside an expression. A procedure runs with CALL.

W5 (7.1) Predict, then run (it creates a small function in bookstore, which the 7.2 demo does not need):

```
DROP FUNCTION IF EXISTS sale_price;
CREATE FUNCTION sale_price(p_price DECIMAL(8, 2))
RETURNS DECIMAL(8, 2) DETERMINISTIC
RETURN ROUND(p_price * 0.8, 2);
SELECT title, price, sale_price(price) AS on_sale
FROM books
WHERE id IN (1, 2)
ORDER BY id;
```

```
+-------+-------+---------+
| title | price | on_sale |
+-------+-------+---------+
| Dune  | 14.99 |   11.99 |
| 1984  |  9.99 |    7.99 |
+-------+-------+---------+
```

If W3 or W4 split the room, take one minute on the board: the client cuts statements, the server runs them; a function returns a value inside an expression, a procedure is CALLed. Both come back in today's handler.

0:10 Checkpoint 2: Modules 5 and 6

30 min

TIMING

2m	Instructions: alone, open notes (lesson pages, their own project files), no neighbor; Part 5 on paper
25m	Work: Parts 1 to 4 in `checkpoint2_starter.sql`, Part 5 in the workbook; you circulate but do not teach
3m	Stop on time; learners save the file and copy the key numbers into the workbook; collect or photograph the page

WHAT LEARNERS GET

The workbook's Checkpoint 2 page and `labs/module-11/checkpoint-2/checkpoint2_starter.sql`. Each part is a TODO line (a placeholder SELECT, so the file runs as shipped); learners replace it with their query and SOURCE the file again. Your reference is `labs/module-11/checkpoint-2/checkpoint2_solution.sql`.

PART	LESSONS	TASK
Part 1	5.1, 5.2	The best seller(s) of each category by copies sold (SUM of order_items.quantity), ties sharing first place: a CTE for the copies and a window function to rank. Columns category, title, copies, sorted by category.
Part 2	5.1	Everyone who reports to Daniel Reyes (employee 2), directly or through someone else, with a level column (1 = reports to Daniel directly), sorted by level and id: a recursive CTE.
Part 3	6.2	The view v_customer_revenue (customer_id, customer, orders, revenue) for customers who have ordered, then a query on the view: customers with revenue above 30, highest first.
Part 4	6.1	The plan of SELECT id, first_name FROM customers WHERE city = 'Chicago' before and after an index that helps it: the first line of each plan.
Part 5	6.3	On paper: the table class_signups below. Name its key, the dependencies that break 2NF and 3NF, and the tables (with keys) you would split it into.

RUNNING IT

- Say: "Thirty minutes, five parts, about five minutes each. A part you cannot finish: write what you would do in a comment. It tells me what to revisit; it is not a grade on you as a person."

- **Do not help with SQL** during the 25 minutes. Allowed: "read the question again", "which lesson is that?", and fixing a broken environment (wrong folder, no bookstore: `SOURCE labs/_states/bookstore.sql`).
- **Watch for**, and note on your list (it drives the Session 12 warm-up): a window function in WHERE (`ERROR 3593`), COUNT(*) counting order lines as orders (Alice 4 instead of 3), a recursive CTE that never adds the level, EXPLAIN typed but not read.
- Learners who finish early check their numbers by hand (Part 3: add Alice's three orders) and write the Exemplary sentence of each part.
- **Marking:** the five-row rubric in the Answer Key (20 points; Proficient in every row, 15 points, is a successful checkpoint). Mark before Session 12 and return it in the warm-up with one sentence of advice per learner.

IF THE ROOM IS NOT READY FOR 30 MINUTES OF ASSESSMENT

Programs that start with Part 2 sometimes have learners who skipped Module 5 or 6 homework. Run the checkpoint anyway, as a diagnostic: the rubric shows exactly which lesson each learner should revisit before the final project, which uses all five ideas.

LESSON PLANS

7.2 Transactions

60 min

OBJECTIVES (FROM THE LESSON PAGE)

- Group changes with START TRANSACTION, COMMIT and ROLLBACK, and explain autocommit and implicit commits
- Undo part of a transaction with SAVEPOINT and ROLLBACK TO SAVEPOINT
- Predict what a second connection sees and when it has to wait, under MySQL's default REPEATABLE READ isolation
- Lock rows on purpose with SELECT ... FOR UPDATE, and read lock errors 1205 and 1213
- Write a stored procedure whose error handler rolls back the whole transaction and passes the error on

TIMING IN THE ROOM

10m	Frame: all or nothing; ACID; autocommit and implicit commits
25m	Live demo: COMMIT, ROLLBACK, two silent traps, two windows, a procedure with a handler
15m	Guided lab: `loyalty_points_starter.sql` in pairs
10m	Start the lesson project: checkout.sql, setup and Step 1

MENTAL MODEL TO SAY

Start with the gift card: "Move 30 dollars of store credit from Alice to Bob: subtract from Alice, add to Bob. If the server stops between the two, 30 dollars vanish. A **transaction** makes the pair one unit: the database keeps both changes or neither."

Then the board line: **START TRANSACTION → your statements → COMMIT (keep all) or ROLLBACK (discard all).** "Every UPDATE you ran in this course was already a transaction: **autocommit** wraps each statement in its own and commits it at once. START TRANSACTION switches that off until you decide."

Then ACID in one breath, pointing at the bookstore: atomicity (an order never exists without its items), consistency (the CHECK holds after every commit), isolation (a report never counts a half-placed order), durability (after COMMIT it survives a power cut). And the warning list on the board: CREATE, ALTER, DROP, TRUNCATE and account statements **commit by themselves**.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

This is the lesson's own sequence, from a fresh `bookstore.sql`. Learners type along in their own window; for the two-window step they open a second terminal. Ask for a prediction before every statement that prints something.

1. **The accounts table (2m).** "Gift-card balances, one row per customer, and a CHECK that makes a negative balance impossible. The lesson creates this table; it is not in bookstore.sql."

```
DROP TABLE IF EXISTS accounts;
CREATE TABLE accounts (
    id      INT PRIMARY KEY,
    name    VARCHAR(100) NOT NULL,
    balance DECIMAL(10, 2) NOT NULL,
    CONSTRAINT chk_balance CHECK (balance >= 0)
);
INSERT INTO accounts (id, name, balance) VALUES
    (1, 'Alice Johnson', 150.00),
    (2, 'Bob Smith', 50.00),
    (3, 'Carol Williams', 0.00);
SELECT * FROM accounts ORDER BY id;
```

```
+----+----------------+---------+
| id | name           | balance |
+----+----------------+---------+
|  1 | Alice Johnson  |  150.00 |
|  2 | Bob Smith      |   50.00 |
|  3 | Carol Williams |    0.00 |
+----+----------------+---------+
```

Learners who type along: this is the lesson's Setup block, run after reloading `bookstore.sql`.

2. **COMMIT: a transfer that sticks (3m).** "Move 30 dollars from Alice to Bob. Two UPDATEs that only make sense together. Watch the Query OK lines: each UPDATE changed one row." Then, before COMMIT is typed, ask: "if the power went out right now, what would the server keep?" (Neither change.)

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 30 WHERE id = 1;
UPDATE accounts SET balance = balance + 30 WHERE id = 2;
COMMIT;
```

```
Query OK, 0 rows affected
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0
Query OK, 0 rows affected
```

3. **Check it (1m).** "After COMMIT the change is permanent and visible to everyone."

```
SELECT * FROM accounts ORDER BY id;
```

```
+----+----------------+---------+
| id | name           | balance |
+----+----------------+---------+
|  1 | Alice Johnson  |  120.00 |
|  2 | Bob Smith      |   80.00 |
|  3 | Carol Williams |    0.00 |
+----+----------------+---------+
```

4 **ROLLBACK: a rehearsal (3m).** "The nightmare: someone deletes every order line. Inside a transaction it is a rehearsal." Predict both counts first.

```
START TRANSACTION;
DELETE FROM order_items;
SELECT COUNT(*) AS items_inside FROM order_items;
ROLLBACK;
SELECT COUNT(*) AS items_after_rollback FROM order_items;
```

```
+--------------+
| items_inside |
+--------------+
|            0 |
+--------------+
+----------------------+
| items_after_rollback |
+----------------------+
|                   18 |
+----------------------+
```

Say the habit aloud: before a risky UPDATE or DELETE by hand, type START TRANSACTION first, check with a SELECT, then decide.

5 **Silent trap 1: a schema change inside a transaction (3m).** "I zero Bob's balance, create a little audit table, change my mind and roll back." Predict Bob's balance.

```
START TRANSACTION;
UPDATE accounts SET balance = 0 WHERE id = 2;
CREATE TABLE demo_audit (note VARCHAR(100));
ROLLBACK;
SELECT id, balance FROM accounts WHERE id = 2;
```

```
+----+---------+
| id | balance |
+----+---------+
|  2 |    0.00 |
+----+---------+
```

The CREATE TABLE committed the open transaction before it ran, so ROLLBACK had nothing to undo. Repair it (no output): `UPDATE accounts SET balance = 80.00 WHERE id = 2; DROP TABLE demo_audit;`

6 **Silent trap 2: COMMIT after an error (3m).** "Carol has 0.00 and sends Bob 40 by hand." Run it as one block, so the COMMIT runs after the error.

```
START TRANSACTION;
UPDATE accounts SET balance = balance + 40 WHERE id = 2;
UPDATE accounts SET balance = balance - 40 WHERE id = 3;
COMMIT;
SELECT * FROM accounts ORDER BY id;
```

```
ERROR 3819 (HY000): Check constraint 'chk_balance' is violated.
+----+----------------+---------+
| id | name           | balance |
+----+----------------+---------+
|  1 | Alice Johnson  |  120.00 |
|  2 | Bob Smith      |  120.00 |
|  3 | Carol Williams |    0.00 |
+----+----------------+---------+
```

In MySQL a failed statement undoes only itself; the transaction stays open with Bob's +40 in it, and COMMIT saved it: 40 dollars from nothing (PostgreSQL would refuse the COMMIT). Repair: `UPDATE accounts SET balance = balance - 40 WHERE id = 2;`. The fix in code is a handler; at the prompt, ROLLBACK after any error.

7 **Two windows (5m).** "Real databases serve many connections at once." Open a second terminal on the projector, side by side, label them A and B on the board. The table continues from the steps above (Alice 120, Bob 80, Carol 0); it comes from two real client sessions:

WINDOW	STATEMENT	WHAT IT SHOWS (REAL OUTPUT)
A	`START TRANSACTION; UPDATE accounts SET balance = balance - 20 WHERE id = 1; SELECT id, balance FROM accounts WHERE id = 1;`	`100.00`: A sees its own change

WINDOW	STATEMENT	WHAT IT SHOWS (REAL OUTPUT)
B	`SELECT id, balance FROM accounts WHERE id = 1;`	`120.00` at once: the committed value, no waiting
B	`UPDATE accounts SET balance = balance + 1 WHERE id = 1;`	waits, then `ERROR 1205 (HY000): Lock wait timeout exceeded; try restarting transaction` (5 s on the test server; 50 s on a default server)
A	`COMMIT;`	releases the row lock
B	`SELECT id, balance FROM accounts WHERE id = 1;`	`100.00`: now B sees it

Say: "readers never wait and never see unfinished work; writers wait for writers." If you commit in A while B is still waiting, B's UPDATE simply finishes: show that once if the room is with you. FOR UPDATE (a read that locks) and deadlocks (1213) stay on the slides.

8 **A procedure that cleans up after itself (3m).** "At the prompt you decide; in a procedure the code must decide. The handler is two lines: ROLLBACK, then RESIGNAL." Paste this one (it is the lesson's; do not type 28 lines live) and walk through the handler, the SIGNALs and FOR UPDATE.

```
DROP PROCEDURE IF EXISTS transfer;
DELIMITER //
CREATE PROCEDURE transfer(IN p_from INT, IN p_to INT, IN p_amount DECIMAL(10,
BEGIN
    DECLARE v_balance DECIMAL(10, 2);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;    -- undo everything this call did
        RESIGNAL;    -- and pass the original error to the caller
    END;
    IF p_amount IS NULL OR p_amount <= 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Amount must be positive';
    END IF;
    START TRANSACTION;
    SELECT balance INTO v_balance FROM accounts WHERE id = p_from FOR UPDATE;
    IF v_balance IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No such source account';
    END IF;
    IF v_balance < p_amount THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient funds';
    END IF;
    UPDATE accounts SET balance = balance - p_amount WHERE id = p_from;
    UPDATE accounts SET balance = balance + p_amount WHERE id = p_to;
    IF ROW_COUNT() = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No such destination accoun
    END IF;
    COMMIT;
END //
DELIMITER ;
```

9 **A good call, and the hardest failing one (2m).** "Alice sends Carol 5. Then Alice sends 5 to account 99, which does not exist: the debit succeeds, the credit finds no row."

```
CALL transfer(1, 3, 5.00);
SELECT * FROM accounts ORDER BY id;
```

```
+----+---------------+---------+
| id | name          | balance |
+----+---------------+---------+
|  1 | Alice Johnson |   95.00 |
|  2 | Bob Smith     |   80.00 |
|  3 | Carol Williams |    5.00 |
+----+---------------+---------+
```

10 **The failing call (1m).** Predict Alice's balance after it.

```
CALL transfer(1, 99, 5.00);
SELECT * FROM accounts ORDER BY id;
```

```
ERROR 1644 (45000): No such destination account
+----+----------------+---------+
| id | name           | balance |
+----+----------------+---------+
|  1 | Alice Johnson  |   95.00 |
|  2 | Bob Smith      |   80.00 |
|  3 | Carol Williams |    5.00 |
+----+----------------+---------+
```

Alice still has 95.00: the handler rolled the debit back before passing the error on.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Committing after an error.** "I got 3819, then typed COMMIT, and Bob still got his 40." Coach: "In MySQL a failed statement undoes only itself. After any error inside a transaction, ROLLBACK, or let a handler do it." (Self-check Q3 and the lab's debug file.)
- **Expecting a lock to block reads.** "Why could B read the row A was changing?" Coach with the two columns on the board: readers get the last committed version and never wait; only writers (and FOR UPDATE) wait for the lock.
- **A transaction left open in a forgotten window.** Someone's UPDATE hangs for 50 seconds. Coach: "Which window has an open transaction? COMMIT or ROLLBACK it." Closing a window rolls back, which also releases the locks.

GUIDED LAB

File: learners open `labs/module-11/l7_2-transactions/loyalty_points_starter.sql`; your reference is `labs/module-11/l7_2-transactions/loyalty_points_solution.sql`. A new `loyalty` table (points per customer, never below zero) and the real orders: each TODO practices one demo idea on new data.

- **SOURCE it as shipped (2m):** `SOURCE labs/module-11/l7_2-transactions/loyalty_points_starter.sql` from a client started in the course folder with `bookstore` as the database. Learners should see unchanged points, 12 orders before and after, and a table of zeros (TODO 3's slip, committed).
- **TODO 1 together (3m), TODOs 2 to 4 in pairs (8m).** Each is a small edit and a new SOURCE. TODO 4 is the implicit-commit trap again: learners must move a line, not add one.

- **Expected:** TODO 1 Frank 15 / Grace 25; TODO 2 inside the rehearsal 7 orders and 10 lines (the cascade took 8 lines with 5 orders), after the ROLLBACK 12 and 18; TODO 3 1: 350, 2: 170, 4: 10, 6: 15, 7: 25; TODO 4 Bob back at 170 and `log_rows` 0 (as shipped: 10 and 1, because the CREATE TABLE ended the transaction and the INSERT autocommitted).
- **Debrief (2m):** "Which TODO would have hurt in production?" (3: a committed slip; 4: a rollback that silently did nothing.)
- **Fast finishers:** `labs/module-11/l7_2-transactions/return_item_starter.sql` (a procedure with a handler; the failing calls are typed at the prompt), then `labs/module-11/l7_2-transactions/debug_lost_points.sql` (three hand-typed transactions: errors 3819 and 1305 and one silent bug).

START THE LESSON PROJECT

The lesson project is **An All-or-Nothing Checkout**: `checkout.sql`, a procedure that places an order and charges store credit, all or nothing. In the room (10 minutes): the setup and Step 1.

- Everyone runs the lesson's Setup block (the `accounts` table) if they did not type along in the demo.
- Step 1: learners create `checkout.sql` in their course folder, paste or type the procedure from the lesson page and SOURCE it. Walk the room for `DELIMITER` problems (a missing `DELIMITER ;` at the end leaves the client waiting).
- **Done in the room** = `CALL checkout(1, 11, 1, @order_id);` runs and Alice's credit drops by 8.49.
- **Goes home:** Steps 2 to 6: the failing calls (out of stock, unknown book, not enough credit, no account), the integrity audit, the gap in the order ids, and a file that runs twice.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- After `START TRANSACTION; UPDATE ...;` who can see the change? **Answer:** only your own session, until COMMIT.
- Which of these ends a transaction by itself: SAVEPOINT, CREATE TABLE, SELECT? **Answer:** CREATE TABLE (an implicit commit).
- B's plain SELECT did not wait for A's lock. Why did B's UPDATE wait? **Answer:** writers need the row lock; readers read the committed snapshot.
- Why RESIGNAL after ROLLBACK in a handler? **Answer:** so the caller still learns the original error.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** B) The server rolls it back when the connection ends; **Q2** Two rows: id 1 qty `4`, id 2 qty `8`; **Q3** C) `ALTER TABLE books ADD COLUMN rating INT;`; **Q4** D) Nothing yet: B waits for A's row lock, and fails with ERROR 1205 if A holds it past the timeout; **Q5** A) So the caller still receives the original error after the half-done

changes are undone; **Q6** D) Run the whole transaction again from the start. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: pair learners who are unsure in the terminal with a confident typist for the two-window step (one laptop, two windows, two people). For the lab, TODO 1 is a two-number edit: start everyone there.

Stretch: `return_item_starter.sql` and `debug_lost_points.sql`; the lesson's Exercise 4 (a safe `cancel_order` procedure) and the Bonus Challenge (a real deadlock, 1213, in two windows, and `SHOW ENGINE INNODB STATUS\G`).

7.3 JSON Columns

60 min

OBJECTIVES (FROM THE LESSON PAGE)

- Add a JSON column and explain how MySQL validates and stores what you put in it
- Read values with ->, ->> and JSON_EXTRACT, and filter rows on them
- Change parts of a document with JSON_SET, JSON_INSERT, JSON_REMOVE and JSON_ARRAY_APPEND
- Turn rows into JSON with JSON_OBJECT and JSON_ARRAYAGG, and JSON back into rows with JSON_TABLE
- Index a JSON path through a generated column, and decide when data belongs in real columns instead

TIMING IN THE ROOM

10m	Frame: columns for what every row has, JSON for the flexible leftovers; paths, -> and ->>
25m	Live demo: the lesson's attributes column from setup to an indexed path, and two silent mistakes
15m	Guided lab: `reader_prefs_starter.sql` in pairs
10m	Start the lesson project: catalog_feed.sql, the setup and Step 1

MENTAL MODEL TO SAY

Use the two columns on the board: "Every book has a title, a price, an author: **columns**. Only some books have a series, some won awards, some are illustrated, and marketing invents a new detail every month. A column per detail would be mostly NULL; a **JSON column** holds one small document per row for the flexible leftovers."

Then the path: "You reach inside a document with a path. **$** is the whole document, **$.format** a key, **$.awards[0]** the first element." And the two arrows, written large: **->** gives JSON (a string keeps its quotes), **->>** gives plain text for people, comparisons and sorting.

Then the rule that decides everything: "If every row has it, you join on it, sum it or protect it with a constraint, it is a column. JSON is for optional, varied, mostly-for-display details. Design the tables first; JSON gets the leftovers."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

The lesson's own sequence, from a fresh `bookstore.sql` (learners reloaded it at the break). Ask for a prediction before each step that prints something.

1. **Add the column and fill it (3m).** "Details that only some books have: a format, a series, awards. One JSON document per row, next to the ordinary columns." Learners reloaded bookstore.sql at the break; these are the lesson's four statements.

```
ALTER TABLE books ADD COLUMN attributes JSON;
UPDATE books SET attributes = '{"format": "paperback", "series": "Dune Chronic
UPDATE books SET attributes = '{"format": "hardcover", "series": "Middle-earth
UPDATE books SET attributes = '{"format": "paperback", "awards": []}' WHERE at
```

```
Query OK, 0 rows affected
Records: 0  Duplicates: 0  Warnings: 0
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0
Query OK, 2 rows affected
Rows matched: 2  Changed: 2  Warnings: 0
Query OK, 8 rows affected
Rows matched: 8  Changed: 8  Warnings: 0
```

2 **What was stored (2m).** Point at Dune: "I typed format, series, awards. MySQL shows awards first: it parsed the text and sorted the keys. Never rely on key order."

```
SELECT id, title, attributes
FROM books
WHERE id IN (1, 2, 6)
ORDER BY id;
```

```
+----+------------+-------------------------------------------------------------
| id | title      | attributes
+----+------------+-------------------------------------------------------------
|  1 | Dune       | {"awards": ["Hugo", "Nebula"], "format": "paperback", "ser
|  2 | 1984       | {"awards": [], "format": "paperback"}
|  6 | The Hobbit | {"format": "hardcover", "series": "Middle-earth", "illustr
+----+------------+-------------------------------------------------------------
```

3 **MySQL checks every document (1m).** "JavaScript lets you write a bare key. JSON does not."

```
UPDATE books SET attributes = '{format: "ebook"}' WHERE id = 2;
```

```
ERROR 3140 (22032): Invalid JSON text: "Missing a name for object member." at
```

Fix: double quotes around every key, or let `JSON_OBJECT('format', 'ebook')` write it for you.

4 **Two arrows (3m).** "Every path starts at $. The single arrow gives JSON, quotes and all; the double arrow gives plain text." Predict which column has quotes.

```
SELECT
    title,
    JSON_EXTRACT(attributes, '$.format') AS extracted,
    attributes->'$.format'               AS arrow,
    attributes->>'$.format'              AS double_arrow
FROM books
WHERE id IN (1, 6)
ORDER BY id;
```

```
+------------+-------------+-------------+--------------+
| title      | extracted   | arrow       | double_arrow |
+------------+-------------+-------------+--------------+
| Dune       | "paperback" | "paperback" | paperback    |
| The Hobbit | "hardcover" | "hardcover" | hardcover    |
+------------+-------------+-------------+--------------+
```

5 **Arrays and missing keys (2m).** "[0] is the first element. 1984 has an empty array; the Hobbit has no awards key at all. A missing key is NULL, not an error."

```
SELECT
    title,
    attributes->>'$.awards[0]'          AS first_award,
    JSON_LENGTH(attributes->'$.awards') AS award_count,
    attributes->>'$.series'             AS series
FROM books
WHERE id IN (1, 2, 6)
ORDER BY id;
```

```
+------------+-------------+-------------+-----------------+
| title      | first_award | award_count | series          |
+------------+-------------+-------------+-----------------+
| Dune       | Hugo        |           2 | Dune Chronicles |
| 1984       | NULL        |           0 | NULL            |
| The Hobbit | NULL        |        NULL | Middle-earth    |
+------------+-------------+-------------+-----------------+
```

6 **Is it in the array? (1m).** "MEMBER OF asks: is this value one of the array's elements?"

```
SELECT title
FROM books
WHERE 'Hugo' MEMBER OF (attributes->'$.awards');
```

```
+-------+
| title |
+-------+
| Dune  |
+-------+
```

7 **Change part of a document (2m).** "JSON_SET returns a changed copy; UPDATE stores it." Mention the family: SET adds or replaces, INSERT only adds, REPLACE only replaces, REMOVE, ARRAY_APPEND.

```
UPDATE books
SET attributes = JSON_SET(attributes, '$.edition', '75th anniversary')
WHERE id = 6;
SELECT attributes->>'$.edition' AS edition, attributes->>'$.format' AS format
FROM books WHERE id = 6;
```

```
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0
+------------------+-----------+
| edition          | format    |
+------------------+-----------+
| 75th anniversary | hardcover |
+------------------+-----------+
```

8 **JSON from rows (2m).** "An app wants JSON? MySQL can build it." Point out that numbers stay numbers and the keys come out sorted again.

```
SELECT JSON_OBJECT('id', id, 'title', title, 'price', price) AS item
FROM books
WHERE id <= 3
ORDER BY id;
```

```
+-------------------------------------------------------------+
| item                                                        |
+-------------------------------------------------------------+
| {"id": 1, "price": 14.99, "title": "Dune"}                  |
| {"id": 2, "price": 9.99, "title": "1984"}                   |
| {"id": 3, "price": 11.99, "title": "To Kill a Mockingbird"} |
+-------------------------------------------------------------+
```

9 **JSON back to rows (3m).** "Arrays are awkward to report on. JSON_TABLE makes one row per element." Read it aloud: for every book, take $.awards, one row per element, call the element award.

```
SELECT b.title, aw.award
FROM books AS b,
     JSON_TABLE(
         b.attributes,
         '$.awards[*]'
         COLUMNS (award VARCHAR(40) PATH '$')
     ) AS aw
ORDER BY b.title, aw.award;
```

```
+-------+--------+
| title | award  |
+-------+--------+
| Dune  | Hugo   |
| Dune  | Nebula |
+-------+--------+
```

10 **Index a path (3m).** "You cannot index the JSON column itself (error 3152), but you can index a generated column that pulls one value out."

```
ALTER TABLE books
    ADD COLUMN format VARCHAR(20)
        GENERATED ALWAYS AS (attributes->>'$.format') VIRTUAL,
    ADD INDEX idx_books_format (format);
SELECT title, format FROM books WHERE format = 'hardcover' ORDER BY title;
```

```
+------------------+-----------+
| title            | format    |
+------------------+-----------+
| Lord of the Rings | hardcover |
| The Hobbit       | hardcover |
+------------------+-----------+
```

11 **Ask the optimizer (1m).** "MySQL even uses that index for the original JSON expression."

```
EXPLAIN SELECT title FROM books WHERE attributes->>'$.format' = 'hardcover'\G
```

```
*************************** 1. row ***************************
EXPLAIN: -> Index lookup on books using idx_books_format (format = 'hardcover
```

12 **Two silent mistakes (2m).** "No error, wrong answer." A capital F, then the single arrow inside CONCAT.

```sql
SELECT title
FROM books
WHERE attributes->>'$.Format' = 'hardcover';
SELECT CONCAT(title, ' (', attributes->'$.format', ')') AS label
FROM books
WHERE id = 1;
```

```
Empty set
+--------------------+
| label              |
+--------------------+
| Dune ("paperback") |
+--------------------+
```

Keys are case-sensitive and a missing key is NULL, so the typo finds nothing and warns nobody. The quotes would end up on the web page: use `->>` whenever a value leaves JSON.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **JSON written like JavaScript** (`{format: "ebook"}`, single quotes inside): error 3140. Coach: "keys and text in double quotes; the whole document inside SQL's single quotes", or `JSON_OBJECT` writes it for you.
- **The single arrow where text was meant**: quotes in labels, comparisons with `'"paperback"'`. Coach: "Is the value leaving JSON (display, CONCAT, WHERE with text)? Then two arrows."
- **A typo in a key, and no error**: `$.Format` returns NULL everywhere. Coach: copy key names from the data (`SELECT attributes FROM books LIMIT 1`), keep one spelling, and protect important keys with a CHECK or a generated column.

GUIDED LAB

File: `labs/module-11/l7_3-json-columns/reader_prefs_starter.sql`; reference `labs/module-11/l7_3-json-columns/reader_prefs_solution.sql`. Customers get a `prefs` column (newsletter, favorite genres, a nested contact object); the same ideas as the demo on new data. The file adds the column and removes it again at the end, so it can be SOURCEd after every edit.

- **TODOs 1 and 2 together (3m), 3 to 6 in pairs (10m).** TODO 1 is the classic: a JSON `true` read with `->>` is the text `'true'`.
- **Expected:** newsletter Alice, Frank, Grace; Classic Alice, Grace; TODO 3 `true` only for Alice; TODO 4 `has_genres` 0 for Dave (prefs without the key) but NULL for Carol, Eve and Hank (no prefs at all); TODO 5 Grace `{"genres": ["Fantasy", "Classic", "Science Fiction"], "newsletter": true}`; TODO 6 NULL 3, false 1, true 4.

- **Circulate for:** `= true` instead of `= 'true'` (no rows: a JSON boolean is not the number 1), `MEMBER OF (prefs->>'$.genres')` with two arrows (an error: MEMBER OF needs a JSON array), and edits that leave the file half-run (`ERROR 1060` next time: reload `bookstore.sql`).
- **Fast finishers:** `labs/module-11/l7_3-json-columns/order_documents_starter.sql` (rows to JSON with JSON_OBJECT and JSON_OBJECTAGG, a supplier's JSON to rows with JSON_TABLE), then `labs/module-11/l7_3-json-columns/debug_bad_json.sql` (three errors, 3140, 3143 and 3152, and two silent bugs).
- **Debrief (2m):** "Which of the six TODOs would a real column have caught with an error?" (TODO 4's missing key, and the debug file's misspelled key: a real column would raise 1054.)

START THE LESSON PROJECT

The lesson project is **The Catalog Feed**: `catalog_feed.sql`, a rerunnable script that tags books, lists award winners and prints a JSON feed. In the room (10 minutes): the setup and Step 1.

- The project builds on the lesson's own setup: the `attributes` column and its three UPDATEs, the generated `format` column and the `chk_books_format` CHECK. Learners who followed the demo have the first two; everyone runs the CHECK from the lesson page.
- Step 1: start `catalog_feed.sql` with `USE bookstore;` , make Brave New World and Fahrenheit 451 ebooks with JSON_SET, run it.
- **Done in the room** = both books show `"format": "ebook"` and a `file_mb` key.
- **Goes home:** Steps 2 to 6 (titles per format, awards with JSON_TABLE, the feed, the series index, and a file that reloads the database and runs twice).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- `attributes->'$.format'` or `attributes->>'$.format'` in a WHERE with `'hardcover'` ? **Answer:** `->>` : the text, without quotes.
- What does a path to a key that is not there return? **Answer:** NULL, no error.
- How do you make `WHERE attributes->>'$.format' = 'hardcover'` fast? **Answer:** a generated column for the path, with an index.
- Price in JSON or in a column? **Answer:** a column: every book has one and every total uses it.

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** C) Use `->>` instead of `->` ; **Q8** `gift` and `3` ; **Q9** A) The document unchanged; **Q10** B) A generated column for $.format, with an index on that column; **Q11** `{"id": 1, "price": 14.99, "title": "Dune"}` ; **Q12** D) An optional gift-wrap note some orders have, with a color and a message. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: keep the two arrows on the board for the whole lab; for learners lost in the nesting, write the TODO 5 UPDATE as two separate UPDATEs first (JSON_ARRAY_APPEND, then JSON_REMOVE), then nest them.

Stretch: `order_documents_starter.sql` and `debug_bad_json.sql`; the lesson's Exercise 4 (price an incoming web order with JSON_TABLE, never trusting its prices) and the Bonus Challenge (a multi-valued index on the awards array).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note. Sort them before Session 12: question 1's "not yet" pile gets the first minute of the Session 12 warm-up.

1. Inside a transaction, an UPDATE fails with error 3819 and you then type COMMIT. What is saved? **Answer:** every change made before the failed statement: a failed statement undoes only itself. ROLLBACK after an error.
2. Write the handler that makes a procedure undo its transaction on any error and still report the error. **Answer:** `DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;`
3. Name one bookstore fact that belongs in JSON and one that must stay a column, with a reason each. **Answer (example):** a book's series (only some books have one, display only) in JSON; its price in a column (every book has one, totals use it).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** checkout.sql Steps 2 to 6 (the failing calls, the integrity audit, the id gap, a file that runs twice) and catalog_feed.sql Steps 2 to 6 (the feed and a file that reloads and runs twice).
- **Lesson exercises.** 7.2: Exercises 1 to 3 (Exercise 3 needs two windows); Exercise 4 is a stretch. 7.3: Exercises 1 to 3; Exercise 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages (answers in the Answer Key).
- **Journal:** both entries. The 7.2 prompt asks learners to explain the two-window experiment as if to a friend.
- **Labs not reached in class:** `return_item_starter.sql`, `debug_lost_points.sql`, `order_documents_starter.sql`, `debug_bad_json.sql`.
- **Checkpoint 2:** anyone who did not finish a part writes it at home and brings it; it is reviewed in the Session 12 warm-up.
- **Before the Next Lesson (from 7.3):** reload `bookstore.sql`: Lesson 7.4 starts from the standard database, without the attributes column.

BRING LIST FOR SESSION 12

- A laptop with MySQL 9.7, `bookstore` freshly reloaded, and the root password: Lesson 7.4 creates accounts as root and logs in as them.
- A terminal you can open twice (two windows) and use outside the mysql client: Lesson 8.1's mysqldump runs at the shell prompt, not at `mysql>`.
- checkout.sql and catalog_feed.sql (finished), and the Checkpoint 2 page.
- The workbook and the journal. Session 12 ends with the final-project brief: bring a calendar for the two studio sessions.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the full checkpoint; the two-window step; the handler; the two arrows.
- **Cut first (saves 5m):** 7.3 demo steps 8 and 9 (JSON from rows and back): the fast-finisher lab covers them.
- **Cut second (saves 3m):** the 7.2 rehearsal step; the lab's TODO 2 does the same.
- **Cut third (saves 10m):** the 7.3 project start: learners run the lesson's setup and Step 1 at home.

RUNNING SHORT: WHAT TO ADD

- FOR UPDATE live in two windows (the lesson's two cashiers): B's locking read waits, A commits, B sees the new balance.
- Run `debug_lost_points.sql` for everyone and fix it together, one transaction per volunteer.
- The 7.3 Bonus Challenge: a multi-valued index on the awards array and its EXPLAIN.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 12: Users, Permissions and Backups

How to teach the twelfth session of MySQL Foundations: user accounts and least privilege (grants at three levels, logging in as each account, roles and the forgotten SET DEFAULT ROLE), then moving data in and out (a mysqldump backup proved by a test restore, the 9.7 GTID trap, a CSV export, a messy file through a staging table), and the hand-out of the final-project brief.

`2 lessons` `3 hrs session` `Lessons 7.4–8.1` `Session 12 of 14 · Part 2` `Final-project brief`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame → live demo → guided lab → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Session 12 is the last teaching session of Part 2: it ends with a 20-minute **final-project brief** before the wrap-up, so each lesson gets 65 minutes in the room (the pages estimate 105 + 105 alone).
- This session has two windows of a different kind: for 7.4 a second **mysql client logged in as another account**, for 8.1 a plain **terminal** for mysqldump. Every output below is real: the logins AS shop_reports, support_desk and tom were made with the mysql client on the kit's MySQL 9.7 test server; the shell commands ran there with the same options (shown as learners type them, `-u root -p`).
- Keep `lesson_19.html` (7.4), `lesson_20.html` (8.1) and `lesson_final_project.html` (8.2, for the brief) open in browser tabs. Labs live in `labs/module-12/`; `_solution.sql` files are yours.
- **Safety.** Never change root's password or privileges in class, and never let learners practice on a server other people depend on. Every lab account is a `kit_` demo account that its own file drops again.

> **THE #1 RISK OF SESSION 12: SOMEONE LOCKS THEMSELVES OUT**
>
> Lesson 7.4 is the first lesson where a typo can lock a learner out of their own server: `ALTER USER CURRENT_USER() IDENTIFIED BY ...`, or a REVOKE aimed at root. **Before:** say it once at the start of 7.4 and write it on the board: "we only ever change kit_ and demo accounts". **In the room:** if someone does lose root anyway, do not fix it live; pair them up and help at the break (recovery means restarting the server with special options, Lesson 7.4's ground rules). **After:** the labs and the checker refuse root and SET GLOBAL, so homework is safe.

BEFORE THE SESSION

- ☐ **Bring list you can assume (from Session 11):** a laptop with MySQL 9.7, `bookstore` freshly reloaded, the root password; a terminal that can be opened twice; checkout.sql and catalog_feed.sql finished; the Checkpoint 2 page; the workbook and the journal.
- ☐ **Checkpoint 2 marked** with the rubric (Session 11's Answer Key), one sentence of advice per learner: you return it in the warm-up.
- ☐ **Files:** copy `labs/module-12/` **without** the `*_solution.sql` files into the shared folder (`new_titles.csv` included). For the brief: the link to `lesson_final_project.html`, or printouts of its Brief, What you hand in, Milestones and How Your Work Is Assessed sections. Keep the kit's `final-assessment.html` (the final-project rubric with instructor notes) for yourself.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-12` from this kit's folder: 10 files, every line PASS.
- ☐ **Projector machine:** MySQL 9.7 with a fresh `bookstore`; a root mysql window and a second terminal at 18 pt, side by side; the lesson's `new_books.csv` and `price_update.csv` saved in your server's `secure_file_priv` folder and in the course folder; a third terminal (plain shell) in the course folder for mysqldump. Find your folder now: `SHOW VARIABLES LIKE 'secure_file_priv';`.
- ☐ **Board, before learners arrive:** "GRANT what ON where TO whom"; the error ladder "1045 password · 1044 database · 1142 table · 1143 column · 3118 locked"; and "mysqldump = terminal · INTO OUTFILE / LOAD DATA = mysql>".

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`ERROR 1819 ... does not satisfy the current policy requirements`	The validate_password component (some installers enable it)	A longer password with mixed case, digits and a symbol; never uninstall the component in class.
Logging in as the demo account says `ERROR 1045`	Typo in the password, or the account was created for another host	Retype; `SELECT user, host FROM mysql.user;` and use the exact pair.
`mysqldump: command not found` / not recognized	The client tools' folder is not on the PATH (Session 1)	Same PATH fix as for `mysql`; Docker: `docker exec mysql97 mysqldump ...`.
`ERROR 3546 ... GTID_PURGED cannot be changed` on restore	A 9.7 dump without `--set-gtid-purged=OFF`	Dump again with the option (the demo shows it on purpose).

SYMPTOM	CAUSE	FIX
Windows: the restored file is gibberish or `ERROR ... ASCII '\0'`	PowerShell's `>` wrote UTF-16	`--result-file=...` for the dump; `--commands -e "SOURCE file"` for the restore.
`ERROR 1290` for every INTO OUTFILE	Not the server's folder	`SHOW VARIABLES LIKE 'secure_file_priv';` and use that folder; NULL means file statements are disabled (the client-side methods still work).
`ERROR 3948` or `2068` with LOAD DATA LOCAL	local_infile OFF on the server, or the client started without `--local-infile=1`	The labs need neither; for the demo file, use the server-side LOAD DATA INFILE.

SESSION 12 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 7.2 and 7.3; Checkpoint 2 returned with one sentence of advice each.
0:10	10	**7.4** Frame and mental model	Why not root; 'user'@'host'; least privilege; GRANT what ON where TO whom.
0:20	30	**7.4** Live demo	The lesson's accounts: grants at three levels, logging in AS each account in a second window, REVOKE, roles and the forgotten SET DEFAULT ROLE, a locked account.
0:50	15	**7.4** Guided lab	`labs/module-12/l7_4-user-management/warehouse_accounts_starter.sql` in pairs.
1:05	10	**7.4** Start the lesson project	team_access.sql Steps 1 to 3; Steps 4 to 6 go home.
1:15	10	Break	Learners reload bookstore.sql and open a plain terminal next to the mysql client for 8.1.
1:25	10	**8.1** Frame and mental model	Whole databases (mysqldump, at the terminal) versus rows (INTO OUTFILE and LOAD DATA, at mysql>); where the server may touch files.
1:35	30	**8.1** Live demo	A backup, a test restore, the 9.7 GTID trap; a CSV export; a messy CSV through a staging table; two refusals and one silent warning.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
2:05	15	**8.1** Guided lab	`labs/module-12/l8_1-importing-exporting/new_titles_import_starter.sql` in pairs.
2:20	10	**8.1** Start the lesson project	The backup kit, Steps 1 and 2 (backup account, a dump with it); Steps 3 to 6 go home.
2:30	20	Final project brief	Hand out Lesson 8.2's brief (the Maple Street Tool Library); read Rosa's needs, find the entities in pairs, walk the milestones and the two studio sessions.
2:50	10	Wrap-up	Exit ticket (3 questions), homework (including the Milestone 1 draft), bring list for Session 13.
Total	180		Ends at 3:00.

In the room. 105 + 105 = 210 minutes alone; the room has 130 after the warm-up, the break, the brief and the wrap-up: 65 per lesson, each with its full shape. The final-project brief takes 20 minutes because Sessions 13 and 14 are studios: learners must arrive at Session 13 with the brief read and a first design draft.

Goes home (about 2 to 3 hours). team_access.sql Steps 4 to 6 and the backup kit Steps 3 to 6; each lesson's Hands-on Exercises; both Quick Quizzes; both journal entries; the labs not reached; and the **final project's Milestone 1 draft** (design notes on paper, no SQL yet).

Protect, in this order: the logins AS another account (7.4's whole idea: test both directions); the test restore and the GTID trap; the staging-table import; the final-project brief. **Cut first:** the 8.1 LOCAL warning step (Common Mistake 5 on the page), then the 7.4 old-advice and two-refusals steps (the debug handout has them), then the 8.1 project start (Step 1 is three statements learners can run at home).

OPENING SEGMENT

0:00 Warm-up: Transactions and JSON — 10 min

TIMING

6m	Five retrieval questions on Lessons 7.2 and 7.3 (workbook Warm-up box), answers by show of hands
4m	Checkpoint 2 returned: each learner reads your sentence of advice; name the one part most of the room should revisit

THE QUESTIONS AND THEIR ANSWERS

1. **W1** (7.2) `START TRANSACTION;`, an UPDATE, an `ALTER TABLE`, then `ROLLBACK;`. Is the UPDATE undone? **Answer:** No. ALTER TABLE caused an implicit commit before it ran, so the UPDATE was already committed and the ROLLBACK had nothing to undo.

2. **W2** (7.2) What is the difference between ERROR 1205 and ERROR 1213? **Answer:** 1205: a lock wait timed out; only the waiting statement failed and the transaction is still open. 1213: a deadlock; InnoDB rolled the whole transaction back, so retry all of it.

3. **W3** (7.3) In `WHERE attributes ? '$.format' = 'hardcover'`, which arrow goes where the ? is, and why? **Answer:** ->> : it returns plain text; -> would return the JSON string "hardcover" with its quotes, and the comparison would fail.

4. **W4** (7.3) What does JSON_TABLE do, in one sentence? **Answer:** It turns a JSON array (in a column or a literal) into rows with typed columns that you can join, group and filter.

W5 (7.2 + 7.3) Predict, then run (in a practice database):

```
CREATE TABLE notes (id INT PRIMARY KEY, doc JSON);
INSERT INTO notes VALUES (1, '{"qty": 2, "tags": ["gift"]}');
START TRANSACTION;
UPDATE notes SET doc = JSON_SET(doc, '$.qty', 5);
ROLLBACK;
SELECT doc->>'$.qty' AS qty, doc->'$.tags' AS tags FROM notes;
```

```
+------+----------+
| qty  | tags     |
+------+----------+
| 2    | ["gift"] |
+------+----------+
```

Checkpoint 2: if more than a third of the room was below Proficient in the same row, assign that lesson's exercises again; Part 3's COUNT(*) slip and Part 1's window function in WHERE come back in the final project's Saturday queries.

7.4 User Management & Permissions

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Create accounts as 'user'@'host' and explain how MySQL 9.7 stores their passwords (caching_sha2_password)
- Grant and revoke privileges at the global, database, table and column level, and read SHOW GRANTS
- Log in as a restricted user and recognize the access-denied errors (1142, 1143, 1044, 3118)
- Group privileges into roles, assign them, and make them active with SET DEFAULT ROLE
- Design least-privilege accounts for an application, a report and staff, and script them

TIMING IN THE ROOM

10m	Frame: why not root; 'user'@'host'; least privilege; GRANT what ON where TO whom
30m	Live demo: grants at three levels, logins as each account, REVOKE, roles, a locked account
15m	Guided lab: `warehouse_accounts_starter.sql` in pairs
10m	Start the lesson project: team_access.sql Steps 1 to 3

MENTAL MODEL TO SAY

Start with the three stories from the lesson: a clerk's DELETE without WHERE, an attacker in the search box, a dashboard password in a shared config file. "None of these needs root. Each needs an account that can do its job and nothing more: the **principle of least privilege**."

Then the building: "root is the master key. A program gets the key to one room; a report gets a window to look through. Every account is a name AND a door: **'user'@'host'**."

Then the sentence pattern on the board, **GRANT what ON where TO whom**, and the levels from wide to narrow: `*.*` (administrators only), `bookstore.*`, `bookstore.books`, a column list. "Grant at the narrowest level that works, and prove it by logging in as the account."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

The lesson's own accounts, from a fresh `bookstore.sql`. Root window on the left, the second window on the right: log in there as each account when the step says so. Learners type along in their own two windows; the passwords are the lesson's placeholders.

1. **An account (2m).** "An account is a user name AND the host it connects from. Always write the host on purpose."

```
CREATE USER 'shop_reports'@'localhost' IDENTIFIED BY 'CHANGE-ME-reports-1';
SELECT user, host, plugin, account_locked
FROM mysql.user
WHERE user = 'shop_reports';
```

```
+--------------+-----------+-----------------------+----------------+
| user         | host      | plugin                | account_locked |
+--------------+-----------+-----------------------+----------------+
| shop_reports | localhost | caching_sha2_password | N              |
+--------------+-----------+-----------------------+----------------+
```

The plugin column: caching_sha2_password, the 9.7 default (Session 1's health check).

2 **A new account can do nothing (1m).** "USAGE means: may connect. That is all."

```
SHOW GRANTS FOR 'shop_reports'@'localhost';
```

```
+-------------------------------------------------+
| Grants for shop_reports@localhost               |
+-------------------------------------------------+
| GRANT USAGE ON *.* TO `shop_reports`@`localhost` |
+-------------------------------------------------+
```

3 **GRANT what ON where TO whom (2m).** Write the four words on the board, then read the statement against them.

```
GRANT SELECT ON bookstore.* TO 'shop_reports'@'localhost';
SHOW GRANTS FOR 'shop_reports'@'localhost';
```

```
+-----------------------------------------------------------+
| Grants for shop_reports@localhost                         |
+-----------------------------------------------------------+
| GRANT USAGE ON *.* TO `shop_reports`@`localhost`          |
| GRANT SELECT ON `bookstore`.* TO `shop_reports`@`localhost` |
+-----------------------------------------------------------+
```

No FLUSH PRIVILEGES needed: grants take effect at once.

4 **The website's account, table by table (3m).** "The site reads the catalog and records orders. It never changes a price or deletes anything."

```
CREATE USER 'shop_app'@'localhost' IDENTIFIED BY 'CHANGE-ME-app-1';
GRANT SELECT ON bookstore.* TO 'shop_app'@'localhost';
GRANT INSERT, UPDATE ON bookstore.customers   TO 'shop_app'@'localhost';
GRANT INSERT         ON bookstore.orders      TO 'shop_app'@'localhost';
GRANT INSERT         ON bookstore.order_items TO 'shop_app'@'localhost';
SHOW GRANTS FOR 'shop_app'@'localhost';
```

```
+-----------------------------------------------------------------------------+
| Grants for shop_app@localhost                                               |
+-----------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `shop_app`@`localhost`                                |
| GRANT SELECT ON `bookstore`.* TO `shop_app`@`localhost`                     |
| GRANT INSERT, UPDATE ON `bookstore`.`customers` TO `shop_app`@`localhost`   |
| GRANT INSERT ON `bookstore`.`order_items` TO `shop_app`@`localhost`         |
| GRANT INSERT ON `bookstore`.`orders` TO `shop_app`@`localhost`              |
+-----------------------------------------------------------------------------+
```

If the site is ever broken into, the attacker can read the catalog and add orders, and nothing else.

5 **Column-level privileges (2m).** "The support desk sees names and emails, and may fix an email. Nothing else."

```
CREATE USER 'support_desk'@'localhost' IDENTIFIED BY 'CHANGE-ME-support-1';
GRANT SELECT (id, first_name, last_name, email), UPDATE (email)
    ON bookstore.customers TO 'support_desk'@'localhost';
```

6 **Test as the user: shop_reports (4m).** In the SECOND window: `mysql -u shop_reports -p bookstore` (the password is typed at the prompt, never on the command line). "A grant you have not tested is a guess: test one thing it must do and one it must not."

```
SELECT CURRENT_USER();
SELECT COUNT(*) AS books FROM books;
DELETE FROM books WHERE id = 11;
```

```
+------------------------+
| CURRENT_USER()         |
+------------------------+
| shop_reports@localhost |
+------------------------+
+-------+
| books |
+-------+
|    11 |
+-------+
ERROR 1142 (42000): DELETE command denied to user 'shop_reports'@'localhost' t
```

7 **Test as the user: support_desk (3m).** `exit`, then `mysql -u support_desk -p bookstore`. Predict each line first.

```
SELECT first_name, email FROM customers WHERE id = 1;
SELECT first_name, city FROM customers WHERE id = 1;
SELECT * FROM orders;
```

```
+------------+-------------------+
| first_name | email             |
+------------+-------------------+
| Alice      | alice@example.com |
+------------+-------------------+
ERROR 1143 (42000): SELECT command denied to user 'support_desk'@'localhost' t
ERROR 1142 (42000): SELECT command denied to user 'support_desk'@'localhost' t
```

1143 names the column that was never granted; `SELECT *` would be refused for the whole table (1142).

8 **REVOKE at the level you granted (2m).** Back in the root window.

```
REVOKE UPDATE ON bookstore.customers FROM 'shop_app'@'localhost';
SHOW GRANTS FOR 'shop_app'@'localhost';
```

```
+-------------------------------------------------------------------+
| Grants for shop_app@localhost                                     |
+-------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `shop_app`@`localhost`                      |
| GRANT SELECT ON `bookstore`.* TO `shop_app`@`localhost`           |
| GRANT INSERT ON `bookstore`.`customers` TO `shop_app`@`localhost` |
| GRANT INSERT ON `bookstore`.`order_items` TO `shop_app`@`localhost` |
| GRANT INSERT ON `bookstore`.`orders` TO `shop_app`@`localhost`    |
+-------------------------------------------------------------------+
```

9 **The old advice, once more (1m).** "Session 1 showed this error; here is where it comes from."

```
CREATE USER 'legacy_app'@'localhost' IDENTIFIED WITH mysql_native_password BY
```

```
ERROR 1524 (HY000): Plugin 'mysql_native_password' is not loaded
```

Fix: update the app's driver, never weaken the server.

10 **Roles (3m).** "Several clerks, same job: grant to a role once, then grant the role to people."

```
CREATE ROLE 'bookstore_reader', 'bookstore_sales';
GRANT SELECT ON bookstore.* TO 'bookstore_reader';
GRANT INSERT, UPDATE ON bookstore.orders      TO 'bookstore_sales';
GRANT INSERT, UPDATE ON bookstore.order_items TO 'bookstore_sales';
CREATE USER 'tom'@'localhost'  IDENTIFIED BY 'CHANGE-ME-tom-1';
CREATE USER 'lena'@'localhost' IDENTIFIED BY 'CHANGE-ME-lena-1';
GRANT 'bookstore_reader', 'bookstore_sales' TO 'tom'@'localhost', 'lena'@'loca
SHOW GRANTS FOR 'tom'@'localhost';
```

```
+-------------------------------------------------------------------------+
| Grants for tom@localhost                                                |
+-------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `tom`@`localhost`                                 |
| GRANT `bookstore_reader`@`%`,`bookstore_sales`@`%` TO `tom`@`localhost` |
+-------------------------------------------------------------------------+
```

11 **The step everybody forgets (4m).** Second window, as tom WITHOUT a database: `mysql -u tom -p`. Then try `mysql -u tom -p bookstore`.

```
SELECT CURRENT_ROLE();
SELECT COUNT(*) FROM bookstore.books;
```

```
+----------------+
| CURRENT_ROLE() |
+----------------+
| NONE           |
+----------------+
ERROR 1142 (42000): SELECT command denied to user 'tom'@'localhost' for table
ERROR 1044 (42000): Access denied for user 'tom'@'localhost' to database 'book
```

The second line is the login with bookstore: refused before any query, because no privilege is active yet.

12 **SET DEFAULT ROLE (2m).** Root window:

```
SET DEFAULT ROLE ALL TO 'tom'@'localhost', 'lena'@'localhost';
SELECT user, default_role_user FROM mysql.default_roles ORDER BY user, default
```

```
+------+-------------------+
| user | default_role_user |
+------+-------------------+
| lena | bookstore_reader  |
| lena | bookstore_sales   |
| tom  | bookstore_reader  |
| tom  | bookstore_sales   |
+------+-------------------+
```

13 **Tom, again (1m).** Second window: `mysql -u tom -p bookstore`.

```
SELECT CURRENT_ROLE();
SELECT COUNT(*) AS books FROM books;
```

```
+-----------------------------------------------+
| CURRENT_ROLE()                                |
+-----------------------------------------------+
| `bookstore_reader`@`%`,`bookstore_sales`@`%` |
+-----------------------------------------------+
+-------+
| books |
+-------+
|    11 |
+-------+
```

14 **Lock an account (1m).** "Staff on leave, a job you are retiring: lock it, keep its grants." Then try to log in as shop_reports.

```
ALTER USER 'shop_reports'@'localhost' ACCOUNT LOCK;
```

```
ERROR 3118 (HY000): Access denied for user 'shop_reports'@'localhost'. Account
```

15 **Two refusals worth reading (1m).** A misspelled account, then the right user with the wrong host.

```
GRANT SELECT ON bookstore.* TO 'reports'@'localhost';
SHOW GRANTS FOR 'shop_reports'@'%';
```

```
ERROR 1410 (42000): You are not allowed to create a user with GRANT
ERROR 1141 (42000): There is no such grant defined for user 'shop_reports' on
```

16 **Clean up (1m).** "Drop what you created; a demo account left on a server is a door left open."

```
DROP USER 'shop_reports'@'localhost', 'shop_app'@'localhost', 'support_desk'@'
          'tom'@'localhost', 'lena'@'localhost';
DROP ROLE 'bookstore_reader', 'bookstore_sales';
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **The right name, the wrong host.** "It says access denied but I created the account!" Coach: `SELECT user, host FROM mysql.user;` and compare the pair; inside a session, `SELECT USER(), CURRENT_USER();`.
- **Granted but never activated.** A role shows in SHOW GRANTS and the user still gets 1142 (or 1044 at login). Coach: "Is the role ACTIVE? `SELECT CURRENT_ROLE();` as that user; the fix is SET DEFAULT ROLE ALL."
- **Revoking at another level.** 1147 on a REVOKE. Coach: "Read SHOW GRANTS: at which level was it granted? Revoke exactly that line."

GUIDED LAB

File: `labs/module-12/l7_4-user-management/warehouse_accounts_starter.sql`; reference `labs/module-12/l7_4-user-management/warehouse_accounts_solution.sql`. Two program accounts (the warehouse scanner, the accounting export) with table and column privileges, an audit through information_schema, a REVOKE and a lock. Every account is `kit_...` and the file drops them at the end.

- **TODO 2 together (4m), TODOs 3 and 5 in pairs (7m), TODO 6 if time (2m):** the login test from a second window; the README has the steps.

- **Expected:** the warehouse's grants merge into `GRANT SELECT, UPDATE (` in_stock `) ON` bookstore `.` books ` ` plus SELECT on authors; accounting gets three customer columns, orders and order_items; the audit lists 4 table and 4 column privileges; after TODO 5 accounting keeps customers and orders and kit_warehouse is locked (Y).
- **TODO 6, as kit_warehouse (real output):** the in_stock UPDATE works, the price UPDATE is refused:

```
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0
ERROR 1143 (42000): UPDATE command denied to user 'kit_warehouse'@'localhost' for
```

- **Circulate for:** column lists in the wrong place (`GRANT SELECT ON customers (id, ...)` : ERROR 1064; the columns go after the privilege), a REVOKE that names `bookstore.*` instead of the table, and anyone typing a statement aimed at root.
- **Fast finishers:** `labs/module-12/l7_4-user-management/staff_roles_starter.sql` (roles: grant, activate, change once for everybody, revoke a bundle), then `labs/module-12/l7_4-user-management/debug_grants.sql` (errors 1410, 1141, 1396 and 1147).

START THE LESSON PROJECT

The lesson project is **Accounts for the Bookstore Team**: `team_access.sql`, a rerunnable script that creates three roles and four accounts. In the room (10 minutes): Steps 1 to 3.

- Step 1: the DROP USER IF EXISTS / DROP ROLE IF EXISTS lines that make the script rerunnable. Step 2: the three roles. Step 3: the website account, table by table.
- **Done in the room** = the file runs twice in a row without an error, and SHOW GRANTS for team_web lists four SELECTs and two INSERTs.
- **Goes home:** Steps 4 to 6 (people and the dashboard with default roles, the inventory query, the two-direction test as team_sofia). Remind learners to set Dune back to 14.99 after Step 6.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- What can an account do right after CREATE USER? **Answer:** connect, nothing else (USAGE).
- 1142 or 1143: which names a column? **Answer:** 1143.
- A role is granted and the user still gets 1142. What is missing? **Answer:** an active role: SET DEFAULT ROLE ALL (or SET ROLE in the session).
- Why does the website get no DELETE anywhere? **Answer:** least privilege: a bug or an attacker using its login can do only what it needs.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** B) Two separate accounts, each with its own password and privileges; **Q2** Three: `GRANT USAGE ON *.*`, `GRANT SELECT ON \`bookstore\`.*`, `GRANT INSERT ON \`bookstore\`.\`orders\``; **Q3** D) The account has column privileges on customers, and city is not one of them; **Q4** A) Right away, both of them, and every future account given the role; **Q5** C) Fails with ERROR 1147: there is no table-level grant on books to take away; **Q6** `ALTER USER 'kit_mailer'@'localhost' ACCOUNT LOCK;`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: two windows can overwhelm; pair learners so one types as root and the other logs in as the demo account, then swap. Keep the error ladder on the board.

Stretch: `staff_roles_starter.sql` and `debug_grants.sql`; the lesson's Exercise 3 (a marketing view granted instead of columns) and Exercise 4 (a remote account with REQUIRE SSL and a query limit), then the Bonus Challenge (the exact privileges mysqldump needs: it leads straight into 8.1).

8.1 Importing & Exporting Data

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Back up a database with mysqldump, choose the right options for MySQL 9.7, and restore the file into a test database
- Explain what secure_file_priv and local_infile control, and find their values on your server
- Export query results to a CSV file with SELECT ... INTO OUTFILE, including a header row
- Import a CSV file through a staging table with LOAD DATA INFILE, converting dates and empty values on the way in
- Recognize and fix the errors file operations raise (1290, 1086, 3948, 2068, 3546)

TIMING IN THE ROOM

10m	Frame: whole databases at the terminal, rows at mysql>; where the server may touch files
30m	Live demo: backup, test restore, the GTID trap; a CSV export; a staging-table import; refusals and a silent warning
15m	Guided lab: `new_titles_import_starter.sql` in pairs
10m	Start the lesson project: the backup kit, Steps 1 and 2

MENTAL MODEL TO SAY

"Two kinds of moving data, two kinds of tools." Point at the board: **whole databases** (backups, a new server) go through **mysqldump**, a program at the **terminal**, which writes a .sql file of CREATE and INSERT statements; the mysql client plays it back. **Rows** (a report for a spreadsheet, a supplier's file) go through **SELECT ... INTO OUTFILE** and **LOAD DATA**, SQL at the **mysql> prompt**.

Then the fence: "INTO OUTFILE and LOAD DATA INFILE run on the SERVER's disk, so the server only touches one folder, secure_file_priv, and only for accounts with the FILE privilege. LOCAL means the client sends a file from your own computer, and both sides must agree to it."

Then the rule that makes a backup a backup: **a dump you have never restored is a hope.** Back up, restore into a test database, compare the counts, write it down.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

From a fresh `bookstore.sql`, with three windows: a plain terminal in the course folder (mysqldump), the root mysql client, and the lesson page for the CSV files.

1. **A backup, at the terminal (3m).** Open a plain terminal (not mysql>) in the course folder. "mysqldump is a separate program. On 9.7 these four options are the habit."

```
mysqldump -u root -p --single-transaction --routines --events \
    --set-gtid-purged=OFF bookstore > bookstore_backup.sql
```

It prints nothing and leaves a file of about 12 KB. Windows PowerShell: `--result-file=bookstore_backup.sql` instead of `>`.

2. **What is inside (2m).** "Plain SQL: a DROP and a CREATE per table, then the rows."

```
grep -c 'INSERT INTO' bookstore_backup.sql
grep 'DROP TABLE IF EXISTS' bookstore_backup.sql
```

```
8
DROP TABLE IF EXISTS `authors`;
DROP TABLE IF EXISTS `book_categories`;
DROP TABLE IF EXISTS `books`;
DROP TABLE IF EXISTS `categories`;
DROP TABLE IF EXISTS `customers`;
DROP TABLE IF EXISTS `employees`;
DROP TABLE IF EXISTS `order_items`;
DROP TABLE IF EXISTS `orders`;
```

Every table starts with DROP TABLE IF EXISTS: a restore replaces tables. Open the file in an editor for the versioned `/*!40101 ... */` comments.

3. **A test restore (3m).** "A backup you have never restored is a hope." Restore into a test database, then compare.

```
mysql -u root -p -e "CREATE DATABASE bookstore_test"
mysql -u root -p bookstore_test < bookstore_backup.sql
mysql -u root -p -t -e "SELECT
    (SELECT COUNT(*) FROM bookstore.order_items) AS items_original,
    (SELECT COUNT(*) FROM bookstore_test.order_items) AS items_copy"
```

```
+----------------+------------+
| items_original | items_copy |
+----------------+------------+
|             18 |         18 |
+----------------+------------+
```

Then `DROP DATABASE bookstore_test;`.

4. **The 9.7 trap (3m).** "Leave out --set-gtid-purged=OFF and watch."

```
mysqldump -u root -p --single-transaction bookstore > plain_dump.sql
mysql -u root -p -e "CREATE DATABASE bookstore_test2"
mysql -u root -p bookstore_test2 < plain_dump.sql
```

```
Warning: A partial dump from a server that has GTIDs will by default include t
ERROR 3546 (HY000) at line 24: @@GLOBAL.GTID_PURGED cannot be changed: the add
```

9.7 starts with GTIDs on: the plain dump carries the server's replication history and refuses to restore on the same server. No tables were created. Take the dump again with the option.

5 **And at the wrong prompt (1m).** Back in the mysql client, the classic slip:

```
mysqldump -u root -p bookstore > bookstore_backup.sql;
```

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual tha
```

Type `exit`, run it at `$` or `PS>`.

6 **Where the server may write files (2m).** "INTO OUTFILE and LOAD DATA INFILE run on the SERVER's disk, inside one folder." On a Linux package or the Docker image (shown: MySQL 8.4's Docker image):

```
SHOW VARIABLES LIKE 'secure_file_priv';
```

```
+------------------+-----------------------+
| Variable_name    | Value                 |
+------------------+-----------------------+
| secure_file_priv | /var/lib/mysql-files/ |
+------------------+-----------------------+
```

Learners run it on their own server and write the answer down: every path below uses that folder. (The kit's test server keeps its folder elsewhere; its outputs below are real, with the path shown as `/var/lib/mysql-files/`.)

7 **Export a CSV (3m).** "Any query, plus INTO OUTFILE and the format, at the end."

```
SELECT b.id, b.title, a.name AS author, b.price, b.published
FROM books AS b
JOIN authors AS a ON a.id = b.author_id
ORDER BY b.id
INTO OUTFILE '/var/lib/mysql-files/books_export.csv'
    FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
    LINES TERMINATED BY '\n';
```

```
Query OK, 11 rows affected
```

The first lines of the file (open it with a text editor or a spreadsheet; on Linux it belongs to the mysql user):

```
1,"Dune","Frank Herbert",14.99,"1965-08-01"
2,"1984","George Orwell",9.99,"1949-06-08"
3,"To Kill a Mockingbird","Harper Lee",11.99,"1960-07-11"
...
```

8 **Outside the folder (1m).** "Anywhere else is refused."

```
SELECT title, price FROM books
INTO OUTFILE '/tmp/books.csv';
```

```
ERROR 1290 (HY000): The MySQL server is running with the --secure-file-priv op
```

9 **Import through a staging table (4m).** Show `new_books.csv` first (on the lesson page): a header, a comma inside "I, Robot", US dates, an empty page count, and an author the store does not know. "Never load a stranger's file straight into a real table."

```
CREATE TABLE import_books (
    title     VARCHAR(200),
    author    VARCHAR(100),
    price     DECIMAL(8, 2),
    pages     INT,
    published DATE
);
LOAD DATA INFILE '/var/lib/mysql-files/new_books.csv'
INTO TABLE import_books
    FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
    LINES TERMINATED BY '\n'
    IGNORE 1 LINES
    (title, author, price, @pages, @published)
SET pages     = NULLIF(@pages, ''),
    published = STR_TO_DATE(@published, '%m/%d/%Y');
```

```
Query OK, 0 rows affected
Query OK, 4 rows affected
Records: 4  Deleted: 0  Skipped: 0  Warnings: 0
```

Read the status line aloud: 4 records, 0 skipped, 0 warnings.

10 **Look at what arrived (1m).**

```
SELECT * FROM import_books;
```

```
+------------------+----------------+-------+-------+------------+
| title            | author         | price | pages | published  |
+------------------+----------------+-------+-------+------------+
| I, Robot         | Isaac Asimov   |  8.99 |   253 | 1950-12-02 |
| Emma             | Jane Austen    |  8.49 |   474 | 1815-12-23 |
| Frankenstein     | Mary Shelley   |  7.49 |  NULL | 1818-01-01 |
| The Silmarillion | J.R.R. Tolkien | 12.99 |   365 | 1977-09-15 |
+------------------+----------------+-------+-------+------------+
```

11 **Check against the real tables (2m).** "Which authors are missing? A LEFT JOIN with IS NULL."

```
SELECT i.title, i.author
FROM import_books AS i
LEFT JOIN authors AS a ON a.name = i.author
WHERE a.id IS NULL;
```

```
+--------------+--------------+
| title        | author       |
+--------------+--------------+
| Frankenstein | Mary Shelley |
+--------------+--------------+
```

12 **Copy the good rows, all or nothing (3m).** "Lesson 7.2 comes back: one transaction."

```
INSERT INTO authors (name, birth_year) VALUES ('Mary Shelley', 1797);
START TRANSACTION;
INSERT INTO books (title, author_id, price, pages, published)
SELECT i.title, a.id, i.price, i.pages, i.published
FROM import_books AS i
JOIN authors AS a ON a.name = i.author
ORDER BY i.title;
SELECT b.id, b.title, a.name AS author, b.pages, b.published
FROM books AS b
JOIN authors AS a ON a.id = b.author_id
WHERE b.id > 11
ORDER BY b.id;
COMMIT;
DROP TABLE import_books;
```

```
+----+------------------+----------------+-------+------------+
| id | title            | author         | pages | published  |
+----+------------------+----------------+-------+------------+
| 12 | Emma             | Jane Austen    |   474 | 1815-12-23 |
| 13 | Frankenstein     | Mary Shelley   |  NULL | 1818-01-01 |
| 14 | I, Robot         | Isaac Asimov   |   253 | 1950-12-02 |
| 15 | The Silmarillion | J.R.R. Tolkien |   365 | 1977-09-15 |
+----+------------------+----------------+-------+------------+
```

13 **The silent LOCAL import (2m).** "With LOCAL the client sends a file from your own computer, and bad values become warnings, not errors." (local_infile is ON on the demo server; Lesson 8.1's Common Mistake 4 shows the 3948 error when it is OFF.)

```
CREATE TABLE price_update (title VARCHAR(200), price DECIMAL(8, 2));
LOAD DATA LOCAL INFILE 'price_update.csv'
INTO TABLE price_update
    FIELDS TERMINATED BY ','
    IGNORE 1 LINES;
SHOW WARNINGS;
```

```
Query OK, 0 rows affected
Query OK, 2 rows affected, 1 warning
Records: 2  Deleted: 0  Skipped: 0  Warnings: 1
+---------+------+-----------------------------------------------------------
| Level   | Code | Message
+---------+------+-----------------------------------------------------------
| Warning | 1366 | Incorrect decimal value: 'N/A' for column 'price' at row 2
+---------+------+-----------------------------------------------------------
```

Emma now costs 0.00 and the only clue was `Warnings: 1`. Read the warnings right after every import (the next statement clears them).

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **The wrong prompt.** mysqldump at `mysql>` (1064) or SQL at `$` ("command not found"). Coach: "Look at the prompt first: `$` / `PS>` runs programs, `mysql>` runs SQL."
- **A path the server cannot use.** 1290 for /tmp or a home folder. Coach: "Whose disk is that path on? The server's. Which folder does it allow?" `SHOW VARIABLES LIKE 'secure_file_priv';`
- **Trusting the status line's first number.** "4 rows affected, done." Coach: read the whole line (Skipped, Warnings), SHOW WARNINGS when it is not 0, and SELECT the staging table before copying anything.

GUIDED LAB

File: `labs/module-12/l8_1-importing-exporting/new_titles_import_starter.sql` (and `labs/module-12/l8_1-importing-exporting/new_titles.csv`, the supplier's file); reference `labs/module-12/l8_1-importing-exporting/new_titles_import_solution.sql`. A European supplier's file (semicolons, day.month.year, an `N/A` price, an empty page count, a new author, a book already sold) through a staging table. The staging table is filled with an INSERT of exactly the text LOAD DATA would store, so the lab needs no file access or server setting.

- **SOURCE it as shipped (2m):** The Two Towers costs 0.00 and every date is NULL, with a screen of warnings (1411, 1292). Ask: "Which of these would have reached the books table?"
- **TODO 1 together (4m), TODOs 2 and 3 in pairs (7m).** TODO 3 is a commented block: learners remove the `--` and put in the date pattern from TODO 1.
- **Expected:** after TODO 1, no warnings and real dates; Step 2 holds back Animal Farm (already sold) and The Two Towers (no price) and shows Bram Stoker's author_id as NULL (new author); after TODO 3, books 12 Dracula, 13 Foundation, 14 Persuasion (pages NULL); 14 books, 11 authors.
- **Real file, optional (README):** with local_infile on, the LOAD DATA LOCAL statement in the lab's header loads `new_titles.csv` into the same staging table (real output, the kit's test server):

```
+----------------+----------------+-----------+-----------+---------------+
| title          | author         | price_raw | pages_raw | published_raw |
+----------------+----------------+-----------+-----------+---------------+
| Animal Farm    | George Orwell  | 8.49      | 112       | 17.08.1945    |
| Dracula        | Bram Stoker    | 8.99      | 418       | 26.05.1897    |
| Foundation     | Isaac Asimov   | 9.99      | 255       | 01.06.1951    |
| Persuasion     | Jane Austen    | 7.49      |           | 20.12.1817    |
| The Two Towers | J.R.R. Tolkien | N/A       | 352       | 11.11.1954    |
+----------------+----------------+-----------+-----------+---------------+
```

- **Fast finishers:** `labs/module-12/l8_1-importing-exporting/backup_check_starter.sql` (a least-privilege backup account and a proof it cannot write, the restore test's reference counts, a CSV-shaped report with a header row), then `labs/module-12/l8_1-importing-exporting/debug_import.sql` (errors 1064, 1290, 1411 and 1292).

START THE LESSON PROJECT

The lesson project is **The Bookstore Backup Kit**: a backup account, a tested dump, a monthly sales CSV and a repeatable import. In the room (10 minutes): Steps 1 and 2.

- Step 1: `backup_user.sql` (reload, CREATE USER IF NOT EXISTS bookstore_backup, the five privileges, SHOW GRANTS).
- Step 2: at the terminal, the dump AS bookstore_backup with `--no-tablespaces --skip-masking-policies` (on 8.4 without the last option).

- **Done in the room** = a dated `bookstore_YYYY-MM-DD.sql` made by the backup account, with no warnings on screen.
- **Goes home:** Steps 3 to 6 (the test restore and restore_log.txt, the sales CSV checked by loading it back, the repeatable import, the scheduled backup with a login path).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- mysqldump: terminal or mysql>? **Answer:** terminal.
- Which option does every 9.7 dump you restore yourself need? **Answer:** --set-gtid-purged=OFF.
- What does a database dump leave out that the server needs? **Answer:** the user accounts and their grants (keep team_access.sql and backup_user.sql).
- Why a staging table? **Answer:** to look, convert and check before the real tables change.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** C) At the terminal's shell prompt ($ or PS>), not inside the mysql client; **Q8** A) `--routines`; **Q9** `1976-07-04` and `NULL`; **Q10** D) Name the target database: `mysql -u root -p bookstore_test < bookstore_backup.sql` (or dump with --databases); **Q11** B) To look at, convert and check the rows (dates, empty values, missing authors) before any real table changes; **Q12** C) ERROR 1086: the file already exists. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners on Windows do the shell part with `--result-file` and the `--commands -e "SOURCE ..."` restore (the README spells it out); learners without file access skip the server-side export and use the lab's on-screen report.

Stretch: `backup_check_starter.sql` and `debug_import.sql`; the lesson's Exercise 2 (signups with IGNORE and NULLIF) and Exercise 4 (a round trip that proves itself), then the Bonus Challenge (`mysqldump --tab`).

FINAL PROJECT BRIEF (HAND-OUT)

2:30 Final Project Brief: the Maple Street Tool Library

20 min

TIMING

3m	Hand out the brief: Lesson 8.2's "The Brief" and "What you hand in" (`lesson_final_project.html`)
4m	Read Rosa's needs aloud; learners underline nouns (tables) and circle rules
6m	In pairs: a first list of entities and relationships; one minute of sharing on the board
5m	The eight milestones, the rubric (How Your Work Is Assessed, 100 points), the two studio sessions
2m	Questions; the homework: the Milestone 1 draft

WHAT TO SAY AND SHOW

- "Until now the bookstore was handed to you. For the last two sessions you start from a client's description and an empty server." The client is **Rosa Delgado**, who coordinates a volunteer tool library: two people borrowed the same ladder, nobody knows who has the drill, someone took the chainsaw without the course.
- Read the brief's eight bullet points aloud. Learners underline the **nouns** (members, tools, categories, safety courses, loans) and circle the **rules** (one open loan per tool, nothing out while in repair, suspended members cannot borrow, at most three tools at once, no course no chainsaw).
- Pairs, 5 minutes: which nouns become tables, which become columns, which relationships are one-to-many and which many-to-many? Collect on the board: five entities plus the junction between members and safety courses (it carries the completion date). Do not show the lesson's reference design: it is hidden on the page for a reason.
- Show "What you hand in" (01_schema.sql to 05_security.sql, a tested backup, README.md) and the eight milestones with their lessons: every Part 2 lesson comes back (5.1 and 5.2 in the Saturday queries, 6.2 and 7.1 and 7.2 in the business logic, 7.4 in security, 8.1 in the backup). Show the rubric: 80 of 100 is a solid, portfolio-ready project.
- Sessions 13 and 14 are **studios**: the room is for building, with you circulating. "Arrive at Session 13 with the brief read and your Milestone 1 draft: you will turn it into tables in the first hour."

INSTRUCTOR NOTES

- Mark the final project with the kit's `final-assessment.html` (rubric and instructor notes with the reference numbers); learners use the lesson page's rubric. Keep the kit file out of the learners' folder.
- Learners who prefer another business can use the lesson's alternative brief (a small clinic, in the page's

Common Questions). Agree on it now, not in Session 13.

- Everyone needs MySQL 9.7 with room for a new database, `tool_library`; the bookstore is not touched. Learners who miss Session 13 can still start from the lesson page alone.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

1. Why does the website's account get INSERT on orders but no UPDATE or DELETE anywhere? **Answer:** least privilege: a bug or an attacker using the website's login can only do what the website needs (read the catalog, add orders).
2. Name the two steps that turn a dump file into a backup you can trust. **Answer:** dump with the right options (on 9.7 with --set-gtid-purged=OFF), then restore it into a test database and compare counts (and write it down).
3. Who is the final project's client, and name two rules her database must enforce. **Answer:** Rosa Delgado of the Maple Street Tool Library; any two of: one open loan per tool, nothing out while in repair, suspended members cannot borrow, at most three tools per member, no safety course no chainsaw.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** team_access.sql Steps 4 to 6 (default roles, the inventory, the test as team_sofia) and the backup kit Steps 3 to 6 (test restore and restore_log.txt, the sales CSV, the repeatable import, backup.sh).
- **Lesson exercises.** 7.4: Exercises 1 to 3; Exercise 4 is a stretch. 8.1: Exercises 1 to 3; Exercise 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages, and **both journal entries**.
- **Labs not reached in class:** `staff_roles_starter.sql`, `debug_grants.sql`, `backup_check_starter.sql`, `debug_import.sql`.
- **Final project, Milestone 1 draft (about 30 minutes):** read the whole brief and the lesson's "From Requirements to Tables" section; on paper, the entities and their columns, the relationships (one or many, both directions), and where each of Rosa's five rules will be enforced (column type or constraint, foreign key, or procedure). No SQL yet.
- **Before the Next Lesson (from 8.1):** skim Lessons 6.3 (design) and 7.4 (users): the final project leans on both.

BRING LIST FOR SESSION 13

- A laptop with MySQL 9.7 and the root password; room for a new database (`tool_library`).
- The Milestone 1 draft (entities, relationships, where each rule lives) and the brief.
- team_access.sql and the backup kit scripts: the final project's security and backup milestones reuse them.

- The workbook and the journal; the course folder with `labs/`.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the logins as another account; the test restore and the GTID trap; the staging-table import; the full 20 minutes of the brief.
- **Cut first (saves 2m):** the 8.1 LOCAL warning step.
- **Cut second (saves 3m):** the 7.4 old-advice and two-refusals steps (`debug_grants.sql` has the refusals).
- **Cut third (saves 10m):** the 8.1 project start: Step 1 is three statements learners run at home.

RUNNING SHORT: WHAT TO ADD

- The 7.4 Bonus Challenge live: a team_backup account with exactly mysqldump's privileges, then a dump made with it at the terminal.
- Run `debug_import.sql` for everyone and fix it together.
- Start Milestone 1 in the room: pairs compare their entity lists.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 13: Final Project Kickoff and Studio I

How to run the first final-project studio. Launch the Maple Street Tool Library brief and its rubric, show the build mistakes that cost the most time, and get every learner from Rosa's eight requirements to a checked design, a re-runnable schema whose constraints are proven by failing tests, and test data that plants the edge cases.

`1 lesson (first half)` `3 hrs session` `Lesson 8.2 · Milestones 1-3` `Session 13 of 14 · Part 2`

HOW TO USE THIS GUIDE

- Session 13 is a **studio**: one lesson (8.2, the final project) and most of the room's time spent building. The shape: warm-up → kickoff (the brief, the files, the rubric) → a short frame and live demo → Milestone 1 in the room → a guided lab on the bookstore → Milestones 2 and 3 in the room. You circulate and run short **check-ins** at the end of each milestone instead of teaching new material.
- Type, do not paste, in the demo. Every SQL block below was run to produce the output printed under it, on MySQL 9.7 (the lesson's own blocks were run as one session, exactly as the page shows them, and match the page). Read error numbers aloud from the left.
- The lesson page is the brief and the reference. Keep `lesson_final_project.html` open in a tab: The Brief, Milestones and Timeline, From Requirements to Tables, How Your Work Is Assessed, Common Mistakes. **Do not open the reference solutions on the projector** before learners have attempted a milestone; the page hides them on purpose.
- The assessment, the rubric with scoring guidance and the instructor answer notes with real reference numbers are in `final-assessment.html` at the kit root (Part 2 section). Print the rubric for learners from there.
- Session 14 continues the same project (Milestones 5 to 7 in the room, then the showcase and Lesson 8.3). Teal blocks are hands-on, amber blocks are watch-outs.

> **THE #1 RISK OF SESSION 13: A DESIGN NOBODY CHECKED**
>
> A learner who starts typing CREATE TABLE before every requirement has a home spends Session 14 rebuilding. The two most expensive design errors are (1) trying to put a rule that looks at other rows into a CHECK (it cannot work, and the learner discovers that at Milestone 5) and (2) storing derived data (an `is_available` or `times_borrowed` column) that the queries then trust. **The defense:** nobody starts Milestone 2 without a passed design check-in (two minutes: point at each of Rosa's eight requirements and say where it lives). Keep a tally on the studio tracker below; visit the learners without a tick first.

BEFORE THE SESSION

- ☐ **Check what Session 12 handed out:** the brief (the lesson page's "The Brief" section, or the printed one-pager) and the reading for this session (the brief and "From Requirements to Tables"). Learners who did not read it can still do Milestone 1 in the room; it takes them ten minutes longer.
- ☐ **Print or share:** the rubric and the milestone checklist (from `final-assessment.html`, Part 2), the workbook (its Milestone tracker), and sticky notes for the check-ins. Optional: A3 paper for design diagrams.
- ☐ **Files to hand out:** `labs/module-13/` without the `*_solution.sql` files. Run `python3 labs/check_labs.py module-13` once (6 files, every line PASS, a few seconds).
- ☐ **Projector machine:** MySQL 9.7 with the bookstore loaded (the guided lab uses it: `SOURCE labs/_states/bookstore.sql`), a terminal in the course folder at 18 pt, a second terminal inside an empty `tool-library` folder (the demo of where to start the client), browser tabs `lesson_final_project.html` and `final-assessment.html`.
- ☐ **Board, before learners arrive:** a three-column table headed "Rosa's requirement · Where it lives · Why", with rows 1 to 8 and the middle column empty; the four error numbers 1064 · 3780 · 3814 · 1824; and "Rules about ONE row → constraint · rules about OTHER rows → procedure + transaction".
- ☐ **Decide the alternative-brief policy:** the lesson's FAQ allows another business (at least five tables, a many-to-many relationship, a rule across rows, two kinds of users; the clinic brief is ready-made). Approve own briefs in writing today; they cannot use the reference numbers in `final-assessment.html`, so they write their own expected results.

KNOWN TRAPS IN THIS SESSION

SYMPTOM	CAUSE	FIX
`SOURCE 01_schema.sql` inside 02_data.sql: `Failed to open file '01_schema.sql', error: 2`	SOURCE paths are relative to the folder the CLIENT was started in, not to the file that contains the SOURCE line (measured)	Start the client inside the project folder: `cd tool-library`, then `mysql -u root -p`. Or write the path from the course folder in both places.

SYMPTOM	CAUSE	FIX
`ERROR 1064 ... near 'condition`	A reserved word as a column name	Rename it (`tool_condition`); not backticks.
`ERROR 3780 ... are incompatible`	INT versus INT UNSIGNED or BIGINT on the two sides of a foreign key	One key type everywhere (INT).
`ERROR 1824 Failed to open the referenced table`	Child table created before its parent	Reorder: lookup tables, then the tables that point at them, junction and history last.
`ERROR 3814 ... contains disallowed function: curdate`	A time rule written as a CHECK	Compare two columns of the row; time rules go in the procedure.
Second run of 01_schema.sql fails with `ERROR 1050 Table ... already exists`	The script has no `DROP DATABASE IF EXISTS` first line	Add it; rerun the whole file.
The data script stops halfway: `ERROR 1452`	A loan or course record points at an id that was not inserted (or inserted later)	Explicit ids, parents' INSERTs first (members, categories, courses, tools, then member_courses and loans).
Someone's dates fail `chk_loans_due` (3819)	due_on typed before loaned_on, or the two dates swapped in the VALUES list	Read the column list of the INSERT against the VALUES; the CHECK just caught a real data error.

SESSION 13 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	15	Warm-up	Five retrieval questions on Lessons 7.4 and 8.1 and the brief from Session 12; a show of hands: who has a design draft?
0:15	15	**8.2** Kickoff: the brief, deliverables and rubric	Rosa's eight requirements read aloud; the six files to hand in; the 100-point rubric; how the studio runs; the tool-library folder.
0:30	20	**8.2** Frame and live demo	From requirements to tables (nouns, verbs, where each rule lives); four build mistakes and the silent LEFT JOIN, typed live.
0:50	30	**8.2** Studio: Milestone 1, Design	Tables, keys, relationships and every requirement mapped to its home; a two-minute design check-in with each learner.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:20	10	Break	Leave the requirement-to-home table on the board. Learners without a passed design check-in see you first after the break.
1:30	15	**8.2** Guided lab	`labs/module-13/l8_2-final-project/gift_cards_starter.sql`, then `debug_gift_card_proofs.sql`: named constraints and the tests that prove them, on the bookstore.
1:45	40	**8.2** Studio: Milestone 2, Build the schema	01_schema.sql, re-runnable, parents first, every constraint named; one failing test per constraint in a rolled-back transaction; schema check-in.
2:25	25	**8.2** Studio: Milestone 3, Test data	02_data.sql rebuilds the schema and plants the edge cases; the row-count query; data check-in.
2:50	10	Wrap-up	Exit ticket, what goes home (Milestones 3 and 4), bring list for Session 14.
Total	180		Ends at 3:00.

In the room. The lesson estimates 240 minutes of focused work for eight milestones (30 + 40 + 25 + 45 + 40 + 20 + 20 + 20). This session gives the room Milestones 1 to 3 (95 minutes of the lesson's estimate) plus the kickoff, a demo and a guided lab on the bookstore. Session 14 takes Milestones 5 to 7 in the room. Milestone 4 (the Saturday queries) and Milestone 8 (self-review and the README) are homework, because they are individual, quiet work that does not need the room.

Goes home (about 3 to 5 hours). Finish Milestones 1 to 3 if the room did not; Milestone 4 (all six Saturday queries with `@today = '2026-06-15'`, two checked by hand); the edge-cases lab (`edge_cases_starter.sql`), which practices exactly the patterns Milestone 4 needs; the journal entry. The lesson's Quick Quiz and Exercises wait until after Milestone 5 (they run on the database as Milestone 5 leaves it).

Protect, in this order: the design check-in for every learner (no Milestone 2 without it); the re-runnable 01_schema.sql with its constraint tests; the demo's Mistake 5 (the silent LEFT JOIN). **Cut first:** the guided lab (hand out `gift_cards_starter.sql` as homework and show the test pattern in the Milestone 2 mini-demo instead), then Mistakes 2 and 4 in the demo (they are on the lesson page), then the Milestone 3 studio (it becomes homework: it is mostly typing).

STUDIO TRACKER (COPY ONE PER GROUP)

Tick each milestone when the check-in passes. A blank cell at the end of the session is a name for the Session 14 warm-up.

LEARNER	M1 DESIGN: EVERY REQUIREMENT HAS A HOME	M2 RUNS TWICE, TESTS REFUSED	M3 COUNTS 8/5/2/7/12/16 OR OWN	NOTE FOR SESSION 14

LEARNER	M1 DESIGN: EVERY REQUIREMENT HAS A HOME	M2 RUNS TWICE, TESTS REFUSED	M3 COUNTS 8/5/2/7/12/16 OR OWN	NOTE FOR SESSION 14

OPENING SEGMENT

0:00 Warm-up and Kickoff — 30 min

TIMING

15m	Warm-up: five retrieval questions (four on Lessons 7.4 and 8.1, one on the brief); hands up for a design draft
15m	Kickoff: the brief, the six deliverables, the rubric, how the studio runs, the tool-library folder

WARM-UP QUESTIONS (ANSWERS IN THE ANSWER KEY)

Learners answer on the workbook's warm-up page (5 minutes), then you take answers by cold call. W1 to W4 are the Session 12 lessons, which the project uses in Milestones 6 and 7; W5 checks the reading.

W1

The treasurer's account was granted the reporting role, but every SELECT gives ERROR 1142. What was forgotten? ***(Lesson 7.4)***

W2

Someone types `mysqldump -u root -p bookstore > backup.sql` at the `mysql>` prompt. What happens? ***(Lesson 8.1)***

W3

A dump of one database restores perfectly on a new laptop, but the web form cannot log in. Why? *(Lesson 8.1)*

W4

Which mysqldump option puts the stored procedures into the dump? *(Lesson 8.1)*

W5

From Rosa's brief: name the tables you would build, and one rule that cannot be a CHECK constraint. *(the brief)*

SEQUENCE AND TALKING POINTS

- **Warm-up debrief (5m).** For each question, one learner answers, one adds. On W5 write the six table names on the board; if nobody names `member_courses`, ask: "a member finishes many courses, a course is finished by many members: where does the date go?" Then: "hands up if you have a design draft from the reading." Note the hands: those learners do the Milestone 1 check-in first.
- **The brief (5m).** Read Rosa's eight requirements aloud from the lesson page while learners follow in the workbook. Say: "This is a real kind of client: she describes problems, not tables. Your job is to turn eight sentences into six tables, a view, two procedures, two roles and a backup." Point at requirement 5: "five rules in one sentence; most of the project's thinking is here."
- **The deliverables (3m).** Show "What you hand in": `01_schema.sql`, `02_data.sql`, `03_queries.sql`, `04_logic.sql`, `05_security.sql`, a tested backup and `README.md`. Say: "Numbered scripts, each one runs with SOURCE, and 02_data.sql runs 01_schema.sql first, so the whole database rebuilds from files. That is what makes it portfolio-ready."

- **The rubric (4m).** Hand out the printed rubric (seven areas, 100 points; 0, half or full points per area; 80 or more is solid). Say: "Notice what earns full points: proof. A constraint is proven when a statement that breaks it is refused. A query is checked when you compared it with the data by hand." Then the timeline: Milestones 1 to 3 today, 4 at home, 5 to 7 in Session 14, 8 and the README at home; submission one week after Session 14.
- **The folder (3m).** On the projector: make a `tool-library` folder next to `labs/`, `cd` into it, start the client THERE. Say: "02_data.sql says SOURCE 01_schema.sql. SOURCE looks in the folder the client was started in, not next to the file. Start the client inside tool-library and it just works." (Measured on 9.7; the lesson page does not say it.)

WATCH-OUTS

- **"Can I look at the reference?"** Yes, after trying: the lesson says so. Ask them to try for at least the milestone's time first, then compare and write one difference into their design notes. Copying the reference wholesale scores poorly on Design ("every rule mapped ... with reasons") because the reasons are missing.
- **Own briefs** need your written approval today (see Before the Session). Hold them to the same structure: five tables, a many-to-many, a rule across rows, two kinds of users.
- **Learners who missed Session 12** have not seen roles (7.4) or mysqldump (8.1). They can do all of today; send them to Lessons 7.4 and 8.1 before Session 14.

LESSON PLAN

8.2 Final Project: Design and Build a Database

140 min (of 240)

OBJECTIVES (FROM THE LESSON PAGE)

- Turn a written brief into entities, relationships and business rules, and draw them as a diagram
- Build a normalized schema in a re-runnable script, with keys, foreign keys and CHECK constraints that you have tested
- Answer business questions with joins, GROUP BY, CTEs and window functions, using a fixed report date
- Enforce rules that span several rows in a stored procedure with a transaction
- Secure the database with roles and least-privilege accounts, then back it up and prove the backup restores

Today covers the first two objectives in the room (design, schema with tested constraints) and starts the third at home (the Saturday queries). Session 14 covers the last two.

TIMING IN THE ROOM

20m	Frame and live demo: from requirements to tables; four build mistakes and the silent LEFT JOIN
30m	Studio: Milestone 1, design, with a check-in per learner
15m	Guided lab: `gift_cards_starter.sql`, then `debug_gift_card_proofs.sql`
40m	Studio: Milestone 2, 01_schema.sql and its constraint tests (5-minute mini-demo of the test pattern first)
25m	Studio: Milestone 3, 02_data.sql with the edge cases
10m	(in the wrap-up) Milestone 4 explained for home

MENTAL MODEL TO SAY (5 MINUTES)

Use the lesson's four moves, pointing at the board's empty table: "**Underline the nouns.** Things with their own facts become tables; facts about a thing become columns. **Read the verbs** and ask 'one or many?' in both directions: one-to-many puts the foreign key on the many side; many-to-many needs a junction table, and here the junction carries its own fact, the completion date."

Then the move that decides the project: "**Sort the rules by where they can live.** A rule about ONE value, a type or a constraint: NOT NULL, UNIQUE, ENUM, CHECK. A rule BETWEEN tables, a foreign key. A rule that has to LOOK AT OTHER ROWS (only one open loan per tool, at most three per member) can never be a CHECK: it goes in the procedure that creates loans, inside a transaction." Fill in two rows of the board table

together (requirement 2: tools columns, ENUM, CHECK > 0; requirement 5: checkout_tool), then leave the rest for the studio.

Last: "**Leave out what can be calculated.** Overdue, days late, available: they depend on today or on other rows, so they are computed in queries, never stored."

LIVE DEMO (YOU TYPE, THEY FOLLOW; 15 MINUTES)

The lesson's Common Mistakes, typed in the `mysql>` client (any folder; the scratch database is dropped at the end). Ask for a prediction before each block. The outputs are the real ones; they match the lesson page.

1. **A scratch database for the mistakes (1m).** "Before you build Rosa's database, five mistakes that cost the most time in projects like this. I make them in a scratch database so nothing collides with your project."

```
CREATE DATABASE pitfall_demo;
USE pitfall_demo;
```

```
Database changed
```

2. **Mistake 1: a reserved word (2m).** Ask first: "tool condition is in the brief. What could go wrong with this column?" Then run it.

```
CREATE TABLE demo_tools (
    id        INT AUTO_INCREMENT PRIMARY KEY,
    condition VARCHAR(10)
);
```

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual tha
)' at line 3
```

CONDITION is reserved (stored programs use it). Read where the parser stopped: `near 'condition`. Fix: `tool_condition`. Same trap: `order`, `group`, `rank`, `key`.

3. **Mistake 2: a foreign key whose type does not match (3m).** "A parent table, then a loans table whose tool_id is BIGINT."

```
CREATE TABLE demo_tools (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50));

CREATE TABLE demo_loans (
    id      INT AUTO_INCREMENT PRIMARY KEY,
    tool_id BIGINT NOT NULL,
    FOREIGN KEY (tool_id) REFERENCES demo_tools (id)
);
```

```
ERROR 3780 (HY000): Referencing column 'tool_id' and referenced column 'id' ir
```

INT and BIGINT clash; so do INT and INT UNSIGNED. Decide one key type now and write it in the design notes.

4 **Mistake 3: today's date in a CHECK (2m).** "Due dates must be in the future. Sounds like a CHECK, right?" Let them say yes, then run it.

```
CREATE TABLE demo_loans (
    id     INT AUTO_INCREMENT PRIMARY KEY,
    due_on DATE NOT NULL,
    CONSTRAINT chk_due_future CHECK (due_on >= CURDATE())
);
```

```
ERROR 3814 (HY000): An expression of a check constraint 'chk_due_future' conta
```

A CHECK must give the same answer for a row forever. Compare columns of the same row (`due_on >= loaned_on`) and put time rules in the procedure.

5 **Mistake 4: the child before the parent (2m).** "The course records table, written before the members table."

```
CREATE TABLE demo_courses_done (
    member_id INT NOT NULL,
    FOREIGN KEY (member_id) REFERENCES demo_members (id)
);
```

```
ERROR 1824 (HY000): Failed to open the referenced table 'demo_members'
```

Parents first, junction and history tables last; drop in the reverse order.

6 **Mistake 5: the silent one (4m).** "June loans per member, members with none should show 0." Ask for a prediction: how many rows? Most say three.

```
CREATE TABLE demo_members (id INT PRIMARY KEY, name VARCHAR(50));
CREATE TABLE demo_member_loans (member_id INT, loaned_on DATE);
INSERT INTO demo_members VALUES (1, 'Rosa'), (2, 'Kevin'), (3, 'Tessa');
INSERT INTO demo_member_loans VALUES (1, '2026-06-06'), (2, '2026-05-02');

SELECT m.name, COUNT(l.member_id) AS june_loans
FROM demo_members AS m
LEFT JOIN demo_member_loans AS l ON l.member_id = m.id
WHERE l.loaned_on >= '2026-06-01'
GROUP BY m.id, m.name
ORDER BY m.name;
```

```
+------+------------+
| name | june_loans |
+------+------------+
| Rosa |          1 |
+------+------------+
```

Kevin and Tessa vanished, with no error. WHERE runs after the join and throws away the NULL rows. This is the mistake the rubric's Queries row is most often lost on.

7 **The fix: the condition moves into ON (1m).** "Conditions on the right-hand table of a LEFT JOIN belong in ON."

```
SELECT m.name, COUNT(l.member_id) AS june_loans
FROM demo_members AS m
LEFT JOIN demo_member_loans AS l
       ON l.member_id = m.id AND l.loaned_on >= '2026-06-01'
GROUP BY m.id, m.name
ORDER BY m.name;

DROP DATABASE pitfall_demo;
```

```
+-------+------------+
| name  | june_loans |
+-------+------------+
| Kevin |          0 |
| Rosa  |          1 |
| Tessa |          0 |
+-------+------------+
```

The DROP at the end removes the scratch database. Leave the four error numbers on the board: 1064, 3780, 3814, 1824.

◆ STUDIO: MILESTONE 1, DESIGN (30 MINUTES)

- **Task (lesson, Milestone 1):** on paper or in a text file, the tables with their columns, types and keys; the relationships (a sketch like the lesson's diagram is enough); and each of Rosa's eight requirements next to where the design enforces or answers it. No SQL yet.
- **Use the workbook's Milestone 1 page:** the eight-row requirement table and the starter schema checklist (primary keys, named foreign keys with matching types and a deliberate ON DELETE, NOT NULL and UNIQUE, ENUM or lookup table with a reason, DECIMAL and DATE, two or more CHECKs, indexes for the open-loan lookups, no derived data, snake_case plural names).
- **Check-in (2 minutes each, from minute 10):** the learner points at each requirement and says where it lives. Pass = all eight have a home and requirement 5's five rules are in a procedure, not in CHECKs. Ask one "why" question: "why an ENUM there?", "what happens to a member's loans if the member is deleted?". Tick the tracker.
- **Common design slips to coach:** a `courses` text column in members ("how do you store the date for each course?"); `available` or `times_borrowed` columns ("what if a return is recorded and the flag is not reset?"); one `name` column for tools AND members in one table; `ON DELETE CASCADE` on loans ("Rosa said nobody may delete history"); no `returned_on` ("how do you know a loan is open?").
- **Fast finishers:** compare with the reference design on the lesson page (open it now) and write one difference they can defend in their notes; then start Milestone 2.

GUIDED LAB (15 MINUTES)

Files: `labs/module-13/l8_2-final-project/gift_cards_starter.sql`, then `labs/module-13/l8_2-final-project/debug_gift_card_proofs.sql`; reference `labs/module-13/l8_2-final-project/gift_cards_solution.sql`. New practice on the bookstore (reload it first: `SOURCE labs/_states/bookstore.sql`): a gift-card feature with two tables, built like Milestone 2 asks: re-runnable, parents first, every constraint named.

- **Run it as shipped (2m):** four constraints come back (two foreign keys on gift_card_uses, the amount CHECK and PRIMARY on gift_cards) and a TODO line for the balances.
- **TODOs 1 to 4 (8m):** one named constraint each: `uq_gift_cards_code`, `fk_gift_cards_customer`, `chk_gift_cards_dates` (two columns, never CURDATE), the junction's composite PRIMARY KEY and `chk_gcu_amount`. **Expected:** nine constraints, three CHECK clauses. TODO 5 (the balance, derived with LEFT JOIN and COALESCE) is for fast finishers.
- **Then the proofs (5m):** learners write the expected error number next to each test in `debug_gift_card_proofs.sql`, then run it. **Expected:** 3819, 3819, 1452, 1062, 1265, 1452 in that order, one row inside the transaction, zero after ROLLBACK. Ask: "which test would have passed silently before your TODOs?" (B, C and D.)
- **Debrief (1m):** "This file is your Milestone 2 test block in miniature. Copy the shape: START TRANSACTION, one good parent row, one bad statement per rule, ROLLBACK, a count."

STUDIO: MILESTONE 2, BUILD THE SCHEMA (40 MINUTES)

- **Mini-demo first (5m), the test pattern from the lesson:** with the reference schema loaded on the projector (run the lesson's `01_schema.sql` in your tool-library terminal), show the transaction and two of the four tests. The real outputs:

```
START TRANSACTION;

INSERT INTO categories (name) VALUES ('Hand Tools');
INSERT INTO members (first_name, last_name, email, joined_on)
VALUES ('Test', 'Member', 'test@example.org', '2026-01-01');
INSERT INTO tools (name, category_id, replacement_value, acquired_on)
VALUES ('Test hammer', 1, 15.00, '2026-01-01');
```

```
INSERT INTO loans (tool_id, member_id, loaned_on, due_on)
VALUES (1, 1, '2026-06-10', '2026-06-03');
```

```
ERROR 3819 (HY000): Check constraint 'chk_loans_due' is violated.
```

```
INSERT INTO tools (name, category_id, replacement_value, acquired_on)
VALUES ('Mystery tool', 99, 10.00, '2026-01-01');
```

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fa
```

```
ROLLBACK;

SELECT COUNT(*) AS members_left FROM members;
```

```
+--------------+
| members_left |
+--------------+
|            0 |
+--------------+
```

- Say: "a failed statement inside a transaction only undoes itself; ROLLBACK removes the good rows too. CREATE TABLE cannot be rolled back, which is why the tests come after the schema."
- **Task (lesson, Milestone 2):** `01_schema.sql` starting with `DROP DATABASE IF EXISTS tool_library;`, tables parents first, every constraint named, the open-loan indexes; then one statement each constraint must reject.
- **Check-in (from minute 25):** the learner runs `SOURCE 01_schema.sql` twice in a row (no ERROR either time), then the test block (every test refused, `members_left` or their equivalent 0). Tick the

tracker.

- **Coach:** a FK with a different type (3780) or a parent created later (1824): the demo's numbers on the board; a test that is NOT refused (a missing constraint: "which rule did this test expect?"); constraints without names (the error then says `loans_chk_1`: ask them to name it so the error explains itself).

STUDIO: MILESTONE 3, TEST DATA (25 MINUTES)

- **Task (lesson, Milestone 3):** `02_data.sql` that first runs `SOURCE 01_schema.sql`, then about 8 members, 5 categories, 2 courses, 12 tools and 15 or more loans over several months, with explicit ids and the planted edge cases: a suspended member, a member who never borrowed, a tool in repair, a tool never lent, a tie for most popular in a category, loans open, overdue and returned late.
- **Learners following the reference data** get exactly these counts from the lesson's last query (real run):

```
+----------------+-----------+
| table_name     | row_count |
+----------------+-----------+
| members        |         8 |
| categories     |         5 |
| safety_courses |         2 |
| member_courses |         7 |
| tools          |        12 |
| loans          |        16 |
+----------------+-----------+
```

- **Check-in:** the counts query, then "show me your suspended member, your never-borrowed member and your overdue loan". Learners with their own data must be able to point at each edge case by id.
- **Coach:** 1452 halfway through (a child row before its parent's INSERT, or an id typo); 3819 from `chk_loans_due` (dates swapped: the CHECK just caught a real data error, say so); loans for the suspended member that start AFTER the suspension (fine, but say how the story goes) and two open loans for the same tool (the procedure will refuse that later, but INSERT does not: ask them to fix the data).

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Rules that look at other rows written as CHECKs.** "CHECK ((SELECT COUNT(*) ...) < 3)" fails (subqueries are not allowed in a CHECK). Coach with the board's last line: "one row or other rows?" and move the rule to the Milestone 5 procedure, written down now in the design notes.
- **Scripts that only run once.** A learner fixes a column with ALTER TABLE at the prompt and the script no longer matches the database. Coach: "change the file, run the file": the rubric's Schema row and Milestone 8's rebuild depend on it.
- **Happy-path test data.** Every member borrowed, nothing overdue. Coach with the edge-cases lab and Q10: "which of your queries would return nothing? Is that the right answer, or is the data hiding a bug?"

CHECKS FOR UNDERSTANDING

Ask these out loud during the check-ins:

- Where does "at most three tools at once" live, and why not a CHECK? **Answer:** in checkout_tool, inside a transaction: it counts other rows, and a CHECK sees only the row being written.
- Which table carries `completed_on`, and why there? **Answer:** member_courses: it is a fact about the pair member + course.
- Why does a failed test INSERT not stop your test block? **Answer:** a failed statement only undoes itself; the transaction stays open until ROLLBACK.
- Which of your tables would you create last, and drop first? **Answer:** loans (it points at tools and members), and member_courses.

Workbook self-check, items Q1 to Q12 (learners do them at home, after Milestone 3).

Answers: **Q1** C) In `checkout_tool`, inside a transaction that locks the tool's row; **Q2** B) `completed_on`, the date the member finished the course; **Q3** One row only: `Ana` with `2` (Ben and Cy are gone); **Q4** B) `ERROR 3780 (HY000): Referencing column 'tool_id' and referenced column 'id' in foreign key constraint 'fk_loans_tool' are incompatible.`; **Q5** A) So the script can be run again from scratch and always gives the same result; **Q6** C) The number of days a loan is overdue; **Q7** `0` (in a column named `courses_left`); **Q8** B) `categories`, `members`, `tools`, `loans`; **Q9** A) The loans foreign keys have no ON DELETE rule (refuse), and no account is granted DELETE; **Q10** D) The test data has no overdue loan and every member has borrowed something; **Q11** The library adds and lists categories (they are data); the conditions are a fixed list the code relies on; **Q12** A) `ERROR 1064 (42000): You have an error in your SQL syntax ... near 'condition VARCHAR(10)`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners who struggle with the design start from the lesson's reference design table (open the hidden block) and write the "why" for each row in their own words: that is honest Design work at the "meets" level. In Milestone 2, pair them for the test block. A learner far behind can type the reference `01_schema.sql` and then write their OWN tests (the tests are where the understanding shows).

Stretch: the lesson's Exercise 4 idea early (a generated column that makes "one open loan per tool" a UNIQUE constraint); the FAQ's alternative clinic brief as a second design; a lookup table for tool conditions instead of the ENUM, with the trade-off written down.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

On the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note. Sort into "got it" / "not yet" before Session 14: question 2's pile is the list for the Session 14 warm-up.

1. Name one of Rosa's rules that must live in a procedure, and say why it cannot be a CHECK. **Answer:** any of: one open loan per tool, at most three per member, course before the chainsaw, not while in repair, not while suspended: each looks at other rows (or another table), and a CHECK sees only the row being written.
2. Your 01_schema.sql ran once. What must be true for it to run a second time without an error? **Answer:** it starts with `DROP DATABASE IF EXISTS tool_library;` (or drops its tables, children first) and creates everything again.
3. Write one test statement that proves a CHECK on your loans table works. What should MySQL answer? **Answer:** for example an INSERT with `due_on` before `loaned_on`; MySQL must answer `ERROR 3819 ... Check constraint 'chk_loans_due' is violated.`

HOMEWORK (ABOUT 3 TO 5 HOURS)

- **Finish Milestones 1 to 3** if the room did not: the design with every requirement mapped, `01_schema.sql` running twice with its tests, `02_data.sql` with the edge cases and the counts query.
- **Milestone 4, the Saturday questions** (`03_queries.sql`): six commented queries, all with `SET @today = '2026-06-15';`, at least two checked by hand against the data. Learners on the reference data can compare with the lesson's hidden reference; everyone writes down the two hand checks.
- **Lab:** `labs/module-13/l8_2-final-project/edge_cases_starter.sql`: four report queries that only edge cases can test (an inner join that should be a LEFT JOIN, a WHERE that should be in ON, ROW_NUMBER that should be RANK). It is Milestone 4's patterns on the bookstore.
- **Optional:** `labs/module-13/l8_2-final-project/debug_build_order.sql` (four build mistakes in a bike-share script); the self-check Q1 to Q12 in the workbook.
- **Journal:** the lesson's prompt can wait for Session 14; tonight write which requirement was hardest to place and where you put it.
- **Read ahead for Session 14:** Milestones 5 to 7 on the lesson page (try them first, the reference stays hidden) and Lesson 7.2's EXIT HANDLER pattern.

BRING LIST FOR SESSION 14

- The `tool-library` folder with `01_schema.sql`, `02_data.sql` and `03_queries.sql` that rebuild the database from scratch (Session 14 starts by running them).
- The design notes and the printed rubric.
- A laptop with MySQL 9.7 and the bookstore loaded (the Session 14 labs use it), and the root password.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** a passed design check-in for every learner; 01_schema.sql running twice with its tests; the demo's

Mistake 5.

- **Cut first (saves 15m):** the guided lab. Hand out the gift-cards files as homework; the Milestone 2 mini-demo shows the test pattern.
- **Cut second (saves 6m):** demo Mistakes 2 and 4. Point at the lesson page and the trap table.
- **Cut third (saves 25m):** the Milestone 3 studio: it becomes homework (it is mostly typing). Make sure everyone has seen the edge-case list on the slide.

RUNNING SHORT: WHAT TO ADD

- Start Milestone 4 together: query 1 (on the shelf now) with NOT EXISTS, checked by hand against the data (7 of 12 tools for the reference data).
- Run `debug_build_order.sql` for everyone; each fix is one volunteer.
- Show the reference design and let pairs argue for one difference in their own design (two minutes each).

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 14: Final Project Studio II, Showcase and Next Steps

How to run the last session of MySQL Foundations. Studio II takes the final project through business logic, least-privilege accounts and a backup proven by a restore; every learner presents in the showcase; Lesson 8.3 turns the course into habits (SQL injection and placeholders, an information_schema health check pointed at their own project) and the course closes with submission, evaluation and next steps.

2 lessons | 3 hrs session | Lessons 8.2–8.3 · Milestones 5–7 | Session 14 of 14 · Part 2

HOW TO USE THIS GUIDE

- Session 14 closes the course. Before the break: **Studio II** (Milestones 5 to 7 of the final project, Lesson 8.2), with a short frame and demo. After the break: the **showcase** (every learner presents for three minutes), then **Lesson 8.3** (best practices, the health check, SQL injection, what to learn next) and the course close.
- Every SQL block below was run for real on MySQL 9.7. The 8.2 outputs come from the lesson's own reference solution run as one session (they match the page); the account logins and the dump and restore were measured by logging in as the accounts and running mysqldump. The 8.3 outputs come from the lesson page's own blocks on a fresh bookstore.
- Keep `lesson_final_project.html`, `lesson_21.html` and `final-assessment.html` open. The final-assessment page holds the rubric, scoring guidance, the reference numbers for every milestone and the submission rules.
- **Prepare the projector copy before class:** in a terminal started inside a folder holding the lesson's reference scripts, build tool_library (Milestones 2, 3 and 5's procedures, not the calls), so the demo's calls answer exactly as above. Do not show the scripts themselves.
- Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes match the course site.

> **THE #1 RISK OF SESSION 14: NO TIME LEFT FOR THE SHOWCASE AND THE CLOSE**
>
> Milestone 5 expands to fill any time you give it. The showcase and Lesson 8.3 are what turn a project into a finished course, so they are fixed. **The defense:** at 1:05 everyone moves to Milestone 6 whatever the state of their procedure (a procedure that enforces three rules is "meets" on the rubric; the rest goes home). At 1:30 the break starts on time. A learner whose procedure is broken presents a design decision and a query instead.

BEFORE THE SESSION

- ☐ **Projector copy of the reference:** tool_library built from the lesson's scripts (see How to Use), and the bookstore loaded fresh in a second terminal for the 8.3 demo and the labs.
- ☐ **Files to hand out:** `labs/module-14/` without the `*_solution.sql` files. Run `python3 labs/check_labs.py module-14` once (10 files, every line PASS).
- ☐ **Showcase order:** a list on the board or a shared document; a timer visible to the room (3 minutes, then 30 seconds of questions). Groups larger than ten: run two showcase rooms or a gallery walk (see the showcase block).
- ☐ **Print or share:** the showcase cards ("one strength, one question", on sticky notes), the workbook (its showcase page and course-close page), the submission instructions from `final-assessment.html`, and the course evaluation link.
- ☐ **Board, before learners arrive:** the five-box procedure shape; "Milestone 6 at 1:05 · break 1:30 · showcase 1:40"; the three showcase prompts (a design decision · a Saturday answer checked by hand · a rule refused live).
- ☐ **Certificates or completion records**, if your program issues them: the completion rule is in `final-assessment.html` (the project is submitted after this session, so completion is confirmed after grading).

KNOWN TRAPS IN THIS SESSION

SYMPTOM	CAUSE	FIX
`CREATE PROCEDURE` ends at the first `;`	No `DELIMITER //` before it (or a missing `DELIMITER ;` after)	Lesson 7.1's DELIMITER lines around every procedure in 04_logic.sql.
A failed check-out leaves the session in an open transaction (later statements behave oddly)	No EXIT HANDLER, or a handler without ROLLBACK	`DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;` as the last DECLARE.
`ERROR 1337 (42000): Variable or condition declaration after cursor or handler declaration`	The handler declared before a variable (measured on 9.7)	Variables first, then the handler; every DECLARE before the first statement.
A call for a member that does not exist fails with 1452 instead of a clear message	No `IS NULL` test: a SELECT ... INTO that finds no row leaves the variable NULL with only a warning, and `NULL <> 'active'` is not true, so the INSERT runs and the foreign key refuses it	Test `IS NULL` first after each SELECT ... INTO, as the reference does ('No such member').

SYMPTOM	CAUSE	FIX
`tl_desk_app` gets 1142 on everything, even SELECT	The role was granted but not activated	`SET DEFAULT ROLE 'tl_desk' TO 'tl_desk_app'@'localhost';` (Lesson 7.4's silent mistake); log in again.
`mysqldump: command not found` at the mysql> prompt, or ERROR 1064	mysqldump typed inside the client	It is a terminal command (Lesson 8.1).
Restore fails with a DEFINER or privilege error	Restoring as a non-root account	Restore as root (the lesson says so): the view and procedures carry `DEFINER=root@localhost`.
Learners with their own data get other counts in Milestone 7	Expected	The check is that the two counts match each other, not that they are 18.

SESSION 14 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Rebuild check: every learner runs 02_data.sql and 03_queries.sql from their tool-library folder; three questions on Milestone 4.
0:10	15	**8.2** Frame and live demo: rules across rows	The shape of checkout_tool (lock, check, SIGNAL, insert, commit; the EXIT HANDLER) and what a failing call per rule looks like.
0:25	40	**8.2** Studio: Milestone 5, Business logic	04_logic.sql: v_open_loans, checkout_tool, return_tool, one failing call per rule; logic check-in. Stuck learners do `add_item_rules_starter.sql` first.
1:05	25	**8.2** Studio: Milestones 6 and 7, Security and backup	05_security.sql and a login as the desk account; mysqldump, restore into tool_library_test, compare counts.
1:30	10	Break	Learners pick the three things they will show. Set up the showcase order on the board.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:40	35	**8.2** Showcase	Three minutes per learner: one design decision, one Saturday answer checked by hand, one rule refused live. Peers write one strength and one question.
2:15	20	**8.3** Frame and live demo	Best practices as habits; SQL injection and the placeholder fix; information_schema checks; safe-update mode.
2:35	15	**8.3** Start the lesson project	health_check.sql pointed at tool_library (Steps 1 to 4); the learning plan goes home.
2:50	10	Wrap-up and course close	Final submission, exit ticket, course evaluation, what comes next.
Total	180		Ends at 3:00.

In the room. Lesson 8.2's Milestones 5, 6 and 7 (the lesson estimates 40 + 20 + 20 minutes) get 65 minutes plus a 15-minute frame; the showcase takes 35. Lesson 8.3 (80 minutes for a solo learner) gets 35 minutes: the frame, the injection and health-check demo, and Steps 1 to 4 of its project on the learner's own tool_library. The rest of 8.3 goes home.

Goes home (about 2 to 3 hours, plus the project). Finish the final project: Milestones 5 to 7 if unfinished, Milestone 8 (rebuild from the files on a clean server, the self-review list) and `README.md`; submit within one week. Lesson 8.2's Exercises 1 to 3 (they run on tool_library as Milestone 5 leaves it), both Quick Quizzes, both journal entries (the 8.2 prompt about a changed design decision; the 8.3 prompt looking back at Lesson 1.1). Lesson 8.3's project Step 5 (`learning_plan.md`) and Exercises 1 and 2.

Protect, in this order: the showcase for every learner; Milestone 6 (accounts that are not root, no DELETE); the 8.3 injection demo; the course close. **Cut first:** the 8.3 health-check demo steps (the project start runs the same checks), then the Milestone 5 frame's success calls, then Milestone 7 in the room (the commands go home with the reference numbers from the slide).

OPENING SEGMENT

0:00 Warm-up: the Rebuild Check

10 min

TIMING

5m	Rebuild check: 02_data.sql, then 03_queries.sql, from the tool-library folder
5m	Three questions on Milestone 4 (answers in the Answer Key)

WARM-UP QUESTIONS

W1

From a client started in your tool-library folder: `SOURCE 02_data.sql`, then `SOURCE 03_queries.sql`. Which line fails, if any? *(Run your scripts)*

W2

Saturday question 3 uses RANK(). What would ROW_NUMBER() lose? *(Milestone 4)*

W3

Why does every Saturday query read `@today` instead of `CURDATE()`? *(Milestone 4)*

SEQUENCE AND TALKING POINTS

- **Rebuild check (5m).** Say: "Start your client inside tool-library and rebuild everything from your files. If anything needs a manual fix, that is your first job today." Walk the room once; learners whose scripts fail go on the tracker for the first visit of the studio.

- **Milestone 4 (5m).** W2 and W3 by cold call; then ask two learners to read a hand check aloud ("7 of 12 tools on the shelf: 2, 4, 7 and 10 are out and 12 is in repair"). Learners on the reference data compare with the hidden reference; learners with their own data compare with their hand checks.
- **Learners who did not do Milestone 4:** they do Milestone 5 today anyway (it does not depend on the queries) and Milestone 4 at home. The showcase can show a query from Milestone 2's tests instead.

8.2 Final Project: Design and Build a Database

115 min (of 240)

OBJECTIVES (FROM THE LESSON PAGE)

- Turn a written brief into entities, relationships and business rules, and draw them as a diagram
- Build a normalized schema in a re-runnable script, with keys, foreign keys and CHECK constraints that you have tested
- Answer business questions with joins, GROUP BY, CTEs and window functions, using a fixed report date
- Enforce rules that span several rows in a stored procedure with a transaction
- Secure the database with roles and least-privilege accounts, then back it up and prove the backup restores

Today: the last two objectives in the room (rules in a procedure with a transaction; roles, accounts, a backup that is proven to restore), and every objective in the showcase.

TIMING IN THE ROOM

15m	Frame and live demo: the shape of checkout_tool; one failing call per rule
40m	Studio: Milestone 5, 04_logic.sql
25m	Studio: Milestones 6 and 7, 05_security.sql, a login as the desk account, the backup and the restore test
35m	Showcase (after the break)

MENTAL MODEL TO SAY

"Last session you sorted the rules: one row → a constraint, other rows → a procedure. Today you write that procedure. It is the only door into the loans table: the desk's account may not INSERT into loans at all, only CALL checkout_tool, and the procedure runs with its definer's rights, so every loan passes through every rule."

"A procedure that checks first and inserts second has a gap: two desk volunteers can both see the drill as free. The transaction and the row lock close the gap; the handler makes sure a refusal undoes everything and tells the caller why."

LIVE DEMO (15 MINUTES; SHOW THE ANSWERS, NOT THE CODE)

1. **The shape, on the board (no code yet) (4m).** Draw five boxes in a column and say each one: "START TRANSACTION · read and LOCK the rows the rules need (SELECT ... FOR UPDATE) · check each rule, SIGNAL a clear message when one fails · INSERT · COMMIT. Around it all, one EXIT HANDLER:

ROLLBACK, then RESIGNAL." Point at the five rules from requirement 5 and ask which row each one reads (member: status; tool: condition and course; loans: open loan for this tool, open loans for this member; member_courses). Do not show the reference procedure: Milestone 5 is the learners' own work. Lesson 7.2's handler and Lesson 7.1's SIGNAL are the two pieces they already have.

2. **What "proven" looks like: one failing call per rule (6m).** "This is the reference procedure answering. Yours must give each rule its own message." Run the calls on your projector copy (built from the lesson's scripts before class).

```
CALL checkout_tool(8, 1, '2026-06-15');   -- Victor is suspended
CALL checkout_tool(1, 12, '2026-06-15');  -- tile cutter in repair
CALL checkout_tool(6, 7, '2026-06-15');   -- ladder is out with Amara
CALL checkout_tool(7, 6, '2026-06-15');   -- Tessa has no chainsaw course
```

```
ERROR 1644 (45000): Member is suspended
ERROR 1644 (45000): Tool is out for repair
ERROR 1644 (45000): Tool is already on loan
ERROR 1644 (45000): Safety course required for this tool
```

Four rules, four messages, error 1644 each (SQLSTATE 45000 is "user-defined").

3. **The limit, and a success (3m).** "Kevin already has the socket set. Two more succeed, the fourth must not."

```
CALL checkout_tool(2, 1, '2026-06-15');
CALL checkout_tool(2, 3, '2026-06-15');
CALL checkout_tool(2, 8, '2026-06-15');   -- a fourth tool for Kevin
```

```
+---------+------------+
| loan_id | due_on     |
+---------+------------+
|      17 | 2026-06-22 |
+---------+------------+
+---------+------------+
| loan_id | due_on     |
+---------+------------+
|      18 | 2026-06-22 |
+---------+------------+
ERROR 1644 (45000): Loan limit reached (3 open loans)
```

The successful call prints the new loan id and due date: the desk's web form can show them.

4. **Returns and the desk view (2m).** "return_tool closes the open loan; the view is the desk's screen."

```
CALL return_tool(1, '2026-06-16');
CALL return_tool(3, '2026-06-16');
CALL return_tool(7, '2026-06-16');

SELECT * FROM v_open_loans ORDER BY due_on, loan_id;

CALL return_tool(7, '2026-06-16');        -- the ladder is already back
```

```
+---------+---------+----------------+--------------+-----------+-----------+
| loan_id | tool_id | tool           | member       | loaned_on  | due_on    
+---------+---------+----------------+--------------+-----------+-----------+
|      14 |       2 | Cordless drill | Rosa Delgado | 2026-06-06 | 2026-06-13
|      15 |       4 | Socket set     | Kevin Brooks | 2026-06-10 | 2026-06-17
|      16 |      10 | Carpet cleaner | Hannah Weiss | 2026-06-12 | 2026-06-19
+---------+---------+----------------+--------------+-----------+-----------+
ERROR 1644 (45000): Tool is not on loan
```

18 loans now: 16 from the data plus Kevin's two. Milestone 7's counts use this number.

STUDIO: MILESTONE 5, BUSINESS LOGIC (40 MINUTES)

- **Task (lesson, Milestone 5):** `04_logic.sql` with the view `v_open_loans`, `checkout_tool(member, tool, date)` enforcing all five rules of requirement 5 inside a transaction with a clear message for each, and `return_tool(tool, date)`; NULL as the date means today. Then a call that must fail for every rule and one that must succeed.
- **Stuck on the shape?** Send them to `labs/module-14/l8_2-final-project/add_item_rules_starter.sql` first (15 minutes): the same shape on the bookstore (lock the order, check, SIGNAL, insert, commit, the handler), with `labs/module-14/l8_2-final-project/debug_add_item_rules.sql` as its failing calls. Then back to the project.
- **Check-in (from minute 25):** the learner runs their failing calls: each rule refused with its own message (five messages), one success, and `SELECT * FROM v_open_loans` afterwards. Ask: "what happens to the transaction when the course check fails?" (the handler rolls it back).
- **Coach:** a rule checked AFTER the INSERT (the loan exists before the refusal: move the checks up); a COUNT that includes returned loans (`returned_on IS NULL` missing); `loaned_on + 7` instead of `+ INTERVAL 7 DAY` (it turns the date into a number: measured, DATE('2026-06-28') + 7 is 20260635, not a date); checking `v_condition = 'repair'` when no tool was found (test IS NULL first).
- **At 1:05 everyone moves on**, finished or not (see the #1 risk).

STUDIO: MILESTONES 6 AND 7, SECURITY AND BACKUP (25 MINUTES)

- **Milestone 6 (12m):** `05_security.sql` with the roles `tl_desk` and `tl_reports` and the two accounts, each with SET DEFAULT ROLE. Then log in as the desk account in a second terminal and try

what it must not do. The reference, measured:

```
SHOW GRANTS FOR 'tl_desk_app'@'localhost' USING 'tl_desk';
```

```
+-------------------------------------------------------------------------------
| Grants for tl_desk_app@localhost
+-------------------------------------------------------------------------------
| GRANT USAGE ON *.* TO `tl_desk_app`@`localhost`
| GRANT SELECT ON `tool_library`.* TO `tl_desk_app`@`localhost`
| GRANT INSERT, UPDATE ON `tool_library`.`members` TO `tl_desk_app`@`localhost`
| GRANT EXECUTE ON PROCEDURE `tool_library`.`checkout_tool` TO `tl_desk_app`@`loc
| GRANT EXECUTE ON PROCEDURE `tool_library`.`return_tool` TO `tl_desk_app`@`local
| GRANT `tl_desk`@`%` TO `tl_desk_app`@`localhost`
+-------------------------------------------------------------------------------
```

```
-- logged in as tl_desk_app:
CALL checkout_tool(4, 1, '2026-06-17');
DELETE FROM loans WHERE id = 1;
INSERT INTO loans (tool_id, member_id, loaned_on, due_on) VALUES (3, 4, '2026-06-1
```

```
+---------+------------+
| loan_id | due_on     |
+---------+------------+
|      19 | 2026-06-24 |
+---------+------------+
ERROR 1142 (42000): DELETE command denied to user 'tl_desk_app'@'localhost' for t
ERROR 1142 (42000): INSERT command denied to user 'tl_desk_app'@'localhost' for t
```

- The CALL works (definer's rights); the direct DELETE and INSERT are refused. The treasurer's account reads (`SELECT COUNT(*) FROM loans` answers) and is refused an UPDATE with the same 1142.
- **Milestone 7 (13m):** in the terminal, the lesson's mysqldump command, `CREATE DATABASE tool_library_test`, the restore, then the comparison in the client. Measured on the reference after Milestone 5 (338-line dump, no errors):

```
+----------------+------------+
| loans_original | loans_copy |
+----------------+------------+
|             18 |         18 |
+----------------+------------+
```

- Both counts 18; `checkout_tool` and `return_tool` are in the copy (defined by root@localhost); the copy's `v_open_loans` lists 3 open loans. Then `DROP DATABASE tool_library_test;` and one

line in the README with the date and the counts.

- **Check-in:** the learner shows SHOW GRANTS for the desk account (no DELETE, no INSERT on loans) and the two matching counts.

SHOWCASE (35 MINUTES, AFTER THE BREAK)

Three minutes per learner (pairs: four minutes per pair), then 30 seconds for one question. Every presenter shows three things, from their own laptop on the projector or as screenshots:

1. **One design decision and why** (an ENUM or a lookup table, a foreign key's ON DELETE rule, where a rule lives).
2. **One Saturday answer, checked by hand** ("7 of 12 on the shelf: here is why").
3. **One rule refused live** (a failing CALL, or a constraint test).

Listeners write one strength and one question on a sticky note per presenter and hand them over; you fill the showcase sheet below (one strength per learner goes into the written feedback). The showcase is practice: it is **not scored**. Rubric scores come from the submitted files.

- **More than ten learners:** split into two rooms with a co-facilitator, or run a gallery walk (laptops open at the three things, half the room presents while the other half walks, then swap: 2 × 15 minutes).
- **A learner whose procedure is broken** shows a design decision, a query and a constraint test instead. Nobody skips the showcase.
- **Close the showcase** with one minute from you: the three strengths you saw most often, and the one fix you saw most often (it goes in everybody's README as a next step).

SHOWCASE SHEET

LEARNER	DESIGN DECISION (AND WHY)	SATURDAY ANSWER, CHECKED BY HAND	RULE REFUSED LIVE	ONE STRENGTH TO WRITE IN THE FEEDBACK

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Checks without a lock.** The procedure checks and inserts correctly, but reads the tool without FOR UPDATE. Coach with two terminals: "open a transaction in each and lend the same drill" (Lesson 7.2's two-session demo). Q5 in the self-check.
- **root everywhere.** The desk account was skipped, or created with ALL PRIVILEGES. Coach with Rosa's requirement 7 and the rubric's Security row ("desk account can only change data through procedures").

- **A backup that was never restored.** "The dump file exists" is the "meets" level; full points need the restore and the matching counts in the README. Coach: "a backup is only a backup once it has been restored."

CHECKS FOR UNDERSTANDING

Ask during the studio check-ins:

- Why can the desk account create a loan it may not INSERT? **Answer:** the procedure runs with its definer's rights (SQL SECURITY DEFINER, the default).
- What does RESIGNAL add to ROLLBACK? **Answer:** the caller still receives the original error and message.
- Your dump restored. What else must you keep to rebuild Rosa's system? **Answer:** 05_security.sql (the accounts are not in the dump) and the numbered scripts.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** C) `ERROR 1644 (45000): Safety course required for this tool`; **Q2** A) So a failed check-out never leaves a half-finished transaction, and the caller still receives the real error message; **Q3** `ERROR 1142 (42000): DELETE command denied to user 'tl_desk_app'@'localhost' for table 'loans'`; **Q4** D) The accounts tl_desk_app and tl_treasurer and their roles; **Q5** B) So a second check-out of the same tool waits until the first commits, then sees the new loan; **Q6** Four rows; loan 13 (the extension ladder, `12` days) first. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: the add_item_rules lab first; then a procedure with three rules ("meets" on Business logic) is a fine result for today. In Milestone 6, the lesson's reference 05_security.sql may be typed and then TESTED by the learner: the login test is where the understanding shows.

Stretch: the lesson's Exercise 4 (a generated column that makes one open loan per tool a UNIQUE constraint), the Bonus Challenge (reservations), or a scheduled backup (Lesson 8.1 Step 6) for tool_library.

8.3 Next Steps & Best Practices

35 min

OBJECTIVES (FROM THE LESSON PAGE)

- Summarize what each module of the course taught and where to look things up again
- Apply a short list of design, query, security and operations practices to any MySQL project
- Audit a database with information_schema queries for missing keys, risky types and old character sets
- Explain SQL injection and write parameterized queries, in SQL and in an application language
- Choose a next learning step that fits your goal

TIMING IN THE ROOM

5m	Frame: the course in one page; habits, not rules
15m	Live demo: SQL injection and the placeholder; information_schema checks; safe-update mode
15m	Start the lesson project: health_check.sql on tool_library (Steps 1 to 4)

MENTAL MODEL TO SAY

Show the lesson's module table ("The Course in One Page") and say: "Every practice in this lesson prevents a problem you met in this course: a missing primary key, FLOAT money, an UPDATE without WHERE, a password in code. Habits are cheap insurance."

Then the one idea that matters most when they leave: "Real users never see a mysql> prompt. Applications send SQL through a driver, and the user's input travels as a PARAMETER, separate from the SQL text. Glue input into SQL and someone else writes your WHERE clause."

LIVE DEMO (YOU TYPE, THEY FOLLOW; 15 MINUTES)

The lesson's own blocks on a freshly loaded bookstore; outputs are real and match the page.

1. **Building SQL from input: the leak (5m).** "A web form looks up a customer by the email someone typed. Here is what gluing the input into the SQL does when the input is crafted." Read the query_sent line aloud.

```
SET @input = 'nobody@example.com'' OR ''1''=''1';

SET @sql = CONCAT('SELECT first_name, email FROM customers WHERE email = ''',
SELECT @sql AS query_sent;

PREPARE lookup FROM @sql;
EXECUTE lookup;
DEALLOCATE PREPARE lookup;
```

```
+-------------------------------------------------------------------------------
| query_sent
+-------------------------------------------------------------------------------
| SELECT first_name, email FROM customers WHERE email = 'nobody@example.com' (
+-------------------------------------------------------------------------------
Statement prepared
+------------+-------------------+
| first_name | email             |
+------------+-------------------+
| Alice      | alice@example.com |
| Bob        | bob@example.com   |
| Carol      | carol@example.com |
| Dave       | dave@example.com  |
| Eve        | eve@example.com   |
| Frank      | frank@example.com |
| Grace      | grace@example.com |
| Hank       | hank@example.com  |
+------------+-------------------+
```

Every customer's email leaks, with no error. Variations delete tables or skip logins.

2. **The fix: a placeholder (3m).** "The value travels separately and can only ever be a value. Every driver does this."

```
PREPARE lookup FROM 'SELECT first_name, email FROM customers WHERE email = ?';
EXECUTE lookup USING @input;
DEALLOCATE PREPARE lookup;
```

```
Statement prepared
Empty set
```

Show one driver example on the lesson page (the language your group uses next): `?`, `%s`, `:name`, `@name` are all the same idea.

3. **information_schema checks a design (5m).** "Every server describes itself in tables. Plant an old-style table, then let a query find it."

```
CREATE TABLE hc_demo_payments (
    paid_on DATE,
    amount  FLOAT,
    note    VARCHAR(100) CHARACTER SET latin1
);

SELECT t.TABLE_NAME AS table_without_pk
FROM information_schema.TABLES AS t
LEFT JOIN information_schema.TABLE_CONSTRAINTS AS c
       ON c.TABLE_SCHEMA = t.TABLE_SCHEMA
      AND c.TABLE_NAME = t.TABLE_NAME
      AND c.CONSTRAINT_TYPE = 'PRIMARY KEY'
WHERE t.TABLE_SCHEMA = 'bookstore'
  AND t.TABLE_TYPE = 'BASE TABLE'
  AND c.CONSTRAINT_NAME IS NULL
ORDER BY t.TABLE_NAME;
```

```
+------------------+
| table_without_pk |
+------------------+
| hc_demo_payments |
+------------------+
```

4 **Risky types (2m).** "FLOAT for money and latin1 text."

```
SELECT TABLE_NAME, COLUMN_NAME, COLUMN_TYPE, CHARACTER_SET_NAME
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = 'bookstore'
  AND (DATA_TYPE IN ('float', 'double')
       OR CHARACTER_SET_NAME NOT IN ('utf8mb4', 'binary'))
ORDER BY TABLE_NAME, ORDINAL_POSITION;
```

```
+------------------+-------------+--------------+--------------------+
| TABLE_NAME       | COLUMN_NAME | COLUMN_TYPE  | CHARACTER_SET_NAME |
+------------------+-------------+--------------+--------------------+
| hc_demo_payments | amount      | float        | NULL               |
| hc_demo_payments | note        | varchar(100) | latin1             |
+------------------+-------------+--------------+--------------------+
```

Then `DROP TABLE hc_demo_payments;`. FLOAT for money: the lesson's demo sums 0.10 + 0.20 + 19.99 to 20.289999775588512.

5 **Safe-update mode (2m).** "A forgotten WHERE, caught."

```
SET SQL_SAFE_UPDATES = 1;

UPDATE books SET price = price * 1.10;
```

```
ERROR 1175 (HY000): You are using safe update mode and you tried to update a t
```

Then `SET SQL_SAFE_UPDATES = 0;`.

START THE LESSON PROJECT: HEALTH CHECK (15 MINUTES)

- **Steps 1 and 2:** `health_check.sql` with `SET @db = 'bookstore';` and the three checks; on the bookstore, checks 1 and 2 print Empty set (the result you want) and the table count is 8.
- **Step 3:** the unindexed-foreign-key check (Empty set on the bookstore).
- **Step 4, the point of today:** `SET @db = 'tool_library';` and run it on their own project. On the reference tool_library (measured): 6 tables; no table without a primary key; no FLOAT or non-utf8mb4 column; every foreign-key column leads an index; and the delete rules:

```
+----------------+-------------------+-----------------------+-------------+
| TABLE_NAME     | CONSTRAINT_NAME   | REFERENCED_TABLE_NAME | DELETE_RULE |
+----------------+-------------------+-----------------------+-------------+
| loans          | fk_loans_member   | members               | NO ACTION   |
| loans          | fk_loans_tool     | tools                 | NO ACTION   |
| member_courses | fk_mc_course      | safety_courses        | NO ACTION   |
| member_courses | fk_mc_member      | members               | CASCADE     |
| tools          | fk_tools_category | categories            | NO ACTION   |
| tools          | fk_tools_course   | safety_courses        | NO ACTION   |
+----------------+-------------------+-----------------------+-------------+
```

- Ask: "why NO ACTION here and RESTRICT on the bookstore?" Both mean "refuse" in InnoDB; the bookstore's script wrote `ON DELETE RESTRICT`, the project wrote no rule at all, and information_schema reports the default as NO ACTION. Only fk_mc_member cascades: course records mean nothing without their member.
- **Goes home:** Step 5 (`learning_plan.md`: a path, a first topic, a project, dates) and the comment block at the top of the file.
- **Fast finishers:** `labs/module-14/l8_3-next-steps/health_check_starter.sql` (two checks the lesson does not have: `_id` columns that no foreign key protects, and the CASCADE/SET NULL list), `labs/module-14/l8_3-next-steps/safe_search_starter.sql` (placeholders on the bookstore) and `labs/module-14/l8_3-next-steps/debug_unsafe_habits.sql` (safe-update mode).

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **"Escaping quotes is enough."** Coach: placeholders cost nothing and cannot be gotten wrong; escaping by hand can. The lesson says so in its Common Mistake 1.
- **Health check reports nothing, so the database must be fine.** Coach: run it on the planted table (the lab plants one) to see that each check CAN find something. An empty result only means something when the check can fail.
- **Learning plans with no dates.** Coach with the lesson's Step 5: a path, one first topic, one small project and a date next to each.

CHECKS FOR UNDERSTANDING

- What makes a parameterized query safe? **Answer:** the value is sent separately from the SQL text, so it can never become part of the statement.
- Which information_schema table tells you what deleting a parent does? **Answer:** REFERENTIAL_CONSTRAINTS (DELETE_RULE).
- Which MySQL version do you install for production next year? **Answer:** the current LTS (9.7 or 8.4), asked for by name.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** D) All eight customers; **Q8** A) They are deleted too (fk_items_order is CASCADE); **Q9** It is refused: `ERROR 1175` (safe update mode: no KEY column in the WHERE); **Q10** C) It is the "find what is missing" pattern: keep every table, then keep only those that found no PRIMARY KEY row; **Q11** B) In an environment variable or a secrets manager, never in the code or the repository; **Q12** `EXECUTE stmt USING @author, @max_price;`: user variables, in the same order as the ? placeholders. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: the health-check starter has check 1 done; Steps 1 to 4 on the bookstore only are enough for today. **Stretch:** the lesson's Exercise 3 (a one-line summary per table with correlated subqueries) and its Bonus Challenge (a trigger, a FULLTEXT index or a scheduled event added to the final project, with one thing the manual taught them).

CLOSING

COURSE CLOSE (10 MINUTES)

1. **Final submission (3m).** On the slide: the zipped `tool-library` folder (the five scripts, the backup file of the test data, README.md with the restore date and counts, the self-scored rubric), within one week, by the channel your program uses. Placeholder passwords only. The full rules: `final-assessment.html`.
2. **Exit ticket (3m).** Three questions below.

3. **Course evaluation (2m).** The link or the paper form; ask for one sentence on what to keep and one on what to change.
4. **What comes next (2m).** Show "Choose a path by goal" from Lesson 8.3 once more, and the course's reference pages (cheat sheet, glossary, common errors, troubleshooting). Thank them; name one thing the group did well.

EXIT TICKET (3 QUESTIONS, NO NOTES, 3 MINUTES)

1. Why can the desk's web form create a loan through checkout_tool although its account may not INSERT into loans? **Answer:** the procedure runs with its definer's rights (SQL SECURITY DEFINER is the default), so it may insert; the caller only needs EXECUTE, and every loan passes the rules.
2. Rewrite `CONCAT('... WHERE email = ''', @input, '''')` safely. **Answer:** `PREPARE s FROM 'SELECT ... WHERE email = ?'; EXECUTE s USING @input;` (a driver: a placeholder plus the value in a list).
3. Name your next step from Lesson 8.3's table and the date you start. **Answer:** any path with a date (the learning plan's first line).

HOMEWORK (ABOUT 2 TO 3 HOURS, PLUS FINISHING THE PROJECT)

- **Finish the final project** and submit it within one week: Milestones 5 to 7 if unfinished; Milestone 8 (rebuild from the files in order on a clean server or after `DROP DATABASE tool_library;`, the self-review list); `README.md` (one page: design, rebuild, backups with the restore date and counts, next steps); score it with the rubric and write the score and one improvement in the README (the lesson's "Before the Next Lesson").
- **Lesson 8.2:** Exercises 1 to 3 (late returns, member activity, value at risk; they run on tool_library as Milestone 5 leaves it, 18 loans); Exercise 4 is a stretch. The Quick Quiz. The journal prompt (a design decision you changed your mind about).
- **Lesson 8.3:** project Step 5, `learning_plan.md` with dates; Exercises 1 and 2 (Exercise 3 is a stretch); the Quick Quiz; the journal prompt (look back at your Lesson 1.1 entry).
- **Optional labs:** `labs/module-14/l8_3-next-steps/safe_search_starter.sql`, `labs/module-14/l8_3-next-steps/health_check_starter.sql`, `labs/module-14/l8_2-final-project/least_privilege_starter.sql`.
- **After the course:** put the first date of the learning plan in a calendar; push the project and health_check.sql to a repository (placeholder passwords only, never a backup of real data, never `.mylogin.cnf`).

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the showcase for every learner; Milestone 6; the injection demo; the course close.

- **Cut first (saves 7m):** the 8.3 demo's information_schema and safe-update steps: the project start runs the same checks.
- **Cut second (saves 3m):** the 8.2 demo's success and return steps.
- **Cut third (saves 13m):** Milestone 7 in the room. Show the slide with the commands and the measured counts; it goes home.

RUNNING SHORT: WHAT TO ADD

- Two-terminal demo of the lock: open a transaction in each terminal and call checkout_tool for the same tool; the second waits, then reports "Tool is already on loan".
- The 8.2 Exercise 3 (value at risk with ROLLUP) together, on the reference data: Cleaning 200.00, Hand Tools 60.00, Power Tools 120.00, all categories 380.00.
- `safe_search_starter.sql` for everyone, then one learner explains the placeholder in their next language.
